

EDK II Build Specification

TABLE OF CONTENTS

EDK II Build Specification

Tables

Figures

1 Introduction

1.1 Overview

1.2 Target Audience

1.3 Terms

1.4 Related Information

1.5 Conventions Used in this Document

2 Design Discussion

2.1 Development Environments

2.2 UEFI/PI Firmware Images

2.3 Boot Sequence

2.4 Typical Flash Part Layout

2.5 Generic Build Process

2.6 Creating EFI Images

2.7 SKU Support

3 UEFI and PI Image Specification

4 EDK II Build Process Overview

4.1 EDK II Build System

4.2 Build Process Overview

4.3 Pre-Build Stage Overview

4.4 Creating Binary EFI Images - \$(MAKE) stage

4.5 Post-Build Stage

4.6 File Specifications

4.7 File Extensions

5 Meta-Data File Specifications

5.1 Build Meta-Data File Formats

5.2 tools_def.txt

5.3 target.txt File

6 Quick Start

6.1 Environment Variables

6.2 Build Scope

7 Build Environment

7.1 Build Scope

7.2 Third Party Tools

7.3 GUIDed Tools

8 Pre-Build AutoGen Stage

8.1 Overview

8.2	Auto-generation Process
8.3	Auto-generated code
8.4	Auto-generated PCD Database File
8.5	Auto-generated Makefiles
8.6	Binary Modules
8.7	Generated AsBuilt INF Files
9	Build or \$(MAKE) Stage
9.1	Overview
9.2	Preprocess/Trim
9.3	Compile/Assembly
9.4	Static Link
9.5	Dynamic Link
9.6	Generate Module Images
9.7	Generate Platform Images
10	Post-Build ImageGen Stage - FLASH
10.1	Overview of Flash Device Layout
10.2	Parsing FDF Meta-Data File
10.3	Build Intermediate Images
10.4	Create the FV Image File(s)
10.5	Create the FD image file(s)
10.6	Post Build Processing
11	Post-Build ImageGen Stage - Other
11.1	EFI PCI Option ROM Images
11.2	UEFI Applications
11.3	Capsules
12	Build Changes and Customizations
12.1	Building for Debug
12.2	Adding Custom Compression Tools
12.3	Using Custom Build Tools
12.4	Customizing Compilation for a Component
12.5	Platform Specific ASL Tools
12.6	Build Reproducibility
13	Build Reports
13.1	Build Report Generation Options
13.2	Sample Launch Steps: NT32 platform
13.3	Output
13.4	Platform Summary
13.5	Mixed PCD Section
13.6	Global PCD Section
13.7	FD Section
13.8	Module Section
13.9	Execution Order Prediction Section

[Appendix A Variables](#)

[Appendix B tools_def.txt](#)

[Appendix C target.txt](#)

[Appendix D build.exe command](#)

[D.1 Overview](#)

[D.2 Makefile actions](#)

[D.3 Build Targets and options](#)

[D.4 Usage](#)

[Appendix E NT32 Platform Emulation](#)

[Appendix F Firmware Volume INF](#)

[F.1 Firmware Volume INF Description](#)

[F.2 \[Attributes\] Section](#)

[F.3 \[Files\] Section](#)

[F.4 \[Options\] Section](#)

[Appendix G VS2005 Team Suite Performance](#)

[G.1 Step 1 - Create a new project](#)

[G.2 Step 2 - Update the project](#)

[Appendix H Module Types](#)

[Appendix I VPD Tool](#)

[I.1 Build System Output File Format](#)

[I.2 VPD Tool Map File Format](#)

[Appendix J Makefiles](#)

[J.1 NMAKE Module Makefile Format](#)

[Appendix K Third Party Tool Flags](#)



EDK II Build Specification

DRAFT FOR REVIEW

07/16/2026 08:51:21

Acknowledgements

Redistribution and use in source (original document form) and 'compiled' forms (converted to PDF, epub, HTML and other formats) with or without modification, are permitted provided that the following conditions are met:

1. Redistributions of source code (original document form) must retain the above copyright notice, this list of conditions and the following disclaimer as the first lines of this file unmodified.
2. Redistributions in compiled form (transformed to other DTDs, converted to PDF, epub, HTML and other formats) must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

THIS DOCUMENTATION IS PROVIDED BY TIANOCORE PROJECT "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL TIANOCORE PROJECT BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS DOCUMENTATION, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Copyright (c) 2008-2017, Intel Corporation. All rights reserved.

Revision History

Revision	Revision History	Date
1.0	Initial release.	February 2008
1.1	Update based on errata	August 2008
1.2	Updates based on enhancement requests	June 2009
1.21	Updates based on errata and enhancement requests	January 2010
	Definitions in DSC file [defines] section are now global to both DSC and FDF files	
	Added language filters: RFC_LANGUAGES and ISO_LANGUAGES	
	Rule processing for file type lists is alphabetical, i.e., files are added in alphabetical order	
	Added warning for VFR file naming convention - cannot use a name that is also used for a C file	
	Use of the IDENTIFIER statement in tools_def.txt is optional	

	Whitespace characters are permitted in the meta-data files, so tools must handle them (use of token based processing is recommended)	
	Tools must support any number of FV_EXT_ENTRY_TYPE statements in an FDF file	
	The build tools must auto compute the ExtHeaderOffset in the FV Header.	
	The build tools must auto compute the ExtHeaderSize in the FV Ext Header based on the size of the FV Ext Header and all the FV Ext Header Entries.	
	The build tools must auto compute the ExtEntrySize field in the FV Ext Header Entry structure based on the size of the file specified by the FILE statement or the number of bytes in the byte array of the DATA statement. If the size is greater than 16-bits, the build should break.	
	Specified parsing priority rules for definitions and values	
	Added report generator syntax as part of the build	
	Add support for "Auto" alignment for PE32 and TE images	
	Add support for specifying block information for capsules	
	FeatureFlagExpression processing should allow C-style expression syntax and follow C rules for processing	
1.22	Grammatical and formatting changes. Added Module types appendix.	May 2010
1.22 w/	Updated to match the implementation at the time of the UDK2010 SR1 release:	December 2011
Errata A	Updated to support UEFI version 2.3.1 and updated spec release dates in Introduction	
	Clarify UEFI's PI Distribution Package Specification	
	Spelling and punctuation fixes	
	Updated document title, removed ADD_BUILD_RULE from target.txt, Added VPD information, Fixed VOID* string format, Break build if EDK II modules uses ISO 639-2 language codes, updated macro usage in tools_def.txt	
	Allow user visible language name to contain space characters	
	Updated language code from RFC 3066 to RFC 4646 as well as adding additional content that matches implementation	
	Update to specify VOID* data must be translated to either a Hex Byte Array, a C Format GUID or a valid C format string	
	Update AutoGen section regarding UEFI and PI specification versions _gUefiDriverRevisions, _gDxeRevision or _gPeimRevision	
	Add rules and file formats for the VPD tools	
	Added description of EDK GLOBAL macro utilization - defined in DSC, used in EDK II DSC, EDK II FDF and EDK INF files only	
	Added description of the SOURCE_OVERRIDE_PATH for EDK INF files only	
	Added ECP_SOURCE system environment variable	
	Remove SET statements from DSC processing	
	Use just the PcdName in conditional directive statements. \$(PcdName) and PCD(PcdName) are not permitted in conditional directive statements defined in the DSC and FDF files	
	Document what the valid numbers are for the debug switch for build.exe, updated EBNF for -D command-line option	

	Specify normal build report items, including the number of warning messages that might come from the EDK II build tools (not the third party tools, just the EDK II build system tools), correct module report for dependency expressions, correct PCD reports, removed notification report section, added Fixed Address Prediction and EOT sections to the reports	
	Prohibit listing multiple library class instances or PCD entries within a single section in EDK II Meta-data files	
	Updated to remove path restriction on macro values, clarified how architectures are selected for the build, clarified macro precedence tables	
	Updated MACRO EBNF inside of tools_def.txt definitions	
	Make sure that generated values in the AutoGen stage have either a u or a ull appended to the value to ensure that they are unsigned (PCD values are always unsigned integers)	
	Update description of the TOOL_CHAIN_TAG so that if it is not specified, the build will break	December 2011
	Remove CREATE_FILE from specification, 10.5.1, as this was never supported	
	Updated Create FFS files from Leaf sections description in Rules section	
	Specify how sections are merged during parsing of the EDK II meta-data files	
	Specify how the maximum size of a <code>VOID*</code> PCD is calculated if it is not specified	
	Allow any non-zero number to evaluate to True	
	Change "should" to say recommended	
	Require MdePkg/MdePkg.dec in the [Packages] section of all INF files listed in the DSC file	
	updated comments and operand notes in table	
	REMOVED "ALL" from the Report Type list, as ALL is not a valid option value	
	Clarify that macros are only expanded when parsing EDK II INF and DEC files, and that Macro values are expanded or evaluated when parsing of EDK II DSC and FDF files	
	Updated 5.2.3 and E.4.2 to allow lower case characters in a MACRO name in tools_def.txt.	
	Updated TOOL_CHAIN_TAG in Appendix C to match definition in chapter 5	
	Removed number of warnings message at the completion of the build - not for SR1 - 13,0	
	Added Support for C++ in auto-generated code	
1.22 w/	Updates:	June 2012
Errata B	Section 1.1, page 1: Added description regarding build system tools and appropriate error conditions for 'forward compatibility'	
	Section 1.4, page 5: Updated Specification Versions to include released Errata	
	Section 5, added note about <code>build_rule.txt</code> updates	
	Section 5.2.3: Added DOS <code><EOL></code> character sequence definition	

	Section 5.3: Added DOS <EOL> character sequence definition	
	Removed Section 5.4, <code>build_rule.txt</code> . This file is for tool usage and not for normal editing purposes.	
	Section 8.5, Removed reference to binary "LIB" - library distribution is not currently supported	
	Section 10, table 19, updated to support <code>EFI_SECTION_FREEFORM_SUBTYPE_GUID</code>	
	Removed section 12.5 for EDK Library INF modifications	
	Removed Appendix_D_Build_rule.txt	
	Appendix I Updated - clarify that Unicode file is a UCS-2 encoded file, and that the Language Code for EDK II modules must be RFC4646 language codes; EDK components support UCS-2 encoded files containing ISO639 language codes	
1.22 w/	Updates:	August 2013
Errata C	Section 1.3, Updated Errata for UEFI Specs.	
	Sections 4.3.4, 4.4, 4.5.3, 8.1, 8.4, 9.7, 10.1, Appendix E.2, Removed 8.4.2, Appendix L.1 and L.2, removing Top level and Architectural level Makefiles - build system has been modified to have Python call the module Makefiles if they exist; also updating figures to remove the 'Platform' Makefile	
	Section 8.5.1.2.4 - removed fds target in the makefile, as the build command will call the GenFds if the fds target is given on the command-line	
	Section 4.1, 5, 5.3, D.4, Allow DSC, FDF and the Conf directory to be located outside of the WORKSPACE directory(--conf=CONF_DIRECTORY)	
	Section 5.2.3, Table 9, Added ADDDEBUGFLAG predefined attribute	
	Section 6.4.4.2 remove line following GenFv --help, as this is not the correct usage	
	Section 7.1.5, State that flags are processed from left to right, with the right most flag overriding a flag that is to the left	
	Added new section 8.5 added wording for generating "As Built" INF files as part of the AutoGen process	
	Added new section 8.4 describing the PEI and DXE Dynamic PCD database generation	
	8.3.5.5.2 Added statement regarding when to generate a #define statement for Fixed At Build PCDs in libraries	
	Build system reports the total number of warning messages emitted by the EDK II tools (not the third party tools invoked by the EDK II build system). Added support for lower case characters in the UEFI Compliant Unicode Token entry.	
	Cleaned up EBNF for HII Tokens to remove ambiguity. Added support for Patchable In Module PCD for overriding a Formset Class GUID in a binary HII driver.	
	Added additional rules for PCD Database generation	
1.22 w/	Updates:	January 2014
Errata D	Added text describing new DSC flag to generate a special version of the PCD database that contains both the PCD token name and the token number. New section, 8.2.4.10 provides text describing this option.	
	Updated section 10.5 to describe how PCDs are used in expressions in	

	the [FD] offset fields.	
	Updated section 10.2 to describe the last statement taking precedence when duplicate statements appear for some entries in the FDF file.	
	Updated Chapter 13 to reflect current implementation	
1.24	Updates:	December 2014
	Changed specification version to 1.24	
	Updated UEFI specification and EDK II meta data specifications in section 1.3; added the EDK II UNI Unicode File Specification and EDK II Expression Syntax Specification	
	Removed Appendix G, EFI Compliant Unicode File Grammar and Appendix I, HII UEFI Compliant Unicode File Grammar - refer to the EDK II UNI Unicode File Specification instead	
	"Clarify that the "As Built" INF [Depex] section contains the full dependency expression including the linked in library classes' Depex section in comments.	
	"Updated options in Appendix D to match implementation (removed several flags); added reserved flag, --tool- profile to build.exe.	
	Added clarification of PCD rules for INF files in 8.4.1	
	Removed example text from tool_def.txt and target.txt appendices and Meta-Data File Spec chapter, adding hyper link to SVN location for the template files used to create these two files	
1.24 w/	Updates:	March 2015
Errata A	Update link to the EDK II Specifications, fixed the name of the Multi-String .UNI File Format Specification	
	Updated 10.4 to describe how tools add an FvNameString in the FV image extension header	
	Adding a VPD PCD Sub-section to the FLASH Report - defined in new section, 13.6.3	
1.25	Updates:	June 2015
	Updated to support UEFI 2.5 and PI 1.4 specifications	
	Add BUILDORDER attribute to tools_def.txt in chapter 5.2.3	
	Updated 7.1.5 to include the <code>MODULE_TYPE</code> modifier in <code>[BuildOptions]</code> sections specified in the DSC file.	
	Clarify 8.4.2.10 wording that <code>VOID*</code> PCDs size reservation, when not present in the DSC file, is determined as the largest of the sizes for content in the DSC, DEC and INF files.	
	Update 8.7.12 [Binaries] section in an As Built INF to allow flexibility when determining file types and to allow module relative subdirectories in the filename.	
	Updated 8.2.4.12 to include the flag that generates the Dynamic and DynamicEx HII PCD Variable check binary in the FV section	
1.24 w/	Updates:	August 2015
Errata A	Updated 10.4 to describe how tools add an FvNameString in the FV image extension header	
	Update 8.7.5 - 8.7.9 to allow the build system to update usage information based on feature flag evaluation during the build	

1.26	Updates :	January 2016
	Specification revision to 1.26	
	Removed data structure definitions (duplicates from PE/ COFF, PI Specifications and TE headers) in Chapter 3 and included references to the industry specifications to remove potential typographical errors and inconsistencies.	
	Removed Setup and Getting Started sections from Quick Start chapter 6 - this information is available on the TianoCore.org web-site.	
	Revised WORKSPACE wording for updated build system that can handle packages located outside of the	
	WORKSPACE directory tree (refer to the TianoCore.org/ EDKII website for additional instructions on setting up a development environment). Added new, optional system environment variables used by the build system in this environment.	
	Provide clarification on VPD data generation and report for VPD data content	
	Clarify precedence of the DPX_SOURCE and [Depex] section.	
	Specify the alignment required for VOID* PCDs based on the string, Unicode string or byte-code array values.	
	Remove Unicode file storage requirement; refer to the Multi-String UNI File Format Specification instead.	
	Clarify BUILDRULEORDER	
	Add support for INF statement in an FD region.	
1.27	Convert to Gitbook	May 2017
	#471 Build spec: only copy the "TianoCore" Userextension section into "As Built" INF	
	#472 [Build Spec] Extend macro usage in the !include statements for DSC/FDF files	
	#483 Build Spec: add description for merge Pre-build and Post-build into build process	
	#490 Build Spec: specify the alignment requirements for VOID* PCDs stored in a PCD section	
	#479 Build spec: add description for nmake long command line handling	
	#485 Build spec: add support for mixed Pcd access method	
	#476 Build spec: add support for HII image package	
	#465 DEC spec: document private definitions	
	#504 Update build report to support -Y HASH option	
	#508 Build Spec: Add sections to PCD report for PCDs only used in conditions PCDs that are not used anywhere	
	#480 Build spec: add description for Override PCD value on the command line	
	#523 Build spec: add EBNF for the --pcd syntax in the Section D.4	
	#517 Build spec: chapter 5.2.2 Guided Tools add description for Pkcs7Sign tool and BrotliCompress tool	
	#481 Build Spec: add clarification for not used Pcd that build tool will	

	not do additional checks on its value	
	#518 Build Spec: Update Precedence of PCD Values	
	#669 Build Spec: Add multi-arg support to PREBUILD/POSTBUILD	
	#717 Build Spec: Build report to display every module's build time	
	#689 Build spec: add description for build with binary cache	
1.28	Add clarification for the binary cache files	Mar 2018
	#775 Build spec: Add description about auto detect thread number	
	Add Flexible PCD value format into spec	
	Update PCD value and SKU, DefaultStore info in build report	
	Clarify structure PCD field value assignment precedence	
	Update description for build handles PCDs for SKU support	
	Add statement about module scoped <code><Pcd*></code> section override scope	
	Add !error statement section	
	#1110 Extend exclamation statement's keyword to case-insensitive	
	#598 update supported operators in 8.2.4.6 Expressions	
1.29	#1453 Update Build spec to remove EDK related contents	Mar 2019
1.30	#1952 Update Build spec to support HOST_APPLICATION MODULE_TYPE	July 2019
1.31	#991 generate ProcessLibraryConstructorList() declaration in AutoGen.h for SEC modules	February 2024
1.32	Document <code>,XIP</code> suffix extension to <code>EFI_FILE_NAME</code> in FV INF <code>[files]</code> section for selective XIP rebase control	July 2026
	edk2 PR #12551 and PR #12807	

Tables

- [Table 1 EDK Build Infrastructure Support Matrix](#)
- [Table 2 EFI Section Types](#)
- [Table 3 Defined FV File Types](#)
- [Table 4 Basic EFI_SECTION Type Codes](#)
- [Table 5 Encapsulation EFI_SECTION Type Codes](#)
- [Table 6 Dependency Section Type Codes](#)
- [Table 7 Predefined Command Codes](#)
- [Table 8 Predefined Attributes](#)
- [Table 9 System Environment Variable Usage](#)
- [Table 10 Reserved Macros Expanded by Tools](#)
- [Table 11 Reserved FDF \[Rule\] Section Macro Strings](#)
- [Table 12 Operator Precedence and Supported Operands](#)
- [Table 13: \[Depex\] Expression Operator Precedence](#)
- [Table 13: \[Depex\] Expression Operator Precedence](#)
- [Table 14 AutoGen Stage Input File Extensions](#)
- [Table 15 VFR Compatibility Matrix](#)
- [Table 16 Access Method Section Tags](#)
- [Table 17 \\$\(MAKE\) Stage Intermediate Output File Extensions](#)
- [Table 18 \\$\(MAKE\) Stage Output File Extensions](#)
- [Table 19 GenFds Image Generation: Intermediate File Extensions](#)
- [Table 20 ImageGen Final Output File Extensions](#)
- [Table 21 Variable Descriptions](#)
- [Table 22 Build Targets and Command-line Options](#)
- [Table 23 EDK II Module Types](#)
- [Table 24 Standard C File Compiler Options](#)
- [Table 25 Assembly Flags](#)
- [Table 26 C Compiler's Preprocessor Options](#)
- [Table 27 C Compiler's Preprocessor Options for VFR files ONLY](#)
- [Table 28 Pre-compiled Header \(PCH\) Creation Flags](#)
- [Table 29 Static Linker Flags](#)
- [Table 30 Dynamic Linker Flags](#)

Figures

- [Figure 1 UEFI/PI Firmware Image Creation](#)
- [Figure 2 EFI PCI Expansion Option ROM and UEFI Application Creation](#)
- [Figure 3 PI Firmware Phases](#)
- [Figure 4 NT32 Flash Device Layout](#)
- [Figure 5 Typical IA32/X64 Flash Device Layout](#)
- [Figure 6 Typical IPF FD Layout](#)
- [Figure 7 General EFI Section Format \(< 16MB\)](#)
- [Figure 8 General EFI Section Format for Large Size Sections.](#)
- [Figure 9 Typical FFS File Layout \(<16MB\)](#)
- [Figure 10 File Header 2 layout for files larger than 16Mb](#)
- [Figure 11 General FV Layout](#)
- [Figure 12 Standard Image to Terse Image Comparison](#)
- [Figure 13 EFI Image Files](#)
- [Figure 14 Depex File](#)
- [Figure 15 Firmware Volume Layout](#)
- [Figure 16 EFI PCI Expansion Option ROM layout](#)
- [Figure 17 EFI Capsule Layout](#)
- [Figure 18 EDK II Platform Build Process Flow](#)
- [Figure 19 EDK II AutoGen Process](#)
- [Figure 20 EDK II Build Process - Platform Point of View \(PoV\)](#)
- [Figure 21 EDK II Build Process - Module PoV](#)
- [Figure 22 FD Image Generation Process](#)
- [Figure 23 Capsule Creation Process.](#)
- [Figure 24 Report.html](#)
- [Figure 25 VS2005 Property Page](#)
- [Figure 26 VS2005 Performance Summary](#)
- [Figure 27 VS2005 Call Tree View](#)

1 INTRODUCTION

1.1 Overview

This document describes the EDK II Build Architecture. This specification was designed to support new build requirements for building EDK II modules and EDK components within the EDK II build infrastructure as well as to generate binary firmware images and Unified Extensible Firmware Image (UEFI) applications.

EDK II Build utilities described in this document use INI style text based meta-data files to describe components, modules, libraries, platforms, firmware volumes and firmware device images.

This document describes the high level EDK II Build Architecture, which has the following goals:

Compatible

The EDK II build environment must maintain backward compatibility with the existing EDK files. This means that the changes made to this specification must not require changes to existing files.

Compatibility is maintained by providing the EDK Tools in the EDK Compatibility package. Also, some INF files may require modification for the EDK II build environment if they are used to access Flash firmware - the PI 1.0 specification modified the flash data structures and defined some new GUID values.

EDK II Build system tools must test the format of the EDK II meta-data files. The EDK II build tools must provide an error if, during parsing of the EDK II meta-data files, a version of the files is encountered that is higher than the version of the files that the tools support.

Simplified platform build and configuration

One goal of this format is to simplify the build setup and configuration for a given platform. It was also designed to simplify the process of adding EDK and EDK II firmware components to a firmware volume on a given platform.

Specification Conformance

The EDK II Build infrastructure supports building UEFI 2.5 and PI 1.4 compliant platforms. Existing EDK components may need to be updated to align with these specifications.

Table 1 EDK Build Infrastructure Support Matrix

	EDK DSC	EDK II DSC	EDK FDF	EDK II FDF	EDK INF	EDK II INF
EDK Build Tools	YES	NO	YES	NO	YES	NO
EDK II Build Tools	NO	YES	NO	YES	YES	YES

1.2 Target Audience

This document is intended for persons performing EFI development and support for different platforms.

1.3 Terms

The following terms are used throughout this document to describe varying aspects of input localization:

BaseTools

The BaseTools are the tools required for an EDK II build.

BDS

Framework Boot Device Selection phase.

BNF

BNF is an acronym for "Backus Naur Form." John Backus and Peter Naur introduced for the first time a formal notation to describe the syntax of a given language.

Component

An executable image. Components defined in this specification support one of the defined module types.

DEC

EDK II Package Declaration File. This file declares information about what is provided in the package. An EDK II package is a collection of like content.

DEPEX

Module dependency expressions that describe runtime process restrictions.

Dist

This refers to a distribution package that conforms to the UEFI Platform Initialization Distribution Packages Specification.

DSC

EDK II Platform Description File. This file describes what and how modules, libraries and components are to be built, as well as defining library instances which will be used when linking EDK II modules.

DXE

Framework Driver Execution Environment phase.

DXE SAL

A special class of DXE module that produces SAL Runtime Services. DXE SAL modules differ from DXE Runtime modules in that the DXE Runtime modules support Virtual mode OS calls at OS runtime and DXE SAL modules support intermixing Virtual or Physical mode OS calls.

DXE SMM

A special class of DXE module that is loaded into the System Management Mode memory.

DXE Runtime

Special class of DXE module that provides Runtime Services

EBNF

Extended "Backus-Naur Form" meta-syntax notation with the following additional constructs: square brackets "[...]" surround optional items, suffix "*" for a sequence of zero or more of an item, suffix "+" for one or more of an item, suffix "?" for zero or one of an item, curly braces "{...}" enclosing a list of alternatives and super/subscripts indicating between n and m occurrences.

EDK

Extensible Firmware Interface Development Kit, the original implementation of the Intel(R) Platform Innovation Framework for EFI Specifications developed in 2007.

EDK II

EFI Development Kit, version II that provides updated firmware module layouts and custom tools, superseding the original EDK.

EDK Compatibility Package (ECP)

The EDK Compatibility Package (ECP) provides libraries that will permit using most existing EDK drivers with the EDK II build environment and EDK II platforms.

EFI

Generic term that refers to one of the versions of the EFI specification: EFI 1.02, EFI 1.10 or any of the UEFI specifications.

FDF

EDK II Flash definition file. This file is used to define the content and binary image layouts for firmware images, update capsules and PCI option ROMs.

FLASH

This term is used throughout this document to describe one of the following:

- An image that is loaded into a hardware device on a platform - traditional ROM image
- An image that is loaded into an Option ROM device on an add-in card
- A boot able image that is installed on removable, boot able media, such as a Floppy, CD-ROM or USB storage device.
- An image that contains update information that will be processed by OS Runtime services to interact with EFI Runtime services to update a traditional ROM image.
- A UEFI application that can be accessed during boot (at an EFI Shell Prompt), prior to hand-off to the OS Loader.

Foundation

The set of code and interfaces that glue implementations of EFI together.

Framework

Intel(R) Platform Innovation Framework for EFI consists of the Foundation, plus other modular components that characterize the portability surface for modular components designed to work on any implementation of the EFI architecture.

GUID

Globally Unique Identifier. A 128-bit value used to name entities uniquely. A unique GUID can be generated by an individual without the help of a centralized authority. This allows the generation of names that will never conflict, even among multiple, unrelated parties. GUID values can be registry format (8-4-4-4-12) or C data structure format.

GUID also refers to an API named by a GUID.

HII

Human Interface Infrastructure. This generally refers to the database that contains string, font, and IFR information along with other pieces that use one of the database components.

HOB

Hand-off blocks are key architectural mechanisms that are used to hand off system information in the early pre-boot stages.

INF

- EDK II Module Information File. This file describes how the module is coded. For EDK, this file describes how the component or library is coded as well as providing some basic build information.
- Source INF - An EDK II Module Information file that contains content in a [Sources] section and it does not contain a [Binaries] section. If the [Binaries] section is empty or the only entries in the [Binaries] section are of type DISPOSABLE, then the [Binaries] section is ignored.
- Binary INF - An EDK II Module Information file that has a [Binaries] section and does not contain a [Sources] section or the [Sources] section is empty.
- Mixed INF - An EDK II Module Information file that contains content in both [Sources] and [Binaries] sections and there are entries in the [Binaries] section are not of type DISPOSABLE
- AsBuilt INF - An EDK II Module Information file generated by the EDK II build system when building source content (listed in a [Sources] section).

IFR

Internal Forms Representation. This is the binary encoding that is used for the representation of user interface pages.

Library Class

A library class defines the API or interface set for a library. The consumer of the library is coded to the library class definition. Library classes are defined via a library class .h file that is published by a package.

Library Instance

An implementation of one or more library classes.

Module

A module is either an executable image or a library instance. For a list of module types supported by this package, see module type.

Module Type

All libraries and components belong to one of the following module types: `BASE`, `SEC`, `PEI_CORE`, `PEIM`, `DXE_CORE`, `SMM_CORE`, `DXE_DRIVER`, `DXE_RUNTIME_DRIVER`, `DXE_SMM_DRIVER`, `DXE_SAL_DRIVER`, `UEFI_DRIVER`, OR `UEFI_APPLICATION`. These definitions provide a framework that is consistent with a similar set of requirements. A module that is of module type `BASE`, depends only on headers and libraries provided in the MDE, while a module that is of module type `DXE_DRIVER` depends on common DXE components. For a definition of the various module types, see Appendix Module Types. The EDK II build system also permits modules of type `USER_DEFINED` and `HOST_APPLICATION`. These modules will not be processed by the EDK II Build system.

Package

A package is a container. It can hold a collection of files for any given set of modules. Packages may be described as one of the following types of modules:

- source modules, containing all source files and descriptions of a module

- binary modules, containing EFI Sections or a Framework File System and a description file specific to linking and binary editing of features and attributes specified in a Platform Configuration Database (PCD).
- mixed modules, with both binary and source modules

Multiple modules can be combined into a package, and multiple packages can be combined into a single package.

PCD

Platform Configuration Database.

PEI

Pre-EFI Initialization Phase.

PEIM

An API named by a GUID.

PPI

A PEIM-to-PEIM Interface that is named by a GUID.

Protocol

An API named by a GUID.

Runtime Services

Interfaces that provide access to underlying platform-specific hardware that might be useful during OS runtime, such as time and date services. These services become active during the boot process but also persist after the OS loader terminates boot services.

SAL

System Abstraction Layer. A firmware interface specification used on Intel(R) Itanium(R) Processor based systems.

SEC

Security Phase is the code in the Framework that contains the processor reset vector and launches PEI. This phase is separate from PEI because some security schemes require ownership of the reset vector.

SKU

Stock Keeping Unit.

SMM

System Management Mode. A generic term for the execution mode entered when a CPU detects an SMI. The firmware, in response to the interrupt type, will gain control in physical mode. For this document, "SMM" describes the operational regime for IA32 and x64 processors that share the OS-transparent characteristics.

UEFI Application

An application that follows the UEFI specification. The only difference between a UEFI application and a UEFI driver is that an application is unloaded from memory when it exits regardless of return status, while a driver that returns a successful return status is not unloaded when its entry point exits.

UEFI Driver

A driver that follows the UEFI specification.

UEFI Specification Version 2.5

Current UEFI version.

UEFI Platform Initialization Distribution Package Specification Version 1.0

The current version of this specification includes Errata B.

UEFI Platform Initialization Specification 1.4

Current version of the PI specification.

Unified EFI Forum

A non-profit collaborative trade organization formed to promote and manage the UEFI standard. For more information, see <http://www.uefi.org>.

VFR

Visual Forms Representation.

VPD

Vital Product Data that is read-only binary configuration data, typically located within a region of a flash part. This data would typically be updated as part of the firmware build, post firmware build (via patching tools), through automation on a manufacturing line as the 'FLASH' parts are programmed or through special tools.

1.4 Related Information

The following publications and sources of information may be useful to you or are referred to by this specification:

- Unified Extensible Firmware Interface Specification, Version 2.5, Unified EFI, Inc, 2015, <http://www.uefi.org>.
- Platform Initialization Specification, Version 1.4, Unified EFI, Inc., 2015, <http://www.uefi.org>.
- UEFI Platform Initialization Distribution Package Specification, Version 1.0 with Errata B, Unified EFI, Inc., 2014, <http://www.uefi.org>.
- Intel(R) Platform Innovation Framework for EFI Specifications, Intel, 2007, <http://www.intel.com/technology/framework/>.
- http://tianocore.sourceforge.net/wiki/EDK_II_Specifications
 - EDK II Module Writers Guide, Intel, 2010.
 - EDK II User Manual, Intel, 2010.
 - EDK II C Coding Standard, Intel, 2015.
 - EDK II DEC Specification, Intel, 2016.
 - EDK II DSC Specification, Intel, 2016.
 - EDK II FDF Specification, Intel, 2016.
 - EDK II INF Specification, Intel, 2016.
 - Multi-String UNI File Format Specification, Intel, 2016.
 - EDK II Expression Syntax Specification, Intel, 2015.
 - VFR Programming Language, Intel, 2015.
 - UEFI Packaging Tool (UEFIPT) Quick Start, Intel, 2015.
 - EDK II Platform Configuration Database Infrastructure Descriptions, Intel, 2009.
- INI file, Wikipedia, <http://www.uefi.org>.
- C Now - C Programming Information, Langston University, Tulsa Oklahoma, J.H. Young, 1999-2011, http://en.wikipedia.org/wiki/INI_file
- NASM, the netwide assembler, <http://www.nasm.us>

1.5 Conventions Used in this Document

This document uses typographic and illustrative conventions described below.

1.5.1 Data Structure Descriptions

Intel(R) processors based on 32 bit Intel(R) architecture (IA 32) are "little endian" machines. This distinction means that the low-order byte of a multi byte data item in memory is at the lowest address, while the high-order byte is at the highest address.

Processors of the Intel(R) Itanium(R) processor family may be configured for both "little endian" and "big endian" operation. All implementations designed to conform to this specification will use "little endian" operation.

In some memory layout descriptions, certain fields are marked reserved. Software must initialize such fields to zero and ignore them when read. On an update operation, software must preserve any reserved field.

The data structures described in this document generally have the following format:

STRUCTURE_NAME

The formal name of the data structure.

Summary

A brief description of the data structure.

Prototype

An EBNF-type declaration for the data structure..

Example

Sample data structure using the prototype.

Description

A description of the functionality provided by the data structure, including any limitations and caveats of which the caller must be aware.

Related Definitions

The type declarations and constants that are used only by this data structure.

1.5.2 Pseudo-Code Conventions

Pseudo code is presented to describe algorithms in a more concise form. None of the algorithms in this document are intended to be compiled directly. The code is presented at a level corresponding to the surrounding text.

In describing variables, a list is an unordered collection of homogeneous objects. A queue is an ordered list of homogeneous objects. Unless otherwise noted, the ordering is assumed to be FIFO.

Pseudo code is presented in a C-like format, using C conventions where appropriate. The coding style, particularly the indentation style, is used for readability and does not necessarily comply with an implementation of the Extensible Firmware Specification.

1.5.3 Typographic Conventions

This document uses the typographic and illustrative conventions described below:

Typographic Convention	Typographic convention description
Plain text	The normal text typeface is used for the vast majority of the descriptive text in a specification.
<u>Plain text (blue)</u>	Any plain text that is underlined and in blue indicates an active link to the crossreference. Click on the word to follow the hyperlink.
Bold	In text, a Bold typeface identifies a processor register name. In other instances, a Bold typeface can be used as a running head within a paragraph.
<i>Italic</i>	In text, an <i>Italic</i> typeface can be used as emphasis to introduce a new term or to indicate a manual or specification name.
BOLD Monospace	Computer code, example code segments, and all prototype code segments use a BOLD Monospace typeface with a dark red color. These code listings normally appear in one or more separate paragraphs, though words or segments can also be embedded in a normal text paragraph.
<u>BOLD Monospace</u>	Words in a <u>BOLD Monospace</u> typeface that is underlined and in blue indicate an active hyper link to the code definition for that function or type definition. Click on the word to follow the hyper link.
\$(VAR)	This symbol VAR defined by the utility or input files.
Italic Bold	In code or in text, words in Italic Bold indicate placeholder names for variable information that must be supplied (i.e., arguments).

Note: Due to management and file size considerations, only the first occurrence of the reference on each page is an active link. Subsequent references on the same page will not be actively linked to the definition and will use the standard, non-underlined **BOLD Monospace** typeface. Find the first instance of the name (in the underlined **BOLD Monospace** typeface) on the page and click on the word to jump to the function or type definition.

The following typographic conventions are used in this document to illustrate the Extended Backus-Naur Form.

Typographic Convention	Typographic convention description
[item]	Square brackets denote the enclosed item is optional.
{item}	Curly braces denote a choice or selection item, only one of which may occur on a given line.
<item>	Angle brackets denote a name for an item.
(range-range)	Parenthesis with characters and dash characters denote ranges of values, for example, (a-zA-Z0-9) indicates a single alphanumeric character, while (0-9) indicates a single digit.
"item"	Characters within quotation marks are the exact content of an item, as they must appear in the output text file.

?	The question mark denotes zero or one occurrences of an item.
*	The star character denotes zero or more occurrences of an item.
+	The plus character denotes one or more occurrences of an item.
item{n}	A superscript number, n, is the number occurrences of the item that must be used. Example: (0-9)8 indicates that there must be exactly eight digits, so 01234567 is valid, while 1234567 is not valid.
item{n,}	A superscript number, n, within curly braces followed by a comma "," indicates the minimum number of occurrences of the item, with no maximum number of occurrences.
item{,n}	A superscript number, n, within curly brackets, preceded by a comma ",", indicates a maximum number of occurrences of the item.
item{n,m}	A super script number, n, followed by a comma "," and a number, m, indicates that the number of occurrences can be from n to m occurrences of the item, inclusive.

2 DESIGN DISCUSSION

This section of the document provides an overview to the build process for UEFI and PI compliant modules. This includes existing EDK components and EDK II modules. EDK II build tools process the following meta-data files:

- EDK II build configuration files
- EDK Component and EDK II Module (INF) Files
- EDK Library (only used by EDK Components) INF Files
- EDK II Package Declaration (DEC) Files
- EDK II Platform Description (DSC) Files
- EDK II Flash Description (FDF) Files

The meta-data file content is used to generate:

- Module specific C files, both .c and .h files
- PI compliant dependency files
- Makefiles used by third party compiler utilities
- PCI Option ROM images
- UEFI compliant image files
- Platform firmware images
- Platform update capsules

Note: Path and Filename elements within the EDK II Meta-Data files and command line arguments are case-sensitive in order to support building on UNIX style operating systems.

Note: The total path and file name length is limited by the operating system and third party tools. It is recommended that for EDK II builds that the project directories under a subst drive in Windows (s:/ build as an example) or be located in either the /opt directory or in the user's /home/username directory for Linux and OS/X. This will minimize the path lengths of filenames for the command-line tools.

Reference Implementation

The EDK II build system is a reference implementation. Its description starts with chapter EDK II Build Process, after discussing the design and architectural elements of UEFI/PI compliant files.

2.1 Development Environments

The EDK II build environment must support development workstations running Microsoft* Windows operating systems, Linux operating systems or Apple\ Mac OS*/X operating systems. In addition, multiple compiler tools chains such as from Microsoft, Intel and GCC, must be supported. All provided source code must be POSIX compliant. Module modules that will be distributed outside of an organization, it is recommended that if assembly source code is used, NASM files should be created. For backward compatibility, both GCC (**GAS**) and Microsoft (**MASM**) style files may be provided. See the EDK II C Coding Standard for additional information.

2.2 UEFI/PI Firmware Images

UEFI and PI specifications define the standardized format for EFI firmware storage devices (FLASH or other non-volatile storage) which are abstracted into "Firmware Volumes". Build systems must be capable of processing files to create the file formats described by the UEFI and PI specifications. The tools provided as part of the EDK II BaseTools package process files compiled by third party tools, as well as text and Unicode files in order to create UEFI or PI compliant binary image files. In some instances, where UEFI or PI specifications do not have an applicable input file format, such as the Visual Forms Representation (VFR) files used to create PI compliant IFR content, tools and documentation have been provided that allows the user to write text files that are processed into formats specified by UEFI or PI specifications.

A Firmware Volume (FV) is a file level interface to firmware storage. Multiple FVs may be present in a single FLASH device, or a single FV may span multiple FLASH devices. An FV may be produced to support some other type of storage entirely, such as a disk partition or network device. For more information consult the Platform Initialization Specification, Volume 3.

In all cases, an FV is formatted with a binary file system. The file system used is typically the Firmware File System (FFS), but other file systems may be possible in some cases. Hence, all modules are stored as "files" in the FV. Some modules may be "execute in place" (linked at a fixed address and executed from the ROM), while others are relocated when they are loaded into memory and some modules may be able to run from ROM if memory is not present (at the time of the module load) or run from memory if it is available.

Files themselves have an internally defined binary format. This format allows for implementation of security, compression, signing, etc. Within this format, there are one or more "leaf" images. A leaf image could be, for example, a PE32 image for a DXE driver.

Therefore, there are several layers of organization to a full UEFI/PI firmware image. These layers are illustrated below in Figure 1. Each transition between layers implies a processing step that transforms or combines previously processed files into the next higher level. Also shown in Figure 1 are the reference implementation tools that process the files to move them between the different layers.

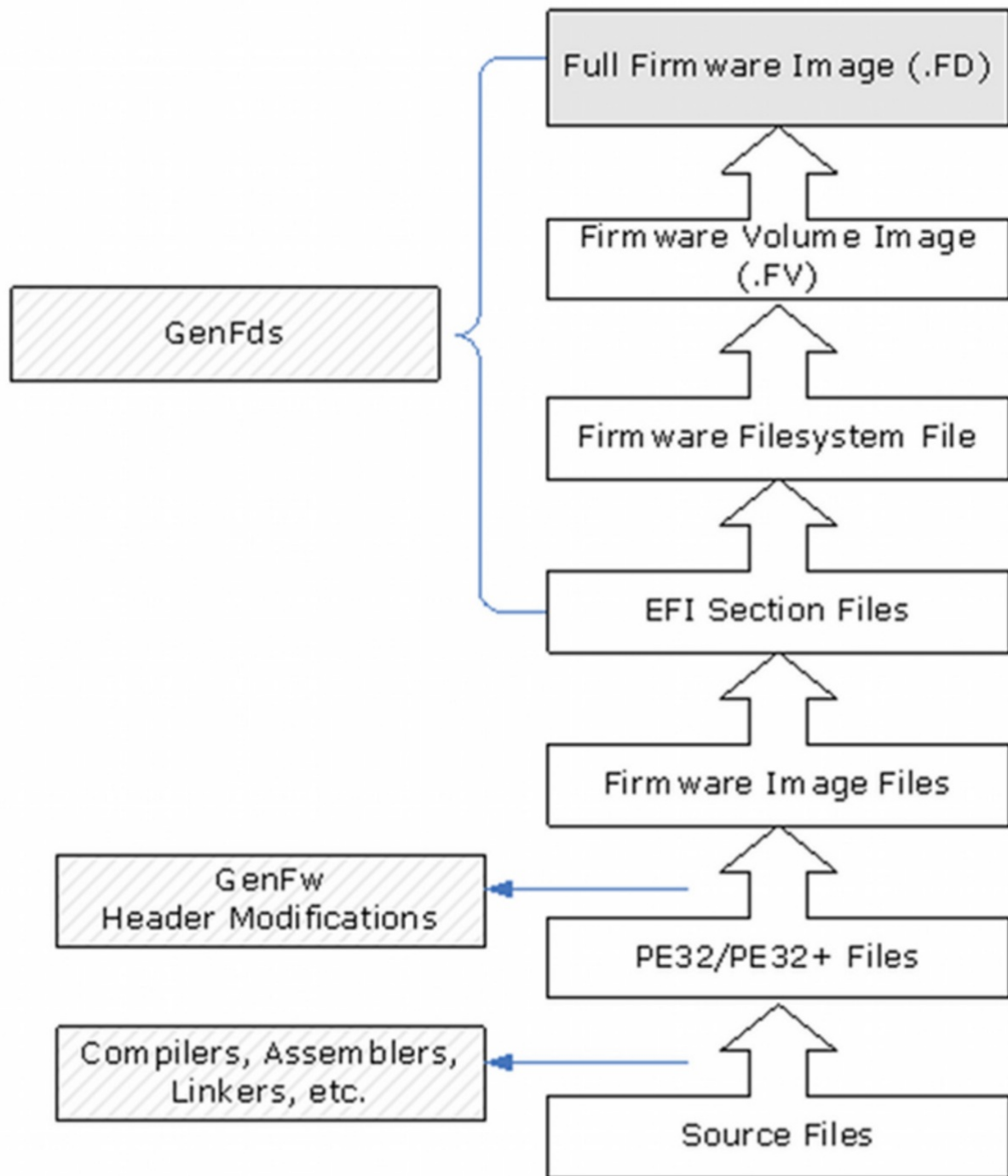


Figure 1 UEFI/PI Firmware Image Creation

In addition to creating images that initialize a complete platform, the build process also supports creation of stand-alone UEFI applications (including OS Loaders) and Option ROM images containing driver code. Figure 2, below, shows the reference implementation tools and creation processes for both of these image types.

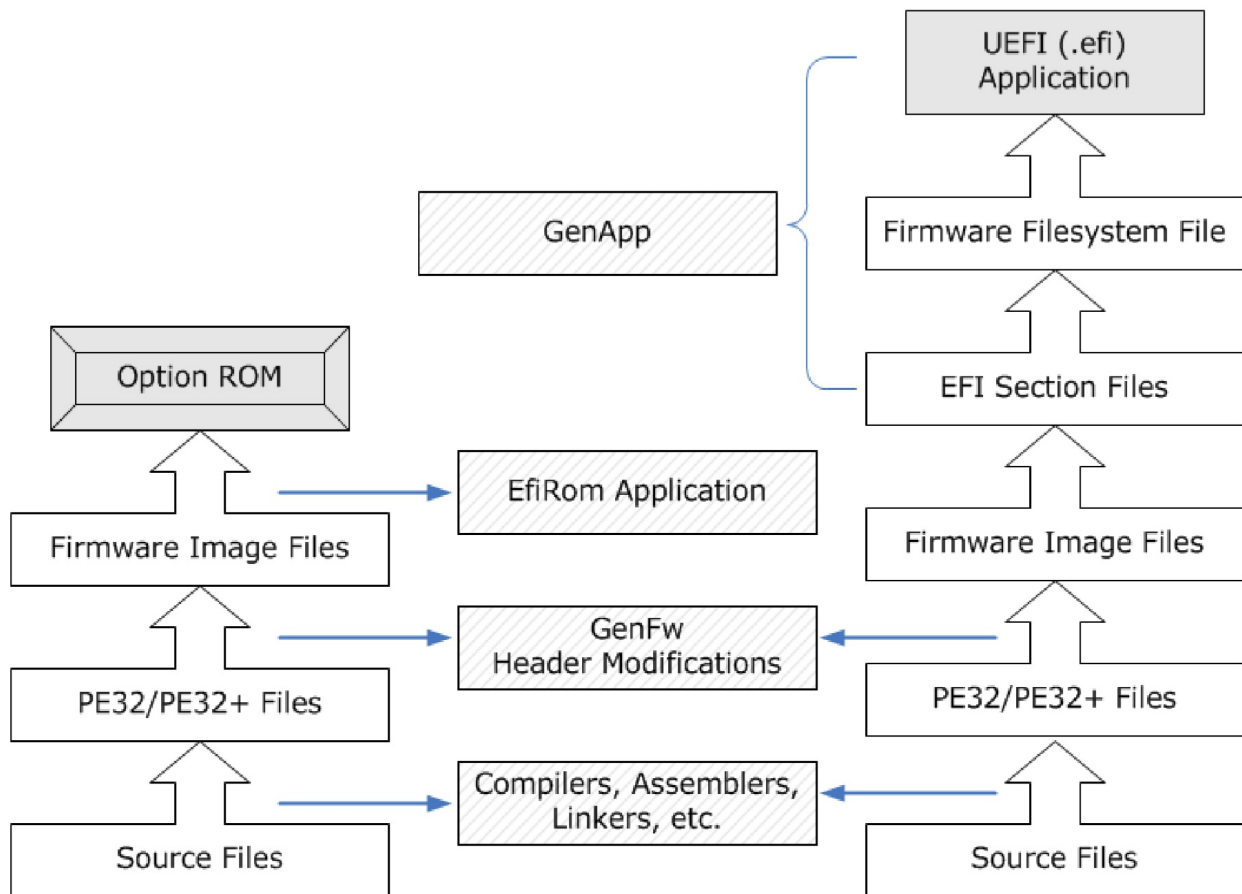


Figure 2 EFI PCI Expansion Option ROM and UEFI Application Creation

The final feature that is supported by the EDK II build process is the creation of Binary Modules that can be packaged and distributed for use by other organizations. Binary modules do not require distribution of the source code. This will permit vendors to distribute UEFI images without having to release proprietary source code.

This packaging process permits creation of an archive file containing one or more binary files that are either Firmware Image files or higher (EFI Section files, Firmware File system files, etc.). The build process will permit inserting these binary files into the appropriate level in the build stages.

2.3 Boot Sequence

PI compliant system firmware must support the six phases: security (SEC), pre-efi initialization (PEI), driver execution environment (DXE), boot device selection (BDS), run time (RT) services and After Life (transition from the OS back to the firmware) of system. Refer to Figure 3 below.

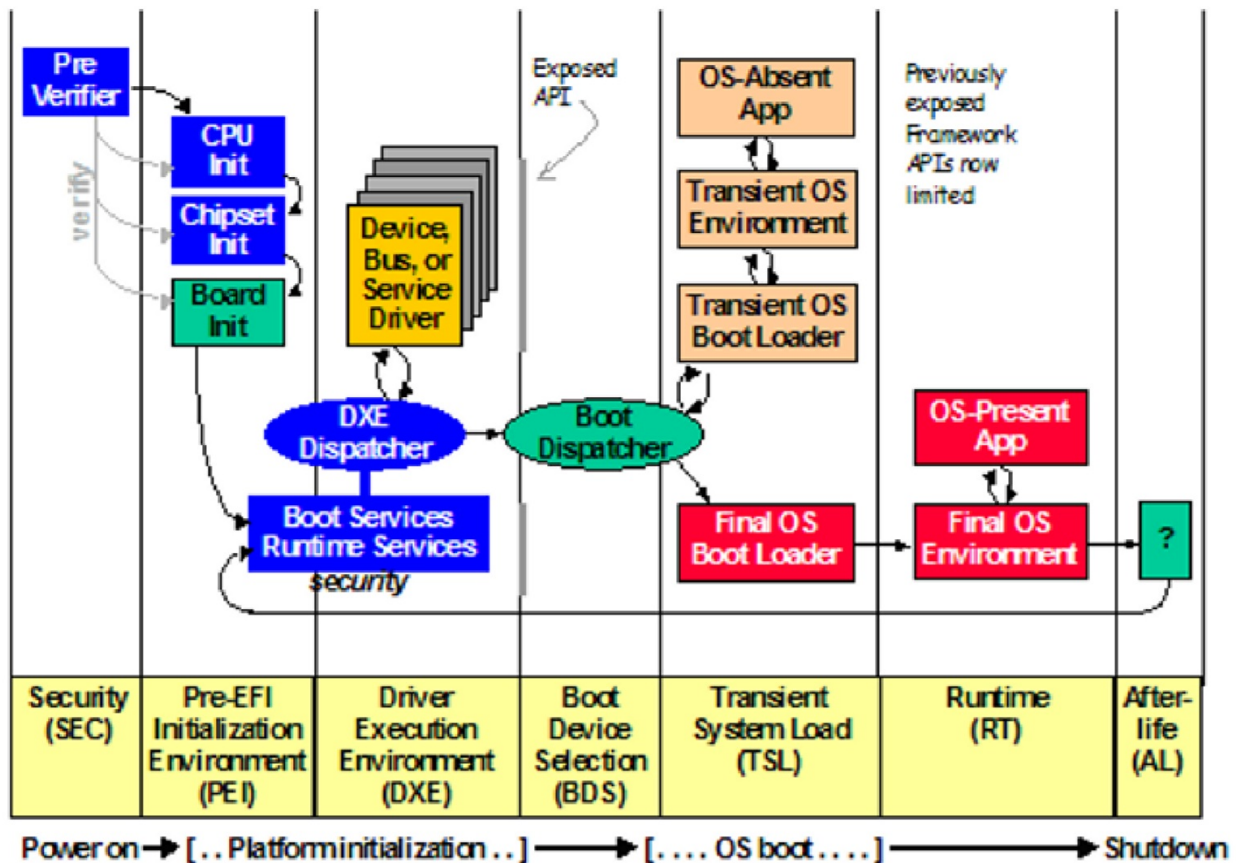


Figure 3 PI Firmware Phases

2.3.1 Security (SEC)

The Security (SEC) phase is the first phase in the PI Architecture and is responsible for the following:

- Handling all platform restart events
- Creating a temporary memory store
- Serving as the root of trust in the system
- Passing handoff information to the PEI Foundation

The security section may contain modules with code written in assembly. Therefore, some EDK II module development environment (MDE) modules may contain assembly code. Where this occurs, both Windows and GCC versions of assembly code are provided in different files.

2.3.2 Pre-EFI Initialization (PEI)

The Pre-EFI Initialization (PEI) phase described in the PI Architecture specifications is invoked quite early in the boot flow. Specifically, after some preliminary processing in the Security (SEC) phase, any machine restart event will invoke the PEI phase.

The PEI phase initially operates with the platform in a nascent state, leveraging only on-processor resources, such as the processor cache as a call stack, to dispatch Pre-EFI Initialization Modules (PEIMs). These PEIMs are responsible for the following:

- Initializing some permanent memory complement
- Describing the memory in Hand-Off Blocks (HOBs)
- Describing the firmware volume locations in HOBs
- Passing control into the Driver Execution Environment (DXE) phase

2.3.3 Drive Execution Environment (DXE)

Prior to the DXE phase, the Pre-EFI Initialization (PEI) phase is responsible for initializing permanent memory in the platform so that the DXE phase can be loaded and executed. The state of the system at the end of the PEI phase is passed to the DXE phase through a list of position independent data structures called Hand-Off Blocks (HOBs). HOBs are described in detail in the Platform Initialization Specification.

There are several components in the DXE phase:

- DXE Foundation
- DXE Dispatcher
- A set of DXE Drivers

2.3.4 Boot Device Selection (BDS)

The Boot Device Selection (BDS) phase is implemented as part of the BDS Architectural Protocol. The DXE Foundation will hand control to the BDS Architectural Protocol after all of the DXE drivers whose dependencies have been satisfied have been loaded and executed by the DXE Dispatcher. The BDS phase is responsible for the following:

- Initializing console devices
- Loading device drivers
- Attempting to load and execute boot selections

2.3.5 Transient System Load (TSL) and Runtime (RT)

The Transient System Load (TSL) is primarily the OS vendor provided boot loader. Both the TSL and the Runtime Services (RT) phases may allow access to persistent content, via UEFI drivers and UEFI applications. Drivers in this category include PCI Option ROMs.

2.3.6 After Life (AL)

The After Life (AL) phase consists of persistent UEFI drivers used for storing the state of the system during the OS orderly shutdown, sleep, hibernate or restart processes.

2.4 Typical Flash Part Layout

While a flash part layout is specific to a given platform, there are some generalizations that apply. The SEC and PEI code is typically put into a "RECOVERY" location, while all remaining sections are put into the "MAIN" location. The MAIN section may be compressed for size optimization, provided the PEI or SEC code contains appropriate decompression drivers. The PI specification defines only standard EFI compression; if other compression mechanisms (or verification mechanisms, such as CRC32) are required, then both the tools for creating a compressed image and a library for decompressing the image must be provided. These non-standard compression, encryption, signing or verification mechanisms are applied to a GUIDED encapsulation section. Each method needs a unique GUID, however the methods may be applied to images more than once per FD image. This is done in order to facilitate recovery and updates (called capsules). Other areas in flash may be reserved for non-volatile (NV) data storage, fault tolerant working (FTW) space or vital product data (VPD) areas. These other regions are not defined in the PI specification, and implementation is left to the platform integrator. The reference design Nt32 Platform emulation environment contains a virtual flash device. The content within this virtual FD is laid out per Figure 4.

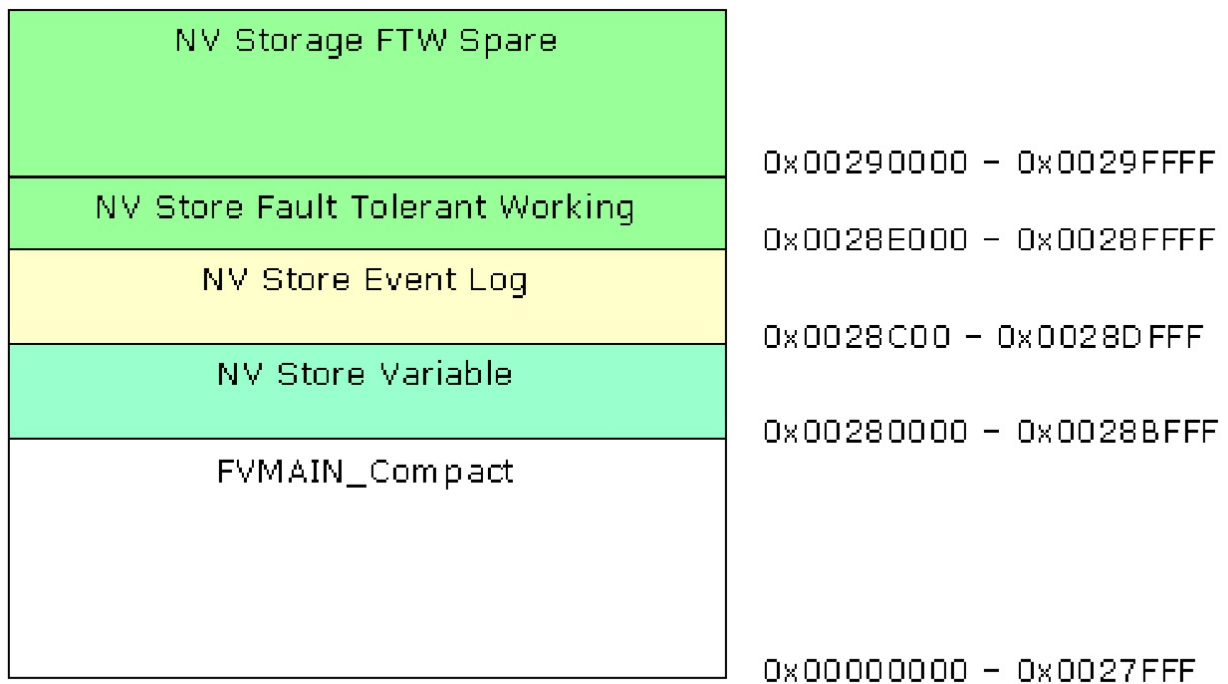


Figure 4 NT32 Flash Device Layout

Figure 5 represents a typical IA32/X64 FD layout, where SEC and PEI code is located in the FV Recovery section, and the remaining drivers are located in a GUIDED encapsulation (compressed) section designated as `FVMAIN_Compact`.

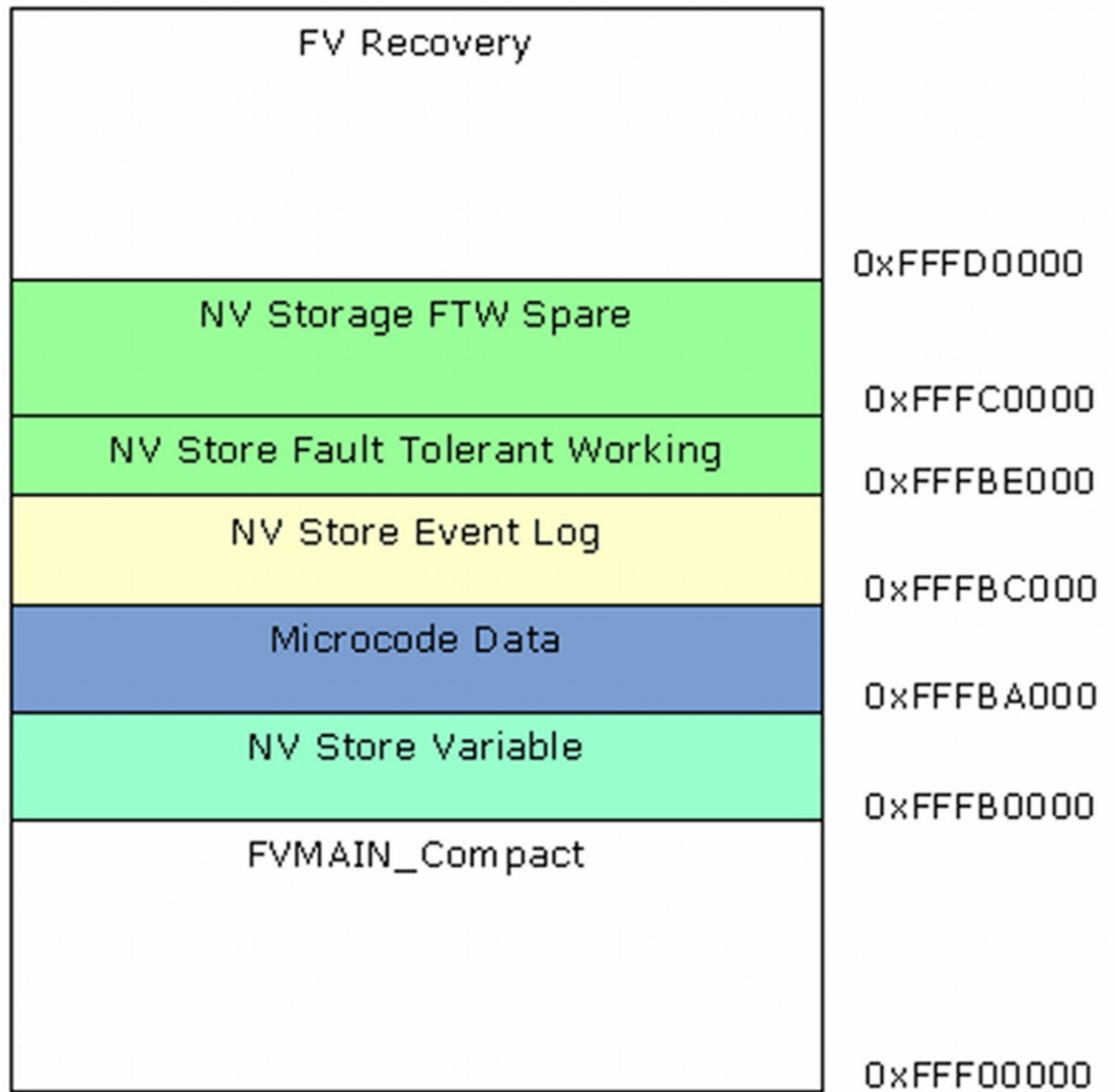


Figure 5 Typical IA32/X64 Flash Device Layout

Figure 6 represents a typical IPF FD layout.

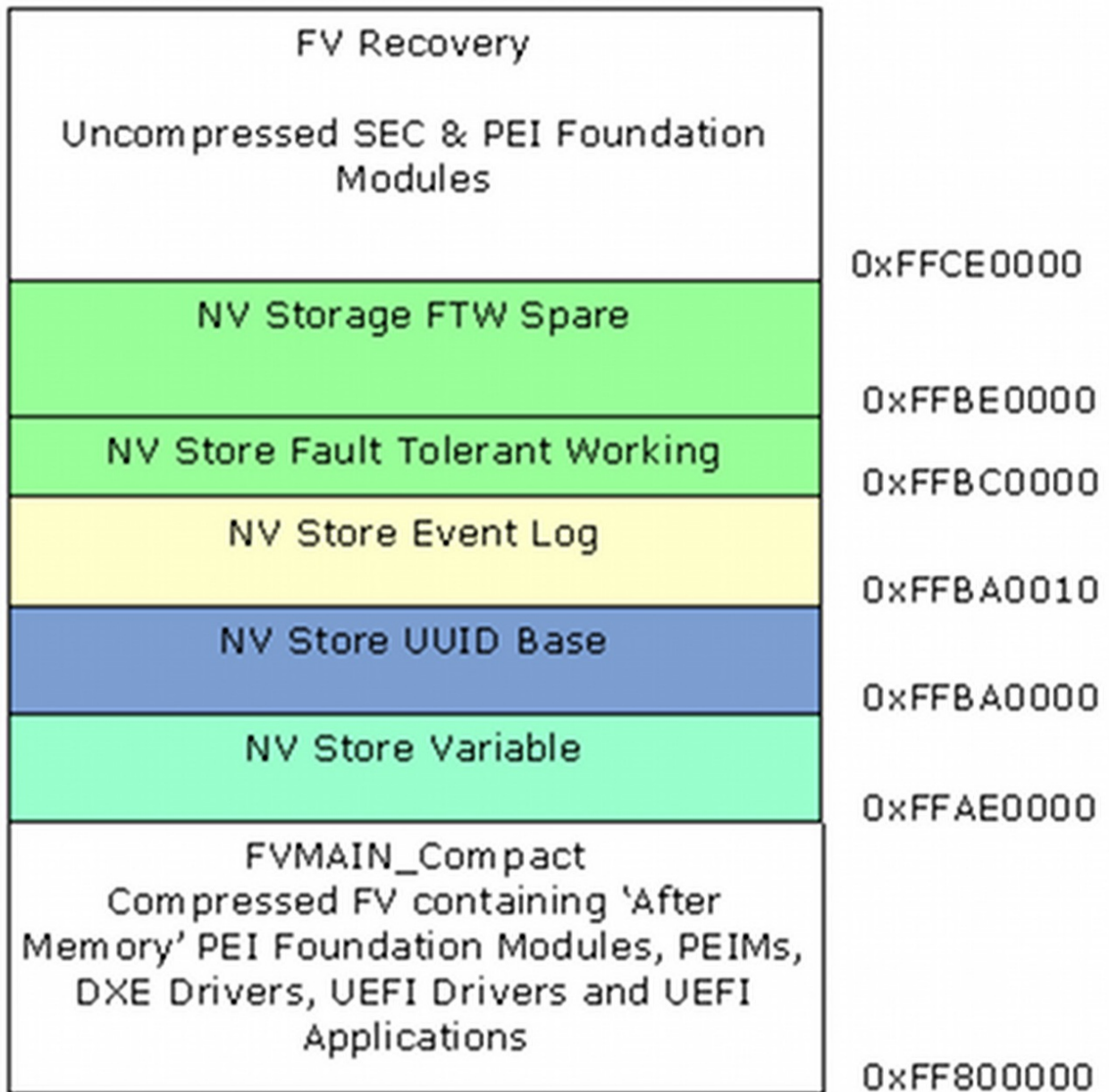


Figure 6 Typical IPF FD Layout

All of these layouts assume only one flash device, with the virtual memory addresses listed for each section within the FD.

Note: More than one flash device may be present within a platform, so the images may be split over multiple devices.

2.5 Generic Build Process

All code starts out as either C sources and header files, assembly sources and header files, UCS-2 HII strings in Unicode files, Virtual Forms Representation files or binary data (native instructions, such as microcode) files. Per the UEFI and PI specifications, the C and Assembly files must be compiled and linked into PE32/PE32+ images.

While some code is designed to execute only from ROM, most UEFI/PI modules are written to be relocate-able. These are written and built different. For example, Execute In Place (XIP) module code is written and compiled to run from ROM, while the majority of the code is written and compiled to execute from memory, which requires that the code be relocate able.

Some modules may also permit dual mode, where it will execute from memory only if memory is available, otherwise it will execute from ROM. Additionally, modules may permit dual access, such as a driver that contains both PEI and DXE implementation code. Code is assembled or compiled, then linked into PE32/PE32+ images, the relocation section may or may not be stripped and an appropriate header will replace the PE32/PE32+ header. Additional processing may remove more non-essential information, generating a Terse (TE) image.

The binary executables are converted into EFI firmware file sections. Each module is converted into an EFI Section consisting of an Section header followed by the section data (driver binary).

2.5.1 EFI SECTION Files

The general section format for sections less than 16MB in size is shown in Figure 7. Figure 8 shows the section format for sections 16MB or larger in size using the extended length field.

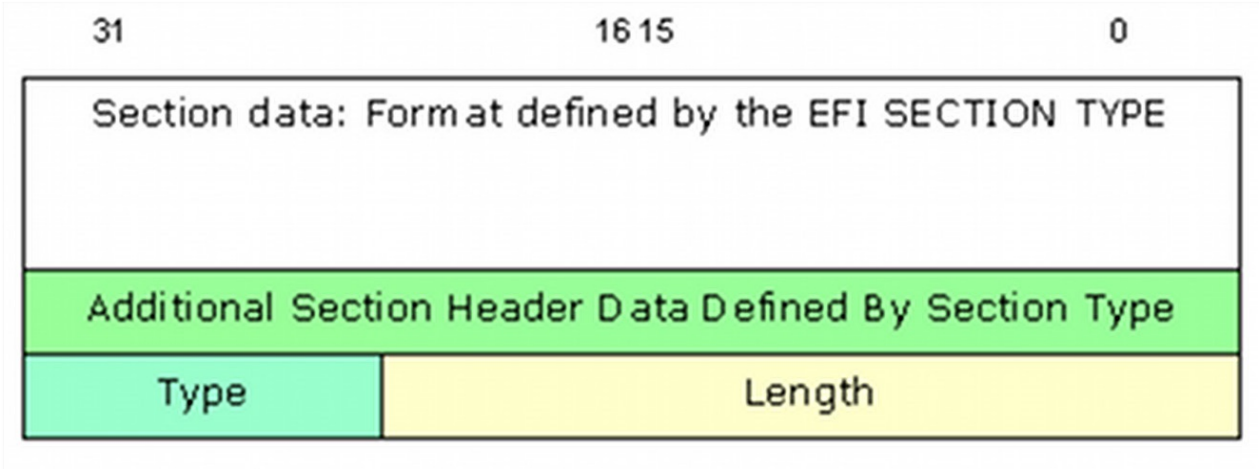


Figure 7 General EFI Section Format (< 16MB)

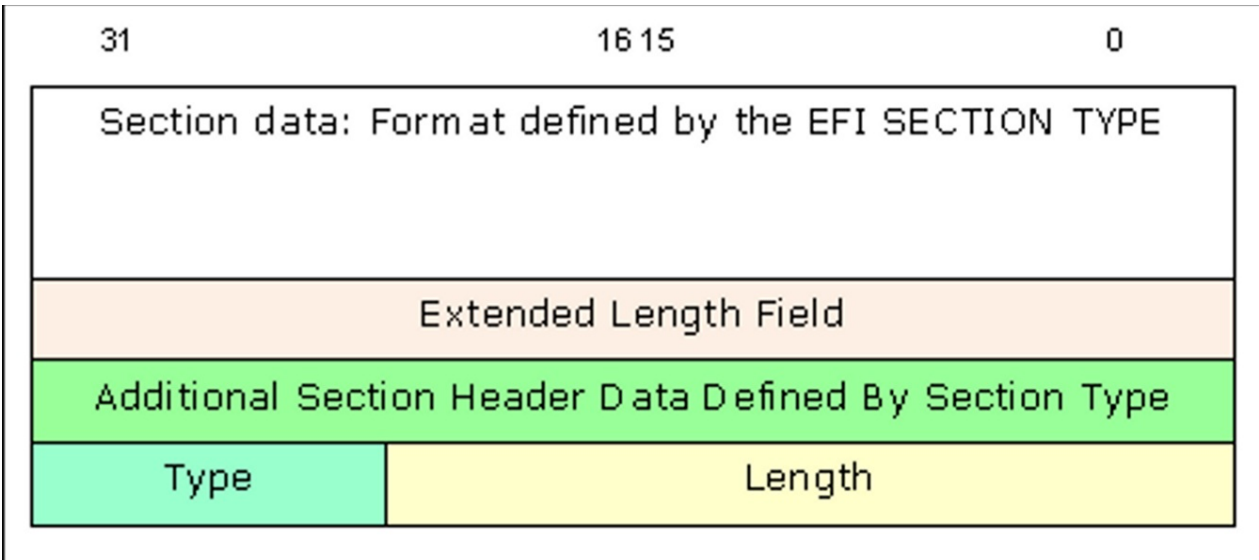


Figure 8 General EFI Section Format for Large Size Sections.

Table 2 below lists the different architecturally defined section types, refer to the PI Specification, Volume 3 for additional details.

Table 2 EFI Section Types

Name	Description
EFI_SECTION_COMPRESSION	Encapsulation section where other sections are compressed
EFI_SECTION_GUID_DEFINED	Encapsulation section where other sections have a format defined by a GUID.
EFI_SECTION_DISPOSABLE	Encapsulation section used during the build process but not required for execution.
EFI_SECTION_PE32	PE32+ Executable Image
EFI_SECTION_PIC	Position-Independent Code.
EFI_SECTION_TE	Terse Executable image.
EFI_SECTION_DXE_DEPEX	DXE Dependency Expression.
EFI_SECTION_VERSION	Version, Text and Numeric (UNICODE)
EFI_SECTION_SMM_DEPEX	Leaf section type for determining the dispatch order for an SMM driver.
EFI_SECTION_USER_INTERFACE	User-Friendly name of the driver (UNICODE)
EFI_SECTION_COMPATIBILITY16	DOS-style 16-bit executable.
EFI_SECTION_FIRMWARE_VOLUME_IMAGE	PI Firmware Volume Image.
EFI_SECTION_FREEFORM_SUBTYPE_GUID	Raw data with GUID in header to define format.
EFI_SECTION_RAW	Raw data (for example, a logo).
EFI_SECTION_PEI_DEPEX	PEI Dependency Expression.

2.5.2 Firmware Files

Multiple EFI Sections are combined into a Firmware file (FFS) which consists of zero or more EFI sections. Each FFS consists of a FFS header plus the data. Figure 9 show the basic FFS File layout and Figure 10 shows the FFS File layout for files of 16MB or larger.

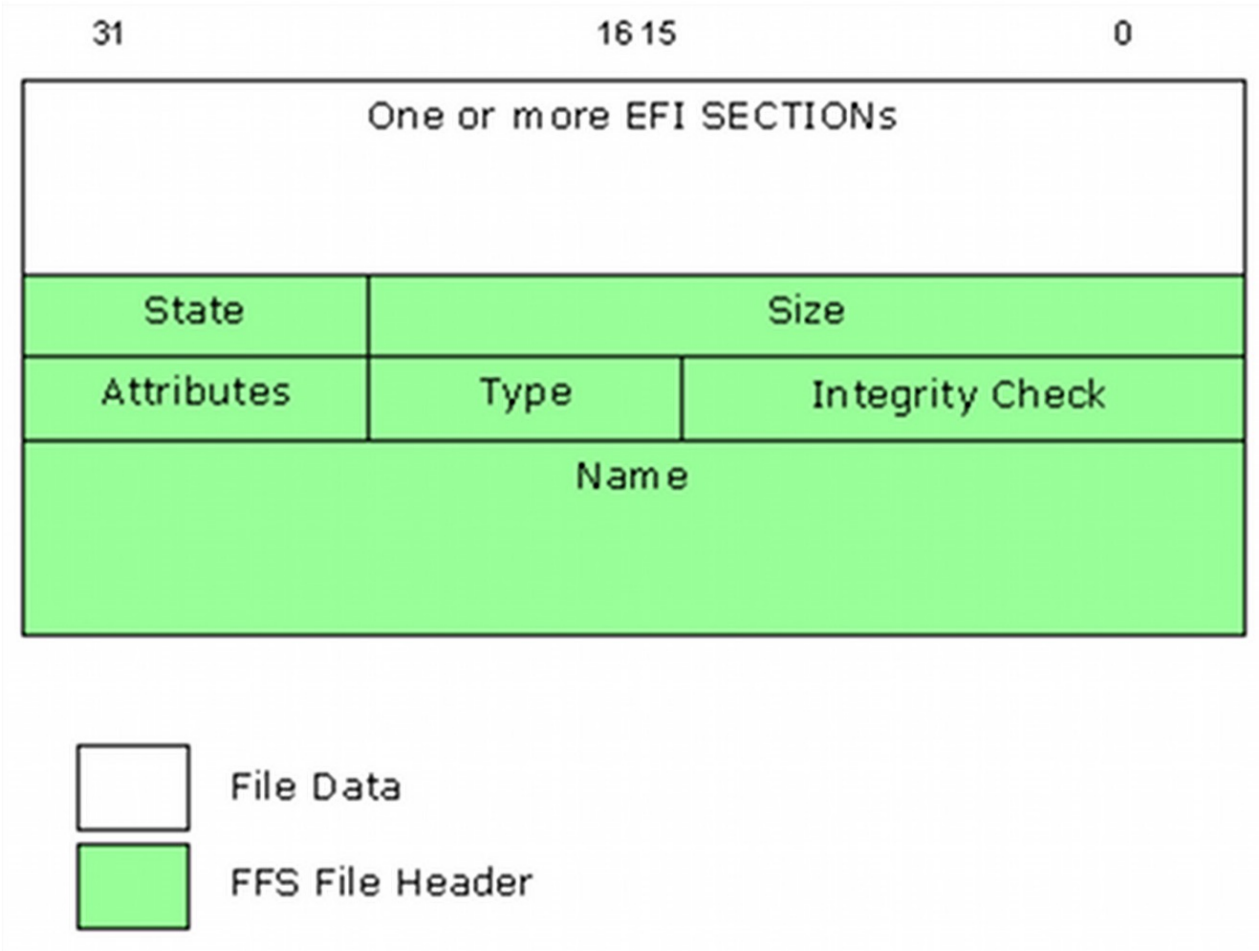


Figure 9 Typical FFS File Layout (<16MB)

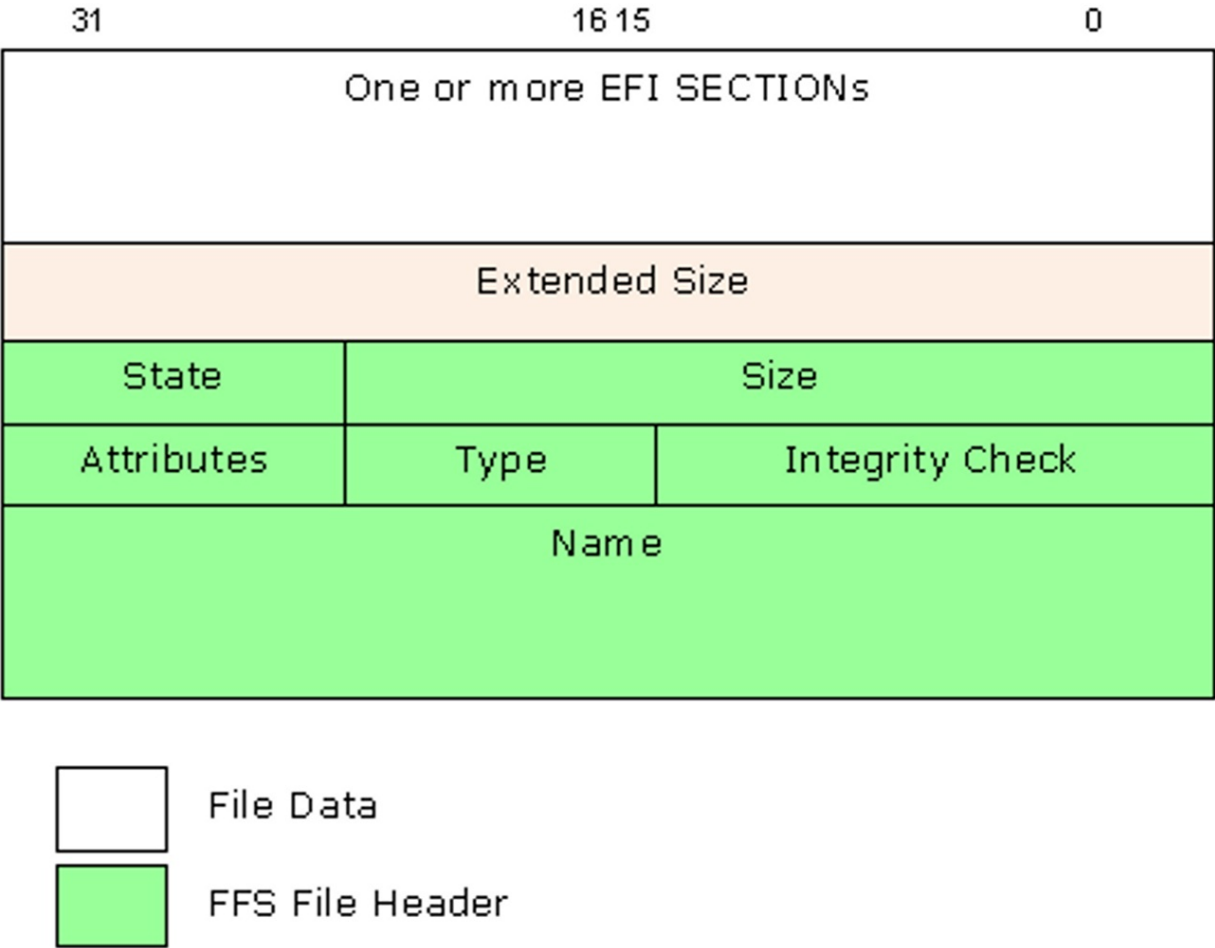


Figure 10 File Header 2 layout for files larger than 16Mb

Table 3 lists the different FV file types architecturally defined in the PI Specification describing the content (FFS) of the Firmware Volume Data.

Table 3 Defined FV File Types

Name	Description	Code
EFI_FV_FILETYPE_RAW	Binary data.	0x01
EFI_FV_FILETYPE_FREEFORM	Sectioned Data.	0x02
EFI_FV_FILETYPE_SECURITY_CORE	Platform core code used during the SEC phase.	0x03
EFI_FV_FILETYPE_PEI_CORE	PEI Foundation code.	0x04
EFI_FV_FILETYPE_DXE_CORE	DXE Foundation code.	0x05
EFI_FV_FILETYPE_PEIM	PEI Module (PEIM)	0x06
EFI_FV_FILETYPE_DRIVER	DXE driver.	0x07
EFI_FV_FILETYPE_COMBINED_PEIM_DRIVER	Combined PEIM/DXE driver	0x08
EFI_FV_FILETYPE_APPLICATION	Application	0x09
EFI_FV_FILETYPE_SMM	Contains a PE32+ image that will be loaded into SMRAM.	0x0A
EFI_FV_FILETYPE_FIRMWARE_VOLUME_IMAGE	An embedded Firmware Volume Image.	0x0B
EFI_FV_FILETYPE_COMBINED_SMM_DXE	Contains PE32+ image that will be dispatched by the DXE Dispatcher and will also be loaded into	0x0C

	SMRAM.	
EFI_FV_FILETYPE_SMM_CORE	SMM Foundation	0x0D
EFI_FV_FILETYPE_OEM_*	OEM File Types	0xC0..0xDF
EFI_FV_FILETYPE_DEBUG_*	Debug/Test File Types	0xE0..0xEF
EFI_FV_FILETYPE_FFS_*	Firmware File System Specific File Types	0xF0..0xFF
EFI_FV_FILETYPE_FFS_PAD	Pad file for FFS.	0xF0

2.5.3 Firmware Volumes

One or more FFS files are combined into a Firmware Volume (FV). The format for an FV is a header followed by an optional extended header, followed by zero or more FFS files. Figure 11 illustrates the layout of the FV.

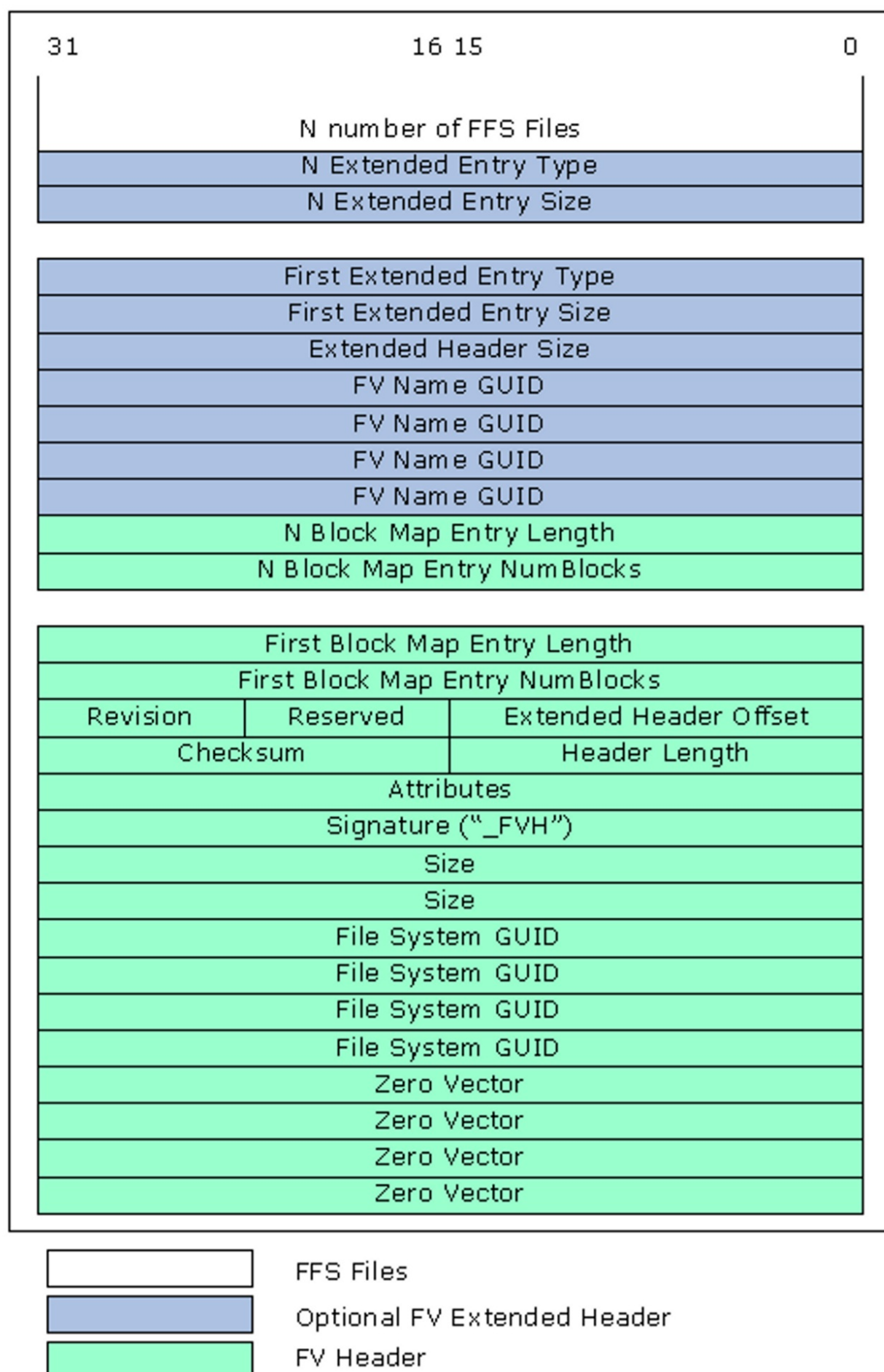


Figure 11 General FV Layout

Multiple FV files, each of which is just a logical firmware device, can be combined into a single FD image.

Within the context of modules, error messages within the code are written in plain text (English - ASCII) while messages that are displayed as part of the menu system or are stored for display later, are written in Unicode (UCS2-LE encoded) format. The UEFI/PI specifications define the structure for Human Interface Infrastructure (HII) as well as Visual Forms Representation (VFR). Vital Product Data (VPD) areas are also supported. The VPD format is unique to a platform implementation, and not defined by any specification. The EDK II build system does provide tools to generate VPD binary data files and text based map files that show the layout of the VPD PCDs.

2.5.4 Special Files - VTF & BSF

The Volume Top File (VTF) is a file that must be located such that the last byte of the file is also the last byte of the firmware volume. Regardless of the actual file type, a VTF file must have the file name GUID of `EFI_FFS_VOLUME_TOP_FILE_GUID`. The file name is a GUID, and `EFI_FFS_VOLUME_TOP_FILE_GUID` is the C define that is used by code and the build system in place of the GUID value.

The build system must be aware of this GUID and insert a pad file if necessary to guarantee the VTF is located correctly at the top of the firmware volume. This is also required for update and write operations.

The Bootstrap file is firmware file that is aligned to the top of the 32-bit address space. It is responsible for encapsulating the reset vector for the Itanium processor family and IA-32. It also contains fixed information, such as the PEIM return link for IA-32 and the entry point to the PEI core. Also of interest, it contains the base of the boot FV to enable successive module discovery in PEI.

2.5.5 EFI_FV_FILETYPE_SECURITY Notes

The security section is always executed from ROM. For size optimization, the relocation (`.reloc`) section of security executables may be stripped.

Security drivers run directly from flash need to have the BaseAddress re-based to the location the driver occupies in ROM prior to putting the driver into a Firmware Volume (FV).

2.5.6 EFI_FV_FILETYPE_PEI_CORE Notes

The last step of the security section was to hand-off execution to the PEI foundation, which is typically executed in three phases, pre-memory, during memory detection and after memory is available. For size optimization, it is recommended to have the prememory and memory detection PEI core modules ROM resident, to have the PE32+ image converted to a terse image, and to have the `.reloc` section stripped. After memory is present, it is recommended that the PEI Core modules be shadowed in memory to speed up execution. These modules can also contain signing, decryption and/or decompression routines to handle verification, uncompressing or decrypting algorithms for GUIDED encapsulation sections or for compressed PEIMs and any remaining FVs that contain the DXE Foundation and all drivers and applications that are used in the DXE phase or later. The decompression must always occur after memory is available.

The PEI Foundation modules that run directly from flash, need to have the BaseAddress re-based to the location it occupies in ROM, prior to putting the driver into an FV. By default, the EDK II build system will strip the `.reloc` section of all modules.

2.5.7 EFI_FV_FILETYPE_PEIM Notes

There are three types of PEIMs:

- XIP must execute from ROM,
- PEIMs that must be executed from memory and
- PEIMs that will execute in from memory if memory is available. If no memory is available, then the PEIMs can execute from ROM.

For PEIMs executed only from ROM, it is recommended that the image be converted to a terse image, the `.reloc` section stripped for size optimization and module cannot be compressed - the images must be re-based to the location in ROM.

PEIMs that execute from memory must never have the `.reloc` section stripped, but may be converted to terse images and may be compressed.

PEIMs that are coded to register for shadow, i.e., they may be run from memory if memory is present, must not have the `.reloc` section stripped. The EDK II build system uses a keyword, `SHADOW`, in the module's INF file to indicate this mode, setting `SHADOW = TRUE`. By default, the EDK II build system will strip the `.reloc` section of PEIMs; PEIMs must specify the `SHADOW = TRUE` in the module's INF file to prevent this. Additional flags in the FDF file, `RELOCS_RETAINED` and `RELOCS_STRIPPED`, are provided to over-ride stripping of the `.reloc` section.

Like the PEI Foundation, it is recommended that PEIMs that are able to run from memory, be shadowed in memory to speed up execution.

Once the PEI Foundation has been loaded, PEIMs are dispatched, and if a PEIM is dependent on the existence of another PEIM, an `EFI_SECTION_PEI_DEPEX` section is used to define the dependency relationship. The PEI Foundation will use this section (if present in an FFS) to ensure the required PEIMs are available prior to dispatch.

2.5.8 EFI_FV_FILETYPE_COMBINED_PEIM_DRIVER Notes

Dual function (PEI/DXE) drivers (PEIMs that are coded to register for shadow) must never have the `.reloc` section stripped. Additionally, compression of these modules may decrease the overall size of the FD image in hardware. Using the terse image format for drivers of this type is not permitted by the PI specification. For this class of driver, one PEI and/or one DXE dependency section can be added to the FFS file containing the image.

2.5.9 DXE, BDS, TLS and AL Notes

Stripping the `.reloc` section from these modules and any UEFI applications is not recommended, but is allowed in certain cases. Additionally, these images cannot be converted to the terse format - only elements of the PEI Foundation (PEI Core) and PEIMs can be converted to use the terse format headers. Compression of the images is permitted, however, as most compression algorithms work better over a larger data set, it is recommended that the images be combined into a Firmware Volume, and the entire FV can be compressed.

The modules (after the DXE Foundation has been given control) may have other dependent drivers. Similar to the `EFI_SECTION_PEI_DEPEX` section, a dependency `EFI_SECTION_DXE_DEPEX` section may be required. These files are used by the DXE foundation to ensure required drivers are available when needed.

Another feature of some of these modules, the BDS is particular, has to do with the Human Interface Infrastructure (HII). The HII uses internal forms representation (IFR) coded files.

2.6 Creating EFI Images

2.6.1 Compiling Code

EDK II modules include both libraries, drivers and applications. Library modules are compiled and linked as static libraries. Drivers and applications are compiled to object files, then linked with the static libraries they require. After the static image has been created, the resulting image is run through the dynamic linker to generate the relocateable binary images (DLL). All EFI images must be formatted PE32/PE32+/COFF.

Note: ELF images created by GCC on UNIX-based systems need additional processing to convert the image into the PE32+/COFF format.

Since UEFI/PI images are not standard executables, these dynamically linked (DLL) files must be processed to become UEFI/PI compliant images. This processing involves replacing the standard header with an EFI header that reflects the `EFI_SECTION` type. Prior to creating the EFI section files, PEI Foundation and PEIM images may be processed into either a terse image, or have the `.reloc` section removed (for images that will always execute directly from ROM).

2.6.2 Creating a Terse Image

The following is an partial view of the process; omissions may exist.

To create a Terse image:

1. The DOS, PE and/or optional headers must be replaced with a minimal header. The TE header will have a signature of "VZ". Per the PE/COFF specification, at offset 0x3C in the file is a 32-bit offset (from the start of the file) to the PE signature, which always follows the MSDOS stub. The PE signature is immediately followed by the COFF file header.
2. After verifying the DOS header's magic number (0x5A4D), the PE signature ("PE\0\0") is verified, then obtains the Machine type from the optional header's subsystem field.
 - Since this process removes bytes from the file, the number of bytes stripped must be calculated based on the location of the PESigOffset (0x3C) plus the 4 bytes containing the offset pointer plus the size of the Coff header plus the size of any Optional Header (from the Coff header's `SizeOfOptionalHeader`).
 - The number of bytes stripped must always be less than 64K bytes. The original file size minus the number of bytes stripped is then inserted into the TE header's `StrippedSize` value.
 - The optional header's magic number is used to determine whether the Optional Header data structure is IPF (`EFI_OPTIONAL_IMAGE_HEADER64`) or non-IPF (`EFI_OPTIONAL_IMAGE_HEADER32`).
 - Using the correct optional header format, the TE header's `AddressOfEntryPoint` is set to the optional header's `AddressOfEntryPoint` . Additionally, the `Subsystem` entry from the optional header's `Subsystem` entry will be packed into one byte.
 - Additional entries, `BaseOfCode` and `ImageBase` in the TE header come from the optional Header. If the optional header's `NumberOfRvaAndSizes` is greater than 0, then the relocation data from the optional header `DataDirectory[0].VirtualAddress` and `Size` is set based on the content of the optional header's `DataDirectory[0]` values.

- Likewise, if the `NumberOfRvaAndSizes` is greater than 1, then the debug data from the optional header's `DataDirectory[1].VirtualAddress` and `Size` is set in the TE header's `DataDirectory[1]` entry.
3. As a last step before creating the image, the COFF header specifies the value of the `NumberOfSections` in the file which needs to be packed into a single byte of the TE header. The number of sections must be less than 255 for this to succeed.
 4. After the header is created, then the rest of the original image - all header information stripped is appended to the TE header.

Figure 10 shows the relationship of original image to the TE image.

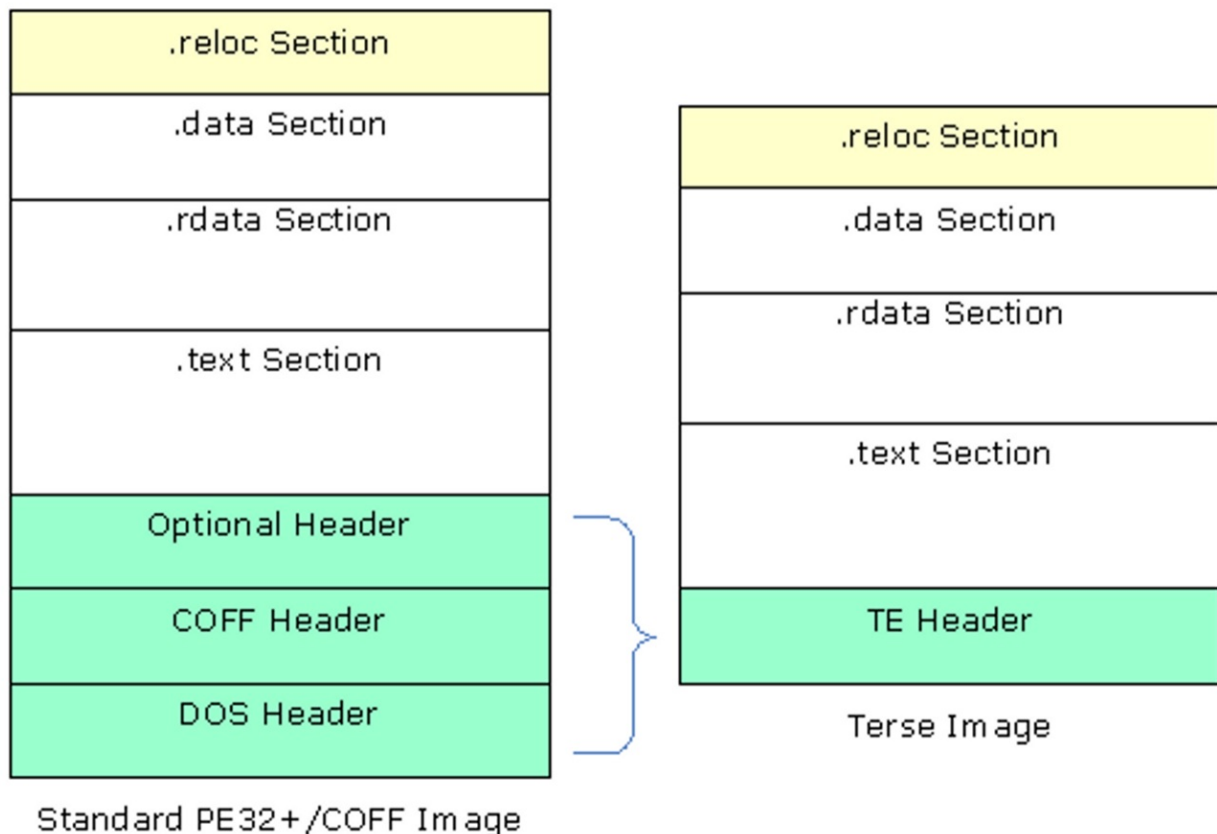


Figure 12 Standard Image to Terse Image Comparison

2.6.3 Removing .reloc sections

Removing the relocation section of either a PE or TE image can only be done if the `.reloc` section is at the end of the file. While most `.reloc` sections are fairly small in comparison to the other sections of the files, removing all of the `.reloc` sections in combination with using Terse images for the PEI foundation and PEIMs that do not register for shadow (see UEFI/PI specs) can shrink a platform image by several hundred bytes.

For a TE image, the file size (`StrippedSize`) is adjusted by subtracting the length of the `.reloc` section. The `DataDirectory[0].VirtualAddress` is set to 0, as is the `Size` parameter.

Removing the relocation section of a PE image is slightly more complicated. The PE32+ image header (which contains both the `EFI_IMAGE_DOS_SIGNATURE` - 0x5A4D and an `EFI_IMAGE_NT_SIGNATURE` - 0x00004545) will be modified by setting the `EFI_IMAGE_FILE_HEADER` Characteristics' `EFI_IMAGE_FILE_RELOCS_STRIPPED` bit. The IPF header uses a similar data structure to IA32, X64 and EBC data structures. The naming within the data structures is consistent. Therefore, regardless of the machine type, both the `SizeOfImage` and `SizeOfInitializedData` are adjusted by subtracting the length of the `.reloc` section. If the `NumberOfRvaAndSizes`

is greater than the `EFI_IMAGE_DIRECTORY_ENTRY_BASERELOC`, then the `DataDirectory[0]` `VirtualAddress` and the `Size` are both set to 0. Finally, the `.reloc` section's header is modified, setting the `Misc.VirtualSize` and `SizeOfRawData` to 0.

2.6.4 Generating LEAF EFI_SECTION Files

This section provides the overview for generating `EFI_SECTION` files. `EFI_SECTION` headers must be present on all leaf sections. The `EFI Section` header (see above) will be prefixed to the file or data section (`VERSION` and `USER_INTERFACE` data can be generated "on-the-fly" rather than creating a separate Unicode file first).

For the files that are PE32 code (`EFI_SECTION_PE32`, `EFI_SECTION_TE` and `EFI_SECTION_PIC`, the `.text`, `.debug`, `.reloc` and `.data` section headers (if they exist) are overwritten with the `EFI_IMAGE_SECTION_HEADER`.

The section name (i.e., `.text`) is copied into the `Name` entry, while the remaining sections are set as follows:

```
Hdr->Misc.VirtualSize    = Size;
Hdr->VirtualAddress      = Offset;
Hdr->SizeOfRawData       = Size;
Hdr->PointerToRawData    = Offset;
Hdr->PointerToRelocations = 0;
Hdr->PointerToLinenumbers = 0;
Hdr->NumberOfRelocations = 0;
Hdr->NumberOfLinenumbers = 0;
Hdr->Characteristics     = Flags;
```

- For a `.text` section, the `Flags` value is a bit-wise OR of `EFI_IMAGE_SCN_CNT_CODE`, `EFI_IMAGE_SCN_MEM_EXECUTE` and `EFI_IMAGE_SCN_MEM_READ` (`0x60000020`).
- For a `.data` section, the `Flags` value is a bit-wise OR of `EFI_IMAGE_SCN_CNT_INITIALIZED_DATA`, `EFI_IMAGE_SCN_MEM_WRITE` and `EFI_IMAGE_SCN_MEM_READ` (`0xC0000040`).
- For a `.reloc` section, the `Flags` value is a bit-wise OR of `EFI_IMAGE_SCN_CNT_INITIALIZED_DATA`, `EFI_IMAGE_SCN_MEM_DISCARDABLE` and `EFI_IMAGE_SCN_MEM_READ` (`0x42000040`).
- For a `.debug` section, the `Flags` value is a bit-wise OR of `EFI_IMAGE_SCN_CNT_INITIALIZED_DATA`, `EFI_IMAGE_SCN_MEM_DISCARDABLE` and `EFI_IMAGE_SCN_MEM_READ` (`0x42000040`).

Once these have been modified, the `EFI_COMMON_SECTION_HEADER` will be prefixed to the file.

Each `EFI_COMMON_SECTION_HEADER` "type" field defines the data that follows. Table 4 lists the section type value. All EFI section files start with the `EFI_COMMON_SECTION_HEADER`.

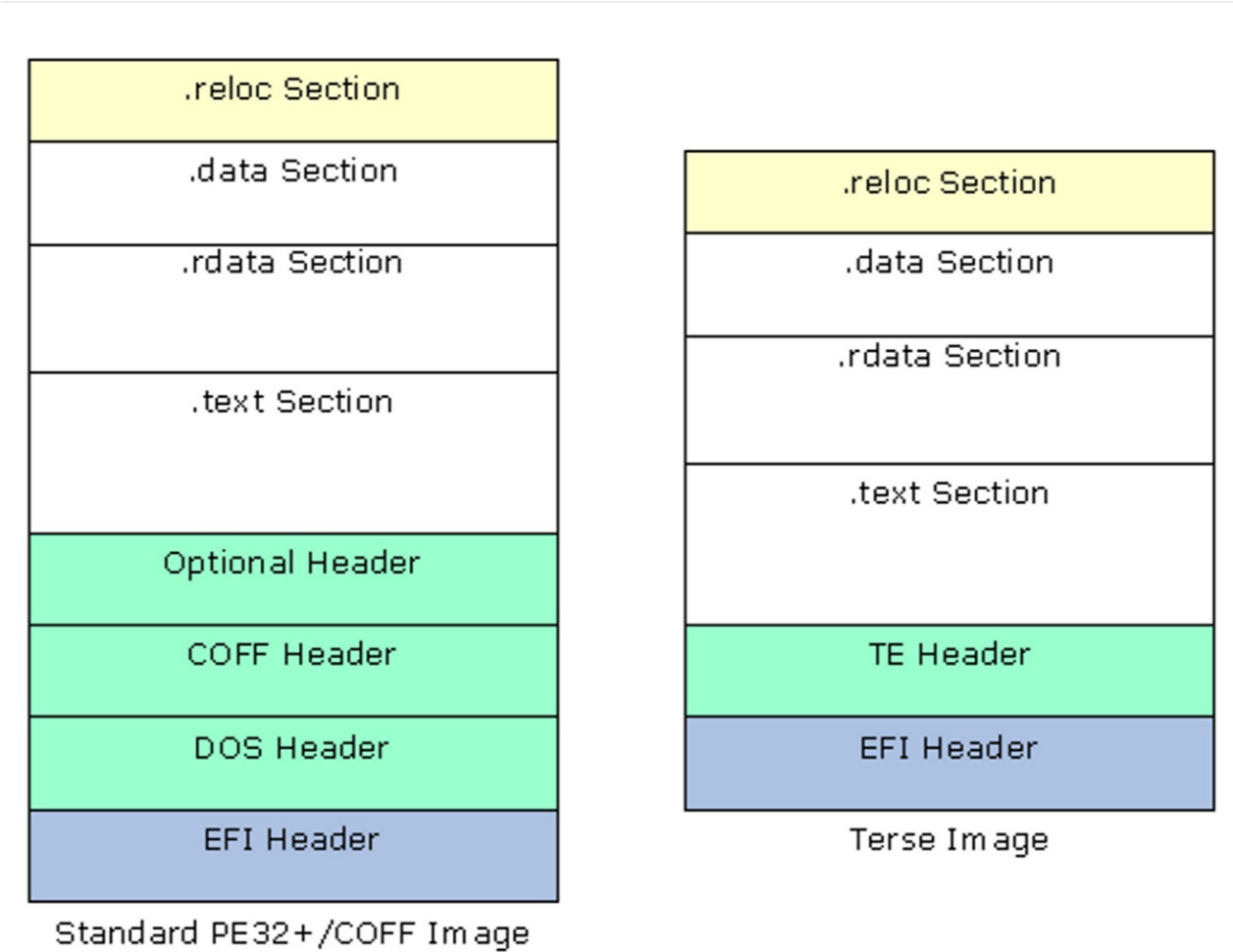


Figure 13 EFI Image Files

Table 4 Basic EFI SECTION Type Codes

Section Type	Value
EFI_SECTION_PE32	0x10
EFI_SECTION_PIC	0x11
EFI_SECTION_TE	0x12
EFI_SECTION_VERSION	0x14
EFI_SECTION_USER_INTERFACE	0x15
EFI_SECTION_COMPATIBILITY16	0x16
EFI_SECTION_FIRMWARE_VOLUME_IMAGE	0x17
EFI_SECTION_FREEFORM_SUBTYPE_GUID	0x18
EFI_SECTION_RAW	0x19

The size for these standard sections is defined as a 24-bit unsigned integer that contains the total size of the section in bytes, including the `EFI_COMMON_SECTION_HEADER`. For example, a zero-length section has a size of 4 bytes.

Except for the `EFI_SECTION_VERSION` and the `EFI_SECTION_USER_INTERFACE`, the format of each section is the `EFI_COMMON_SECTION_HEADER` prefixed to a file containing data. Refer to the definitions for `EFI_SECTION_VERSION` and `EFI_SECTION_USER_INTERFACE` in the UEFI specifications for more information.

2.6.5 Generating Encapsulation EFI_SECTION Files

This section provides the overview for generating the two Encapsulation `EFI_SECTION` files. The `EFI_SECTION` header must be present along with additional header information. The encapsulation EFI Section header will be prefixed to the file or data section. There are three encapsulation `EFI_SECTION` types. The first two types listed have extended header information.

Table 5 Encapsulation EFI_SECTION Type Codes

Section Type	Value
<code>EFI_SECTION_COMPRESSION</code>	0x01
<code>EFI_SECTION_GUID_DEFINED</code>	0x02
<code>EFI_SECTION_DISPOSABLE</code>	0x03

A compression section uses the `EFI_SECTION_COMPRESSION` header, while the GUID defined section uses an `EFI_SECTION_GUID_DEFINED` header.

The size for these sections is defined as a 24-bit unsigned integer that contains the total size of the section in bytes, including the size of the header.

For the `EFI_SECTION_COMPRESSION`, the `CompressionType` field must be set to 0x01 for standard compression, or 0x00 if the image is not compressed.

Note: In the specification, only `PI_STD` compression is supported for this section type.

For the `EFI_GUID_DEFINED_SECTION`, which is used for non-standard compression (see above) the named GUID that defines the section follows the `EFI_COMMON_SECTION_HEADER`. After this GUID are two additional `UINT16` parameters, the first is the `DataOffset` which contains the offset in bytes from the beginning of the common header to the first byte of the data. An **Attributes** parameter is a bit field code which declares specific characteristics of the section contents.

These headers are prefixed to the data files, which may include the standard PE32 headers.

2.6.6 Generating DEPEX EFI_SECTION Files

This section provides the overview for creating PEI, DXE and SMM DEPEX sections. The DEPEX grammar is defined in the PI Specification Volume 1, Dependency Expression Grammar chapter, while the OP codes for the Mnemonic are defined in Volume 2, DXE Dispatcher chapter. The translation of the mnemonic and/or GUID involves creating a binary file using postfix notation. The file does not conform to PE32+/COFF, and no header information is attached prior to generating the `EFI_SECTION` files. The format of the binary data is 8-bit aligned, with a single byte per op-code, with op-codes that require a GUID value (`BEFORE`, `AFTER` and `PUSH`) being followed by 16 bytes to contain the GUID value. See Table 6 below.

Table 6 Dependency Section Type Codes

Section Type	Value
<code>EFI_SECTION_PEI_DEPEX</code>	0x1B
<code>EFI_SECTION_DXE_DEPEX</code>	0x13
<code>EFI_SECTION_SMM_DEPEX</code>	0x1C

Once the binary file is created an `EFI_SECTION` file can be created, and the `EFI_COMMON_SECTION_HEADER` will be prefixed to the file.

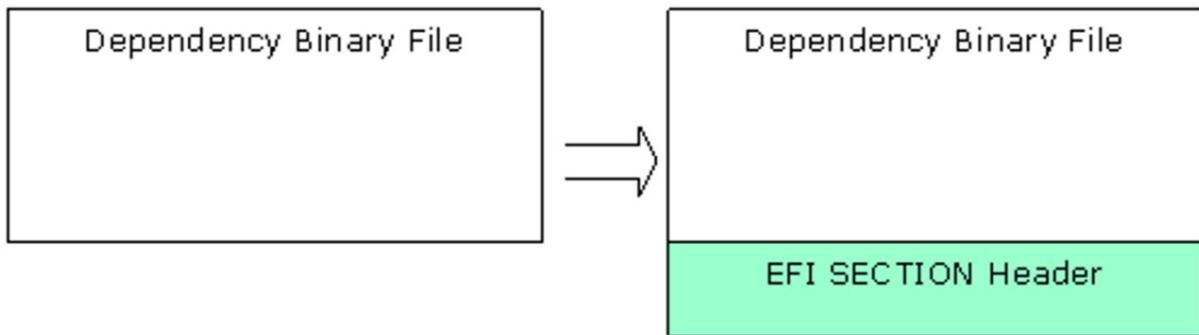


Figure 14 Depex File

2.6.7 Generating Visual Forms (IFR - HII) Files

This section covers the generation of the Human Interface Infrastructure (HII) format files used for displaying information on the console. While all error messages from the EFI drivers are written in English, displaying data on the console - selection and configuration menus - is performed using HII formats. This permits the user to select an alternate language for these displays.

Strings intended for these displays must be written in Unicode (UCS-2LE) format, rather than plain ASCII text. The Unicode strings for these forms must be kept in separate files (.uni extension), or optionally, within C code (either .c source or .h header) files. Forms, strings, fonts and images are stored in an HII database encoded to an Internal Forms Representation (IFR) - with each object and attribute a byte stream.

All HII files are included as part of a driver module's code - the data that makes up IFR content is compiled into standard object code and linked in to the driver.

2.6.8 Generating EFI FFS Files

This section provides the overview for generating an FFS file. Once the EFI Section files have been created, they need to be placed within an FFS file. An FFS file contains an FFS header and one or more section files. The ordering of the section files within the FFS is not specified by the PI specification, so sections may appear in any order. The Name of the FFS file, which is placed in the `FfsFileHeader` data structure is a GUID value with a structure of `UINT64`, `UINT32`, `UINT32`, `UINT8[8]`.

The alignment of data within the FFS must match the alignment specified for a given section, so padding may be required between the FFS header and the section headers. Alignment must be set and padding inserted prior to calculating the size or performing the Integrity check (checksum on the header itself and all of the section data).

The size of the FFS, in the `FfsFileHeader.Size` array is computed using the size of all files, including all pad files, plus the size of the header. The size value must be less than `0x01000000` (16MB).

The `FfsFileHeader.IntegrityCheck.Checksum.Header` is set to 0, as are the `Checksum.File` and `FileHeader.State`, prior to calculating (and setting) the checksum of the header. If the `FFS_ATTRIB_CHECKSUM` bit is set in the `FfsFileHeader.Attributes`, then the checksum for the remainder of the FFS content must be generated and placed in the `Checksum.File` part of the `FfsFileHeader.IntegrityCheck` structure.

The `FfsFileHeader.State` is zeroed, the `EFI_FILE_HEADER_CONSTRUCTION`, `EFI_FILE_HEADER_VALID` and `EFI_FILE_DATA_VALID` bits are set.

2.6.8.1 FDF file

The build system uses the FDF file to specify construction of the FD, FVs and FFS files, as well as how to construct the different EFI Sections (what content is put into each section). Flags for attributes and alignment values are specified in the FDF file. These values are used to set the bits in FFS Header. As an example, if multiple sections are specified with different alignment values, only the maximum value of the alignment is used, and all sections are aligned to that value. Additionally, the sections are placed into the FFS in the order they appear in the FDF or specified by the Rules section of the FDF configuration file. Each driver is put into an FFS of its own. Also, EDK II expects the ordering of PEIM and DXE FFS files to start with an optional dependency section, followed by the PE32, user interface and finally the version sections.

2.6.9 APRIORI Files

At most, there can be at most one PEI `APRIORI` and one DXE `APRIORI` file in a given firmware volume.

The PEI file, named by GUID of `PEI_APRIORI_FILE_NAME_GUID`, will specify the order of invocation of PEIMs by the PEI foundation. This is a special file, of the type, `EFI_FV_FILETYPE_FREEFORM` with a single `EFI_SECTION_RAW` and has the format:

```
typedef struct {
    EFI_GUID  FileNamesWithinVolume[NumberOfModulesInVolume];
} PEI_APRIORI_FILE_CONTENTS;
```

The DXE file, named by GUID of `DXE_APRIORI_FILE_NAME_GUID`, will specify the dispatch order of drivers by the DXE foundation. This is a special file, of the type `EFI_FV_FILETYPE_FREEFORM` with a single `EFI_SECTION_RAW` and has the same format as the `PEI_APRIORI_FILE_CONTENTS`.

2.6.10 Generating EFI Firmware Volume (FV) Files

This section provides the overview for generating an FV file, which contains an FV header and a sequence of FFS files. FVs are usually implemented so that the SEC and PEI Foundation are not compressed, while most PEIMs are executed from ROM (see above). As a result, these images are typically placed in a separate FV, with post-PEI phase modules placed in one or more FVs that are compressed. Reference Section 2.6.11 below.

The FV files are combinations of FFS files. For SEC, PEI Foundation and most PEIMs that execute directly from ROM, will need to have the BaseAddress re-based to the location of the driver in ROM. There are three different types of rebase actions. The first action is for the initial Boot drivers, while the most common is for execute in place (XIP) drivers. Some Runtime drivers may also need to be re-based. As part of the rebase these execute from ROM drivers may need to be aligned to the natural alignment of the machine architecture (or section alignment).

2.6.10.1 Combining FFS files into FV files

The build system uses the Flash Description File (FDF) to describe how to combine FFS files into different FV files, as well as the layout of the FD files within an FD description. Each FV definition within the FDF is used to complete a data structure for constructing the FV. The `FvName` in the `FV_INFO` structure is used to identify the name of the files that will be created in the FV directory.

```
typedef struct {
    BOOLEAN          BaseAddressSet;
    EFI_PHYSICAL_ADDRESS BaseAddress;
    EFI_GUID          FvFileSystemGuid;
    BOOLEAN          FvFileSystemGuidSet;
    CHAR8            FvExtHeaderFile[_MAX_PATH];
    UINTN            Size;
```

```

EFI_FVB_ATTRIBUTES    FvAttributes;
CHAR8                 FvName[_MAX_PATH];
EFI_FV_BLOCK_MAP_ENTRY FvBlocks[MAX_NUMBER_OF_FV_BLOCKS];
CHAR8                 FvFiles[MAX_NUMBER_OF_FILES_IN_FV][_MAX_PATH];
UINT32                SizeOfFvFiles[MAX_NUMBER_OF_FILES_IN_FV];
BOOLEAN               IsPiFvImage;
INT8                  ForceRebase;
} FV_INFO;

```

The FV file header (see `EFI_FIRMWARE_VOLUME_HEADER` definition in Section 3) is constructed using the following information.

The first 16 bytes (`ZeroVector`) are set to zero. The `FvFileSystemGuid` is assigned a PI Specification defined GUID (`EFI_FIRMWARE_FILE_SYSTEM2_GUID`) that identifies it as a PI compliant Firmware Volume. The `Signature` is set to " _FVH " and the reserved byte is set to zero. The PI Specification defined `Revision` is set to `0x02` .

As FFS files are added to the FV, the length of the FFS is added to the `FvLength` field, such that the final `FvLength` is complete length of the firmware volume, including the header (and extended header information). Also, as an FFS file is added to the FV, if the driver executes from ROM, the base address of the driver will be adjusted (re-based) within the FFS file to the physical location in ROM (`BaseAddress + offset`).

`Attributes` (defined in the FDF file) are set that define the capabilities and power-on defaults of this FV. These come from the `FvAttributes` of the `FV_INFO` data structure. The `HeaderLength` is set to size of header, including the size of the `{0,0}` terminated `BlockMap` data array.

The `BlockMap` data array is a mapping of the FFS files - giving the length (in blocks) and block size for each FFS file in the FV, starting with the first FFS file. This is an index of the blocks, and does not specify each FFS by name. If an extended header is required, it must be placed immediately following the `BlockMap` data array. The `ExtHeaderOffset` is set to the location of the extended header. Each block will be aligned on the largest value specified by the `EFI_FVB2_ALIGNMENT` attribute. Note that it is permissible to use variable block length devices, and as such, each block entry would have the `BlockMap[index].NumBlocks = 1` , while the `BlockMap[index].BlockLength` would vary according to the size of the FFS file (plus any padding value needed to align the next block on a natural boundary).

If the extended header is not included, then the `ExtHeaderOffset` is set to zero. If an extended header is present, it is followed by zero or more variable length extension (`EFI_FIRMWARE_VOLUME_EXT_ENTRY`) entries.

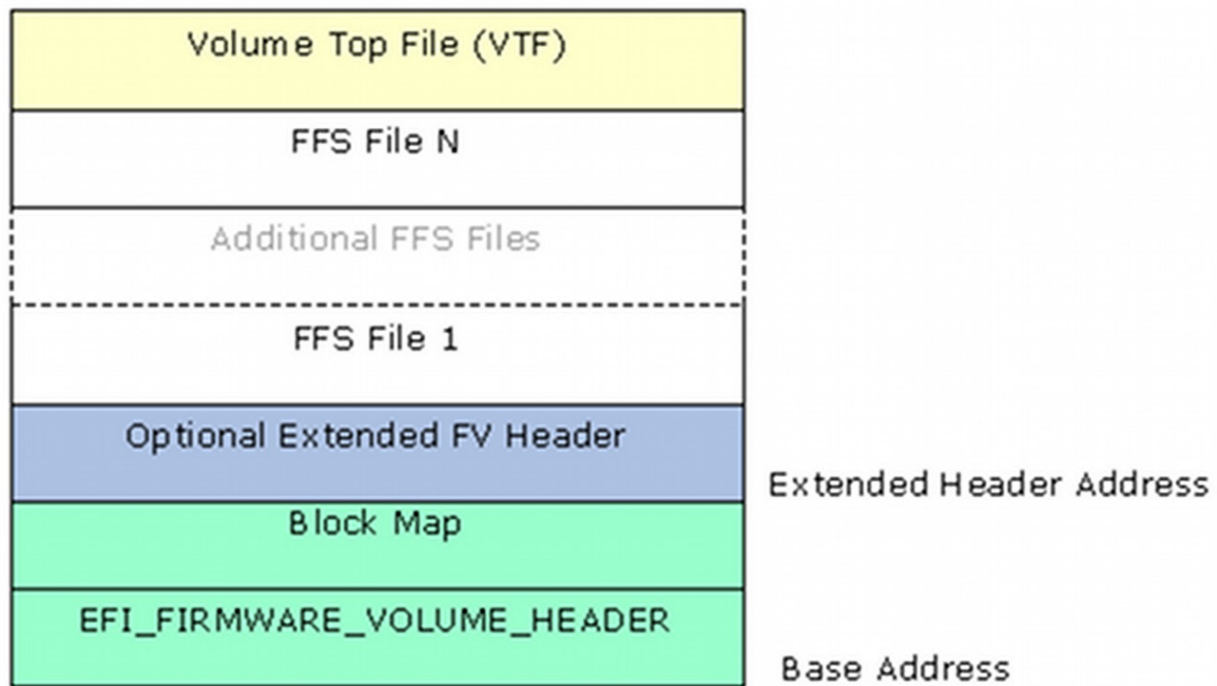


Figure 15 Firmware Volume Layout

Prior to adding the last FFS file (as specified in the FDF file), the FFS file must be modified to comply with the Volume Top File specification.

After the last FFS file has been added (so that the `FvLength` is complete), the `Checksum` field is set to zero and a checksum is calculated on the header so that a valid header will sum to zero (and placed into the `Checksum` field).

2.6.11 Implementing Compression

As stated in earlier sections, images that are executed from ROM may not be compressed. Images that contain .reloc sections, or that are executed post PEI phase may be compressed to save space in ROM. For best space savings and performance, an entire FV (containing all post PEI phase code) can be compressed, rather than compressing individual drivers. Decompression routines take a finite amount of time which can be additive - especially on copy commands - which can result in an unacceptable boot speed. Additionally, most compression algorithms provide better compression over larger data sets.

The EDK II Build system supports EFI standard compression as well as CRC32 validation within the tools.

For any other form of compression, encoding or signing must be through external tools. TianoCompress and LZMA tools are provided with the EDK II build system.

The GUIDED encapsulation section method is used to control these additional tools. By definition (UEFI/PI specifications) a named GUID for a GUIDED encapsulation section is used to provide information about how to process the section.

Assigning a GUID to a tool, such as **TianoCompress**, in the `tools_def.txt` file (refer to Section 5) the **TianoCompress.exe** application can be used to compress an EFI image - FV, FFS and/or SECTION. The following shows two lines that are in the `tools_def.txt` file to identify the **TianoCompress** tool.

```
*_MYTOOLS_*_TIANO_PATH = TianoCompress.exe
*_MYTOOLS_*_TIANO_GUID = A31280AD-481E-41B6-95E8-127F4C984779
```

2.6.11.1 GUIDED encapsulation sections

When the Build system encounters a GUIDED encapsulation section in the FDF file, the GUID is tested against GUIDs registered in the `tools_def.txt` file. If a match is found, then the executable tool associated in the GUID is executed on the encapsulated data defined in the FDF file. Since the tool is not present during a system boot, any optional tool must be able to provide code that can be used by any decompression, signing or verification drivers during boot. The following shows the use of the `TianoCompress` GUID in a sample FDF file for an FVMAIN image that contains all post-PEI modules.

```
FILE FV_IMAGE = E76CB2EC-A71A-42e8-8F34-56237870BD12 DEBUG_MYTOOLS_IPF {  
  SECTION GUIDED A31280AD-481E-41B6-95E8-127F4C984779 {  
    SECTION FV_IMAGE = FVMAIN  
  }  
}
```

In the example above, the first GUID (starting with `E76CB2EC`) is the EFI Name for the firmware volume, while the second (following the `SECTION GUIDED` statement) identifies the tool (**TianoCompress.exe**) that will be used on the FV section specified within the curly brackets after the GUID.

The EDK II build system requires that all such tools take a minimum of three options on the command line. The **-e** option specifies that the tool will encrypt, compress, encode or sign the file specified on the command line. The **-o** option specifies the name of the output file to be created when using the **-e** option. A third option, **-d**, is used to decrypt, decompress, decode or verify the file specified on the command line.

Note: Additional options may be included with the tool, however, the build system only requires these three options.

2.6.12 Implementing Encryption or Signing

The same techniques used for implementing custom compression can be used for tools that are used for signing or encrypting an image.

2.6.13 Generating an FD image file

This section provides the overview for generating a platform flash file. When generating the FD file, the flash device is assumed to be "partitioned" into different areas, with the content of each added in sequence (with zero filled padding of any partial blocks to the next specified offset).

2.6.13.1 Data structures

The EDK II build system will create an FD file in the FV output directory, and using the information in the FDF file, will add all FV files, as well as any data structures that are specified in the FDF file. Each FV will be added to the FD file in the order specified by the `[FD]` section at the location specified. Data structures in the FDF are typically used to initialize the data area for use by EFI drivers, and as such, may require an FV header to identify the region (such as NV storage) in Flash.

2.6.14 Generating Applications

This section provides an overview to generating UEFI applications which may or may not be resident in a flash device. Applications fall into three different types, applications that execute from within a flash image, applications that execute from the EFI Shell and applications that execute from an Operating System (accessing UEFI runtime services or need to access UEFI System Table fields). The build only

provides support for UEFI applications that execute from within the flash image and applications that execute from the EFI Shell. These statically linked applications cannot make use of OS standard libraries or headers.

Applications that are executed within the flash image must be stored in an FFS file, along with the optional version and user interface sections. These applications are installed as part of the standard shell commands. The only way to execute a command that is executed within the flash image is to install it along with the shell commands. Otherwise, they cannot be executed.

Applications that execute from the EFI shell are PE32/COFF applications that have a modified header, and do not need to be placed within an FFS file. The .efi file generated by the \$(MAKE) stage is capable of being executed from the shell command prompt.

2.6.15 Generating an Option ROM file

This section provides the overview for generating an external PCI Option ROM, where the driver is on a PCI add-in card. PCI devices that are laid down on a platform board, rather than on an add-in card (Ethernet, Video, Audio, etc. devices), will most likely have the driver resident in an FFS/FV/FD with the device vendor providing the driver code to the board vendor. A PCI Option ROM is typically discovered during system initialization, and the driver will be dispatched by the DXE Foundation. PCI Option ROM drivers are constructed from either EFI files or Binary files or a combination of both.

Most EFI implementations of PCI Option ROMs can use EFI compression for the driver, so that the ROM image fits into a smaller size device on the PCI add-in card. These drivers can use NV storage space in the primary on-board flash device, provided they register the system table data. If the driver is compressed, the size of the compressed file must be an even multiple of 512 bytes.

Note: The maximum size for the driver code is 16MB, so drivers that are larger than 16MB must be compressed.

The EFI file (PE32+ with modifications to the .data, .text, .reloc and .debug sections - see Generating Leaf `EFI_SECTION` Files above) will have an `EFI_PCI_EXPANSION_ROM_HEADER` prefixed to the EFI file (aligned on a 4-byte boundary). The header structure is defined in the PCI industry standard specification, and is shown below.

```
typedef struct {
    UINT16  Signature;
    UINT16  InitializationSize;
    UINT32  EfiSignature;
    UINT16  EfiSubsystem;
    UINT16  EfiMachineType;
    UINT16  CompressionType;
    UINT8   Reserved[8];
    UINT16  EfiImageHeaderOffset;
    UINT16  PciOffset;
} EFI_PCI_EXPANSION_ROM_HEADER;
```

The `Signature` value of the PCI 3.0 version header is defined as `0xAA55`, and the `EfiSignature` is defined as `0x0EF1`. The `InitializationSize` is the number of 512-byte blocks of the driver image plus the size of this header. The `EfiSubsystem` is set to the value of PE32 Optional Header's Subsystem value, while the `EfiMachineType` is set to the `EFI_IMAGE_FILE_HEADER`'s Machine Type. The `CompressionType` field is set to either `0x0000` for no compression, or `0x0001` for standard EFI compression - no other compression types are permitted. The reserved bits are typically set to 0. However they may be used. The `EfiImageHeaderOffset` is set to the size of this header, while the `PciOffset` is the offset to the EFI header, (the Option ROM header size plus any padding bytes to align the driver on its natural alignment boundary). Additionally,

the PCI Data Structure (PCI 3.0 compliant is the default) is also inserted. The Vendor ID and Device ID are inserted into the PCI Data Structure. The `ClassCode` and `CodeRevision` are determined from the input file header information, while the `ImageLength` field is set to the Option ROM Header's `InitializationSize` field. All other fields are currently set to 0 by the reference implementation's `EfiRom` tool.

PCI device Expansion ROMs may contain code for multiple processor architectures. This may be implemented in a single physical ROM image, which can contain as many code images as desired for different system and processor architectures, Figure 16, below.

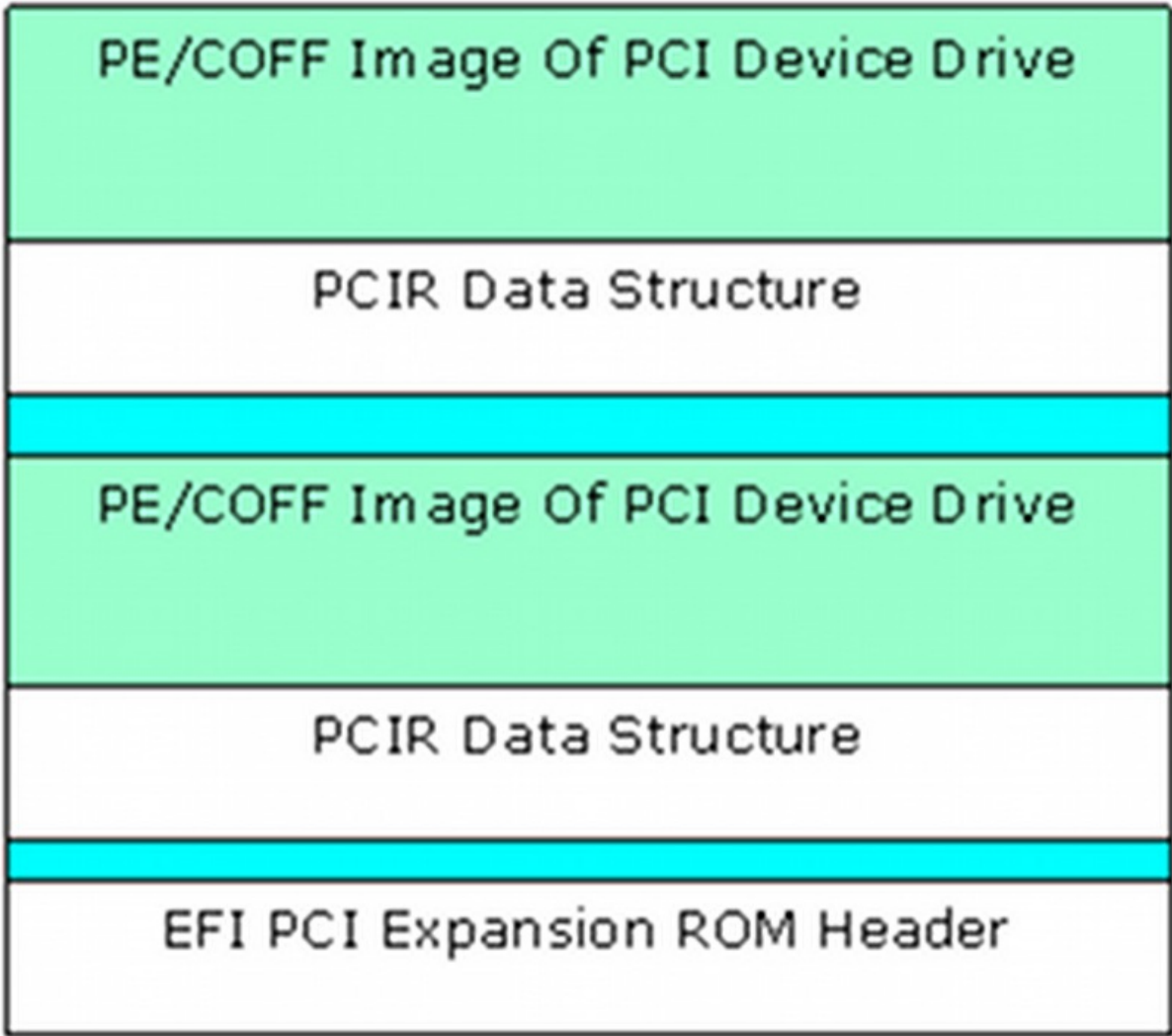


Figure 16 EFI PCI Expansion Option ROM layout

Each image must start on a 512-byte boundary and must contain the PCI Expansion ROM header. The starting point of each image depends on the size of previous images.

The last image in a ROM has a special encoding in the header to identify it as the last image (PCI Firmware Specification, Revision 3.0).

Legacy Option ROM images must be the first image in the ROM image. The following is a list of the image combinations that may be placed in a PCI option ROM. This is not an exhaustive list. Instead, it provides what will likely be the most common PCI option ROM layouts. EFI complaint system firmware must work with all of these PCI option ROM layouts, plus any other layouts that are possible within the PCI Specification. The format of a Legacy Option ROM image is defined in the PCI Specification.

- Legacy Option ROM image

- Legacy Option ROM image + IA-32 EFI driver
- Legacy Option ROM image + Itanium Processor Family EFI driver
- Legacy Option ROM image + IA-32 EFI driver + Itanium Processor Family EFI driver
- Legacy Option ROM image + IA-32 EFI driver + x64 EFI driver
- Legacy Option ROM image + EBC Driver
- IA-32 UEFI driver
- Itanium Processor Family EFI driver
- IA-32 UEFI driver + Itanium Processor Family EFI driver
- EBC Driver

It is also possible to place an application in a PCI Option ROM. The exact mechanism by which applications can be loaded and executed from a PCI Option ROM is outside the scope of this document.

2.6.15.1 Binary Option ROM code

Pre-existing Binary Option ROM code can also be provided by hardware vendors. These images are verified to be of the correct format and length. If the length of the provided image is not an exact 512-byte multiple, padding bytes are added to ensure the image is an exact multiple of 512 bytes. If this occurs, a new checksum is calculated and replaces an existing checksum value.

2.6.16 Generating Capsule Update Files

This section provides the overview for generating Capsule files. Capsules are formatted variable length data structures that are passed from runtime back to the pre boot phases (PEI, DXE, BDS). They are intended to be the major vehicle for delivering firmware volume changes. Capsules are constructed with a capsule header and the capsule volume. Content within the capsule volume usually includes a Firmware Volume as well as a Configuration Results (CR) file. The CR file is a string of Internal Forms Representation (IFR) name, value pairs as defined by the Human Interface Infrastructure (HII).

After identifying and creating the Firmware Volume that will be included in the capsule, the capsule header will be constructed. The header is constructed as follows.

The `CapsuleGuid` defines the format of the capsule data - including any optional header information. The format for a capsule is shown in Figure 15.

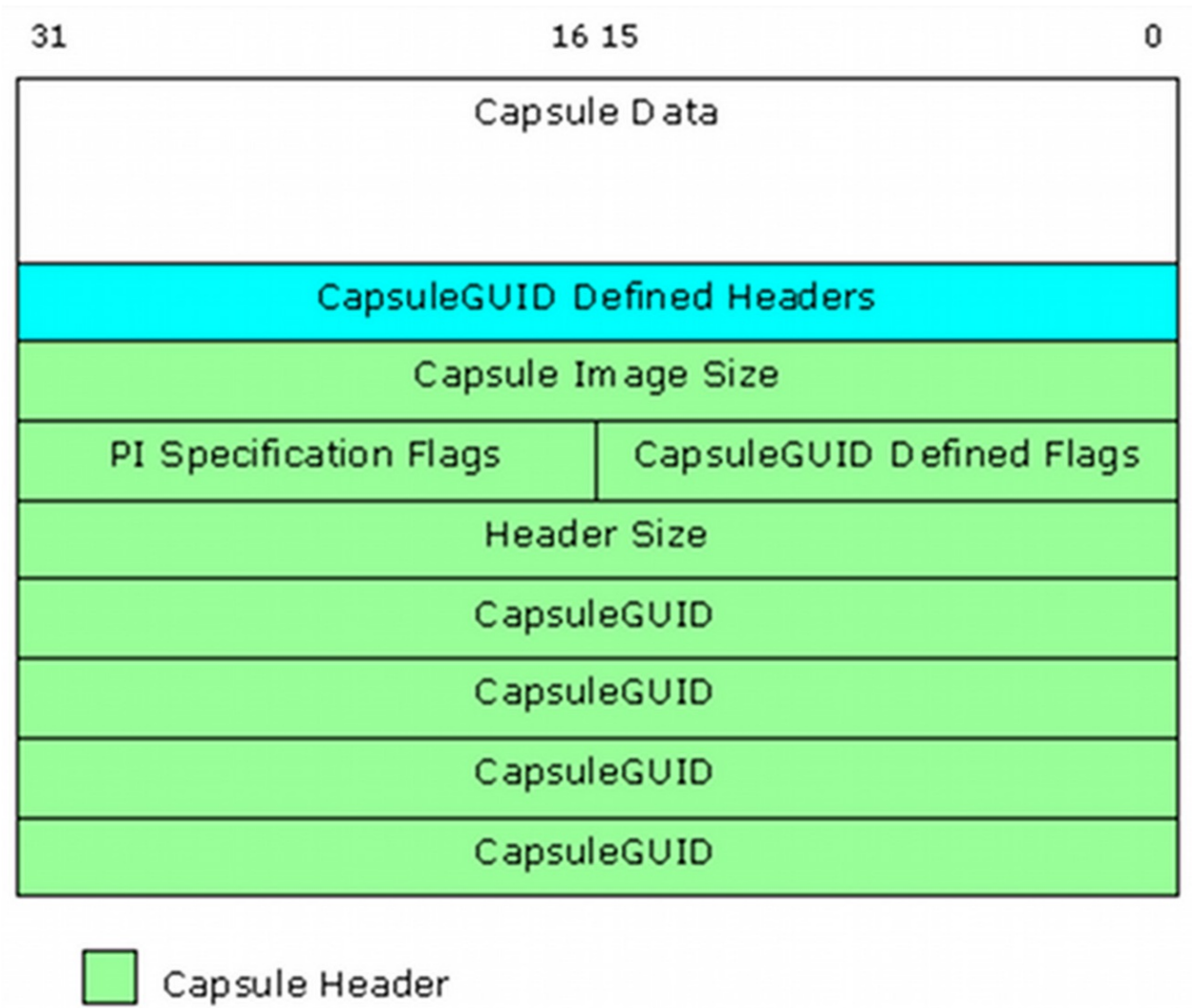


Figure 17 EFI Capsule Layout

2.6.16.1 Capsule generation and content

The EDK II build system provides functionality to generate capsules. The capsule data content is defined in the FDF file for a given platform

2.7 SKU Support

The EDK II build system provides the capability of supporting multiple SKUs in a single firmware image. SKU selection is a runtime option that can be set from a platform driver. The build system also supports building a specific SKU.

The SKU enabled PCD sections (defined by a PCD section tag in the DSC file that contains a SKUID modifier that is not DEFAULT) are a sparsely populated set of configuration settings. The platform developer may specify one or more PCDs that will have different values than the PCD values specified by the default SKU. Additional PCDs not listed in a default PCD section may also be specified under a section with a SKUID modifier.

During runtime, the PCD drivers will automatically return a default SKU value if no specific SKU value was specified after a platform driver calls SetSku(). The configuration elements, PCDs, must be accessed using either Dynamic or DynamicEx PCD access methods. When building an image that supports multiple SKUs, the Feature Flag, Fixed At Build and Patchable In Module PCDs will only use the default SKU configuration settings. The default configuration settings are identified by PCD section tags that have either a Default SKUID modifier or have not SKUID modifier. The set of SKUs that can be included is configurable either through setting the list in the DSC file `[Defines]` section's `SKU_IDENTIFIER` element or through setting one or more `-x SKUID` command-line options to the build command.

When building a single SKU, it is possible to use SKU specific Feature Flag, Fixed At Build and Patchable In Module configuration elements along with the Dynamic and DynamicEx PCD for the specific SKU. The build tools will automatically adjust the SKU specific Dynamic and DynamicEx PCD values overriding the default values. This is equivalent of running SetSku immediately on reset.

Note: If there are no PcdsDynamic or PcdsDynamicEx section tags that use a Skuld modifier, then only the DEFAULT values will be placed into the PCD Database. The platform drivers must not call SetSku() for this single SKU, as the 'DEFAULT' SKU will be the only SKU available when built by this method.

The following examples show the three types of builds.

DEFAULT SKUID Build One or more SKUID | SKUIDENTIFIER entries may appear in the DSC file's `[Skulds]` section as in the following example:

```
[SkuIds]
0 | DEFAULT
1 | ScsiSku
2 | SataSku
3 | iScsiSku
```

Only the `DEFAULT` SKU will be built as identified by a single entry in the `SKU_IDENTIFIER` in the DSC file's `[Defines]` section as in the following example:

```
SKU_IDENTIFIER = DEFAULT
```

The user is not required to specify:

```
-x DEFAULT
```

FeatureFlag, PatchableInModule and FixedAtBuild PCD values from the values in the default PCD sections.

Dynamic and DynamicEx PCD values from the default PCD sections (sections that do not contain a Skuld modifier in the section tag or that contain a Skuld modifier of DEFAULT) must be used. These values will then be placed in the DEFAULT table of the PCD Database.

WARNING: The platform drivers must not call SetSku() for this single SKU, as the 'DEFAULT' SKU will be the only SKU available when built by this method.

Single SKUID Build This method is useful for debugging as well as for size-optimization.

More than one `SKUID | SKUIDENTIFIER` entry must appear in the DSC file's `[SkuIds]` section as in the following example:

```
[SkuIds]
0 | DEFAULT
1 | ScsiSku
2 | SataSku
3 | iScsiSku
```

Only one of the possible SKUs will be built as identified by a single entry in the `SKU_IDENTIFIER` in the DSC file's `[Defines]` section as in the following example:

```
SKU_IDENTIFIER = ScsiSku
```

Note: A `SKU_IDENTIFIER = DEFAULT | ScsiSku` must be treated by tools as exactly the same as if just `SKU_IDENTIFIER = ScsiSku` had been specified.

If the users specifies the following option on the build command-line, only SataSku SKUID will be included:

```
-x SataSku
```

The `-x SKUIDENTIFIER` command-line option overrides the `SKU_IDENTIFIER` statement in the `[Defines]` section.

FeatureFlag, PatchableInModule and FixedAtBuild PCD values from the PCD section that contains the matching SKUID modifier will override the values in the default PCD sections.

Dynamic and DynamicEx PCD values from PCD sections that contain the matching SKUID modifier will override the values from the default PCD section. These values will then be placed in the DEFAULT table of the PCD Database. This is equivalent of executing a SetSku() immediately on reset/power-on.

WARNING: The platform drivers must not call SetSku() for this single SKU build, as the 'DEFAULT' SKU will be the only SKU available when built by this method.

Multiple SKUID Build The DEFAULT SKU is always included in a multi-SKU platform build as these are the default values returned by PcdGet statements until such time as a platform driver executes a SetSku() call.

More than two `SKUID | SKUIDENTIFIER` entries must appear in the DSC file's `[SkuIds]` section as in the following example:

```
[SkuIds]
```

0		DEFAULT
1		ScsiSku
2		SataSku
3		iScsiSku

Only two of the possible SKUs will be built as identified by a list of `SKU_IDENTIFIER` in the DSC file's `[Defines]` section as in the following example:

```
SKU_IDENTIFIER = ScsiSku | SataSku
```

Note: A `SKU_IDENTIFIER = DEFAULT | ScsiSku | SataSku` must be treated by tools as exactly the same as if this `SKU_IDENTIFIER = ScsiSku | SataSku` statement had been specified.

If the users specifies the following options on the build command-line, all of the SKUIDs will be included (DEFAULT is always included):

```
-x ScsiSku -x SataSku -x iScsiSku
```

FeatureFlag, PatchableInModule and FixedAtBuild PCD values must come from the default PCD sections; PCD sections for these access types that have a SKUID modifier must be ignored by the Build Tools. If a PCD listed in a PCD section with a SKUID modifier is NOT listed in the default PCD section, the PCD cannot be used by any module included in the build.

Dynamic and DynamicEx PCD values from the DEFAULT SKU as well as values from the specified SKUs will be put into tables in the PCD Database. Prior to a platform driver is calling `SetSku()`, drivers accessing the PCD database will get values from the DEFAULT SKU. Once a platform driver calls `SetSku()`, the values for the specific SKU will be returned (unless there is no entry for the PCD in the specific SKU table, in which case, the value will come from the DEFAULT SKU table).

3 UEFI AND PI IMAGE SPECIFICATION

See the Microsoft Portable Executable and Common Object File Format Specification for information on PE/COFF images.

The Platform Initialization Specification, Volume 1 defines the data structure for terse executables.

The Platform Initialization Specification, Volume 3 defines the data structures for UEFI/ PI images.

The build tools are required to support IFR op-codes defined in the UEFI specifications. The EDK II implementation for Visual Forms Representation (VFR) syntax is documented in the VFR specification available from the TianoCore EDK II web-site, under EDK II Specifications.

4 EDK II BUILD PROCESS OVERVIEW

The EDK II build system is used to process EDK II meta-data files, EDK II source and/or binary files and some legacy EDK components and libraries. The code-base for EDK II content can be obtained from various sources or various distribution methods. The EDK II build system provides the UEFI Distribution Packaging Tool (UEFIPT) that can be used to create, install or remove UEFI distribution packages. The UEFI distribution package format does not depend on any specific build system. However, the UEFIPT must be used within the context of the EDK II build system.

The EDK II `EdkCompatibilityPkg` in the EDK II source tree provides backward compatibility for existing EDK components and platforms; using EDK processes and tools will not be described in this document. Some EDK components may be built using the EDK II build tools, where those components are included in an EDK II platform file. The exact list of EDK components, or the compatible component types are not provided here - other EDK II documentation contains information on using EDK components with EDK II.

4.1 EDK II Build System

EDK II build system produces binary images that conform to UEFI and PI specification file formats. In some cases, the tools have been extended to follow the Intel Innovation Framework for EFI Specifications. Some binary content may also follow other industry standard specifications, such as ACPI and PCI specifications.

While the build system may seem complex, it was designed to be extremely flexible.

The original build system worked on files within a development `WORKSPACE`. All content for the build had to be located within the `WORKSPACE` directory tree.

The build system has been updated to allow the setting of multiple paths that will be searched when attempting to resolve the location of EDK II packages. This new feature allows for more flexibility when designing a tree layout or combining sources from different sources. The new functionality is enabled through the addition of a new environment variable: `PACKAGES_PATH`.

The `PACKAGES_PATH` variable is an ordered list of additional search paths using the default path separator of the host OS between each entry (";" on Windows, ":" on Linux and OS/X). The path specified by the `WORKSPACE` variable always has the highest search priority over any `PACKAGE_PATH` entries. The first path (left to right) in the `PACKAGES_PATH` list has the highest priority and the last path has the lowest priority.

As soon as a match has been found the build tools will stop searching.

The output of the build system may be located outside of the development workspace. The `WORKSPACE`, `PACKAGES_PATH` and `EDK_TOOLS_BIN` system environment variables contain directory paths that must never contain space characters even though they are permitted by the operating system.

The build system works in the context of a platform, using the Platform Description (DSC) file to define what will get built. When building a single driver, or an application, the DSC file is used to define what will be built, along with the libraries, configuration settings and custom build flags.

All ASCII source files (see Table 14) in the EDK II code base must use the DOS CRLF character sequence for the end-of-line terminator except those that are strictly for GCC, such as assembly files that are only to be processed by *NIX tools that use an extension of ".s" (lower case). Unicode files use the UCS-2 character set.

The Base Tools ASCII source files (C and Python) must use the DOS CRLF character sequence for the end-of-line terminator as well as the DOS batch files with an extension of `.bat`. The *NIX shell scripts identified by an extension of `.sh` as well as the scripts in `BaseTools/BinWrappers/PosixLike` and the `BaseTools/Bin/CYGWIN_NT-5.1-i686` directories must always use the Linux LF character for the end-of-line terminator. Apple's Mac* OS/X operating system correctly translates the Linux LF character into the native CR character for the end-of-line terminator.

4.1.1 Development Environments

The EDK II development environments include Windows*, Linux* and OS/X* development workstations. Development workstations must be running an IA32 or X64 operating system. Intel(R) Itanium Processor Family workstations are not supported.

The new build tools allow directories containing EDK II packages to be located anywhere on a developer's workstation. The recommended method for setting up a development structure on a Windows workstation is create a directory in the root of a drive:

```
mkdir C:\Work
cd C:\Work
set WORKSPACE=%CD%
```

The `edk2` directory can then be placed in this directory and additional directories for platforms and tools should be placed in the top level directory as well:

```
C:\Work\edk2
C:\Work\MyPlatform
C:\Work\edk2-BaseTools-win32
set PACKAGES_PATH=C:\Work\edk2;C:\Work\MyPlatform
set EDK_TOOLS_BIN=C:\Work\edk2-BaseTools-win32
```

In order to complete the setup:

```
C:\Work> mkdir Conf
C:\Work> edk2\edksetup.bat
```

After running this command, the configuration files, `target.txt`, `tools_def.txt` and `build_rule.txt` will be placed in the `C:\Work\Conf` directory.

The EDK II Build output directory is typically created in the `WORKSPACE` directory (based on configuration set in the DSC files). After executing a **build.exe** command, the `C:\Work\Build` directory will be created.

When using this feature, remember the system environment variables, `WORKSPACE`, `PACKAGES_PATH` and `EDK_TOOLS_BIN` must be set before running the `edksetup.bat` script.

4.1.2 Supported Development Tools

The list of validated Third-Party Compiler Tool chains that can be used with the EDK II build system is documented in the `tools_def.template` file in the EDK II source tree's `BaseTools/Conf` directory.

Install the tool chains for compilers and/or additional tools prior to building any image.

4.1.3 Build Process Restrictions

The build process for all development environments must be identical, with the caveat that only applicable EDK II Packages need compile for any given operating system.

Note: All EDK II content is built in the context of a Platform, using a Platform Description (DSC) file to describe what needs to be built, as well as any customization needed for a build. The DSC file is required even though the target may be only an application, a PCI Option ROM or a binary UEFI driver.

System Environment Variables will not be overridden by tools. System Environment Variable names cannot be overloaded - only the value of the System Environment Variable will be used.

There is no restriction on the location of the `EDK_TOOLS_PATH`, it may be located within a directory identified as the `WORKSPACE` directory, or in any other location that is accessible on the development workstation.

When using multi-directory feature, the system environment variables, `WORKSPACE`, `PACKAGES_PATH` and `EDK_TOOLS_BIN` must be set before running the `edksetup.bat` script.

4.2 Build Process Overview

Prior to executing a build command, specific system environment variables must be initialized: `WORKSPACE`, `EDK_TOOLS_PATH` are required for all builds. Additionally, the provided EDK II tool set must be present in a directory that is in the system environment variable: `PATH`. The `edksetup` scripts provided in the root directory of the EDK II development tree will set the `WORKSPACE` and `EDK_TOOLS_PATH`, as well as modify the system environment variable, `PATH` to ensure that the tools can execute. Refer to "Build Environment" for more information.

Command-line options to the build command will override values defined in meta-data files.

The EDK II Build Process is handled in three major stages:

- Pre-build or AutoGen stage: parse meta-data files, UCS-2LE encoded files and VFR files to generate some C source code files and the Makefiles.
- Build or \$(MAKE) stage: process source code files to create PE32/PE32+/COFF images that are processed to EFI format using `NMAKE` (Microsoft operating system development platforms) or `MAKE` (for UNIX style operating system development platforms).
- Post-build or ImageGen stage: takes the binary, EFI format files and creates EFI "FLASH" images, EFI update capsules, UEFI applications or PCI Option ROMs.

Figure 18 shows the relationship of these three stages.

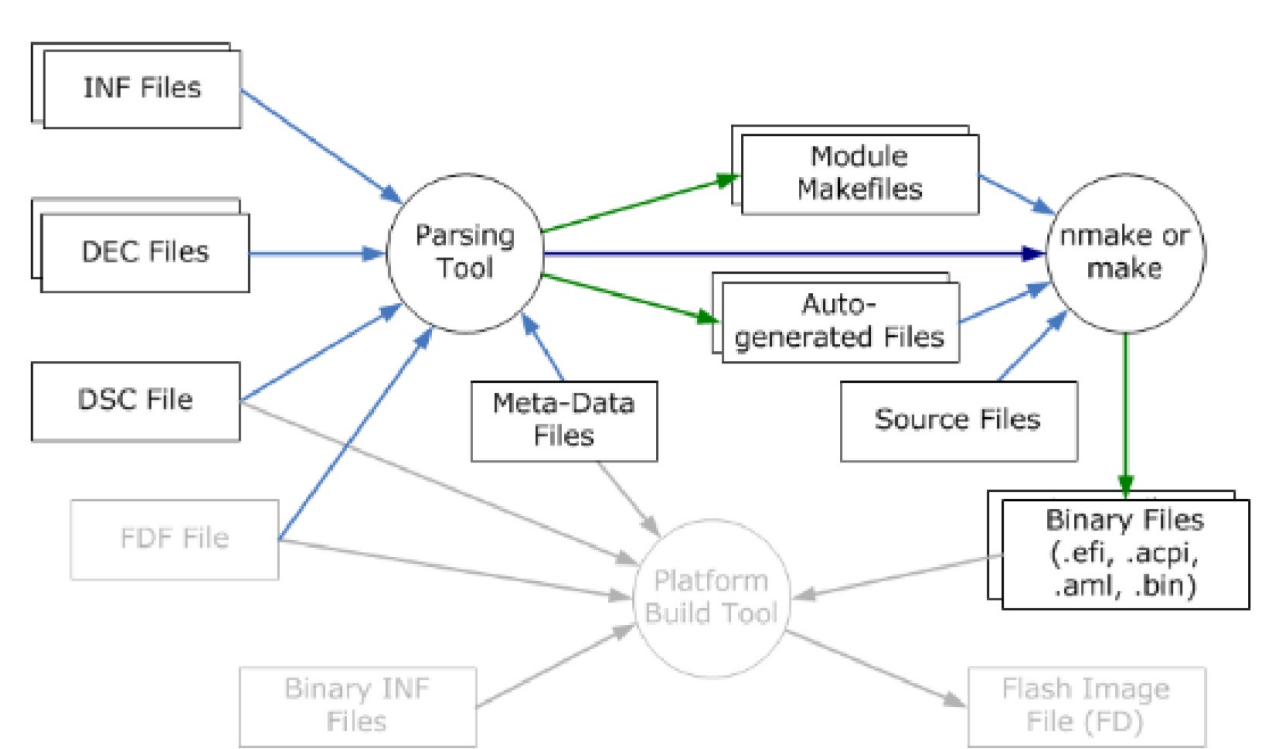


Figure 18 EDK II Platform Build Process Flow

Note: In Figure 18, Meta-Data Files indicates build tool meta-data files: `buildrule.txt`, `tools_def.txt`, `target.txt` and the like.

The Build process is organized so that if a FLASH image file is not required, such as in generating a Binary Module that will be distributed to other end-users, stage three can be skipped. Drivers, Option ROM and/or UEFI applications can also be distributed in this fashion.

Note: The `Nt32Pkg` (`Nt32Pkg/Nt32Pkg.dsc`) emulation platform requires Windows header files. In order to include Windows header files, execute the `edk2setup.bat` utility with the `--nt32` option. This option will detect the Microsoft Visual Studio installation and will execute it's setup command, for example, `vsvars32.bat` .

4.3 Pre-Build Stage Overview

This section provides an overview of the first three meta-data files that are used to control different aspects of the build. There are three files of interest, `build_rule.txt`, `tools_def.txt` and `target.txt`. The next chapter defines the format of the `tools_def.txt` and `target.txt` files. The `build_rule.txt` file is not documented here, as it is internal to the EDK II build system. See the `build_rule.txt` file for additional information.

4.3.1 target.txt

The `$(WORKSPACE)/Conf/target.txt` file is created the first time `edksetup` script is run. The default version of this file is the `$(EDK_TOOLS_PATH)/Conf/target.template` file.

The `target.txt` file is used as a filter, allowing a developer to build items of interest without having to build everything. The variables set in this file include the target platform, the target architecture, the tool chain and pointers to the tool chain configuration and build rule files. If no options are provided on the build command-line, values from this file are used to determine what to build, what tool chain will be used and where to obtain the rules for processing the files.

4.3.2 tools_def.txt

The `$(WORKSPACE)/Conf/tools_def.txt` file is created the first time the `edksetup` script is run. The default version of this file is the `$(EDK_TOOLS_PATH)/Conf/tools_def.template`.

The `tools_def.txt` describes sets of user configurable paths, commands and default flags for external tools (as well as optional tools provided with the build system). Since advanced developers may have multiple versions of tool chains installed, this file allows setting up paths and flags for different tool chains. Each tool chain is identified by a unique name.

4.3.2.1 "Best Known Safe" flags

The `tools_def.txt` file flags for the supported (tested) compiler tool chains that contain "Best Known Safe" flags for generating libraries, drivers and applications. These flags are set for speed optimization. The build system does provide for modifying flags for individual modules and platforms, so flags may be modified for better debug ability.

4.3.3 build_rule.txt

The `$(WORKSPACE)/Conf/build_rule.txt` file is created the first time the `edksetup` script is run. The default version of this file is the `$(EDK_TOOLS_PATH)/Conf/build_rule.template`.

The `build_rule.txt` file is used by the tools to define how different file types are processed. This includes how different files and module types are compiled, as well as how the build tools manipulate the binary image files. Normally, users should not make changes to this file.

4.3.4 Parse EDK II Meta-Data - AutoGen stage

Once the build system knows the basic information needed for the build, the build system parses the additional EDK II meta-data files. The meta-data in the EDK II code base is stored in text-based, INI-style, documents. Refer to the EDK II INF Specification, EDK II DSC Specification, EDK II DEC Specification, and EDK II Flash Description File Specification to see the format of these files.

The DSC file is the first of the EDK II meta-data files that gets parsed. This file provides a list of the other EDK II meta-data files that need to be parsed. The DSC file may be parsed twice in order to resolve PCD values that are used in conditional directive statements.

The contents of current working directory (at the time the build command is executed) may alter what gets built. If the working directory contains one INF file, only the module gets built. This is useful when debugging a driver, as only the one module will be rebuilt. If more than one INF file exists, you will need to use an command-line option to select which INF file you want to build. (The INF filename must be listed in the `ACTIVE_PLATFORM` DSC file's `[Components]` section.)

Note: More than one INF file may exist in a module directory, however the `BASENAME` and `FILE_GUID` for these INF files must be different if both modules will be included in a single FV for platform.

Note: The build system has also been modified to support building multiple versions of a single INF using the format defined in the DSC specification. This permits having multiple versions of a module linked against different libraries.

The second file to be parsed will be the FDF file if one is specified in the DSC file or on the command-line. While the FDF file specified what content gets assembled into the final firmware device image, PCD values from this file may be required for building specific modules specified in the DSC file.

Note: INF files that are listed in the DSC file must include the package, `MdePkg/MdePkg.dec` in order to build properly (even if the module does not contain C files).

The parse stage creates individual module and library `AutoGen.c`, `AutoGen.h` and Makefiles. Since EDK II supports Microsoft, Intel and GCC compiler tool chains, the Microsoft Build Tool, **NMAKE**/`Makefile` is for Windows developer Workstations using Microsoft or Intel tool chains on a Microsoft Windows operating system development workstation. For UNIX based development workstations, the GCC build tool, **MAKE**/`GNUmakefile` is used.

All third party tools and flags for those tools get expanded in the generated Makefiles. The following is an example of makefile statements that support this mode.

```
...
PP      = C:\Program Files\Microsoft Visual Studio .NET 2003\vc7\bin\cl.exe
CC      = C:\Program Files\Microsoft Visual Studio .NET 2003\vc7\bin\cl.exe
APP     = C:\Program Files\Microsoft Visual Studio .NET 2003\vc7\bin\cl.exe
VFRPP   = C:\Program Files\Microsoft Visual Studio .NET 2003\vc7\bin\cl.exe
DLINK   = C:\Program Files\Microsoft Visual Studio .NET 2003\vc7\bin\link.exe
PCH     = C:\Program Files\Microsoft Visual Studio .NET 2003\vc7\bin\cl.exe
ASM     = C:\Program Files\Microsoft Visual Studio .NET 2003\vc7\bin\ml.exe
TIANO   = TianoCompress.exe
SLINK   = C:\Program Files\Microsoft Visual Studio .NET 2003\vc7\bin\lib.exe
ASMLINK = C:\WINDDK\3790.1830\bin\bin16\link.exe ASL = C:\ASL\iasl.exe
...
DEFAULT_PP_FLAGS      = /nologo /E /TC /FI$(DEST_DIR_DEBUG)/AutoGen.h
DEFAULT_CC_FLAGS      = /nologo /W4 /WX /Gy /c /D UNICODE /O1b2 /GL /FI$(DEST_DIR_DEBUG)/AutoGen.h /EHs-c- /GF /Gs8192 /Zi /
Gm
DEFAULT_APP_FLAGS     = /nologo /E /TC
DEFAULT_VFRPP_FLAGS   = /nologo /E /TC /DVFRCCOMPIL /FIAutoGen.h
DEFAULT_DLINK_FLAGS    = /NOLOGO /NODEFAULTLIB /IGNORE:4086 /OPT:REF /OPT:ICF=10 /MAP /ALIGN:32 /MACHINE:I386 /LTCG /DLL /ENTRY
:$(ENTRYPOINT) /SUBSYSTEM:CONSOLE /SAFESEH:NO /BASE:0 /DRIVER /DEBUG /PDB:$(DEST_DIR_DEBUG)/$(BASE_NAME).pdb
DEFAULT_PCH_FLAGS     = /nologo /W4 /WX /Gy /c /D UNICODE /O1b2 /GL /FI$(DEST_DIR_DEBUG)/AutoGen.h /EHs-c- /GF /Gs8192 /Fp$(D
```

```

EST_DIR_OUTPUT)/AutoGen.h.gch /Yc /TC /Zi /Gm
DEFAULT_ASM_FLAGS      = /nologo /W3 /WX /c /coff /Cx /Zd /w0 /Zi
DEFAULT_TIANO_FLAGS    =
DEFAULT_SLINK_FLAGS     = /nologo /LTCG
DEFAULT_ASMLINK_FLAGS  = /link /nologo /tiny
DEFAULT_ASX_FLAGS      =
...
$(OUTPUT_DIR)\Ia32\WriteGdtr.obj : $(COMMON_DEPS)
$(OUTPUT_DIR)\Ia32\WriteGdtr.obj :
$(WORKSPACE)\MdePkg\Include\Library\DebugLib.h
$(OUTPUT_DIR)\Ia32\WriteGdtr.obj : $(WORKSPACE)\MdePkg\Include\Library\BaseLib.h
$(OUTPUT_DIR)\Ia32\WriteGdtr.obj :
$(WORKSPACE)\MdePkg\Include\Library\TimerLib.h
$(OUTPUT_DIR)\Ia32\WriteGdtr.obj :
$(WORKSPACE)\MdePkg\Include\Library\BaseMemoryLib.h
$(OUTPUT_DIR)\Ia32\WriteGdtr.obj : $(WORKSPACE)\MdePkg\Include\Library\PcdLib.h
$(OUTPUT_DIR)\Ia32\WriteGdtr.obj :
$(WORKSPACE)\MdePkg\Library\BaseLib\Ia32\WriteGdtr.c
$(OUTPUT_DIR)\Ia32\WriteGdtr.obj :
$(WORKSPACE)\MdePkg\Library\BaseLib\BaseLibInternals.h
"$(CC)" /Fo$(OUTPUT_DIR)\Ia32\WriteGdtr.obj $(CC_FLAGS) $(INC) $(WORKSPACE)\MdePkg\Library\BaseLib\Ia32\WriteGdtr.c

$(OUTPUT_DIR)\Ia32\WriteDr3.obj : $(COMMON_DEPS)
$(OUTPUT_DIR)\Ia32\WriteDr3.obj :
$(WORKSPACE)\MdePkg\Library\BaseLib\Ia32\WriteDr3.c
"$(CC)" /Fo$(OUTPUT_DIR)\Ia32\WriteDr3.obj $(CC_FLAGS) $(INC)
$(WORKSPACE)\MdePkg\Library\BaseLib\Ia32\WriteDr3.c
...

```

One makefile is generated for each module under a combination of `$(TARGET)_$(TOOL_CHAIN_TAG)` and `$(ARCH)`, package name, module directory, directory name and the `BASE_NAME` of the module's INF file.

Parse DSC file

1. Obtain platform FixedAtBuild and FeatureFlag PCD values and Macro values that are used in the Conditional Directives - if the value of a Macro, FixedAtBuild PCD or FeatureFlag PCD used in the conditional directives cannot be determined, the build will break with an appropriate error message. The use of FixedAtBuild or FeatureFlag PCD names which are defined by SET statements in the FDF file cannot be used in conditional directive statements in the DSC file.
2. Second pass over the DSC files will crush out any conditional directives where the feature flag expression used in the conditional directive is False.
3. Obtain platform PCD values which will go into the individual module AutoGen.h files where needed.
 - If `VPD_TOOL_GUID` was specified in the DSC file's `[Defines]` section, the build processes is suspended while the tool specified by the GUID is called after the build process generates a text file containing the VPD PCDs. If the tool returns successfully (an exit code of 0), the build system parses the name of the 'map' file that contains an ordered list of VPD PCDs.
 - There are some PCD values that get set on the command line, in the FDF file, listed in binary INF files or listed in source INF files, so generating the C files is delayed until all PCD values have been finalized.
4. Obtain the FDF filename and obtain the Flash related PCDs from the FDF file. FeatureFlag and FixedAtBuild PCD names which are defined in the DSC file can be used in conditional directives within the FDF file.
5. For each component listed in the DSC file, parse the Module's INF file
 - Create a directed graph list of the EDK II Library Instances that will be used for the EDK II modules.
 - Create the PEI, DXE or SMM DEPEX file

- Create the Library Instance's AutoGen.c files containing PCD, Guid, Protocol, Ppi and EntryPoint definitions and data structures. PCD values come from command line, FDF file, DSC's INF scoped section, DSC's global PCD sections, default values in the INF file's PCD section, or the DEC file's default values.
- Create the module's library instance Makefiles
 - Individual modules may require different compilation options, over-riding any global definitions.
- Create the AutoGen.c files containing PCD, Guid, Protocol, Ppi and EntryPoint definitions and data structures.
- Create any Strings header file required for VFR processing.
 - The VFR file name cannot be same as a C file name in a module directory. If so, the same output files will be generated and overwritten. (A.vfr --> A.c --> A.obj, A.c --> A.obj)
- Create the module Makefiles

Individual modules may require different compilation options, over-riding any global definitions. If an INF file is not listed in the DSC file and is listed in the FDF file, the parsing tools must check if the INF in the FDF file contains `PatchableInModule` or `DynamicEX` entries. If the INF lists other PCD access methods (`FeatureFlag`, `FixedAtBuild` or `Dynamic`), and the INF contains files listed in a `[Sources]` section and does not contain a `[Binaries]` section, then the build tools must break the build with an appropriate error message.

6. The tools are also responsible for creating binary files containing all `DynamicEx` PCDs that are listed in the DSC, FDF and Binary INF files (listed in the FDF file). These binaries are automatically placed into the (PEIM and DXE) PCD driver FFS files.
7. If the build option, `--ignore-sources` is present on the build command-line, none of the source files listed in a `[Sources]` section will be processed, even if the module is listed in the DSC file and no files (`AutoGen.h`, `AutoGen.c` or `Makefile`) will be generated; the INF must be treated as a Binary only INF file.

4.4 Creating Binary EFI Images - \$(MAKE) stage

Binary EFI images are created in two steps; the first step uses 3rd party assemblers, compilers and linkers to generate a PE32/PE32+/COFF image file, while the second step uses code from the GenFw application provided in EDK II to modify the PE32/PE32+/COFF image file to create an EFI file with an `EFI_IMAGE_SECTION_HEADER` structure. Since different `EFI_SECTION` types may require different values for alignment offsets, the GenFw tool must specify the component type, which is derived from the INF metadata's `ModuleType` statement.

This stage is executed by the build tool calling either the `NMAKE` or the `MAKE` tool for each module. The Makefiles (`Makefile` or `GNUMakefile`) are created during the metadata processing stage. The Makefiles specify the compiler, linker, assembler, and `GenFw` tool commands and options. Once all modules have been built by the 3rd party tools, the build tool will call the `GenFds` application to initiate the third stage of a build, if there is a Flash Definition File (FDF) specified in the DSC file. If no FDF file is specified, then the build will terminate with the creation of individual module EFI formatted (EFI) images.

4.5 Post-Build Stage

4.5.1 Assemble FLASH Images - ImageGen stage

Once all the modules' EFI image files have been created, the final stage of a build process is called. For FLASH images, this stage uses the FDF file, and some parts of the DSC file to create the final binary image files. This stage processes the individual EFI files, formatting them into leaf `EFI_SECTION` types and combining them using implied rules (or custom rules) into different firmware files (FFS), firmware volumes (FVs) and the final FLASH images (FDs). The construction of these images is based on the content of the FDF file (with a very limited amount of data being obtained from the DSC file).

A binary file with a file type of `DISPOSABLE` will not be placed into a `EFI_SECTION_DISPOSABLE` encapsulation section. This keyword is used by UEFI Packaging Tool to ensure that files, such as debug symbol files, get packaged correctly.

The default build rules specify removal of `.reloc` sections of the PE32/PE32+ file for all `SEC`, `PEI_CORE` and `PEIM` modules and components. To prevent removal of the `.reloc` section, a module developer will need to specify a keyword, `SHADOW` in the INF file.

Assuming that all FD and FV images are going to be generated (based on the default value of the top-level build command), for each FV image specified the following must occur.

1. The FDF file is parsed to create a directed graph structure for each FV image, so that all leaf EFI sections are created first. During this stage, INF files that contain a `[Binaries]` section and that do not contain a `[Sources]` section will be processed. An INF file that contains a `[Binaries]` section that contains an entry that starts with `DISPOSABLE`, that entry must be ignored - these files are not to be placed into an `EFI_SECTION_DISPOSABLE` encapsulation section.
2. If an INF not listed in the DSC file, but is listed in the FDF file and the INF contains a `[PatchPcd]` section, the tools must test to determine if the PCD is listed in the DSC (or FDF) file, and whether the value listed in the DSC (or FDF) file is different from the value in the INF file. If the value is different, the tools must patch the binary `.efi` file with the value from the FDF or DSC file prior to creating the EFI leaf section.
3. The tools are also responsible for creating binary files containing all DynamicEx PCDs that are listed in the DSC, FDF and Binary INF files (listed in the FDF file). These binaries are automatically placed into the (PEIM and DXE) PCD driver FFS files.
4. These leaf sections are either put into encapsulated sections or put directly into an FFS file following the implied rules (or user defined rules) defined later in this document.
5. As each FFS File is created, the file is either encapsulated into another FFS file or appended to an FV image.
6. Once all of the FFS files have been placed into an FV image file, the FV file is put into an FD file at the location specified by the FD section of the FDF file.

4.5.2 EFI PCI Expansion Option ROM Images

There are two methods for creating an option ROM image, when the FDF file is specified and when an FDF file is not present. To build from source and no FDF file is present, if the module's INF file contains the keywords, `PCI_DEVICE_ID`, `PCI_VENDOR_ID` and `PCI_CLASS_CODE`, the build will terminate after creating EFI files - there will be no call to the GenFds tool. These key words also force the creation of an option ROM image, after the EFI files have been created, using the EfiRom program to create the EFI PCI Expansion ROM image. If an FDF is present, then the build tools will parse the FDF file looking for an `[OptionRom]`

section, and create the option ROM based on the contents of this section. Note that the FDF specification permits adding binary images, such as the legacy option rom binary, as well as support for multiple architecture driver images to the option ROM image.

A binary flag, `PCI_COMPRESS`, when set to true, tells the tools to use EFI standard compression to compress the entire option ROM image.

Option ROM images are always created in the output FV directory.

4.5.3 UEFI Applications

When building only UEFI applications, no FDF file is specified in the DSC file; the build would normal terminate after creating EFI files - there would be no call to the GenFds tool. Using an option on the build tool command line to specify building a UEFI Application forces the parsing stage to generate the module's `Makefile` with an alternate path. This path will force the creation of a UEFI application, after the EFI files have been created, using UEFI application specific arguments for the `GenFds` tool.

4.6 File Specifications

The EDK II Build is used to generate UEFI and PI compliant images. Additional reference modules may conform to Intel Framework Specifications only if there are no applicable UEFI or PI specification modules.

The EDK II Build Tools will only generate UEFI/PI compliant images.

This build specification does not cover the tools or build processes for EDK builds nor tools provide by the EDK II Compatibility Package.

The binary image files generated at the end of the \$(MAKE) stage conform to the UEFI Images section of the UEFI specification. UEFI uses a subset of the PE32+ image format with a modified header signature. The PE32/PE32+ files are modified by the GenFw application.

Note: This application will also modify an ELF image and generate a PE32/PE32+ image.

Each PE32/PE32+ file will have sections of the original "DOS Header" over-written, a new NT_HEADER (for the PeHeader) and possibly one Optional Header for 32-bit or 64-bit options.

4.7 File Extensions

The EDK II build system is designed to process files in the AutoGen stage with specific extensions for use in the \$(MAKE) stage, producing files with intermediate extension names. For some "final" targets, such as UEFI applications, the intermediate extension is the "final" extension. The ImageGen makes use of the files with intermediate extensions to generate the final images.

5 META-DATA FILE SPECIFICATIONS

This chapter defines the format of two files used by the build. The two files are: `tools_def.txt`, which defines the location and options for third party tools and `target.txt`, which defines the top level default configuration. A third file, `build_rule.txt`, which specifies the rules for creating binary files, will not normally be modified by users, however since this file is closely coupled with the build system, certain changes to build tools will require updating (overwriting) the active copy. The format for `build_rule.txt` is not included in this document.

Templates for these files are in the `$(EDK_TOOLS_PATH)/Conf` directory. The `edksetup` script installs the active copies of these files into the `$(WORKSPACE)/Conf` directory only if they do not exist. It is permissible to have the `Conf` directory (the directory containing `target.txt`) located outside of the `WORKSPACE` directory, however either the absolute or `WORKSPACE` relative directory must be specified on the build command-line using the "--conf" option when they are not in the active `WORKSPACE/Conf` directory.

5.1 Build Meta-Data File Formats

The following subsections describe the different parts of the build system's meta-data files. These files are specific to the build process. Other EDK II meta-data file formats are specified in their corresponding documents (see Related Information in the Introduction.)

5.1.1 Comments

Within a meta-data file, comments are encouraged, with the hash "#" character identifying a comment. In line comments terminate the processing of a line. In line comments must be placed at the end of the line, and may not be placed within the section ("[" , "]" , "<" or ">") tags. Comment characters can be at the start of a line, or after a data element (there must be one or more white space characters between the data element and the comment character. Examples:

```
# this is a comment line
[Unicode-Text-File] # This is also a valid comment.

[Unicode-Text-File # This is not valid]
```

The last example is not valid, as the section header data element format is `[text]` with the square brackets included as part of the data element.

Hash characters within a quoted string are permitted, and do not signify a comment.

5.1.2 Valid Entries

All entries must appear on a single line, with entries terminated by either a new line, or a comment.

5.2 tools_def.txt

This file describes the tools used by a developer, providing the flexibility to have multiple tool chains and different profiles for each tool chain. In the simplest of terms, the file provides a variable mapping of compiler tool chains and flags. The structure of this text file is described below.

There are three types of statements, the `IDENTIFIER` statement which defines a "User Interface" name for identifying this file. The second statement type is the `DEFINE` statement which is used to identify a fully qualified path macro, while the third type of statement is a record statement containing mappings that are processed by the build tools to generate `Makefile` and `GNUMakefile` commands that are executed by a compiler's "make" utility or function.

The left side of the record is subdivided into five groups, defined below. The build tools will process the file and assign the following priority during the parsing. After parsing the right hand `<string>` is substituted into the makefile using the `build_rule.txt` templates.

If a wildcard value is permitted, the wildcard character is the star "*" character.

For tool chains that expect to use a Windows-style nmake utility one entry, the **NMAKE** `COMMANDTYPE` is required. The NIX-based make and MAKE utilities are typically in a developer's path environment (`/usr/bin`). Specifying a **MAKE** command that will use an alternate make utility for NIX-based tool chains is optional.

```
format: TARGET_TOOLCHAIN_ARCH_COMMANDTYPE_ATTRIBUTE = <string>
priority:
    TARGET_TOOLCHAIN_ARCH_COMMANDTYPE_ATTRIBUTE (Highest)
    *****_TOOLCHAIN_ARCH_COMMANDTYPE_ATTRIBUTE
    TARGET_*****_ARCH_COMMANDTYPE_ATTRIBUTE
    *****_*****_ARCH_COMMANDTYPE_ATTRIBUTE
    TARGET_TOOLCHAIN_****_COMMANDTYPE_ATTRIBUTE
    *****_TOOLCHAIN_****_COMMANDTYPE_ATTRIBUTE
    TARGET_*****_****_COMMANDTYPE_ATTRIBUTE
    *****_*****_****_COMMANDTYPE_ATTRIBUTE
    TARGET_TOOLCHAIN_ARCH_*****_ATTRIBUTE
    *****_TOOLCHAIN_ARCH_*****_ATTRIBUTE
    TARGET_*****_ARCH_*****_ATTRIBUTE
    *****_*****_ARCH_*****_ATTRIBUTE
    TARGET_TOOLCHAIN_****_*****_ATTRIBUTE
    *****_TOOLCHAIN_****_*****_ATTRIBUTE
    TARGET_*****_****_*****_ATTRIBUTE
    *****_*****_****_*****_ATTRIBUTE (Lowest)
```

All entries in this file are case-sensitive.

5.2.1 Macros and Other Variable Statements (tools_def.txt only)

The use of MACRO statements is limited in EDK II `tools_def.txt` meta-data file to be local to the meta-data file. The format and usage for the macro statements is:

```
DEFINE MACRO = Value
DEF(MACRO)/filename.foo
```

Any defined MACRO will be expanded by tools when they encounter the entry in the section.

The macro statements are positional, in that only statements following a macro definition are permitted - a macro cannot be used before it has been defined.

MACRO statements are permitted in DSC and FDF files to reference PATH statements, assign values to PCDs and to provide a minimum level of directive statements - refer to the corresponding specification for additional details.

System environment variables may be used in value portion of statements. The system environment value is specified using the following format:

```
ENV(OsEnvironmentVariableName)
```

The following variables, `$(MODULE_NAME)`, `$(IMAGE_ENTRY_POINT)`, `$(MODULE_ENTRY_POINT)`, `$(ARCH_ENTRY_POINT)`, `$(DEBUG_DIR)`, `$(BASE_NAME)`, `$(DEST_DIR_DEBUG)`, `$(EDK_TOOLS_PATH)`, `$(ARCHASM_FLAGS)`, `$(PLATFORM_FLAGS)`, `$(ARCHCC_FLAGS)`, `$(ARCHDLINK_FLAGS)`, `$(DLINKPATH_FLAG)`, `$(ASMPATH_FLAG)`, `$(CCPATH_FLAG)` and `$(SLINKPATH_FLAG)` are never expanded when data is emitted to Makefiles.

These variables are used in values for statements having the `FLAG` attribute or in macros that are used in the value fields of entries with the `FLAG` attribute.

5.2.2 Guided Tools

There are four GUIDed tools that are provided by the EDK II build system.

- **CRC32** - FC1BCDB0-7D31-49AA-936A-A4600D9DD083
 - This tool provides CRC32 (Cyclic Redundancy Check) methods for error detection using the `GenCrc32` tool.
- **TIANO** - A31280AD-481E-41B6-95E8-127F4C984779
 - This tool provides Tiano Compression using the `TianoCompress` application.
- **LZMA** - EE4E5898-3914-4259-9D6E-DC7BD79403CF
 - This tool provides LZMA Compression using the `LzmaCompress` application.
- **VPD TOOL** - 8C3D856A-9BE6-468E-850A-24F7A8D38E08
 - This tool provides VPD binary data and map file generation using the `BPDG` application.
- **LZMAF86** - D42AE6BD-1352-4bfb-909A-CA72A6EAE889
 - `LzmaF86Compress` tool definitions with converter for x86 code. It can improve the compression ratio if the input file is IA32 or X64 PE image. Note: If X64 PE image is built based on GCC44, it may not get the better compression.
- **RSA2048SHA256SIGN** - A7717414-C616-4977-9420-844712A735BF
 - This tool definition uses a test signing key for development purposes only. The tool `Rsa2048Sha256GenerateKeys` can be used to generate a new private/public key and the `gEfiSecurityPkgTokenSpaceGuid.PcdRsa2048Sha256PublicKeyBuffer` PCD value. A custom tool/script can be implemented using the new private/public key with the `Rsa2048Sha256Sign` tool and this tool definition can be updated to use a custom tool/script.
- **BROTLI** - 3D532050-5CDA-4FD0-879E-0F7F630D5AFB
 - This tool provides Brotli Compression using the `BrotliCompress` application.
- **PKCS7SIGN** - 4AAFD29D-68DF-49EE-8AA9-347D375665A7
 - This tool provide PKCS7 signing using the `Pkcs7Sign` application. This tool definition uses a test signing key for development purposes only. New keys can be generated and be used to set the `gEfiSecurityPkgTokenSpaceGuid.PcdPkcs7CertBuffer` PCD value. A custom tool/script can be implemented using the new keys with the `Pkcs7Sign` tool and this tool definition can be updated to use a custom tool/script.

Additional GUIDed tools may be added. If the GUID value is used in the FDF file's GUIDed Encapsulation, the tool, named by the GUID, will be called using a `-e` option to encode the content.

5.2.3 tools_def.txt EBNF Definition

Summary

EDK II tools will not expand `<MacroVal>` statements that appear within quotation marks; the expectation is that external tools or the operating system will expand them during execution.

When specifying Macros for paths for Windows tools, paths that contain space characters do not need to be quoted. When specifying a path in a `FLAGS` section, any path that contains a space character will need to be enclosed with double quotation marks.

After the `IDENTIFIER = UiString` entry and Macro definition statements, all other entries consist of `Token = Value` pairings. The Token is actually a token that is constructed of five fields which are separated by an underscore character.

Comments are only allowed on separate lines and may not be appended appear on actual entry lines.

The following EBNF defines the valid entries in the `tools_def.txt` file.

Prototype

```

<ToolsDef>          ::= "IDENTIFIER" <Eq> <UiString> <EOL>
                    <DefineStatements>*
                    <ToolChainEntries>* <GuidedEntries>*

<TS>                ::= <TabSpace>*
<MTS>               ::= <TabSpace>+
<Tab>               ::= 0x09
<Space>             ::= 0x20
<TabSpace>          ::= {<Tab>} {<Space>}
<UiString>          ::= (a-zA-Z0-9)<Chars>* <EOL>
<Chars>             ::= (0x20-0x7E)
<PathChars>         ::= {0x20} {0x28} {0x29} {(0-9a-zA-Z_-)} {0x2E}
<Eq>                ::= <TS> "=" <TS>
<AsciiChars>        ::= (0x21 - 0x7E)
<AsciiString>       ::= [ <TS>* <AsciiChars>* ]*
<FlagString>        ::= <AsciiString>
<DefineStatements>  ::= <TS> "DEFINE" <MTS> <MACRO> <Eq> <Value> <EOL>
<MACRO>             ::= (A-Z)(a-zA-Z0-9_)*
<Value>             ::= {<Path>} {<FlagString>} {<Numbers>}
<Path>              ::= {<DosPath>} {<NixPath>} {<EnvPath>} {<MacroPath>}
<DosPath>           ::= {<AbsPath>} {<RelPath>}
<AbsPath>           ::= <A-Za-z> ":" ["\" <PathChars>+]*
<RelPath>           ::= ["\" <PathChars>+]*
<NixPath>           ::= ["/" <PathChars>+]*
<Numbers>           ::= (0-9)+ [ "." (0-9)* ]*
<MacroVal>          ::= "DEF(" <MACRO> ")"
<MacroPath>         ::= <MacroVal> {<NixPath>} {<RelPath>}
<EnvPath>           ::= "ENV(" <SysEnvVar> ")"
                    [[["\" {"/"}] <PathChars>*]+
<SysEnvVar>         ::= (A-Z)(A-Z0-9_)* # System Environment Variable
<ToolChainEntries>  ::= <RequiredEntry>
                    [<TS> <OptionalEntry>]*
<Wildcard>          ::= "*"
<RequiredEntry>     ::= <TS> <MakeEntry>
                    <TS> <FamilyEntry>
<MakeEntry>         ::= <Field1> "_" <Tagname> "_" <Arch> "_" <MakePath> <EOL>
<FamilyEntry>       ::= <Wildcard> "_" <Tagname> "_" <Arch> <FamilyType>
<FamilyType>        ::= "_" <Wildcard> "_" <Family>
<Family>            ::= "FAMILY" <Eq> <SupFamily> <EOL>
<SupFamily>         ::= {"ARMGCC"} {"MSFT"} {"INTEL"} {"GCC"} {"RVCT"}
                    {"RVCTCYGWIN"} {"XCODE"} {<NewFamily>}
<NewFamily>         ::= (A-Z) (A-Z0-9)+
<MakePath>          ::= "MAKE_PATH" <Eq> <EXECPATH> <Command> <EOL>
<OptionalEntry>     ::= <Field1> "_" <Field2> "_" <Field3>
<Field1>            ::= {<Target>} {<Wildcard>}
<Target>            ::= {<PreDefinedTargets>} {(A-Z) (A-Za-z0-9)*}
<PreDefinedTargets> ::= {"DEBUG"} {"RELEASE"} {"NOOPT"}

```

```

<Field2> ::= {<TagName>} {<Wildcard>}
<TagName> ::= {<PreDefinedTags>} {(A-Z) (A-Za-z0-9)*}
<PreDefinedTags> ::= {"ARMGCC"} {"ARMLINUXGCC"} {"CYGGCC"}
{"CYGGCCxASL"} {"DDK3790"} {"DDK3790xASL"}
{"ELFGCC"} {"GCC44"} {"GCC45"} {"GCC46"}
{"GCC47"} {"GCC48"} {"GCC49"} {"ICC"}
{"ICC11"} {"ICC11x86"} {"ICC11x86xASL"}
{"ICC11xASL"} {"ICCx86"} {"ICCx86ASL"}
{"ICCx86xASL"} {"ICCxASL"} {"MYTOOLS"}
{"RVCT"} {"RVCTCYGWIN"} {"RVCTLINUX"}
{"UNIXGCC"} {"VS2003"} {"VS2003xASL"}
{"VS2005"} {"VS2005x86"} {"VS2005x86xASL"}
{"VS2005xASL"} {"VS2008"} {"VS2008x86"}
{"VS2008x86xASL"} {"VS2008xASL"} {"VS2010"}
{"VS2010x86"} {"VS2010x86xASL"}
{"VS2010xASL"} {"VS2012"} {"VS2012x86"}
{"VS2012x86xASL"} {"VS2012xASL"} {"VS2013"}
{"VS2013x86"} {"VS2013x86xASL"}
{"VS2013xASL"} {"XCLANG"} {"XCODE32"} {"XCODE5"}

<Field3> ::= <Arch> "_" <Field4> "_" <Attributes>
<Arch> ::= {"IA32"} {"X64"} {"IPF"} {"EBC"} {"ARM"} {<wildcard>}
{(A-Z) (A-Z0-9)*}
<Field4> ::= {<CommandCode>} {"*"}
<CommandCode> ::= {"APP"} {"ASL"} {"ASLCC"} {"ASLDLINK"}
{"ASLPP"} {"ASM"} {"ASM16"} {"ASMLINK"}
{"ASMPATH"} {"CC"} {"CCPATH"} {"CRC32"}
{"DLINK"} {"DLINKPATH"} {"DSYMTIL"} {"FLAGS"}
{"FROMELF"} {"FROMELFPATH"} {"GENFW"} {"LZMA"}
{"LZMAF86"} {"MAKE"} {"MTOC"} {"NASM"}
{"OBJCOPY"} {"OPTROM"} {"PP"} {"PPPATH"} {"RC"}
{"RSA2048SHA256SIGN"} {"SLINK"} {"SLINKPATH"}
{"SYMRENAME"} {"TIANO"} {"VFR"} {"VFRPP"}
{"VFRPPPATH"} {"VPDPTOOL"}

<Attributes> ::= {<ExecAttrs>} {<FlagAttr>} {<MiscAttrs>}
<ExecAttrs> ::= "PATH" "=" <EXECPATH> <Command> <EOL>
<MiscAttrs> ::= [<DllPath>] {<UserDefined>} {<RuleOrder>}
<DllPath> ::= {"DLL"} {"DPATH"} <Eq> <EXECPATH> <EOL>
<EXECPATH> ::= {<Definition>} {<Environ>} {<AbsolutePath>}
<Command> ::= <Word> [" " <Ext>]
<Definition> ::= "DEF(" <MACRO> ")" <Sep> [<Path>]*
<RuleOrder> ::= "BUILDRULEORDER" <Eq> <ExtensionList> <EOL>
<ExtensionList> ::= <Word> [<SP> <Word>]*
<GuidedEntries> ::= <TS> <GuidDef>
<GuidPath>
[<GuidFlags>]
<GuidAttrs>*

<GuidedEntry> ::= <Field1> "_" <Field2> "_" <Arch> "_" <Code>
<GuidDef> ::= <GuidedEntry> "_" <Guid>
<Code> ::= {"VPDPTOOL"} {"LZMA"} {"TIANO"} {"CRC32"}
{"LZMAF86"} {"RSA2048SHA256SIGN"} {<NewTool>}

<NewTool> ::= (A-Z)*
<Guid> ::= "GUID" <Eq> <RegistryFormatGUID> <EOL>
<RegistryFormatGUID> ::= <RHex8> "-" <RHex4> "-" <RHex4> "-" <RHex4> "-"
<RHex12>
<RHex4> ::= <HexDigit> <HexDigit> <HexDigit> <HexDigit>
<RHex8> ::= <RHex4> <RHex4>
<RHex12> ::= <RHex4> <RHex4> <RHex4>
<RawH2> ::= <HexDigit>? <HexDigit>
<GuidPath> ::= <GuidedEntry> "_PATH" <Eq> [<EXECPATH>]
<Command> <EOL>
<GuidFlags> ::= <GuidedEntry> "_" <FlagAttr>
<GuidAttrs> ::= <GuidedEntry> "_" <UserDefined>
<UserDefined> ::= <Word> "=" <UserDefinedValues> <EOL>
<FlagAttr> ::= "FLAG" "=" <FlagValues> <EOL>
<EOL> ::= <TS> 0x0D 0x0A

```

Parameters

No space characters are permitted on the left side of the expression (before the equal sign). All of the keywords that make up the left side of the expression must be alphanumeric only - no special characters are permitted.

FlagValues

This is a string of zero or more tool specific flags. All flags must be printable characters. The flag string starts with the character following the first "=" sign in the line and terminates with the end of line.

Paths

The paths specified in the `tools_def.txt` file must be valid path names for the workstation OS that will be using the tool chain identified by the tag name. Since this file can contain numerous tool chains for multiple operating systems, only the tool chain name specified in `target.txt` or on the command-line needs to be valid paths.

Target

A keyword that uniquely identifies the build target; the first field, where fields are separated by the underscore character. Three values, `NOOPT`, `DEBUG` and `RELEASE` have been pre-defined. This keyword is used to bind command flags to individual commands.

Users may want to add other definitions, such as, `PERF`, `SIZE` or `SPEED`, and define their own set of `FLAGS` to use with these tags. The wildcard character, "*", is permitted after it has been defined one time for a tool chain.

TagName

A keyword that uniquely identifies a tool chain group; the second field. Wildcard characters are permitted only if a command is common to all tools that will be used by a developer. As an example, if the development team only uses IA32 Windows workstations, the ACPI compiler can be specified as

```
DEBUG_*_*_ASL_PATH and RELEASE_*_*_ASL_PATH .
```

Arch

A keyword that uniquely identifies the tool chain target architecture; the third field. This flag is used to support the cross-compiler features, such as when a development platform is IA32 and the target platform is X64 Using this field, a single tag name can be setup to support building multiple target platform architectures with different tool chains.

For example, if a developer is using Visual Studio .NET 2003 for generating IA32 platform and uses the WINDDK version 3790.1830 for X64 or IPF platform images, a single tag. See the `MYTOOLS PATH` settings in the generated `Conf/tools_def.txt` or provided `BaseTools/Conf/tools_def.template` file.

The wildcard character, "*", is permitted only if the same tool is used for all target architectures.

CommandCode

A keyword that uniquely identifies a specific command; the fourth field. Several CommandCode keywords have been predefined, however users may add additional keywords, with appropriate modifications to `build_rule.txt`. See Table 7 below for the pre-defined keywords and functional mappings. The wildcard character, "", is permitted only for the `FAMILY`, `DLL` and `DPATH` attributes (see *Attributes below).

Table 7 Predefined Command Codes

CommandCode	Function
APP	C compiler for applications.
ARCHASM ¹	Flags for a macro assembler that is specific to an architecture
ARCHCC ¹	Flags for a C compiler that is specific to an architecture
¹	

ARCHDLINK ¹	Flags for a dynamic linker that is specific to an architecture
ASL	ACPI Compiler for generating ACPI tables.
ASLCC	A C compiler for ACPI code prior to running the ASL compiler
ASLDLINK	A dynamic linker for the ACPI code
ASLPP	A C Pre-processor for the ACPI code
ASM	A Macro Assembler for assembly code in some libraries.
ASM16	A 16-bit assembler for SEC assembly code in some libraries
ASMLINK	The Linker to use for assembly code generated by the ASM tool.
ASMPATH ¹	This command code is specific to the RVCT31CYGWIN tool chain tag.
CC	C compiler for PE32/PE32+/Coff images.
CCPATH ¹	This command code is specific to the RVCT31CYGWIN tool chain tag.
CRC32	This tool provides CRC32 (Cyclic Redundancy Check) methods for error detection using the GenCrc32 tool.
DLINK	The C dynamic linker.
DLINKPATH ²	This command code is specific to the RVCT31CYGWIN tool chain tag.
DSYMTIL	This command code is specific to the XCODE32 and XCLANG tool chain tags.
FROMELF ¹	This command code is specific to the RVCT31CYGWIN tool chain tag.
FROMELFPATH ¹	This command code is specific to the RVCT31CYGWIN tool chain tag.
GENFW	This command is for the EDK II build system GenFw utility, and allows user customization of the tool's flags.
LZMA	This tool provides LZMA Compression using the LzmaCompress application.
LZMAF86	LzmaF86Compress tool definitions with converter for x86 code.
MAKE	Required for tool chains. This identifies the utility used to process the Makefiles generated by the first phase of the build.
MTOC	This command code is specific to the XCODE32 and XCLANG tool chain tags.
OBJCOPY	This system command is specific to GCC tool chains, it is used to covert ELF images to PE32+ images.
OPTR0M	This command is for the EDK II build system EfiRom utility, and allows user customization of the tool's flags.
PLATFORM ¹	This command is for ARM based tool chains
PP	The C pre-processor command.
PPPATH ¹	This command code is specific to the RVCT31CYGWIN tool chain tag.
RC	This is the command code for resource compilers.
RSA2048SHA256SIGN	This tool definition uses a test signing key for development purposes only.
SLINK	The C static linker.
SLINKPATH ¹	This command code is specific to the RVCT31CYGWIN tool chain tag.
SYMRENAME	This command code is by some of the GCC family tool chains.
TIANO	This tool provides Tiano Compression using the TianoCompress application.
VFR	This command is for the EDK II build system Visual Forms Representation tool, VfrCompile

VFRPP	The C pre-processor used to process VFR files.
VFRPPPATH ¹	This command code is specific to the RVCT31CYGWIN tool chain tag.
VPDTool	This tool provides VPD binary data and map file generation using the BPDG.

¹ These command codes are only used for `FLAG` attribute statements and are not related to actual executable applications.

² This is the path to standard Microsoft libraries (`.dll`).

Attribute

A keyword to uniquely identify a property of the command; the fifth and last field. Several pre-defined attributes have been defined: `DLL`, `FAMILY`, `FLAGS`, `GUID`, `OUTPUT` and `PATH`. Use quotation marks only if the quotation marks must be included in the flag string. The following example shows the format for the required quoted string, `"C:\Program Files\Intel\EBC\Lib\EbcLib.lib"`. Normally, the quotation characters are not required as everything following the equal sign to the end of the line is used for the flag.

```
*_*_EBC_DLINK_FLAGS = "C:\Program Files\Intel\EBC\Lib\EbcLib.lib" /NOLOGO
```

Table 8 Predefined Attributes

Attribute	Description
ADDDEBUGFLAG	This flag is used by objcopy to set the option: <code>--add-gnu-debuglink</code>
BUILDRULEFAMILY	This flag is used by some tool chain tags to set a special <code>FAMILY</code> value when processing the <code>build_rule.txt</code> file. Normally, the <code>FAMILY</code> attribute is used to identify the type of makefile the tools need to generate. Tools such as <code>XCODE</code> will use <code>GCC</code> as the <code>FAMILY</code> , but uses different (from <code>GCC</code>) processing rules. If present and if a build rule (in <code>build_rules.txt</code>) contains an attribute with the value specified in this entry, that rule will be processed and the rule with the <code>FAMILY</code> attribute will be ignored.
DLL	The path to the 3rd party tool's required DLLs - required for some tools to generate debug files.
FAMILY	A flag to the build command that will be used to ensure the correct commands and flags are used in the generated Makefile or GNUMakefile, as well as to use the correct options for independent tools, such as the ACPI compiler. This is typically used to identify the type of Makefile that needs to be generated.
FLAG or FLAGS	The arguments for individual CommandCode tools.
GUID	This defines the Registry Format GUID (8-4-4-12). The tool is identified by the GUID value specified which is also specified in the DSC file. These GUID tools call other tools that modify the code outside of the normal EDK II build system process flow.
OUTFLAGS	This specified an output flag for ACPI (ASL and IASL) tools.
OUTPUT	This specifies an output flag for the Assembler (ASM) command.
PATH	This is the full path and executable name for a command code. For executables that are in the BaseTools paths (or that are in directories specified in the OS PATH environment variable) only the name of the executable is required.
BUILDRULEORDER	This attribute is used by tools to process files listed in INF [Sources] sections in priority order. If a filename is listed with multiple extensions, the tools will use only the file that matches the first extension in the space separated list.

5.2.3.1 BUILDRULEORDER Example

The following is an example use of the `BUILDRULEORDER` attribute.

```
[Sources]
Foo.S
```

```
Foo.asm  
Foo.nasm
```

The tools_def.txt file has the entry.

```
*_*_*_BUILDRULEORDER = nasm asm Asm ASM S s
```

The Foo.nasm file will be processed, and the Foo.s and Foo.asm files will be ignored during the build. If a file is listed in the [Sources] section and the file extension is not listed a section that is specified for a build FAMILY (or BUILDRULEFAMILY if specified as an attribute in the build_rule.txt file) for the selected tool chain (GCC for example) in the build_rule.txt file, then the file is ignored. For example, if the INF has the following section listed: UefiCpuPkg/Library/BaseUefiCpuLib/BaseUefiCpuLib.inf

```
[Sources.IA32]  
  Ia32/InitializeFpu.asm  
  Ia32/InitializeFpu.S  
  
[Sources.X64]  
  X64/InitializeFpu.asm  
  X64/InitializeFpu.S
```

If the tool chain is a GCC tool chain, then only the .S files would be processed and the .asm files will be ignored.

5.3 target.txt File

This file is used to filter the build so that only required components are used. It also provides pointers to the `tools_def.txt` file, and the active `build_rule.txt` files. All file names are relative to the system environment variable, `WORKSPACE`. No wildcard characters are permitted in this file. All entries in this file are case-sensitive.

While the values in this file filter what will be built, the `TARGET`, `TARGET_ARCH` and `TOOL_CHAIN_TAG` values may also be overridden on the build tool's command line.

The following well known macro names may be used in other EDK II meta-data files, `$(TARGET)`, `$(ARCH)` and `$(TOOL_CHAIN_TAG)` and are mapped to the `TARGET`, `TARGET_ARCH` and `TOOL_CHAIN_TAG` in this file or from options specified on the command line which override the settings in this file.

Prototype

```
<TargetText> ::= [<Platform>
                  [<Target>]
                  [<TargetArch>]
                  [<ToolsDef>]
                  [<ToolTagName>]
                  [<ThreadEnable> <EOL>]
                  [<BldRuleConf>]
<TabSpace>    ::= {0x09} {0x20}
<TS>          ::= <TabSpace>*
<MTS>        ::= <TabSpace>+
<Eq>         ::= <TS> "=" <TS>
<Platform>    ::= "ACTIVE_PLATFORM" <Eq> [PlatformFile] <EOL>
<Target>      ::= "TARGET" <Eq> [<Targets>] <EOL>
<Targets>     ::= TargetVal [" " TargetVal]*
<TargetArch>  ::= "TARGET_ARCH" <Eq> [<Archs>] <EOL>
<Archs>       ::= Arch [" " Arch]*
<ToolsDef>    ::= "TOOL_CHAIN_CONF" <Eq> ToolDefsFile <EOL>
<ToolTagName> ::= "TOOL_CHAIN_TAG" <Eq> TagName <EOL>
<ThreadEnable> ::= "MAX_CONCURRENT_THREAD_NUMBER" <Eq> [<NumThrs>]
<NumThrs>     ::= (1-9) [(0-9)]*
<BldRuleConf> ::= "BUILD_RULE_CONF" <Eq> BuildRulesFile <EOL>
<Paths>       ::= (a-zA-Z0-9)(a-zA-Z0-9\_\\-)* "/"
<FileNames>   ::= <Paths>* (a-zA-Z0-9)(a-zA-Z0-9\_\\-)* ["." <Ext>]
<Ext>         ::= (a-zA-Z0-9)+
<EOL>        ::= <TS> 0x0D 0x0A
```

Parameters

PlatformFile

Specify the `WORKSPACE` relative Path and Filename of the platform DSC file that will be used for the build. This line is required only if the current working directory does not contain one or more DSC files.

TargetVal

Zero or more of the following: `NOOPT`, `DEBUG`, `RELEASE`, a user defined word in the `tools_def.txt` file; separated by a space character. If the line is missing or no value is specified, all valid targets specified in the DSC file will attempt to be built.

Arch

The target architectures that are specified on the command-line override the `TARGET_ARCH` entry in the `target.txt` file. The resulting architecture must also be listed as one of the architectures in the `SUPPORTED_ARCHITECTURES` entry in the DSC file's `[Defines]` section. If the target architecture is not specified on

the command-line and the `TARGET_ARCH` entry does not exist in the `target.txt` file, then all valid architectures specified in the DSC file, for which tools are available, will be built. The architectures are space separated.

ToolDefsFile

Specify the name of the filename to use for specifying the tools to use for the build.

If not specified, the file: `WORKSPACE/Conf/tools_def.txt` will be used for the build. The path and file name must be relative to the `WORKSPACE` directory.

TagName

Specify the name of the `tools_def.txt` tool chain tag name to use. If not specified in this file and it is not specified using the `-t` option on the command-line, then the build will break.

Integer

The number of concurrent threads. If not specified or set to zero, tool automatically detect number of processor threads. Recommend setting this value to one less than the number of computer cores or CPUs of the development workstation. When value set to 1, means disable multi-thread build, and value set to more than 1, means user specify the thread number to build.

BuildRulesFile

Specify the file name to use for the build rules that are followed when generating Makefiles. If not specified, the file: `WORKSPACE/Conf/build_rule.txt` will be used.

The path and file name must be relative to the `WORKSPACE` directory.

6 QUICK START

This chapter describes the build environment. Additional chapters describe how the build system parse files, creates C files and assembles binary images into PI compliant firmware images. The EDK II build system uses multiple threads during the build process. The maximum number of threads that will be spawned is controlled in the `Conf/target.txt` file. Typically, this value will be one more than the number of cores that are on the development workstation. Increasing the number beyond the N+1 value will not offer any performance benefit.

Note: Path and Filename elements within the Build Meta-Data files and command-line arguments are case-sensitive in order to support building on UNIX style operating systems.

A build is always performed within the context of a "platform" defined in a single workspace. Multiple platforms can be defined in any one workspace. While some developers will not be building actual platform firmware; the platform definition file (DSC) format is suitable for Option ROM and stand-alone application development, as well as flexible enough to create binary distribution code for individual modules as well as a full platform firmware file.

One set of EDK II build tools is required on a development system. The source code for these tools is written in either generic C or Python.

Refer to the TianoCore.org Getting Started with EDK II web page for additional information on setting up and using the EDK II build system.

6.1 Environment Variables

There are two required system level environment variables that must be set, and several optional environment variables.

6.1.1 Required Environment Variables

The first of the two required variables is `WORKSPACE`. This variable points to a directory that will contain a Conf directory (containing the text files that are used to control build options) and the typical Build output directory tree. The following two lines are an example of setting this variable, the first in a Microsoft Windows* Command Prompt Window, while the second represents setting the variable in a UNIX terminal bash shell.

```
set WORKSPACE=C:\MyWork\Proj1\edk2
export WORKSPACE=/usr/local/src/proj1/edk2
```

The second required environment variable, `EDK_TOOLS_PATH`, required points to the directory containing the Conf directory for the BaseTools directory. The EDK II project contains a BaseTools directory, that contains setup scripts, template files and XML Schema files. Only one copy of the `BaseTools` directories needs to be installed on a workstation (although multiple copies are permitted, such as having one in each workspace). The `EDK_TOOLS_PATH` variable must point to the directory containing the `BaseTools/Conf` directory. The following lines are an example of setting this variable in a Microsoft Windows* Command Prompt window. The first line sets an absolute path to single location, outside of the workspace, while the second line uses tools located within the workspace.

```
set EDK_TOOLS_PATH=C:\Tools
set EDK_TOOLS_PATH=%WORKSPACE%\BaseTools
```

If assembly code is used by the modules and the NASM assembler is used, the system environment variable, `NASM_PREFIX` must be set as shown below and must include the trailing backslash character:

```
set NASM_PREFIX=C:\nasm\
```

6.1.2 Optional Environment Variables

When EDK II Packages are distributed within different directory trees on a developer's workstation, the `PACKAGES_PATH` environment variable is used to list directories (prioritized from left to right) that contain EDK II Package directories. The operating system delimiter, such as the semi-colon character for Microsoft operating systems, is used to separate the directory names. If all development is performed under the root of the edk2 source tree, this variable is not required. The edk2 reference build system will look for EDK II packages in the directory specified in the `WORKSPACE`, then search for the package directory in the directories listed in the `PACKAGES_PATH`; the first occurrence of an EDK II package found will be used.

For Microsoft windows environments, the `EDK_TOOLS_BIN` environment variable can be used to point to the directory that contains the Win32 BaseTools binaries. If these Win32 binaries are located in edk2 directory tree under the `BaseTools\Bin\Win32` directory; this variable is not required. Since developers using *NIX operating systems must build the 'C'-based tools prior to using them and run the Python based tools from source, this environment variable is not required. The edksetup script is used to add the path to the binaries to the system PATH environment variable.

The final optional environment variable, `ECP_SOURCE`, is used to define the location of the EDK Compatibility Package content for building EDK modules. If these values are not set, the build system will automatically set the value to the `EdkCompatibilityPkg` directory in the `WORKSPACE`.

6.1.3 Configuring the Environment Variables

If all development will be done within the root of the edk2 directory tree, and the Win32 BaseTools binaries are in the BaseTools\Bin\Win32 directory, then the edksetup script may be used to setup the development workspace by setting system environment variables, `WORKSPACE` and `EDK_TOOLS_PATH`.

If a more complex development environment is used (multiple directories containing EDK II Packages), then the `WORKSPACE`, `PACKAGES_PATH` and `EDK_TOOLS_BIN` environment variables must be set before running the edksetup script.

The script must be executed prior to building in a new command prompt window or new terminal shell.

Another feature of the script is that it adds the path of the build system tools into the OS environment variable, `PATH`.

6.2 Build Scope

The EDK II build process was designed for maximum flexibility. The meta-data files and command line options enable the developer to build only what they need, rather than having to build a single platform from scratch. Multiple versions of a platform and/or module can be built, as well as just a single module within the context of a platform. This section of the document describes the techniques provided to limit what is built.

Typically, the `target.txt` file is used to limit the scope of a build, restricting the build to specified values for Platform, Architectures, Targets and Tool Chains. With all values in this file commented out, the build system will build all valid targets for all architectures where tools exist. However, options specified on the build tool's command line will override the `TARGET`, `TARGET_ARCH` and `TOOL_CHAIN_TAG` values.

7 BUILD ENVIRONMENT

This chapter details the supported build environments (developer workstations) for EDK II development. It also covers the tool configuration files that describe the developer's tool environments. Note that the term "development platform" referenced throughout this document means the workstation a developer using to write code and build a target binary. The architecture for the target binary does not have to match the same architecture as the developer's workstation.

Note: Path and Filename elements within the Build Meta-Data files and command-line arguments are case-sensitive in order to support building on UNIX style operating systems.

7.1 Build Scope

This section of the document describes the some of the rules that control what gets built and rules that the build system uses when parsing meta-data files.

7.1.1 The precedence of what (platform or module) gets built

1. Content of the current working directory. If the current working directory contains an INF file, then only the module is built in the context of an `ACTIVE_PLATFORM`, otherwise the following apply.
2. **build.exe** option statements (command-line options)
3. `target.txt` file's `ACTIVE_PLATFORM` statement

Note: There are two different options, the `-p` option specifies the `ACTIVE_PLATFORM` to be used for a build, so that if the current working directory contains a module INF file, then the module will be built in the context of the `ACTIVE_PLATFORM`.

Note: If the INF file is not listed in the `ACTIVE_PLATFORM`'s DSC file, the build will result in an error. The `-m` option is used to specify building an individual module (in the context of the `ACTIVE_PLATFORM`).

If the `ACTIVE_PLATFORM` value is not set using the methods above, then, if the current working directory contains a DSC file, then the platform is built (unless a specific module is specified by the `-m` option on the command line).

If the user attempts to build a module that is not part of the current `ACTIVE_PLATFORM`, the build system should provide an appropriate error message and the build should break.

7.1.2 The precedence of the TARGET value

It is possible to build more than one `TARGET` (i.e., `DEBUG`, `RELEASE`, `NOOPT`, etc.) with a single build command. The precedence of the `TARGET` value is:

1. **build.exe -b TARGET** option statements. One or more of the **-b TARGET** options may be specified on the command line of the **build** tool.
2. `TARGET` statement in the `target.txt` file
3. DSC file's `BUILD_TARGETS` statement

7.1.3 The precedence of the TARGET_ARCH values

The target architectures that are specified on the command-line override the `TARGET_ARCH` entry in the `target.txt` file. The resulting architecture must also be listed as one of the architectures in the `SUPPORTED_ARCHITECTURES` entry in the DSC file's `[Defines]` section. If the target architecture is not specified on the command-line and the `TARGET_ARCH` entry does not exist in the `target.txt` file, then all valid architectures specified in the DSC file, for which tools are available, will be built.

Note: If a module's INF file does not contain a `[Sources]` or `[Sources.common]` section, and does contain a `[Sources.IA32]` section, then the module is only valid for IA32 builds. The module will not be built for other architectures.

If an architecture set on a command line or specified in the `target.txt` file is not in the list of the DSC file's `SUPPORTED_ARCHITECTURES` statement, the command will fail.

7.1.4 Third Party tools using -t TOOL_CHAIN_TAG

It is possible to specify a different set of third party tools using the `-t TOOL_CHAIN_TAG` option to the build command. This option takes precedence over the `target.txt` file setting.

1. build command `-t TOOL_CHAIN_TAG` option
2. `target.txt` file `TOOL_CHAIN_TAG` statement

If the `TOOL_CHAIN_TAG` is not specified on the command-line nor in the `target.txt` file, the build system will break with an error.

7.1.5 Precedence of Build Option FLAGS values

The flags needed by third party tools can be specified on a file, module or platform basis. The default flags provided in the `tools_def.txt` file are for size optimization. These flags may be modified to provide better debugging capability. The precedence of the `FLAGS` values for third party tools follows. The reasoning behind this precedence is that flags are appended to a single line from the lowest to highest, with third party tools using the right most option. If a flag line for the Microsoft compiler contains `/O1` (specified in the `tools_def.txt` file) and `/Od` (for example, from the DSC file's `[BuildOptions]` section), then the compiler only recognizes the `/Od` flag.

Flag entries can be defined in the INF and DSC files to replace all previous flags by using two equal signs as in the following example:

```
GCC:*_*_X64_NASM_FLAGS == -f elf32
```

The following is the precedence list for flag entries, and as such, they would be processed in reverse order.

1. Highest - DSC file, INF `<BuildOptions>` section statements
2. DSC file, `[BuildOptions.<arch>.<codebase>.<moduletype>]` section statements
3. DSC file, `[BuildOptions.<arch>.<codebase>]` section statements
4. DSC file, `[BuildOptions.<arch>]` section statements
5. DSC file, `[BuildOptions]` section statements
6. INF file, `[BuildOptions]` section statements
7. Lowest - `tools_def.txt` file `_FLAGS` statements

The following demonstrates the way tools process flags statements.

```
CCFLAGS = ToolsDef.CC_FLAGS + INF.BuildOptions + DSC.BuildOptions.CC_FLAGS + DSC.Inf.BuildOptions
```

The DSC and INF specifications define the `"=="` character string as a replacement rather than append. This allows the INF file to replace the all options specified in the `tools_def.txt` file, and also allows the platform DSC to override all options specified in either the INF or `tools_def.txt` file.

Note: Most tools will process the flag values from left to right, with the right most of a duplicated flag taking priority over identical flags that are to the left. This includes the `-D` option of the build command.

7.2 Third Party Tools

The `tools_def.txt` file provides various flags for third party tools. Refer to the Appendix, "Third Party Tool Flags" for additional information.

7.3 GUIDed Tools

The `tools_def.txt` file also allows for specifying tools by GUID. For custom guided sections specified in the FDF file, the `tools_def.txt` file must specify a GUID that matches the GUID used in the FDF. Using compression, such as LZMA, CRC32 or TianoCompress, as an example, the entries in the `tools_def.txt` must define an entry for the tool as well as implementing a decompression library in the code-base. Since the build system provides these, the entries in the `tools_def.txt` file are:

```
*_*_*_LZMA_PATH = LzmaCompress
*_*_*_LZMA_GUID = EE4E5898-3914-4259-9D6E-DC7BD79403CF
*_*_*_CRC32_PATH = GenCrc32
*_*_*_CRC32_GUID = FC1BCDB0-7D31-49AA-936A-A4600D9DD083
*_*_*_TIANO_PATH = TianoCompress
*_*_*_TIANO_GUID = A31280AD-481E-41B6-95E8-127F4C984779
```

The GUIDed compression tools must use the **-e** option to encode (compress) a file and the **-d** option to decode (decompress) a file.

7.3.1 PCD VPD Data

PCDs defined in the DSC file may be defined as Dynamic VPD or DynamicEx VPD. VPD data is typically located in a separate data section of the FDF file. VPD data also requires that each PCD be located at a known offset from the start of the data region.

VPD data is common to all modules, therefore one and only one value can be defined for a given PCD/SKU in the DSC file.

The DSC file permits automatic assignment for these VPD PCDs when using an external tool, such as the provided **BPDG** tool, by specifying a offset value of "*".

Just calling the tool to create the binary data file is not enough, as the offset value must also be used in a header file for the PEI and DXE PCD drivers. In order to interrupt the build system prior to auto-generating the header files, a special entry in the DSC file, `VPD_TOOL_GUID` identifies the GUIDed tool in defined in the `tools_def.txt` file. The format for the VPD GUIDed tool in the `tools_def.txt` file is:

```
*_*_*_VPD_TOOL_PATH = BPDG
*_*_*_VPD_TOOL_GUID = 8C3D856A-9BE6-468E-850A-24F7A8D38E08
```

When using the BPDG tool provided, the build tools will create the 8C3D856A-9BE6468E-850A-24F7A8D38E08.txt file with the PCD names, SKU names, offsets, values and size, in the build output FV directory. The VPD_TOOL will be called using the two flags and the file generated by the build system as an argument. Any custom tool that is used to create the VPD data must support these two flags and take the input text file that was generated by the build system.

```
<Tool> -o Filename.bin -m Filename.map Filename.txt
```

Once the tool completes, if the PCD offsets have been calculated by the tool, the map file generated by the tool will be read by the build system and will be used to create the header files required by the PCD drivers and binary file to be included in the flash image.

Note: The word "Filename" in the above example will be replaced by the `VPD_TOOL_GUID` value, as in:

```
8C3D856A-9BE6-468E-850A-24F7A8D38E08.txt .
```

The format for the file created by the build system and the format of the map file required by the build system is provided in the appendix, VPD Tool.

7.3.1.1 Using VPD Data for PCD Values

1. Modify the DSC file:

- Add "VPD_TOOL_GUID = 8C3D856A-9BE6-468E-850A-24F7A8D38E08" to the [Defines] section.
- Add at least one [PcdsDynamicVpd] or [PcdsDynamicExVpd] section.
- For numeric or boolean PCDs allowing the tool to determine offsets automatically, add an entry for each PCD using the following format:

```
<PcdTokenSpaceGuidCName>.<PcdCName> | * | <Value>
```

- For `VOID*` PCDs allowing the tool to determine offsets and reserved size automatically, add an entry for each PCD using the following format:

```
<PcdTokenSpaceGuidCName>.<PcdCName> | * | <MaxSize> | <Value>
```

If using automatic offset feature, the build tools byte-align numeric values, while `VOID*` PCD types will be aligned using the following rules:

- ASCII strings, "string" or 'string', will be byte aligned.
- Unicode strings, L"string" or L'string' will be two-byte aligned.
- Byte arrays, {0x00, 0x01} will be 8-byte aligned.

If the developer manually assigns offset values in the DSC file, the developer must follow the same rules.

Note: If a developer manually sets the offset of a `VOID*` PCD with Unicode string, L"string"/L'string' style to a value that is not 2-byte aligned, then an error is generated and the build halts.

Note: If a developer manually sets the offset of a `VOID*` PCD with byte array { } style to a value that is not 8-byte aligned, then a warning is generated, but the build will continue.

2. Modify the FDF file:

- Create a Region for storing the VPD binary data in the [FD] section, the offset and size of the region must be specified.
- The starting address of the VPD data region must be 8-byte aligned (the BaseTools must halt with an appropriate error message if the address is not correctly aligned).
- The PcdVpdBaseAddress PCD must be specified immediately after the region declaration.
- Add the FILE statement to the region with the name of the VPD binary generated by the VPD tool.

```
# VPD Data Region
0x0026D000|0x00001000
gEfiMdeModulePkgTokenSpaceGuid.PcdVpdBaseAddress

FILE = $(OUTPUT_DIRECTORY)/$(TARGET)_$(TOOL_CHAIN_TAG)/FV/8C3D856A-9BE6468E-850A-24F7A8D38E08.bin
```


8 PRE-BUILD AUTOGEN STAGE

For the remainder of this document, unless otherwise specified (using system environment variable, `WORKSPACE`), references to the `WORKSPACE` and `$(WORKSPACE)` refer to the ordered list of directories specified by the combination of `WORKSPACE` and `PACKAGES_PATH`. The build system will automatically join the directories and search these paths to locate content, with the first match terminating the search. For example given the following set of environment variables, and the `MdeModulePkg` is located in both the `edk2` and `edk2Copy` directories, the build system would use the `C:\work\edk2\MdeModulePkg` when attempting to locate the `MdeModulePkg.dec` file.

```
set WORKSPACE=c:\work
set PACKAGES_PATH=c:\work\edk2;c:\work\edk2Copy
```

8.1 Overview

This chapter describes in detail the steps that are accomplished by the AutoGen stage, which is the first step of building a platform or a module.

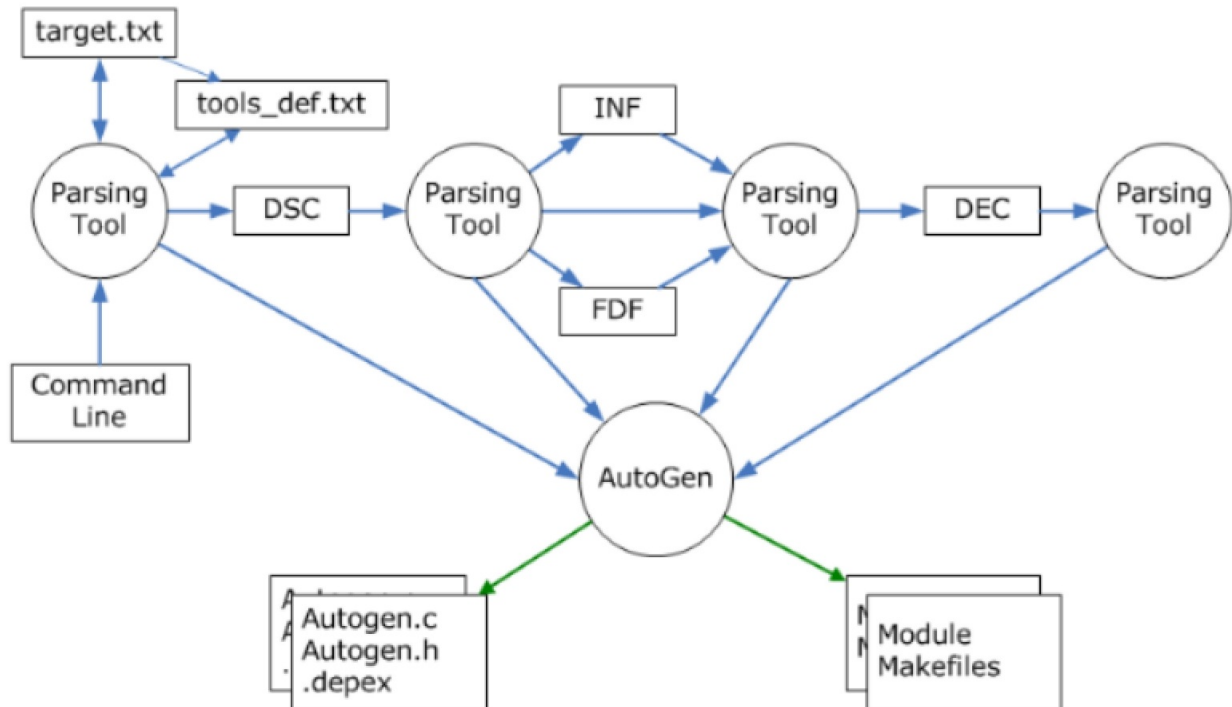


Figure 19 EDK II AutoGen Process

The first file the build tool is looking for in AutoGen stage is `target.txt` in directory `$(WORKSPACE)/Conf`. All the configurations in `target.txt` can be overridden by command line options of build tool. If no platform description file is specified in either `target.txt` and command line, the build tool will try to find one in current directory. And if build tool finds a description file of a module (INF file) in current directory, it will try to build just that module only rather than building a whole platform.

Once the build tool gets what to build and how to build, it starts to parse the platform description file (DSC). From the DSC file, the build tools will locate the INF files for all modules and libraries, as well as other settings of the platform (including DEC specified default values for PCDs used by modules and libraries that do not have values specified in the DSC file).

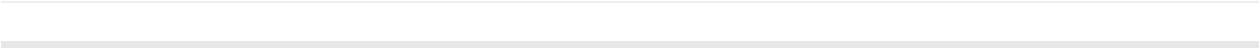
From module description files, the build tool will find out what package description files the module depends on. In this way, the build tool will find out and parse all modules and packages that make up a platform.

The next thing to do in the AutoGen stage is to generate files required to build a module. The files include: `AutoGen.h`, `AutoGen.c`, `$(BASENAME).depex` and `Makefile`.

`AutoGen.c` and `$(BASENAME).depex` files will not be generated for library modules, and `$(BASENAME).depex` file is generated only if there's `[Depex]` section found in the module's INF file.

Each module found in DSC file will have a makefile generated for it. Once all of the makefiles have been generated, the build tool will call **nmake** (or **make**) for each module's `Makefile`.

Note: When building a module, only the module's makefile will be called.



8.2 Auto-generation Process

This section covers, in sequence, the steps taken by the `build.exe` tool. When creating the auto-generated files, the build system must include either the "u" suffix or the "ull" suffix (UINT64 only) to indicate that the values are unsigned for all numeric values specified for PCDs.

8.2.1 Determine What to Build

The build tool will use following algorithm to determine what will be built. The first step the build system performs is to open the `Conf/target.txt` file.

Note: The build system tools allow for specifying an alternate location and filename for `Conf/target.txt` on the command-line that can be either inside or outside of the `WORKSPACE` directory tree.

The following pseudo-code demonstrates how the tools obtain command-line overrides of the information specified in `Conf/target.txt`.

```
If ("-t <DscFile>") {
    // Command line option specified
    ActivePlatform = <DscFile>;
} ElseIf (<ACTIVE_PLATFORM> specified in $ (WORKSPACE)/Conf/target.txt) {
    ActivePlatform = <ACTIVE_PLATFORM>;
} ElseIf (one <DscFile> found in current working directory) {
    ActivePlatform = <DscFile>;
} Else {
    // Unable to determine the Active Platform
    if (Number of DscFiles > 1) {
        PrintError (
            "There are %s DSC files in the folder. "
            "Use '-p' to specify one.", NumDscFiles
        );
    } else {
        PrintError (
            "No active platform specified in target.txt "
            "or command line!\n Nothing to build."
        );
    }
    BreakTheBuild();
}

// Determine whether this is a module only build or the full platform
If ((" -m <InfFile>") || (one <InfFile> found in working directory)) {
    // Either a command line option was specified, or one and only
    // one INF file was found in the current working directory.
    ActiveModule = <InfFile>;
    BuildMode = "SingleModuleBuild";
} Else {
    ActiveModule = NONE;
    BuildMode = "PlatformBuild";
}

Parse ( $ (WORKSPACE) / Conf / target.txt );
Parse ( ActivePlatform );

// Determine Architectures to build
If ("-a <ArchListFromCommandLine>") {
    // command line option given
    ActiveArchList = Intersection (
        <ArchListFromCommandLine>,
        <ArchListFrom (ActivePlatform)>
    )
}
```

```

        );
    } Else {
        ActiveArchList = Intersection (
            <ArchListFromTarget.Txt>,
            <ArchListFrom (ActivePlatform)>
        );
    }

    If (ActiveArchList == NULL) {
        if (ArchListFromCommandLine != NULL) {
            PrintError (
                "The architecture(s) specified on the command line "
                "(%s) are not valid for the active platform (%s\n",
                ArchListFromCommandLine,
                ArchListFrom (ActivePlatform)
            );
        } else {
            PrintError (
                "The active platform cannot be built, the "
                "architectures (%s) are not supported.\n",
                ArchListFrom (ActivePlatform)
            );
        }
        BreakTheBuild();
    }

    // Determine the target type, such as DEBUG and/or RELEASE
    If ("-b <TargetListFromCommandLine>") {
        // command line option given
        ActiveTargetList = Intersection (
            <TargetListFromCommandLine>,
            <TargetListFrom (ActivePlatform)>
        );
    } Else {
        ActiveTargetList = Intersection (
            <TargetListFromTarget.Txt>,
            <TargetListFrom (ActivePlatform)>
        );
    }

    If (ActiveTargetList == NULL) {
        if (TargetListFromCommandLine != NULL) {
            PrintError (
                "Target (%s) specified on the command line is not "
                "valid for this platform (%s).\n",
                TargetListFromCommandLine,
                TargetListFrom (ActivePlatform)
            );
        } else {
            PrintError (
                "Target (%s) is not specified in the target.txt file.\n",
                TargetListFrom (ActivePlatform)
            );
        }
        BreakTheBuild();
    }

    // Determine the tool chain to use for the build
    If ("-t <ToolChainTag>") {
        // command line option given
        ActiveToolChain = <ToolChainTag>
    } ElseIf (<TOOL_CHAIN_TAG> specified in $(WORKSPACE)/Conf/target.txt) {
        ActiveToolChain = <TOOL_CHAIN_TAG>
    } Else {
        if (ToolChainTag != NULL) {
            PrintError (
                "Tool chain specified on the command line (%s) is "
                "not specified in the tools_def.txt file.\n",
                ToolChainTag
            );
        } else {
            PrintError (

```

```

"Tool chain specified in target.txt (%s) is not "
"specified in the tools_def.txt file.\n", TOOL_CHAIN_TAG
);
}
BreakTheBuild();
}

Build (ActivePlatform, ActiveModule, ActiveArchList, ActiveTargetList, ActiveToolChain, BuildMode);

```

8.2.2 Parse File Pointed to by TOOL_CHAIN_CONF

The file specified by `TOOL_CHAIN_CONF` (in `target.txt`) is the tool chain definition file (`tools_def.txt`) that contains all the definitions of external tools used to build modules and platforms, in the form of "name=value". The definition of a tool includes the path of the executable, the path of dynamic libraries the executable needs, and command line options. Each set of tools can be referenced by a tag name either in the command line or in `target.txt`. For example, `WINDDK3790x1830` is used to refer a set of tools from `WINDDK` of version `3790x1830`.

The parser of the tool chain definition file needs to expand macros and wild cards ("*") in the tool definitions. The expanded definitions are put in a database for easier access later. For example, if one overrides a tool's options in `DSC` or `INF` file, the tool will look up the tool's definition in the database and append the options to the end of options in the file specified by `TOOL_CHAIN_CONF`.

Note: The supported third party compiler tools will use the right most (or last) option it encounters, permitting appended options to override options specified first. For example, specifying a compiler option (FLAG) line: `/Od /c /Og` will result the compiler only processing `/c /Og`, ignoring the `/Od` flag.

The final result after `AutoGen` stage is that macros named by `<TOOLCODE>` and

`<TOOLCODE>_FLAGS` will be generated in module's makefile. For example, "CC" and "CC_FLAGS" macros will be generated in the makefile for the compiler tool. The path of dynamic libraries will be prefixed to system's `PATH` environment by the build tools, so that the tools used in the `Makefile` can be called correctly.

8.2.3 Parse build_rule.txt

The file specified by `BUILD_RULE_CONF` (in `target.txt`) contains command steps used to build the source files into intermediate files and then intermediate files into final image files to be put into `FV/FD`. The type of source files and intermediate files are determined by the file extension. That means the same extension cannot be used to represent different file types. But one type of file can have more than one file extension. A single file can only have a single extension.

The parser of this file will convert the contents of the file into a build rule database. Each item in this database will have tool chain family, input file information, output file information and command information. Whenever a source file is found in module's `INF` file, the build tools will attempt to find a build rule in the database corresponding to the input file's extension, and then use the output file as input file information to find another build rule, until no build rule uses the output file information as its input file. If there's no build rule for a type of source file, the build tools just skip it. But if there's build rule for it, one or more makefile targets will be generated for it.

The sequence of build rules applied to source files and intermediate files determines the dependency relationship between targets in makefile. One type of file cannot be used in more than one build rule as an input file and the build rules must not be cyclic.

8.2.4 Parse DSC, FDF, INF, DEC files

The platform description (DSC) file is used to instruct the build system what modules need to be processed in order to generate the PE32/PE32+ image files.

The EDK II build system tools must be located in either the path pointed to by the `EDK_TOOLS_BIN` system environment variable (on Microsoft* operating systems) or located under a subdirectory of the `Bin` directory of the `EDK_TOOLS_PATH` directory.

- All EDK II content used to create PE32/PE32+ images must reside in the directory tree pointed to by the `WORKSPACE`.
- The build system's output directory is not required to be within the `WORKSPACE`.

From the DSC file, the build tools collect the mapping between library classes and library instances (INF files), PCD data for the whole platform, the list of modules (INF files) specified for the platform, and the build output directory. Optionally, the name of the flash image layout description (FDF) file and build options specific to the platform are also obtained. Parsing FDF file at this time is just for the PCD information which might be used by some modules, and merge these PCD values into the information set of PCDs in DSC file.

A PCD entry must only be listed once per section in the DSC or FDF files.

Multiple library class instances for a single library class must not be specified in the same `[LibraryClasses]` or `<LibraryClasses>` section in the DSC file.

8.2.4.1 !include Files

The DSC and FDF file can use `!include` statements to include text files that contain content that would appear in the DSC file. When gathering the content from the DSC (or FDF) file, the file pointed to by the `!include` statement is read before any other information that appears later in the file.

The build system does not parse the files as the lines are read, but rather the lines are all read into a buffer prior to parsing the content. Therefore, the directory and file names for `!include` statements may not contain MACROS.

If only a filename is provided, the file must be located in the same directory as the DSC or FDF file. Use of `$(WORKSPACE)/<Path>/<Filename>` is allowed for include files outside of the directory tree containing the DSC or FDF file, or `<Path>/<Filename>` if the include file is in the directory tree containing the DSC or FDF file.

The keyword `!include` is case-insensitive.

8.2.4.2 INF and DEC Parsing

The build tools try to parse the INF file one by one, including the INF file for library instances. From the INF file, the build tools collect information such as source file list, library class list, package list, GUID/Protocol/PPI list, PCD list, etc.

After all INF files are parsed, the build tools retrieve the list of all of the dependent DEC files and then parse them. From the DEC file, the build tools will get the information such as common include folders, the values of GUID/Protocol/PPI, the default setting of all PCDs in the package, etc.

The `[Packages]` section of the INF file is used by the build tools during the generation of the Makefiles. The `[Includes]` section of the DEC file specified in the `[Packages]` section will be added to the command-lines for compiler tools. The `MdePkg/MdePkg.dec` file must be included in all INF files listed in the DSC file.

EDK II INF files must contain a valid name in the `MODULE_TYPE` element of their `[Defines]` sections. If the module type is not recognized, the build tools should break the build with an appropriate error message.

EDK INF files must contain a valid name in the `COMPONENT_TYPE` element of their `[defines]` sections. If the component type is not recognized, the build tools should break the build with an appropriate error message.

For entries in the [Sources] section of the INF file, in addition to the required file name field, there are optional fields for Family, Tool chain tag name and Tool Code that may contain modifiers that limit the scope of the file to a specific tool chain family, such as GCC, or tool code, such as ASM. If these fields are blank, then there is no restriction to what tools, tagname or tool chain family will process the file. The final field is for a FeatureFlag Expression. This field is an expression that must evaluate to True or False. If the field cannot be evaluated (such as an undeclared PCD used in the expression) the build parser must provide an appropriate error message and stop the build. If the field evaluates to False, the line is ignored. If the field evaluates to True, the build will use this line.

For entries in the [Binaries] section of the INF file, in addition to the file type and name fields, there are optional fields for the target (DEBUG, RELEASE, etc.) and a FeatureFlagExpression field. This field is an expression that must evaluate to True or False. If the field cannot be evaluated (such as an undeclared PCD used in the expression) the build parser must provide an appropriate error message and stop the build. If the field evaluates to False, the line is ignored. If the field evaluates to True, the build will use this line.

The [Binaries] section of an INF file may list files with a FileType of DISPOSABLE. The build tools must ignore files of this type.

8.2.4.3 Build.exe --ignore-sources option

When the --ignore-sources option is present on the build.exe command-line, all modules specified in the DSC and FDF files must be either Binary INFs or Mixed INFs (that contain binary images). The build tools will ignore any content in a Mixed INF [Sources] section. If a Source INF is listed in the DSC file, the build must break during parsing with an appropriate error message. If an INF file is listed in the DSC file that does not contain a [Binaries] section, the build must break during parsing with an appropriate error message. The only code that will be generated during this build is the binary external PCD database file that will be added to the PEIM and DXE PCD driver FFS files.

8.2.4.4 Macros

The build and GenFds tools use the -D, --define command line options with an argument formatted: MACRO_NAME "=" value. If the "=" value is omitted, the MACRO_NAME is assigned a value of 0.

Token names (words defined in the EDK II meta-data file specifications) cannot be used as macro names. As an example, using PLATFORM_NAME as a macro name is not permitted, as it is a token defined in the DSC file's [Defines] section.

Macros defined in INF files are local to the INF file. EDK II INF files must not use global macros except in build option flags. In INF files, macros can only be used for filenames, paths and, in the [BuildOptions] section, on the right (value) side of the statements.

Macros can be defined or used in the INF file's [Defines], [LibraryClasses], [Sources], [Binaries], [Packages] and [BuildOptions] sections.

Macros defined in DEC files are local to the DEC file. DEC files must not use global macros. In DEC files, macros can only be used for filenames and paths.

Macros can be defined or used in the DEC file's [Defines], [Includes] or [LibraryClasses] sections.

System environment variables may be referenced, however their values must not be altered.

Table 9 System Environment Variable Usage

Macro Style Used in Meta-Data files	Windows Environment Variable	Linux & OS/X Environment Variable
\$(WORKSPACE)	%WORKSPACE%	\$WORKSPACE

<code>\$(EDK_TOOLS_PATH)</code>	<code>%EDK_TOOLS_PATH%</code>	<code>\$EDK_TOOLS_PATH</code>
---------------------------------	-------------------------------	-------------------------------

Note: The `PACKAGES_PATH` and `EDK_TOOLS_BIN` system environment variables shall not be referenced in EDK II meta-data files.

There are also four global MACRO statements that may be used in different portions of the DSC and FDF files, `$(TARGET)`, `$(TOOL_CHAIN_TAG)`, `$(OUTPUT_DIRECTORY)` and `$(ARCH)`.

Macros defined in the FDF file are local to the FDF file. Macros are permitted in the entire FDF file.

Note: In the `[Rules]` section of the FDF, the macros listed in that section must match macro names defined for the `build_rule.txt` file.

Macros defined in the DSC file's `[Defines]` section can be used in either the DSC file or in the FDF file. Macros defined in other sections of the DSC file can only be used in the DSC file - they cannot be used in the FDF file. Macros in the DSC file can be used for file names, paths, PCD values, in the `[BuildOptions]` section, on the right (value) side of the statements and in conditional directives. Macros can also be defined or used in the `[Defines]`, `[LibraryClasses]`, `[Libraries]`, `[Components]` and all PCD sections.

Macros defined by the user may be used in the `!include` statements in DSC and FDF files.

Macro values must be defined prior to using them in directive statements or for PCD values. The following provides the precedence (high to low) for obtaining macro values.

- Command-line, `-D` flags (left most has higher priority)
- FDF file, `DEFINE` statements override previous definitions in the `[Defines]` section
- FDF file, `DEFINE` statements in the `[Defines]` section
- DSC file, Component INF `DEFINE` statements embedded in `<subsections>`
- DSC file, `DEFINE` statements in sections following the `[Defines]` section
- DSC file, `DEFINE` statements in the `[Defines]` section

Note: Macros defined in the DSC file's `[Defines]` section are common to both the DSC and FDF file. Macros defined in the FDF file are local to the FDF file. Macros defined in other sections of the DSC file are local to the section types that define them.

Note: Macros defined in INF and DEC files are local to the file that defined them.

Note: Note that all command line options for the build tool are passed to the GenFds tool after the make portion of the build completes.

Macros defined in common sections may be used in the architecturally modified sections of the same section type. Macros defined in architectural sections cannot be used in other architectural sections, nor can they be used in the common section. Section modifiers in addition to the architectural modifier follow the same rules as architectural modifiers.

When used in a `!if` or `!elseif` conditional expression statement or in an expression used in a value field, a macro that has not been defined has a value of 0.

The remaining MACRO definitions will be expanded by tools when they encounter the entry in the section except when the macro is within double quotation marks in build options sections. The expectation is that macros in the quoted values will be expanded by external build scripting tools, such as `nmake` or `make`; they will not be expanded by the build tools. If a macro that is not defined is used in locations that are not expressions or value fields (where the tools would just do macro expansion as in C flags in a `[BuildOptions]` section), nothing will be emitted. If the macro, `MACRO1`, has not been defined, then:

```
MSFT:*_*_*_CC_FLAGS = /c /nologo $(MACRO1) /Od
```

After macro expansion, the logical result would be equal to:

```
MSFT:*_*_*_CC_FLAGS = /c /nologo /Od
```

It is recommended that tools remove any excess space characters when processing these types of lines.

The following table lists reserved global macro names that are completed by the internal build tools. These macros must not be redefined.

Table 10 Reserved Macros Expanded by Tools

Macro String	Description
<code>\$(ARCH)</code>	Architecture of current module
<code>\$(BASE_NAME)</code>	The file name of the module binary
<code>\$(BUILD_DIR)</code>	All files for building a platform will be put in this directory
<code>\$(BUILD_NUMBER)</code>	Used in FDF file <code>[Rules]</code> sections to identify a build number used in a UEFI Version section. This is a value that is defined in the DSC file.
<code>\$(EDK_TOOLS_PATH)</code>	The system environment variable that points to the path of build tools
<code>\$(INF_OUTPUT)</code>	Used in FDF file <code>[Rules]</code> sections to identify the location of UEFI compliant binary leaf section content
<code>\$(INF_VERSION)</code>	Used in FDF file <code>[Rules]</code> sections to identify the version string used in a UEFI Version section.
<code>\$(MODULE_NAME)</code>	Current module name
<code>\$(MODULE_TYPE)</code>	Current module type
<code>\$(MODULE_GUID)</code>	Current module GUID
<code>\$(NAMED_GUID)</code>	Used in FDF file <code>[Rules]</code> sections this macro is used by the build tools to create an FFS file named by the Module's GUID value.
<code>\$(OUTPUT_DIRECTORY)</code>	This directory is where the output binary files will be generated, either an absolute path or relative to the <code>WORKSPACE</code> .
<code>\$(TARGET)</code>	Target of current module (<code>DEBUG</code> / <code>RELEASE</code> / <code>NOOPT</code>)
<code>\$(TOOL_CHAIN_TAG)</code>	Tool chain used to build current module
<code>\$(WORKSPACE)</code>	The system environment variable that points to the current Workspace directory.

The following table lists special Macros that may only be used in an FDF file's `[Rules]` section. Like the Macros in the previous table, they must never be redefined.

1. The `#{d_*}` macros always mean `OutputPath + ModuleGuid + .ffs`

- When starting to generate FFS, the `${s_*}` macros mean source INF file full path, but in `EfiSection.py`, it is changed to the full path of efi file.

Table 11 Reserved FDF [Rule] Section Macro Strings

Variable String	Description
"\${src}"	Source file(s) to be built (full path)
"\${s_path}"	Source INF file directory (absolute path)
"\${s_dir}"	Source file relative directory within a module. NOTE: <code>\${s_dir}</code> is always equals to "." if source file is given in absolute path.
"\${s_name}"	Source file name without path.
"\${s_base}"	Source file name without extension and path.
"\${s_ext}"	Source file extension.
"\${dst}"	Destination file(s) built from <code>\${src}</code> (full path)
"\${d_path}"	Destination file directory (OutputPath + ModuleGuid.ffs)
"\${d_name}"	Destination file name without path.
"\${d_base}"	Destination file name without extension and path
"\${d_ext}"	Destination file extension

Macro evaluation is done at the time the macro is used in an expression, conditional directive or value field, not when a macro is defined. Macros in quoted strings will not be expanded by parsing tools; all other macro values will be expanded, without evaluation, as other elements of the build system will perform any needed tests.

Example

```
[LibraryClasses.common]
  DEFINE MDE = MdePkg/Library
  BaseLib|$(MDE)/BaseLib.inf

[LibraryClasses.X64, LibraryClasses.IA32]
  # Can use $(MDE), cannot use $(MDEMEM)
  DEFINE PERF = PerformancePkg/Library
  TimerLib|$(PERF)/DxeTscTimerLib/DxeTscTimerLib.inf

[LibraryClasses.X64.PEIM]
  # Can use $(MDE) and $(PERF)
  DEFINE MDEMEM = $(MDE)/PeiMemoryAllocationLib
  MemoryAllocationLib|$(MDEMEM)/PeiMemoryAllocationLib.inf
```

8.2.4.5 Conditional Directive Blocks

Additional build scoping can be implemented using the DSC and FDF directive statements in combination with command line options for the build tool. Conditional directive blocks are not permitted in the EDK II DEC and INF files.

Conditional directive statements are used by the build tools preprocessor function to include or exclude statements in the DSC and FDF files. A limited number of statements are supported, and nesting of conditionals is also supported. Statements are prefixed by the exclamation "!" character. Conditional statements may appear anywhere within the DSC and FDF files. They are not permitted in the DSC and INF files.

Refer to the Macro Statement section for information on using Macros in conditional directives.

Conditional directive statements are only permitted in the DSC and FDF files.

Macro and PCD Names can be used in conditional directive statements.

Only macros can be used in the `!ifdef` and `!ifndef` statements, PCDs are code elements which have been declared in the DEC files.

When testing if a Macro has been defined, the only the Macro name is required, while in testing for values, the Macro name must be enclosed: `$(MacroName)`. When the Macro is a string value, the `$(MacroName)` must not be encapsulated in quotation marks, only string literals in directive statements need to be enclosed by double quotation marks.

When testing values for PCDs, only the PCD name is required: `TokenSpaceGuidCName.PcdCName`; enclosing the PCD name in `"$(" and ")"` is not permitted.

Supported statements are: `!ifdef`, `!ifndef`, `!if`, `!else`, `!elseif` and `!endif`, and those keywords are case-insensitive. These control statements are used to either include or exclude lines as the parsing tool processes these files. The `!ifdef` and `!ifndef` statements test whether a Macro has been defined or not defined (PCDs are always defined - the build will break if a PCD is used by a module specified in the DSC file that cannot be located in any of the dependent DEC files, from the `[Packages]` section of an INF specified in the DSC file). `FeatureFlag` and `FixedAtBuild` access methods are the only PCDs that can be used in conditional directives.

The build system will process the DSC and FDF files more than once. The first pass is to pick up all macros and PCD values for macros and PCDs used in conditional directives, then on the second pass, process the conditional directive content. This second pass is required as there is no required order for sections within these files, and some PCD values may be defined in sections that follow the use of the PCD in a conditional directive. Macros and PCDs used in conditional directives must not be encapsulated in a conditional comparison (`!if`) directive block. It is permissible to use an undefined macro prior to the definition of the macro, as in the following example.

```
!ifndef F00
DEFINE F00=TRUE
!endif
```

When using PCDs in conditional directive statements or expressions, only the PCD name is required. Do not encapsulate the PCD name in the `"$(" and ")"` required for macro values as shown in the example below.

```
!if ( gTokenSpaceGuid.PcdCName == 1 ) AND ( $(MY_MACRO) == TRUE )
DEFINE F00=TRUE
!endif
```

In the above example, `F00` must not be used in a conditional directive statement.

When testing strings, the strings must to be encapsulated by double quotation marks, as shown in the following example.

```
!if $(SETUP) == "SETUP"
DEFINE F00=TRUE
!endif
```

For backward compatibility, the EDK II build system will process strings that are not encapsulated by the double quotation marks, however this will not be supported in future releases.

Strings can only be compared to strings of a like type (testing an ASCII string against a Unicode format string must fail), numbers can only be compared against numbers and boolean objects can only evaluate to `TRUE` or `FALSE`. See the Operator Precedence table, below for a list of restrictions on comparisons.

Refer to the DSC and FDF file form specifications "Conditional Directive Blocks" section for additional details of how directives must be processed.

8.2.4.6 Expressions

Expressions can be used in conditional directive comparison statements and in value fields for PCDs in the meta-data files.

Expressions follow C relation, equality, logical and bitwise precedence and associativity. Not all C operators are supported, only operators in the following list can be used.

Note: Due to the flexibility of the build system, a new operator, `IN` has been added that can be used to test whether an element is in a list. The format for this is `<Value> IN <MACRO_LIST>`, where `MACRO_LIST` can only be one of `$(ARCH)`, `$(TOOL_CHAIN_TAG)` and `$(TARGET)`.

Use of parenthesis is encouraged to remove ambiguity.

Additional scripting style operators may be used in place of C operators as shown in the table below.

Table 12 Operator Precedence and Supported Operands

Operator	Use with Data Types	Notes	Priority
<code>? :</code>	All	Conditional operator	Lowest
<code>or , OR , </code>	Number, Boolean		
<code>XOR , xor</code>	Number, Boolean		
<code>and , AND , &&</code>	Number, Boolean		
<code> </code>	Number, Boolean	Bitwise OR	
<code>^</code>	Number, Boolean	Bitwise XOR	
<code>&</code>	Number, Boolean	Bitwise AND	
<code>== , != , EQ , NE , IN</code>	All	The <code>IN</code> operator can only be used to test a quoted unary literal string for membership in a list.	
		Strings compared to boolean or numeric values using <code>"=="</code> or <code>"EQ"</code> will always return <code>FALSE</code> , while using the <code>"!="</code> or <code>"NE"</code> operators will always return <code>TRUE</code>	
<code><= , >= , < , > , LE , GE , LT , GT</code>	All		

<< , >>	Number, Boolean		
+, -	Number, Boolean	Cannot be used with strings - the system does not automatically do concatenation.	
*, / , %	Number, Boolean	Cannot be used with strings	
!, not , NOT , ~	Number, Boolean		Highest

The `IN` operator can only be used to test a literal string against elements in the following global variables:

\$(FAMILY)

`$(FAMILY)` is considered a list of families that different `TOOL_CHAIN_TAG` values belong to. The `TOOL_CHAIN_TAG` is defined in the `Conf/target.txt` or on the command-line. The `FAMILY` is associated with the `TOOL_CHAIN_TAG` in the `Conf/tools_def.txt` file (or the `TOOLS_DEF_CONF` file specified in the `Conf/target.txt` file). While different family names can be defined, `ARMGCC` , `GCC` , `INTEL` , `MSFT` , `RVCT` , `RVCTCYGWIN` and `XCODE` have been predefined in the `tools_def.txt` file.

\$(ARCH)

`$(ARCH)` is considered the list of architectures that are to be built, that were specified on the command line or come from the `Conf/target.txt` file.

\$(TOOL_CHAIN_TAG)

`$(TOOL_CHAIN_TAG)` is considered the list of tool chain tag names specified on the command line

\$(TARGET)

`$(TARGET)` is considered the list of target (such as `DEBUG` , `RELEASE` and `NOOPT`) names specified on the command line or come from the `Conf/target.txt` file.

For logical expressions, any non-zero value must be considered `TRUE` .

Invalid expressions must cause a build break with an appropriate error message.

8.2.4.7 DEPEX processing

EDK II modules that have dependencies must use the `[Depex]` section to define the dependency expressions. If the file specified, the complete dependency expression must be defined in the file. For EDK II modules, the build tools will create the complete dependency expression using the information in the `[Depex]` section along with all `[Depex]` sections from the linked in library instances. Depex expressions listed in an INF file's `[Depex]` section are written as in-fix expressions, while the output of the `GenDepex` tool generating the EFI Depex section is a post-fix expression. If an INF file specifies a `DPX_SOURCE` entry in the INF file's `[Defines]` section, the file must also use an in-fix expression. The table below lists the operator precedence for dependency expressions.

Table 13: [Depex] Expression Operator Precedence

Operator	Use with Data Types	Notes	Priority
()	TRUE, FALSE, Expression, GUID, CName or Encapsulation	Encapsulated items are processed from inner-most to outer-most	Highest
NOT	TRUE, FALSE, Expression,	After identifying encapsulation parameters, the NOT operator must take precedence over any other	

NOT	GUID, CName or Encapsulation	items.	
AND , and	TRUE, FALSE, GUID or Encapsulation	These operators are used to create an expression	
OR , or	TRUE, FALSE, GUID or Encapsulation	These operators are used to create an expression	Lowest
SOR	TRUE, FALSE, GUID or Encapsulation	Only valid for DXE and SMM dependency expressions and must be the first statement followed by either a GUID, encapsulation or an expression	
AFTER , BEFORE	GUID	Only valid for DXE and SMM dependency expressions. These must be the only operator in the dependency expression. Only one of these is permitted per dependency expression	

8.2.4.8 PCD Access Methods

A PCD is defined as `TokenSpaceGuidCName.PcdCName`. Each PcdCName must be unique to the Token Space declaring the PCD. The token space is a name space that is unique to the GUID known as the TokenSpaceGuidCName.

The following list defines the five PCD access methods.

- **FeatureFlag PCD** - used in conditional directive statements in code.
- **PatchableInModule PCD** - a volatile variable that can be updated either during a build or by a tool that knows the offset and data size of the variable.
- **FixedAtBuild PCD** - a static variable that is set during the build.
- **Dynamic** - a PCD that will use the standard PcdGet/PcdSet macros; the values for these PCDs are common to all modules in a platform and must be listed (with the storage method and value) in the DSC file.
- **DynamicEx** - a PCD that uses the PcdGetEx/PcdSetEx macros; the values for these PCDs are common to all modules in a platform and must be listed (with the storage method and value) in the DSC file.

How a PCD is coded also makes a difference as to how code is generated by the build system.

FeatureFlag PCDs can only be used as FeatureFlag PCDs; very straight forward. Modules can code the remaining types of PCDs to be either FixedAtBuild (a const which is accessible via a `PcdGet` function), PatchableInModule (which can be modified using an external tool), Dynamic which is accessible via a `PcdSet`, `PcdGet` or `DynamicEx` which uses the token space GUID and token number of a PCD in the `PcdGetEx` and `PcdSetEx` access methods. The build system will record all (FixedAtBuild, PatchableInModule, Dynamic and DynamicEx) PCD data into one of the two PCD databases implemented in EDK II. Dynamic PCD definitions are an amalgamation of FixedAtBuild, PatchableInModule and DynamicEx.

It is recommended that developers code their modules to use the Dynamic form. The Dynamic form allows the platform integrator to select how they want to use the PCD; selecting how they want to expose the data; FixedAtBuild, PatchableInModule, Dynamic or (PI compliant) DynamicEx. If the platform integrator selects the Dynamic or DynamicEx form for any PCD, then the platform must also contain a PEI and/or DXE PCD driver to maintain a volatile database of values that can be set or retrieved.

Dynamic and DynamicEx PCD values are common to all modules in a platform and the storage mechanism for these PCDs must be defined by the platform developer, so the PCD values must be specified in the DSC file under a section that specifies the storage mechanism (Default, VPD or HII).

The DynamicEx PCDs correspond to the PI Specification, while the other PCD forms are associated with EDK II.

Modules that will be distributed in binary form must use either PatchableInModule or DynamicEx PCDs.

PatchableInModule PCDs also require the build system to generate a map file for each module that is using PatchableInModule PCDs. This map file contains the offset from the start of the file to the location of the first byte of the PCD.

If the Platform Integrator does not specify the format, and all of the INF files that use the PCD state that they have been coded to use the Dynamic PCD form, the tool must examine the methods available for a PCD that have been declared in the DEC file. The build system uses the following priority for the default method.

- If the PCD is listed in the DEC's `PcdsFixedAtBuild` , then use FixedAtBuild, otherwise,
- If the PCD is listed in `PcdsPatchableInModule` , then use PatchableInModule.
- If the PCD is not listed in either of the previous two sections, and it is listed in a `PcdsDynamicEx` section, then use DynamicEx.
- If not listed in any of the previous sections, and the PCD is listed in the `PcdsDynamic` section, then use Dynamic.

If the Platform Integrator does not specify the format, and the PCD state is in Platform DSC file `[Components]` section module scoped `<Pcd*>` section, then the PCD used in `[Components]` section module or its linked library INF files will use the module scoped `<Pcd*>` type as default type.

Build tools are required to process PCD values for `VOID*` PCDs into byte arrays, C format GUIDs or as C format strings (either ASCII or `[L]"string"`) prior to autogenerating the code.

PCD values stored in VPD regions are processed prior to completing the final PCD parsing. Refer to Section 8.4 for additional rules for processing PCDs to create a platform scoped PCD Database.

8.2.4.9 Precedence of PCD Values

The values that are assigned to individual PCDs required by a build may come from different locations and different meta-data files. The following provides the precedence (high to low) to assign a value to a PCD.

- Command-line, `--pcd` flags (left most has higher priority)
- DSC file, Component INF `<Pcd*>` section statements
- FDF file, grammar describing automatic assignment of PCD values
- FDF file, SET statements within a section
- FDF file, SET statement in the `[Defines]` section
- DSC file, global `[Pcd*]` sections
- INF file, PCD sections, Default Values
- DEC file, PCD sections, Default Values

In addition to the above precedence rules, PCDs set in sections with architectural modifiers take precedence over PCD sections that are common to all architectures.

When listed in the same section. If listed multiple times, the last one will be used. If PCD field value is listed, it will override PCD value even if PCD value is after PCD field value.

A PCD value set on the command-line has the highest precedence. It overrides all instances of the PCD value specified in the DSC or FDF file. The following is the syntax to override the value of a PCD on the command line, please refer to appendix D.4 Usage for detail EBNF format.

```
--pcd [<TokenSpaceGuidName>].<PcdName>[.Field]=<Value>
```

<Value> supports the following syntax:

- ASCII string value for a PCD

```
--pcd [<TokenSpaceGuidName>.<PcdCName>[.Field]="String" --pcd [<TokenSpaceGuidName>.<PcdCName>[.Field]='String'"
```

- Unicode string value for a PCD

```
--pcd [<TokenSpaceGuidName>.<PcdCName>[.Field]=L"String" --pcd [<TokenSpaceGuidName>.<PcdCName>[.Field]=L"String'"
```

- Byte array value for a PCD

```
--pcd [<TokenSpaceGuidName>.<PcdCName>[.Field]= H"{0x1, 0x2}"
```

Note: The EDK II meta-data specs have changed to permit a PCD entry (or any other entry) to be listed only one time per section.

If the maximum size of a `VOID*` PCD is not specified in the DSC file, then the maximum size is calculated based on the largest size of 1) the string or array in the DSC file, 2) the string or array in the INF file and 3) the string or array in the DEC file. If the value is a quoted text string, the size of the string will be incremented by one to handle string termination. If the quoted string is preceded by L, as in `L"This is a string"`, then the size of the string will be incremented by two to handle unicode string termination. If the value is a byte array, then the size of the byte array is not modified. If the value is a single quoted string, as in `'string'` or `L'string'`, the size of the string doesn't need to include string null termination character.

For example, if the string in the DSC file is `L"DSC Length"`, the INF file has `L"Module Length"` and the DEC file declares the default as `L"Length"`, then the maximum size that will be allocated for this PCD will be 28 bytes (`L"Module Length"` 26 bytes, 2 bytes for null termination character).

`VOID*` PCDs must be byte aligned if the value is an ASCII string, two-byte aligned if the value is a Unicode string or 8-byte aligned in the value is a byte array.

8.2.4.10 Section Handling

The INF and DEC file parsing routines must process the sections so that common architecture sections are logically merged with the architecturally specific sections. The architectural sections need to be processed so that they are logically after the common section. It is recommended that EDK II developers use a logical ordering of the sections.

Other section modifiers must also be logically appended to the merged sections (for INFs that have architectural and common architecture sections) after the merge.

For `[BuildOptions]` sections in the INF and DSC file, the entries with a common left side (of the "=") will be either appended or replace previous entries based on the "==" replace or "=" append assignment character sequence.

```
Common Section + Architectural Section + Common Section w/extra Modifier + Architectural Section w/extra Modifier
```

Example

```
[BuildOptions.Common]
MSFT:*_*_*_CC_FLAGS = /nologo

[BuildOptions.IA32]
MSFT:*_*_IA32_CC_FLAGS = /D EFI32
```

For IA32 architecture builds of an EDK II INF file would logically be:

```
MSFT:*_*_IA32_CC_FLAGS = /noLogo /D EFI32
```

8.2.4.11 PCD INFO Generation

The UEFI Platform Initialization specification defines a PEIM and Protocol that can retrieve the PCD Token number and the PCD Token Name (the PCD C Name) information from the PCD Database. In order to support these modules, a `PCD_INFO_GENERATION` entry in the DSC file's `[Defines]` section is used to enable generate the PCD Database with the required information (normally, only the PCD Token number is available). This feature does increase the size of the PCD drivers that contain the PCD database, so this capability is added as an optional feature rather than always generating the content.

If the `[Defines]` section has the `PCD_VAR_CHECK_GENERATION` entry set to TRUE, then a binary file will be created in the FV directory for Dynamic and DynamicEx PCD HII Variable checking.

8.2.4.12 Pre Build Processing

The DSC file is parsed after the tool meta-data files. If the `[Defines]` section of the DSC file contains a `PREBUILD = entry` statement, processing of the DSC file is suspended and the script specified in the `PREBUILD` statement is executed. The entry of `PREBUILD` support multiple arguments. And Tool will convert arguments that are `WORKSPACE` or `PACKAGES_PATH` relative paths to absolute paths. If the script file is not found, the **build** command exits with an appropriate error message. If the script fails, it must terminate with a non-zero exit code and the **build** command terminates with the exit value from the pre-build script. The script is required to generate error messages that provide the reason for the termination.

All of the command line options passed into the **build** command are also passed into the script along with the options for `TARGET`, `ARCH`, `TOOL_CHAIN_TAG`, `ACTIVE_PLATFORM`, `Conf Directory`, and `build target`.

If the script terminates successfully (exit value of 0), parsing of the DSC file continues, and build tools may retrieve environment variables that have been updated by the script.

Note: This entry may be wrapped in a conditional directive that uses the value of the `TOOL_CHAIN_TAG` determined earlier. Using a MACRO value other than `$(TOOL_CHAIN_TAG)` is prohibited, as the DSC file has not been processed at the time the ENTRY was found.

Note: Quotes are needed when the script's additional options are present. Quotes are also required if the path to the pre-build command contains space or special characters. Quotes may be used for arguments that have spaces or special characters.

8.2.4.13 NMAKE Command line limitation handling

`NMAKE` is limited to command-line length of 4096 characters. Due to the large number of `/I` directives specified on command line (one per include directory), the path length of `WORKSPACE` is multiplied by the number of `/I` directives and can exceed this command-line length limitation. When this issue occurs, the build tools pass the command line options via a response file instead of directly on the command line. The contents of the response file is combination of `FLAGS` options and `INC` options. If a build fails, the build tools print the response file's file location and the contents of the response file.

The **build** command supports the options `-l` and `--cmd-len` to set the maximum command line length. The default value is 4096.

Note: The following `FLAGS` options are included in the response file: `PP_FLAGS` , `CC_FLAGS` , `VFRPP_FLAGS` , `APP_FLAGS` , `ASLPP_FLAGS` , `ASLCC_FLAGS` , and `ASM_FLAGS` .

8.2.4.14 Build with Binary Cache

build tool provides three new options for binary cache feature. `--hash` enables hash-based caching during build process. When `--hash` is enabled, build tool will base on the module hash value to do the incremental build, without `--hash`, build tool will base on the timestamp to do the incremental build. `--hash` option use md5 method to get every hash value, DSC/FDF, `tools_def.txt`, `build_rule.txt` and build command are calculated as global hash value, Package DEC and its include header files are calculated as package hash value, Module source files and its INF file are calculated as module hash value. Library hash value will combine the global hash value and its dependent package hash value. Driver hash value will combine the global hash value, its dependent package hash value and its linked library hash value. When `--hash` and `--binary-destination` are specified, build tool will copy each module's "As Built" inf file, binary files that in "As built" inf file's [Binaries] section and hash value file into the directory specified by `binary-destination` at the build phase. When `--hash` and `--binary-source` are specified, build tool will try to get the binary files from the binary source directory at the build phase. If the cached binary has the same hash value, it will be directly used. Otherwise, build tool will compile the source files and generate the binary files.

8.2.4.15 !error Statement

The DSC and FDF file can use `!error` statement. The argument of this statement is an error message, it causes build tool to stop at the location where the statement is encountered and error message following the `!error` statement is output as a message.

The keyword `!error` is case-insensitive.

8.2.5 Post processing

Once all files are parsed, the build tools will do following work for each EDK II module:

- Resolve the library classes to library instances, inherit and resolve library classes from them recursively, until no new library instances are found.
- Re-order the library instances according to the consuming relationship and their constructors. For each EDK II module, the tools must select one library instance per required library class (with the exception of the NULL library class keyword) using the following precedence (high to low):
 - The DSC file's component INF scoping `<LibraryClasses>` section
 - The DSC file's `[LibraryClasses.arch.module_type]` section tags with both architecture and module type modifiers
 - The DSC file's common arch with a module type modifier,

`[LibraryClasses.common.module_type]`
 - DSC file's architecture specific modifier only `[LibraryClasses.arch]`
 - The DSC file's common `[LibraryClasses]` section

Note: For modules of type **USER_DEFINED_**, if a `NULL` library class is required, the library instance should be listed in the INF scoping `<LibraryClasses>` section of the component.

- Inherit GUIDs, Protocols and PPIs from all library instances obtained above, and determine values or type of them. The value of a GUID, Protocol or PPI is defined in DEC file.

Note: If GUID, Protocol or PPI is listed in a DEC file, where the `Private` modifier is used in the section tag (`[Guids.common.Private]` for example), only modules within the package are permitted to use the GUID, Protocol or PPI. If a module or library instance outside of the package attempts to use the item, the build must fail with an appropriate error message.

- Inherit PCDs from all library instances obtained above and determine values and type. The value and type of a PCD are obtained from a DSC file, INF file or DEC file if it cannot be found in the DSC or INF file. For each EDK II module, the tools must obtain unique PCD values using the following precedence (high to low):
 - Command-line, `--pcd` flags (left most has higher priority)
 - The DSC file's component INF scoping `<Pcds*>` sections
 - FDF file, grammar describing automatic assignment of PCD values
 - FDF file, SET statements within a section
 - FDF file, SET statement in the `[Defines]` section
 - The DSC file's `[Pcd*.arch.skuid]` sections
 - The DSC file's `[Pcd*.common.skuid]` sections
 - The DSC file's `[Pcd*.arch]` sections
 - The DSC file's `[Pcd*.common]` sections
 - The INF file's PCD sections
 - The DEC file's PCD sections

Note: Values of PCDs using the `FeatureFlag`, `PatchableInModule` and `FixedAtBuild` access methods set for this INF file are local to the INF file and do not pertain to any other INF files. Dynamic and `DynamicEx` access method PCD values are global to a platform and should not be overridden by specifying them here. If, however, the dynamic PCDs are only valid for this INF, it is permissible to set them here.

- Inherit library instance dependency (`[Depex]` sections) expressions if a module does not list a separate dependency file.
- If the DSC file contains PCD sections for `DynamicVpd` or `DynamicExVpd` access methods, special processing is required. Refer to the appendix "VPD PCD Intermediate Files" for additional details.
- Determine if a module has specified Unicode file names, designated by the `.uni` file extension, in the INF file.
- Determine if a module has specified Image definition file names, designated by the `.idf` file extension, in the INF file.
- Any Visual Forms Representation (`.vfr`) files found during the pre-processing steps will be processed during the \$(MAKE) stage. Refer to the "VFR Programming Language" document for additional details.
- Generate the Build Output Directory structure
- Generate the code files
- Generate the `Makefiles`

- Generate the "AsBuilt" INF files

8.3 Auto-generated code

The section covers, in sequence, the processes used to generate code files that will be used during the build.

8.3.1 AutoGen Stage File Extensions

The following table provides the extension and a description of files processed during the AutoGen stage of the build. The `build_rule.txt` file describes the processing rules for generating the Makefiles for the \$(MAKE) stage.

Table 14 AutoGen Stage Input File Extensions

Extension	Description	File Format
.c, .cpp	C code files	ASCII Text, DOS EOL
.h	C header files	ASCII Text, DOS EOL
.asm	32 and 64-bit Windows assembly files	ASCII Text, DOS EOL
.s	32 and 64-bit GCC assembly files	ASCII Text, DOS EOL
.S	IPF GCC and Windows assembly files	ASCII Text, DOS EOL
.nasm	32 and 64-bit NASM assembly files	ASCII Text, DOS EOL
.i	IPF Assembly include files	ASCII Text, DOS EOL
.vfr	Visual Forms Representation files	ASCII Text, DOS EOL
.uni	HII Unicode string files	UCS-2 Characters
.idf	HII Image Definition files	ASCII Text, DOS EOL
.asl	C formatted ACPI code files - these files are processed independent from the C code files	ASCII Text, DOS EOL
.asi	ACPI Header Files	ASCII Text, DOS EOL
.aslc	C formatted ACPI table files - these files are processed independent from the C code files	ASCII Text, DOS EOL
.txt	Microcode text files	ASCII Text, DOS EOL
.map	VPD tool intermediate file	ASCII Text, DOS EOL
.bin	Binary files	Binary
.bmp	Logo files used in the ImageGen stage	Binary
.ui	Unicode User Interface files	UCS-2 Characters

.ver	Unicode Version files	UCS-2 Characters
------	-----------------------	------------------

8.3.2 Dependency expression file

The dependency expression file (`.depex`) is generated from the `[Depex]` section in module's INF file, if the section presents, or `.dxs` file if `DPX_SOURCE` definition is found in INF file. If both the `DPX_SOURCE` definition and `[Depex]` section content is present, the content in the file specified in the `DPX_SOURCE` definition is used and the `[Depex]` section content will be ignored. The GUID used in `[Depex]` section must be the GUID C name.

First, the GUID C name in the dependency expression string will be converted into its value in C structure format. Then the expression string will be converted into postfix notation. Before saving to a file, the operator and GUID value in the postfix notation will be converted to their binary value.

Dependency expression sections listed in an INF file may be scoped via feature flag expressions (logical expressions which typically utilize PCDs using `FeatureFlag` or `FixedAtBuild` access methods). It is the module writer's responsibility to ensure the different sections are mutually exclusive. It is the platform integrator's responsibility to ensure that they do not override this exclusivity.

For example, the following dependency expression

```
NOT (gEfiHiiDatabaseProtocolGuid AND gEfiHiiStringProtocolGuid) OR gPcdProtocolGuid
```

will be converted to

```
include_statement($(MODULE_BUILD_DIR)/OUTPUT/$(BASE_NAME).dxs, "
// PUSH
02
// gEfiHiiDatabaseProtocolGuid
72 c1 9f ef b2 a1 93 46 b3 27 6d 32 fc 41 60 42
// PUSH
02
// gEfiHiiStringProtocolGuid
74 69 d9 0f aa 23 dc 4c b9 cb 98 d1 77 50 32 2a
// AND
03
// NOT
05
// PUSH
02
// gPcdProtocolGuid
06 40 b3 11 5b d8 0a 4d a2 90 d5 a5 71 31 0e f7
// OR
04
// END
08
");
```

The binary dependency expression file will be generated in `$(MODULE_BUILD_DIR)/OUTPUT` with `.depex` file extension.

8.3.2.1 Guidelines

Use of a separate file for describing the dependencies is discouraged. Grammar of the INF, DSC and FDF files permit specifying the dependency expressions. Libraries may also have a dependency, `[Depex]`, section. These dependencies must be appended to the module's `DEPEX` sections unless the module includes a `depex (.dxs)` file - even if the module does not contain a `[Depex]` section. When a developer chooses to write the `.dxs` file, the developer is responsible for specifying all dependencies in the `.dxs` file.

Libraries that are linked to a UEFI DRIVER may have `DEPEX` sections. There are three 'rules' for the tools.

- Tools are coded so that for a given module the `[Depex]` sections of all linked-in library instances are logically AND 'd with the `DEPEX` section of the module
 - If no `DEPEX` section is specified in the module, then only the library instances `DEPEX` sections are logically AND 'd to create the `DEPEX` section for the module
- Tools are also coded to ignore the depex sections of libraries that are linked to `UEFI_DRIVER` or PCI Option ROM code
- The tools will break the build if one module, using one of the noted module types, contains a depex section in the INF file.

8.3.3 VFR

The EDK II build system provides tools for processing formatted Unicode files and Visual Forms Representation (VFR) files in order to create the IFR files. Refer to the EDK II User's Manual for more information regarding the use of the Unicode and VFR files. Refer to the VfrCompiler description and the VFR Programming Language document for more detailed information on the provided implementation. Additionally, the EDK II build AutoGen tools are used to process Unicode files listed in a module's INF file. Also note that the IFR code is not compatible - UFI compliant IFR code is different from the IFR code defined by early Intel Framework documents.

8.3.3.1 Reference Implementation: Compatibility

The EDK II Vfr compiler tools can process EDK II VFR and Unicode files and to generate UEFI/PI compliant IFR files. EDK II Unicode files can use the UEFI defined Unicode extended grammar. Table 15 shows the compatibility matrix.

Table 15 VFR Compatibility Matrix

Code	non-UEFI Compliant VFR Tools	UEFI Compliant VFR Tools
pre-UEFI 2.1 Unicode	Yes	Yes
pre-UEFI 2.1 VFR Source	Yes	Yes
pre-UEFI 2.1 IFR - binaries	Yes	No
UEFI 2.1 Unicode	No	Yes
UEFI 2.1 Vfr	No	Yes
UEFI 2.1 IFR - binaries	No	Yes

8.3.4 HII String Pack

The human-readable HII string pack data consists of UCS-2 characters in .uni files. The build tools will do following steps to convert the strings information into HII string pack data structure.

- The build tools will get all the string IDs, the associated string and language code from the `.uni` files. Note that the DSC file or options on the command-line may be used to filter the languages used for generating the AutoGen code. The `RFC_LANGUAGES` is a semi-colon separated, doubled quoted string of RFC 4646 language codes, while the `ISO_LANGUAGES` (for EDK components only) is a nonseparated double quoted string of three character ISO 639-2 language codes.
- For EDK II modules, their Unicode files must use RFC 4646 language codes. If an EDK II module's Unicode file contains a three character ISO 639-2 language code, the build will break with an appropriate warning message.

Note: Tools must not refactor the EDK component ISO 639-2 language codes to RFC 4646 language codes, as the DXE drivers are responsible for handling the different language code formats.

- Search all source files in the include path of the module to find out which string IDs are used. Macros will be generated in `AutoGen.h` for the string IDs used. Those string IDs not used will be generated but commented out. They are just for debug purposes. For example:

```
include_statement (AutoGen.h, "
//
//Unicode String ID
//

// #define $LANGUAGE_NAME 0x0000 // not referenced
// #define $PRINTABLE_LANGUAGE_NAME 0x0001 //not referenced
#define STR_BOOT_FAILED 0x0002
#define STR_BOOT_SUCCEEDED 0x0003
#define STR_PERFORM_MEM_TEST 0x0004
// #define STR_INTERNAL_EFI_SHELL 0x009E // not referenced
// #define STR_LEGACY_BOOT_A 0x009F // not referenced
// #define STR_PROCESSED_ALL_BOOT_OPTIONS 0x00A0 // not referenced
");
```

- The font attribute specifies the default font that will be used for the characters in string. If `#font` is not specified, then the default font identifier will be used.
- If the `#font` attribute appears before the first `#language` identifier, then it applies to all characters for all languages. If the `#font` attribute appears after a `#language` identifier, it applies only to the string characters in that language. It is permissible for `#font` to appear in more than one place, in which case the language-specific font identifier will have priority.
- The HII string package data will be generated in `AutoGen.c` in the form of a data array, with array name `<ModuleBaseName> Strings`. For example:

```
include_statement (AutoGen.c, "
//
//Unicode String Pack Definition
//
unsigned char PlatformBdsDxeStrings[] = {
// Start of string definitions for fra
0x20, 0x1A, 0x00, 0x00, 0x02, 0x00, 0x8E, 0x02,
0x00, 0x00, 0x96, 0x02, 0x00, 0x00, 0x9E, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
// offset 0x16
0x8E, 0x02, 0x00, 0x00, // offset to string $LANGUAGE_NAME (0x0000)
0x96, 0x02, 0x00, 0x00, // offset to string
                        //$PRINTABLE_LANGUAGE_NAME (0x0001)
...
...
// string $LANGUAGE_NAME offset 0x0000028E
0x66, 0x00, 0x72, 0x00, 0x61, 0x00, 0x00, 0x00,
// string $PRINTABLE_LANGUAGE_NAME offset 0x00000296
0x46, 0x00, 0x72, 0x00, 0x61, 0x00, 0x6E, 0x00,
0xE7, 0x00, 0x61, 0x00, 0x69, 0x00, 0x73, 0x00,
0x00, 0x00,
...
...
// strings terminator pack
0x00, 0x00, 0x00, 0x00, 0x02, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
};
");
```

8.3.4.1 More than One Unicode File

If more than one Unicode file is required by a module, the rules for including these files are as follows. If one Unicode file uses a `#include` statement to include other Unicode files, these secondary Unicode files must also be listed in the INF file's [Sources] section.

8.3.5 HII Image Pack

The HII Image package data is stored in `.idf` files. The build tools perform the following steps to convert the image information into an HII Image package data structure.

- The build tools retrieve all the image IDs, the optional `TRANSPARENT` setting and the associated image file name from the `.idf` files. The `TRANSPARENT` setting is optional. If it is specified, build tools apply the `TRANS` image block type to the input image file. The UEFI Specification does not define the `TRANS` block type for JPG or PNG images. The `TRANSPARENT` setting is ignored for JPG and PNG images. The image file name should be listed in the [Sources] section of the INF file, and the extension of the image file must be one of `.bmp`, `.jpg`, or `.png`. The extension is case insensitive.
- Search all source files in the include path of the module to find out which image IDs are used. Macros are generated in `AutoGen.h` for the image IDs used. For example:

```
include_statement(AutoGen.h, "
//
//Image ID
//
#define IMG_FULL_LOGO  0x0001
#define IMG_OEM_LOGO   0x0002
");
```

- The HII Image package data is generated in `<ModuleBaseName>Idf.hpk` or in `AutoGen.c` in the form of a data array, with array name `<ModuleBaseName>Images`. For example:

```
include_statement(AutoGen.c, "
//
//Image Pack Definition
//
unsigned char HelloWorldImages[] = {
// STRGATHER_OUTPUT_HEADER
0xD9, 0xCA, 0x01, 0x00,
// Image PACKAGE HEADER
0xD5, 0xCA, 0x01, 0x06, 0x0C, 0x00, 0x00, 0x00, 0x97, 0xC7, 0x01, 0x00,
// Image DATA
// 0x0001: IMG_FULL_LOGO: 0x0001
0x12, 0x01, 0x90, 0x01, 0xDC, 0x00, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF,
...
...
// 0x0002: IMG_OEM_LOGO: 0x0002
0x14, 0x02, 0x25, 0x01, 0xDC, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
...
...
// End of the Image Info
0x00,
// Palette Header
0x03, 0x00,
// Palette Data
// 0x0001: IMG_FULL_LOGO: 0x0001
0x00, 0x00, 0x00, 0x00, 0x00, 0x80, 0x00, 0x80, 0x00, 0x00, 0x80, 0x80, 0x80, 0x00, 0x00, 0x80,
0x00, 0x80, 0x80, 0x80, 0x00, 0x80, 0x80, 0x80, 0xC0, 0xC0, 0xC0, 0x00, 0x00, 0xFF, 0x00, 0xFF,
...
...
// 0x0002: IMG_OEM_LOGO: 0x0002
0x00, 0x00, 0x00, 0x00, 0x00, 0x80, 0x00, 0x80, 0x00, 0x00, 0x80, 0x80,
...
...
};
```

```
};
");
```

- If more than one image definition file is required by a module, the build tools combine the images from the multiple `.idf` files into a single HII Image Pack.

8.3.6 AutoGen.h file

The code generated in AutoGen.h includes:

- Prototypes of constructor and destructor from the library instances the module will link against
- Prototypes of entry and unload image entry points
- Global variable definitions for GUID/Protocol/PPI values used as well as extern definitions
- Global variable definitions and the database of PCDs used
- Unicode string database definitions.
- Image package database definitions.

The file will contain:

8.3.6.1 Header prologue

The macro name is composed with GUID value of INF file.

```
include_statement (AutoGen.h, "
  #ifndef _AUTOGENH_6987936E_ED34_44db_AE97_1FA5E4ED2116
  #define _AUTOGENH_6987936E_ED34_44db_AE97_1FA5E4ED2116

  #ifdef __cplusplus
  extern "C" {
  #endif
  ");
```

8.3.6.2 Header file inclusion.

Only one header file is included.

```
include_statement (AutoGen.h, "
  #include <Base.h>
  ");
```

8.3.6.3 Caller ID GUID definition.

The GUID value is the same as INF file GUID. The macro, `EFI_CALLER_ID_GUID`, is generated only for non - library module.

```
include_statement (AutoGen.h, "
  extern GUID gEfiCallerIdGuid;
  // following definition is not needed for library module
  #define EFI_CALLER_ID_GUID \
  { 0x6987936E, 0xED34, 0x44db, { 0xAE, 0x97, 0x1F, 0xA5, 0xE4, 0xED,
    0x21, 0x16 } }
  ");
```

8.3.6.4 PCD definitions

There are differences in the generated code for library and non-library modules, which are illustrated in pseudo-code below.

8.3.6.4.1 Non-library Module

```

include_statement(AutoGen.h, "
#define _PCD_TOKEN_<TokenCName> <TokenNumber>
");

If ((PCD_type == FIXED_AT_BUILD) || (PCD_type == FEATURE_FLAG)) {
  If ((DatumType == 'VOID*') &&
      (
        (PcdValue == array) ||
        (PcdValue == C_FormatGuid) ||
        (PcdValue == C_String)
      )
  ) {
    include_statement (AutoGen.h, "
      #define _PCD_PATCHABLE_<TokenCName>_SIZE <MaxDatumSize>"
    );
  }
  include_statement (AutoGen.h, "
#define _PCD_VALUE_<TokenCName> <PcdValue>
extern const <DatumType> _gPcd_FixedAtBuild_<TokenCName>;
#define _PCD_GET_MODE_<DatumSize>_<TokenCName> _gPcd_FixedAtBuild_<TokenCName>
");
}
If (PCD_type == PATCHABLE_IN_MODULE) {

  If ((DatumType == 'VOID*') &&
      (
        (PcdValue == array) ||
        (PcdValue == C_FormatGuid) ||
        (PcdValue == C_String)
      )
  ) {
    include_statement (AutoGen.h, "
      #define _PCD_PATCHABLE_<TokenCName>_SIZE <MaxDatumSize>
    ");
  }

  include_statement (AutoGen.h, "
    #define _PCD_VALUE_<TokenCName> <PcdValue>
    extern <DatumType> _gPcd_BinaryPatch_<TokenCName>;
    #define _PCD_GET_MODE_<DatumSize>_<TokenCName> _gPcd_BinaryPatch_<TokenCName>
  ");

  If ((DatumType == 'VOID*') &&
      (
        (PcdValue == array) ||
        (PcdValue == C_FormatGuid) ||
        (PcdValue == C_String)
      )
  ) {
    include_statement (AutoGen.h, "
      #define _PCD_SET_MODE_<DatumSize>_<TokenCName>(SizeOfBuffer,Buffer) \
        LibPatchPcdSetPtr (_gPcd_BinaryPatch_<TokenCName>, \
          (UINTN)_PCD_PATCHABLE_<TokenCName>_SIZE, \
          (SizeOfBuffer), \
          (Buffer) \
        )
    ");
  }
  Else {
    include_statement (AutoGen.h, "
      #define _PCD_SET_MODE_<DatumSize>_<TokenCName>(Value) \
        (_gPcd_BinaryPatch_<TokenCName> = (Value))
    ");
  }
}
If (PCD_type == DYNAMIC) {
  include_statement (AutoGen.h, "
    #define _PCD_GET_MODE_<DatumSize>_<TokenCName> \
      LibPcdGet<DatumSize>(_PCD_TOKEN_<PcdTokenCName>)
  ");
}

```

```

If (DatumType == 'VOID*') {
    include_statement (AutoGen.h, "
        #define _PCD_SET_MODE_<DatumSize>_<TokenCName>(SizeOfBuffer, Buffer) \
        LibPcdSet<DatumSize> ( \
            _gPcd_BinaryPatch_<TokenCName>, \
            (SizeOfBuffer), \
            (Buffer) \
        )
    ");
} Else {
    include_statement (AutoGen.h, "
        #define _PCD_SET_MODE_<DatumSize>_<TokenCName>(Value) \
        LibPcdSet<DatumSize>(_gPcd_BinaryPatch_<TokenCName>, (Value))
    ");
}
}
If (PCD_type == DYNAMIC_EX) {
    include_statement (AutoGen.h, "
        #define _PCD_GET_MODE_<DatumSize>_<TokenCName> \
        LibPcdGetEx<DatumSize>(&<TokenSpaceGuidCName>, \
            _PCD_TOKEN_<PcdTokenCName> \
        )
    ");
}
If (DatumType == 'VOID*') {
    include_statement (AutoGen.h, "
        #define _PCD_SET_MODE_<DatumSize>_<TokenCName>( \
            SizeOfBuffer, Buffer) \
        LibPcdSetEx<DatumSize>(&<TokenSpaceGuidCName>, \
            _gPcd_BinaryPatch_<TokenCName>, \
            (SizeOfBuffer), \
            (Buffer) \
        )
    ");
} Else {
    include_statement (AutoGen.h, "
        #define _PCD_SET_MODE_<DatumSize>_<TokenCName>(Value) \
        LibPcdSetEx<DatumSize>(&<TokenSpaceGuidCName>, \
            _gPcd_BinaryPatch_<TokenCName>, \
            (Value) \
        )
    ");
}
}
}

```

8.3.6.4.2 Library Module

```

nclude_statement(AutoGen.h, "
    #define _PCD_TOKEN_<TokenCName> <TokenNumber>
");

If ((PCD_TYPE == FIXED_AT_BUILD) &&
    (ALL_MODULES_LINKED_W_THIS_LIB_USE_PCD_TYPE == FIXED_AT_BUILD) &&
    (ALL_MODULES_LINKED_W_THIS_LIB_USE_PCD_VALUES == <ThisPcdValue>)) {
    #define _PCD_VALUE_<TokenCName> <PcdValue>
}

If ((PCD_type == FIXED_AT_BUILD) || (PCD_type == FEATURE_FLAG)) {
    include_statement (AutoGen.h, "
        extern const <DatumType> _gPcd_FixedAtBuild_<TokenCName>;

        #define _PCD_GET_MODE_<DatumSize>_<TokenCName> \
        _gPcd_FixedAtBuild_<TokenCName>
    ");
}

If (PCD_type == PATCHABLE_IN_MODULE) {
    include_statement (AutoGen.h, "
        extern <DatumType> _gPcd_BinaryPatch_<TokenCName>;

        #define _PCD_GET_MODE_<DatumSize>_<TokenCName> \

```

```

    _gPcd_BinaryPatch_<TokenCName>
#define _PCD_SET_MODE_<DatumSize>_<TokenCName>(Value) \
    (_gPcd_BinaryPatch_<TokenCName> = (Value))

");
}

If (PCD_type == DYNAMIC) {
    include_statement (AutoGen.h, "
        #define _PCD_GET_MODE_<DatumSize>_<TokenCName> \
            LibPcdGet<DatumSize>(_PCD_TOKEN_<PcdTokenCName>)
    ");

    If (DatumType == 'VOID*') {
        include_statement (AutoGen.h, "
            #define _PCD_SET_MODE_<DatumSize>_<TokenCName>( \
                SizeOfBuffer, Buffer) \
                LibPcdSet<DatumSize>(_gPcd_BinaryPatch_<TokenCName>, \
                    (SizeOfBuffer), \
                    (Buffer) \
                )
        ");
    } Else {
        include_statement (AutoGen.h, "
            #define _PCD_SET_MODE_<DatumSize>_<TokenCName>(Value) \
                LibPcdSet<DatumSize>(_gPcd_BinaryPatch_<TokenCName>, (Value))
        ");
    }
}

If (PCD_type == DYNAMIC_EX) {
    include_statement (AutoGen.h, "
        #define _PCD_GET_MODE_<DatumSize>_<TokenCName> \
            LibPcdGetEx<DatumSize>( &<TokenSpaceGuidCName>, \
                _PCD_TOKEN_<PcdTokenCName> )
    ");

    If (DatumType == 'VOID*') {
        include_statement (AutoGen.h, "
            #define _PCD_SET_MODE_<DatumSize>_<TokenCName>( \
                SizeOfBuffer, Buffer ) \
                LibPcdSetEx<DatumSize>(&<TokenSpaceGuidCName>, \
                    _gPcd_BinaryPatch_<TokenCName>, \
                    (SizeOfBuffer), \
                    (Buffer) \
                )
        ");
    } Else {
        include_statement (AutoGen.h, "
            #define _PCD_SET_MODE_<DatumSize>_<TokenCName>(Value) \
                LibPcdSetEx<DatumSize>(&<TokenSpaceGuidCName>, \
                    _gPcd_BinaryPatch_<TokenCName>, \
                    (Value) \
                )
        ");
    }
}
}

```

8.3.6.4.3 HII string pack definitions,

These are generated only if `.uni` files are found. For details, please refer to section 7.3.2.

```

include_statement (AutoGen.h, "
    //
    //Unicode String ID
    //
    // #define $LANGUAGE_NAME 0x0000 // not referenced
    // #define $PRINTABLE_LANGUAGE_NAME 0x0001 // not referenced
    #define STR_MISC_BASE_BOARD_MANUFACTURER 0x0002
    #define STR_MISC_BASE_BOARD_PRODUCT_NAME 0x0003
    #define STR_MISC_BASE_BOARD_VERSION 0x0004

```

```
// ...
// ...
// ...
extern unsigned char MiscSubclassStrings[];

#define STRING_ARRAY_NAME MiscSubclassStrings

");
```

8.3.6.4.4 HII image pack definitions

These are generated only if `.idf` files are found.

```
include_statement(AutoGen.h, "
//
//Image ID
//
#define IMG_FULL_LOGO 0x0001
#define IMG_OEM_LOGO 0x0002

extern unsigned char HelloWorldImages[];

#define IMAGE_ARRAY_NAME HelloWorldImages
");
```

8.3.6.5 ProcessLibraryConstructorList function declaration

Entry point functions of non-library SEC modules do not have standardized parameter lists, therefore `edk2` does not offer an `MdePkg/Include/Library/*EntryPoint.h` header file for such modules. Consequently, `ProcessLibraryConstructorList` is not declared for them either, despite the fact that they must call `ProcessLibraryConstructorList` explicitly. Historically, each such module has declared `ProcessLibraryConstructorList` internally for itself, compensating for the lack of a central declaration.

The `ProcessLibraryConstructorList` function is declared in `AutoGen.h` for non-library SEC modules, if `INF_VERSION` in the module's INF file is greater than or equal to `1.30`.

```
include_statement (AutoGen.h, "
VOID
EFIAPI
ProcessLibraryConstructorList (
    VOID
);
");
```

8.3.6.6 AutoGen Epilogue

```
#ifdef __cplusplus
}
#endif

#endif
```

8.3.7 AutoGen.c file

The code generated in `AutoGen.c` includes:

- Calling of constructor and destructor of library instances against which the module will link
- The module load and unload entry points
- Global variables for GUID/Protocol/PPIs value used, global variables and database for PCDs used
- Unicode string pack definition.
- Image pack definition.

AutoGen.c file is only generated for EDK II non-library modules. The following sections identify what lines of information are included in the file as well as pseudo-code to references on to how a variable () might be generated.

The file will contain:

8.3.7.1 Header files inclusion.

Which files are included is determined by module type.

```
Switch MODULE_TYPE {
  case "BASE":
  case "HOST_APPLICATION":
  case "USER_DEFINED":
    include_statement (AutoGen.c, "
      #include <Base.h>

    );
    break;
  case "SEC":
  case "PEI_CORE":
  case "PEIM":
    include_statement (AutoGen.c, "
      #include <PiPei.h>
      #include <Library/DebugLib.h>

    );
    break;
  case "DXE_CORE":
    include_statement (AutoGen.c, "
      #include <PiDxe.h>
      #include <Library/DebugLib.h>

    );
    break;
  case "DXE_DRIVER":
  case "DXE_SMM_DRIVER":
  case "DXE_RUNTIME_DRIVER":
  case "DXE_SAL_DRIVER":
  case "UEFI_DRIVER":
  case "UEFI_APPLICATION":
    include_statement (AutoGen.c, "
      #include <PiDxe.h>
      #include <Library/BaseLib.h>
      #include <Library/DebugLib.h>
      #include <Library/UefiBootServicesTableLib.h>

    );
    break;
  default:
    PrintError ( "%s\n", message);
    BreakTheBuild();
}
```

The following will be inserted in AutoGen.c after the header files have been included.

```
GLOBAL_REMOVE_IF_UNREFERENCED CHAR8 *gEfiCallerBaseName = "<ModuleName>";
```

Where the <ModuleName> is the value of the BASE_NAME from the module INF file's [Defines] section.

8.3.7.2 Caller ID GUID variable definition.

Because not all GUID variables are required, a link-time optimization removes items that are not referenced by other parts of the code to save on space in the image.

```
include_statement (AutoGen.c, "
```

```
GLOBAL_REMOVE_IF_UNREFERENCED GUID gEfiCallerIdGuid = {0x4A9B9DB8,
0xEC62, 0x4A92, {0x81, 0x8F, 0x8A, 0xA0, 0x24, 0x6D, 0x24, 0x6E}};
");
```

8.3.7.3 Library Constructor Statements

If there are `CONSTRUCTOR`s defined in `[Defines]` section in INF file of the library instances that are being linked to.

```
If (CONSTRUCTOR defined in INF) {

  If ((MODULE_TYPE == "BASE") || (MODULE_TYPE == "SEC")) {
    include_statement (AutoGen.c, "
      EFI_STATUS
      EFI_API
      <CONSTRUCTOR> (
        VOID );

      ");
  }

  If ((MODULE_TYPE == "PEI_CORE") || (MODULE_TYPE == "PEIM")) {
    include_statement (AutoGen.c, "
      EFI_STATUS
      EFI_API
      <CONSTRUCTOR> (
        IN EFI_PEI_FILE_HANDLE  FileHandle,
        IN EFI_PEI_SERVICES     **PeiServices
      );

      ");
  }

  If ((MODULE_TYPE == 'DXE_CORE') || (MODULE_TYPE == 'DXE_DRIVER') ||
      (MODULE_TYPE == 'DXE_SMM_DRIVER') ||
      (MODULE_TYPE == 'DXE_RUNTIME_DRIVER') ||
      (MODULE_TYPE == 'DXE_SAL_DRIVER') ||
      (MODULE_TYPE == 'UEFI_DRIVER') ||
      (MODULE_TYPE == 'UEFI_APPLICATION')) {

    include_statement (AutoGen.c, "
      EFI_STATUS
      EFI_API
      <CONSTRUCTOR> (
        IN EFI_HANDLE           ImageHandle,
        IN EFI_SYSTEM_TABLE     *SystemTable
      );

      ");
  }

} // End CONSTRUCTOR defined in INF

If ((MODULE_TYPE == "BASE") || (MODULE_TYPE == "SEC")) {
  include_statement (AutoGen.c, "
    VOID
    EFI_API
    ProcessLibraryConstructorList (
      VOID
    )
  ");
}

If ((MODULE_TYPE == "PEI_CORE") || (MODULE_TYPE == "PEIM")) {
  include_statement (AutoGen.c, "
    VOID
    EFI_API
    ProcessLibraryConstructorList (
      IN EFI_PEI_FILE_HANDLE  FileHandle,
```

```

    IN EFI_PEI_SERVICES    **PeiServices
  )

  ");
}

If ((MODULE_TYPE == 'DXE_CORE') || (MODULE_TYPE == 'DXE_DRIVER') ||
    (MODULE_TYPE == 'DXE_SMM_DRIVER') ||
    (MODULE_TYPE == 'DXE_RUNTIME_DRIVER' ||
    (MODULE_TYPE == 'DXE_SAL_DRIVER') ||
    (MODULE_TYPE == 'UEFI_DRIVER') ||
    (MODULE_TYPE == 'UEFI_APPLICATION')) {
  include_statement (AutoGen.c, "
  VOID
  EFIAPI
  ProcessLibraryConstructorList (
    IN EFI_PEI_FILE_HANDLE  ImageHandle,
    IN EFI_PEI_SERVICES     **SystemTable
  )
  ");
}

include_statement (AutoGen.c, "
{
");

If (CONSTRUCTOR defined in INF) {
  If ((MODULE_TYPE == "BASE") || (MODULE_TYPE == "SEC")) {
    include_statement (AutoGen.c, "
      EFI_STATUS Status;

      Status = <CONSTRUCTOR> ();
      ASSERT_EFI_ERROR (Status);

    ");
  }

  If ((MODULE_TYPE == "PEI_CORE") || (MODULE_TYPE == "PEIM")) {
    include_statement (AutoGen.c, "
      EFI_STATUS Status;

      Status = <CONSTRUCTOR> (FileHandle, PeiServices);
      ASSERT_EFI_ERROR (Status);

    ");
  }
}

If ((MODULE_TYPE == 'DXE_CORE') || (MODULE_TYPE == 'DXE_DRIVER') ||
    (MODULE_TYPE == 'DXE_SMM_DRIVER') ||
    (MODULE_TYPE == 'DXE_RUNTIME_DRIVER' ||
    (MODULE_TYPE == 'DXE_SAL_DRIVER') ||
    (MODULE_TYPE == 'UEFI_DRIVER') ||
    (MODULE_TYPE == 'UEFI_APPLICATION')) {
  include_statement (AutoGen.c, "
    EFI_STATUS Status;

    Status = <CONSTRUCTOR> (ImageHandle, SystemTable);
    ASSERT_EFI_ERROR (Status);

  ");
}
}

include_statement (AutoGen.c, "
}
");

```

8.3.7.4 Library Destructor Statements

Contained if there are `DESTRUCTOR`s defined in `[Defines]` section in INF file of the library instances that are being linked to.

```

If (DESTRUCTOR defined in INF) {
  If ((MODULE_TYPE == "BASE") || (MODULE_TYPE == "SEC")) {
    include_statement (AutoGen.c, "
      EFI_STATUS
      EFIAPI
      <DESTRUCTOR> (
        VOID
      );
    ");
  }
  If ((MODULE_TYPE == "PEI_CORE") || (MODULE_TYPE == "PEIM")) {
    include_statement (AutoGen.c, "
      EFI_STATUS
      EFIAPI
      <DESTRUCTOR> (
        IN EFI_PEI_FILE_HANDLE  FileHandle,
        IN EFI_PEI_SERVICES     **PeiServices
      );
    ");
  }
  If ((MODULE_TYPE == 'DXE_CORE') || (MODULE_TYPE == 'DXE_DRIVER') ||
      (MODULE_TYPE == 'DXE_SMM_DRIVER') ||
      (MODULE_TYPE == 'DXE_RUNTIME_DRIVER') ||
      (MODULE_TYPE == 'DXE_SAL_DRIVER') ||
      (MODULE_TYPE == 'UEFI_DRIVER') ||
      (MODULE_TYPE == 'UEFI_APPLICATION')) {
    include_statement (AutoGen.c, "
      EFI_STATUS
      EFIAPI
      <DESTRUCTOR> (
        IN EFI_HANDLE          ImageHandle,
        IN EFI_SYSTEM_TABLE    *SystemTable
      );
    ");
  }
} // End DESTRUCTOR defined in INF

If ((MODULE_TYPE == "BASE") || (MODULE_TYPE == "SEC")) {
  include_statement (AutoGen.c, "
    VOID
    EFIAPI
    ProcessLibraryDestructorList (
      VOID
    )

  ");
}

If ((MODULE_TYPE == "PEI_CORE") || (MODULE_TYPE == "PEIM")) {
  include_statement (AutoGen.c, "
    VOID
    EFIAPI
    ProcessLibraryDestructorList (
      IN EFI_PEI_FILE_HANDLE  FileHandle,
      IN EFI_PEI_SERVICES     **PeiServices
    )

  ");
}

If ((MODULE_TYPE == 'DXE_CORE') || (MODULE_TYPE == 'DXE_DRIVER') ||
    (MODULE_TYPE == 'DXE_SMM_DRIVER') ||
    (MODULE_TYPE == 'DXE_RUNTIME_DRIVER') ||
    (MODULE_TYPE == 'DXE_SAL_DRIVER') ||
    (MODULE_TYPE == 'UEFI_DRIVER') ||
    (MODULE_TYPE == 'UEFI_APPLICATION')) {
  include_statement (AutoGen.c, "
    VOID
  ");
}

```

```

EFIAPI
ProcessLibraryDestructorList (
    IN EFI_PEI_FILE_HANDLE  ImageHandle,
    IN EFI_PEI_SERVICES     *SystemTable
)

");
}

include_statement (AutoGen.c, "
{

");

If (DESTRUCTOR defined in INF) {
    If ((MODULE_TYPE == "BASE") || (MODULE_TYPE == "SEC")) {
        include_statement (AutoGen.c, "
            EFI_STATUS Status;

            Status = <DESTRUCTOR> ();
            ASSERT_EFI_ERROR (Status);

        ");
    }

    If ((MODULE_TYPE == "PEI_CORE") || (MODULE_TYPE == "PEIM")) {
        include_statement (AutoGen.c, "
            EFI_STATUS Status;

            Status = <DESTRUCTOR> (FileHandle, PeiServices);
            ASSERT_EFI_ERROR (Status);
        ");
    }

    If ((MODULE_TYPE == 'DXE_CORE') || (MODULE_TYPE == 'DXE_DRIVER') ||
        (MODULE_TYPE == 'DXE_SMM_DRIVER') ||
        (MODULE_TYPE == 'DXE_RUNTIME_DRIVER') ||
        (MODULE_TYPE == 'DXE_SAL_DRIVER') ||
        (MODULE_TYPE == 'UEFI_DRIVER') ||
        (MODULE_TYPE == 'UEFI_APPLICATION')) {
        include_statement (AutoGen.c, "
            EFI_STATUS Status;

            Status = <DESTRUCTOR> (ImageHandle, SystemTable);
            ASSERT_EFI_ERROR (Status);
        ");
    }
}

include_statement (AutoGen.c, "
}
");

```

8.3.7.5 Module Entry Point Statements

Contained if there are `ENTRY_POINT`s defined `[Defines]` section in INF file.

```

If (ENTRY_POINT defined in INF) {
    If (MODULE_TYPE == 'PEI_CORE') {
        include_statement (AutoGen.c, "
            EFI_STATUS
            <ENTRY_POINT> (
                IN CONST EFI_SEC_PEI_HAND_OFF    *SecCoreData,
                IN CONST EFI_PEI_PPI_DESCRIPTOR  *PpiList,
                IN VOID                            *OldCoreData
            );

            EFI_STATUS
            EFIAPI
            ProcessModuleEntryPointList (

```

```

    IN CONST EFI_SEC_PEI_HAND_OFF    *SecCoreData,
    IN CONST EFI_PEI_PPI_DESCRIPTOR  *PpiList,
    IN VOID                          *OldCoreData
  )

  {
    return <ENTRY_POINT> (SecCoreData, PpiList, OldCoreData);
  }

");
}

If (MODULE_TYPE == 'DXE_CORE') {
  include_statement (AutoGen.c, "
    const UINT32 _gUefiDriverRevision = 0;

    VOID
    <ENTRY_POINT> (
      IN VOID *HobStart
    );

    VOID
    EFIAPI
    ProcessModuleEntryPointList (
      IN VOID *HobStart
    )

    {
      <ENTRY_POINT> (HobStart);
    }

  ");
}

If (MODULE_TYPE == 'PEIM') {
  include_statement (AutoGen.c, "
    GLOBAL_REMOVE_IF_UNREFERENCED const UINT32 _gPeimRevision = 0;
  ");
  If (Number of ENTRY_POINT == 0) {
    include_statement (AutoGen.c, "
      EFI_STATUS
      EFIAPI
      ProcessModuleEntryPointList (
        IN EFI_PEI_FILE_HANDLE  FileHandle,
        IN EFI_PEI_SERVICES     **PeiServices
      )
      {
        return EFI_SUCCESS;
      }

    ");
  }

  If (Number of ENTRY_POINT == 1) {
    include_statement (AutoGen.c, "
      EFI_STATUS
      <ENTRY_POINT> (
        IN EFI_PEI_FILE_HANDLE  FileHandle,
        IN EFI_PEI_SERVICES     **PeiServices
      );

      EFI_STATUS
      EFIAPI
      ProcessModuleEntryPointList (
        IN EFI_PEI_FILE_HANDLE  FileHandle,
        IN EFI_PEI_SERVICES     **PeiServices
      )
      {
        return <ENTRY_POINT> (FileHandle, PeiServices);
      }

    ");
  }
}

```

```

}

If (Number of ENTRY_POINT > 1) {
  include_statement (AutoGen.c, "
    <ENTRY_POINT1> (
      IN EFI_PEI_FILE_HANDLE FileHandle,
      IN EFI_PEI_SERVICES **PeiServices
    );

    <ENTRY_POINT2> (
      IN EFI_PEI_FILE_HANDLE FileHandle,
      IN EFI_PEI_SERVICES **PeiServices
    );

    EFI_STATUS
    EFIAPI
    ProcessModuleEntryPointList (
      IN EFI_PEI_FILE_HANDLE FileHandle,
      IN EFI_PEI_SERVICES **PeiServices
    )

    {
      EFI_STATUS Status;
      EFI_STATUS CombinedStatus;

      CombinedStatus = EFI_LOAD_ERROR;

      Status = <ENTRY_POINT1> (FileHandle, PeiServices);
      if (!EFI_ERROR (Status) || EFI_ERROR (CombinedStatus)) {
        CombinedStatus = Status;
      }

      Status = <ENTRY_POINT2> (FileHandle, PeiServices);
      if (!EFI_ERROR (Status) || EFI_ERROR (CombinedStatus)) {
        CombinedStatus = Status;
      }

      return CombinedStatus;
    }

    ");
  }
}

If (MODULE_TYPE == 'DXE_SMM_DRIVER') {
  If (Number of ENTRY_POINT == 0) {
    include_statement (AutoGen.c, "
      EFI_STATUS
      EFIAPI
      ProcessModuleEntryPointList (
        IN EFI_HANDLE ImageHandle,
        IN EFI_SYSTEM_TABLE *SystemTable
      )

      {
        return EFI_SUCCESS;
      }

      ");
  }

  If (Number of ENTRY_POINT == 1) {
    include_statement (AutoGen.c, "
      EFI_STATUS
      <ENTRY_POINT> (
        IN EFI_HANDLE ImageHandle,
        IN EFI_SYSTEM_TABLE *SystemTable
      );

      static BASE_LIBRARY_JUMP_BUFFER mJumpContext;
      static EFI_STATUS mDriverEntryPointStatus = EFI_LOAD_ERROR;

```

```

VOID
EFIAPI
ExitDriver (
    IN EFI_STATUS  Status
)

{
    if (!EFI_ERROR (Status) || EFI_ERROR (mDriverEntryPointStatus)) {
        mDriverEntryPointStatus = Status;
    }
    LongJump (&mJumpContext, (UINTN) - 1);
    ASSERT (FALSE);
}

EFI_STATUS
EFIAPI
ProcessModuleEntryPointList (
    IN EFI_HANDLE      ImageHandle,
    IN EFI_SYSTEM_TABLE *SystemTable
)

{
    if (SetJump (&mJumpContext) == 0) {
        ExitDriver (<ENTRY_POINT> (ImageHandle, SystemTable));
        ASSERT (FALSE);
    }

    return mDriverEntryPointStatus;
}

");
}
}

If ((MODULE_TYPE == 'DXE_RUNTIME_DRIVER') ||
    (MODULE_TYPE == 'DXE_DRIVER') ||
    (MODULE_TYPE == 'DXE_SAL_DRIVER') ||
    (MODULE_TYPE == 'UEFI_DRIVER') ||
    (MODULE_TYPE == 'UEFI_APPLICATION')) {
    include_statement (AutoGen.c, "
        const UINT32 _gUefiDriverRevision = 0;

    ");

    If (Number of ENTRY_POINT == 0) {
        include_statement (AutoGen.c, "
            EFI_STATUS
            EFIAPI
            ProcessModuleEntryPointList (
                IN EFI_HANDLE      ImageHandle,
                IN EFI_SYSTEM_TABLE *SystemTable
            )
            {
                return EFI_SUCCESS;
            }

        ");
    }

    If (Number of ENTRY_POINT == 1) {
        include_statement (AutoGen.c, "
            EFI_STATUS
            ${Function} (
                IN EFI_HANDLE      ImageHandle,
                IN EFI_SYSTEM_TABLE *SystemTable
            );

            EFI_STATUS
            EFIAPI
            ProcessModuleEntryPointList (
                IN EFI_HANDLE      ImageHandle,
                IN EFI_SYSTEM_TABLE *SystemTable

```

```

    )
    {
        return <ENTRY_POINT> (ImageHandle, SystemTable);
    }

VOID
EFIAPI
ExitDriver (
    IN EFI_STATUS  Status
)
{
    if (EFI_ERROR (Status)) {
        ProcessLibraryDestructorList (gImageHandle, gST);
    }
    gBS->Exit (gImageHandle, Status, 0, NULL);
}

");
}

If (Number of ENTRY_POINT > 1) {
    include_statement (AutoGen.c, "
        <ENTRY_POINT1> (
            IN EFI_HANDLE      ImageHandle,
            IN EFI_SYSTEM_TABLE *SystemTable
        );

        <ENTRY_POINT2> (
            IN EFI_HANDLE      ImageHandle,
            IN EFI_SYSTEM_TABLE *SystemTable
        );

        EFI_STATUS
        EFIAPI
        ProcessModuleEntryPointList (
            IN EFI_HANDLE      ImageHandle,
            IN EFI_SYSTEM_TABLE *SystemTable
        )
        {
            if (SetJump (&mJumpContext) == 0) {
                ExitDriver (<ENTRY_POINT1> (ImageHandle, SystemTable));
                ASSERT (FALSE);
            }

            if (SetJump (&mJumpContext) == 0) {
                ExitDriver (<ENTRY_POINT2> (ImageHandle, SystemTable));
                ASSERT (FALSE);
            }

            return mDriverEntryPointStatus;
        }

        static BASE_LIBRARY_JUMP_BUFFER mJumpContext;
        static EFI_STATUS mDriverEntryPointStatus = EFI_LOAD_ERROR;

        VOID
        EFIAPI
        ExitDriver (
            IN EFI_STATUS Status
        )
        {
            if (!EFI_ERROR (Status) || EFI_ERROR (mDriverEntryPointStatus)) {
                mDriverEntryPointStatus = Status;
            }
            LongJump (&mJumpContext, (UINTN) - 1);
            ASSERT (FALSE);
        }

    ");
}
}
}

```

8.3.7.6 Module Unload Image Statements

The following algorithm is used to process potential `UNLOAD_IMAGE` statements that might be defined in the `[Defines]` section in the INF file.

```

If (Number of UNLOAD_IMAGE in INF == 0) {
  include_statement (AutoGen.c, "
    GLOBAL_REMOVE_IF_UNREFERENCED const UINT8 _gDriverUnloadImageCount = 0;

    EFI_STATUS
    EFIAPI
    ProcessModuleUnloadList (
      IN EFI_HANDLE ImageHandle
    )
    {
      return EFI_SUCCESS;
    }

  ");
}

If (Number of UNLOAD_IMAGE in INF == 1) {
  include_statement (AutoGen.c, "
    GLOBAL_REMOVE_IF_UNREFERENCED const UINT8 _gDriverUnloadImageCount = 1;
    EFI_STATUS
    <UNLOAD_IMAGE> (
      IN EFI_HANDLE ImageHandle
    );

    EFI_STATUS
    EFIAPI
    ProcessModuleUnloadList (
      IN EFI_HANDLE ImageHandle
    )
    {
      return <UNLOAD_IMAGE> (ImageHandle);
    }

  ");
}

If (Number of UNLOAD_IMAGE in INF > 1) {
  include_statement (AutoGen.c, "
    GLOBAL_REMOVE_IF_UNREFERENCED const UINT8 _gDriverUnloadImageCount = <NumberOfUnloadImage>;

    EFI_STATUS
    <UNLOAD_IMAGE1> (
      IN EFI_HANDLE ImageHandle
    );

    EFI_STATUS
    <UNLOAD_IMAGE2> (
      IN EFI_HANDLE ImageHandle
    );

    EFI_STATUS
    EFIAPI
    ProcessModuleUnloadList (
      IN EFI_HANDLE ImageHandle
    )
    {
      EFI_STATUS Status;

      Status = EFI_SUCCESS;

      if (EFI_ERROR (Status)) {
        <UNLOAD_IMAGE1> (ImageHandle);
      } else {
        Status = <UNLOAD_IMAGE1> (ImageHandle);
      }
    }
  ");
}

```

```

    }

    if (EFI_ERROR (Status)) {
        <UNLOAD_IMAGE2> (ImageHandle);
    } else {
        Status = <UNLOAD_IMAGE2> (ImageHandle);
    }

    return Status;
}

");
}

```

8.3.7.7 Global variables

These are generated from "Guids", "Protocols", "Ppis", "xxxPcd" sections of the `.inf` file and `.uni` and `.idf` files.

```

InfList = [];

add (ModuleInf, InfList);

foreach LibraryInstance {
    add (LibraryInf, InfList);
    foreach DependentLibraryInstance {
        add (LibraryInf, InfList);
    }
}

foreach INF in InfList {
    If ("[Guids]" defined in INF) {
        foreach GuidCName {
            include_statement (AutoGen.c, "
                GLOBAL_REMOVE_IF_UNREFERENCED EFI_GUID <GuidCName> = <GuidValue>;
            ");
        }
    }

    If ("[Protocols]" defined in INF) {
        foreach ProtocolGuidCName {
            include_statement (AutoGen.c, "
                GLOBAL_REMOVE_IF_UNREFERENCED EFI_GUID <ProtocolGuidCName> = <GuidValue>;
            ");
        }
    }

    If ("[Ppis]" defined in INF) {
        foreach PpiGuidCName {
            include_statement (AutoGen.c, "
                GLOBAL_REMOVE_IF_UNREFERENCED EFI_GUID <PpiGuidCName> = <GuidValue>;
            ");
        }
    }

    If ("[Pcd]" defined in INF) {
        foreach PcdCName {
            If ((PcdDatumType == 'VOID*') &&
                (
                    (PcdValue == array) ||
                    (PcdValue == C_FormatGuid) ||
                    (PcdValue == C_String)
                )
            ) {
                include_statement (AutoGen.c, "
                    GLOBAL_REMOVE_IF_UNREFERENCED UINT8 <PcdCName> = <PcdValueMacro>;
                ");
            } Else {
                include_statement (AutoGen.c, "

```

```
        GLOBAL_REMOVE_IF_UNREFERENCED <PcdDatumType> <PcdCName> = <PcdValueMacro>;
    ");
}
}
}

If (.UNI file found in INF SourcesSection) {
    include_statement (AutoGen.c, "
        unsigned char MiscSubclassStrings[] = {
            .....
        }
    ");
}

If (.IDF file found in INF SourcesSection) {
    include_statement (AutoGen.c, "
        unsigned char HelloWorldImages[] = {
            .....
        }
    ");
}
}
```

8.4 Auto-generated PCD Database File

The EDK II code-base provides platform configuration data that can be modified at runtime. The two PCD data types are `Dynamic` PCDs scoped to only the platform drivers and `DynamicEx` PCDs, which may be accessed by other modules. There are two drivers, a PEIM and DXE driver that are used to provide access to these configurable items.

Since binary modules may need to add additional `DynamicEx` PCDs, the EDK II drivers and the EDK II build system create external binary (`PeiPcdDataBase.raw` and `DxePcdDataBase.raw`) database files. These files are generated by the build system based on PCDs listed in the FDF and DSC files, as well as from INF files listed in the DSC and FDF files. The files are a union of all of the `Dynamic` and `DynamicEx` PCDs found from these EDK II meta-data files. During the ImageGen stage, the files will be put into the FFS file (`EFI_SECTION_RAW`) for both the `PEIM` and `DXE` driver. Each driver has been coded to locate the file. The rule for the `PEI_PCD_DRIVER` module and `DXE_PCD_DRIVER` module is integrated into the build system. The EDK II build system limits the offset of `Dynamic` and `DynamicEx` PCDs that are defined in the DSC file using the subtype of HII to a `UINT16` value.

No special rules are required to add the FFS raw section in the FDF file to process these drivers. Standard `PEIM` and `DXE_DRIVER` rules can be specified, as the build system will always insert the database raw sections in to these drivers if the database file exists.

If `Dynamic` or `DynamicEx` PCDs are used by the platform and no database file is created by the build, the build tools must break with an appropriate error message.

Table 16 Access Method Section Tags

Access Method	INF File	DEC File	DSC File
FeatureFlag	[FeaturePcd]	[PcdsFeatureFlag]	[PcdsFeatureFlag]
FixedAtBuild	[FixedPcd]	[PcdsFixedAtBuild]	[PcdsFixedAtBuild]
PatchableInModule	[PatchPcd]	[PcdsPatchableInModule]	[PcdsPatchableInModule]
Dynamic	[Pcd]	[PcdsDynamic]	[PcdsDynamicDefault] [PcdsDynamicVpd] [PcdsDynamicHii]
DynamicEx	[PcdEx]	[PcdsDynamicEx]	[PcdsDynamicExDefault] [PcdsDynamicExVpd] [PcdsDynamicExHii]

The FDF file does not have specific sections for setting PCD values. PCD values are either automatically set in the [FD] sections (region offset and region size) or using SET statements.

8.4.1 PCD Rules:

The subsections that follow cover the rules for processing PCDs defined in FDF, DSC, INF or DEC files.

8.4.1.1 General Rules:

1. A FeatureFlag PCD cannot use be listed under any other access method in the DEC file. If a PCD name is listed in an FeatureFlag section, and also in another section type, the build must break.
2. For PCDs using `Dynamic` or `DynamicEx` access methods, the PCD must be listed in the DSC file. The build parser must break with an appropriate error message if a `Dynamic` or `DynamicEx` PCD is not specified in the DSC.
3. For a given platform build, a PCD can only use one access method. Any INF files in a platform that specifically limit the PCD access method for a given PCD must all list the same access method OR for source INF files only, the list the PCD in a [Pcd] section.

4. BINARY INF files (that do not list files under a `[Sources]` section) can only contain `[PcdEx]` and `[PatchPcd]` Sections - if they contain any other type of PCD, break the build.
5. If a PCD has a Token Space GUID specified in DEC file and the `[Guids]` section tag contains the `Private` modifier (`[Guids.common.Private]` for example), the PCD may only be used by modules in the package containing the DEC file. If a module outside of that package attempts to use the PCD, the build must break with an appropriate error message.

8.4.1.2 Precedence Rules for PCDs not listed in the DSC or FDF Files:

This subsection covers PCDs that are used by modules listed in the DSC file, but the PCD itself is not listed in any PCD section (module scoped or global) within the DSC file. The following general rules are processed in order until one of them is satisfied. If none of these rules can be satisfied, then other rules (below this list) will be tested.

1. If all modules that use a PCD list it in a `[Pcd]` section and the DEC file declares `PcdsFixedAtBuild` as a PCD access method, then the build will use `PcdsFixedAtBuild` for the PCD.
2. If all modules that use a PCD list it in a `[Pcd]` section and the DEC file declares `PcdsPatchableInModule` as a PCD access method, then the build will use `PcdsPatchableInModule` for the PCD.
3. If all modules that use a PCD list it in a `[Pcd]` section and the DEC file declares `PcdsDynamic` as a PCD access method, then the build will use `PcdsDynamicDefault` for the PCD.
4. If all modules that use a PCD list it in a `[Pcd]` section and the DEC file declares `PcdsDynamicEx` as a PCD access method, then the build will use `PcdsDynamicExDefault` for the PCD.

Certain rules in this section assume that the EDK II package creator omitted some entries in the DEC file on purpose. These rules cover the case where a module does not follow the DEC file's access method declarations.

1. PCD access method assignment from Binary INF files take precedence over any access method assignment from Source INF files;
 - If a Binary INF listed only in the FDF file and the PCD access method is listed under a `[PatchPcd]` section and the Source INF files list the PCD in either `[PatchPcd]` or `[Pcd]` sections, then the build system must assign the PCD to use the `PcdsPatchableInModule` access method for all INF files that use the PCD.
 - If a Binary INF listed only in the FDF file and the PCD access method is listed under a `[PcdEx]` section and the Source INF files list the PCD in either `[PcdEx]` or `[Pcd]` sections, then the build system must assign the PCD to use the `PcdsDynamicExDefault` access method for all INF files that use the PCD. The PCD must be added to the Platform's PCD Database.
2. When building modules from source INFs, a PCD can only use one access method for all modules in a platform; a PCD cannot use the patch access method in one source module and fixed access method in another source module in the same platform. The build parser must break with an error message if this occurs.
3. Binary modules included in a platform build are permitted to use the `PatchableInModule` or `DynamicEx` access methods (the Binary module must specify which of these two methods were used to create the binary module) regardless of the method used for a given PCD in modules built from source. The build supports binary modules that use the same or different PCD access method than the source modules or other binary modules. The build parser must break with an error if a PCD is listed as `FixedAtBuild` or `Dynamic` (not `DynamicEx`) in the Binary INF.
4. If the PCD is listed under different access methods in all source INF files in the platform that use the PCD, the build parser must break with an appropriate error message.

5. If the PCD is listed in a `[Pcd]` section in all of the source modules using that PCD that are listed in the DSC file, AND the PCD is listed in the DEC file under `[PcdsDynamicEx]` and/or `[PcdsDynamic]` and/or `[PcdsPatchableInModule]` and `[PcdsFixedAtBuild]` sections, the build must use the `PcdsFixedAtBuild` access method for this PCD in all source modules in the platform that use this PCD.
6. If the PCD is listed in a `[Pcd]` section in all of the source modules using that PCD that are listed in the DSC file, AND the PCD is listed in the DEC file under `[PcdsDynamicEx]` and/or `[PcdsDynamic]` and `[PcdsPatchableInModule]` sections, the build must use the `PcdsPatchableInModule` access method for this PCD in all source modules in the platform that use this PCD.
7. If the PCD is listed in a `[Pcd]` section in all of the source modules using that PCD that are listed in the DSC file, AND the PCD is listed in the DEC file under `[PcdsDynamicEx]` and `[PcdsDynamic]` sections, the build must use the `PcdsDynamicDefault` access method for this PCD in all source modules in the platform that use this PCD.
8. If the PCD is listed in a `[Pcd]` section in all of the source modules using that PCD that are listed in the DSC file, AND the PCD is listed in the DEC file under `[PcdsDynamicEx]` sections, the build must use the `PcdsDynamicExDefault` access method for this PCD in all source modules in the platform that use this PCD.
9. If multiple source modules set the `Dynamic` or `DynamicEx` PCD to the different value in the same platform, and the PCD is not listed in the DSC file, the build should break with an appropriate error message.
10. If a PCD is used in a module listed in the DSC or FDF file and the PCD is not declared in any of the DEC files that the module depends on (listed in the `[Packages]` section) the build must break with an appropriate error message.
11. If a PCD is listed in the DSC or FDF file and the PCD is not declared in any of the DEC files AND the PCD is not used by any of the modules listed in the DSC or FDF file, the build must break with an appropriate error message.

8.4.1.3 Precedence Rules

The rules are listed in order, such that the first match stops any additional processing. The following rules apply to Binary modules listed in a platform DSC file.

1. PCD value assignment from command-line using `--pcd` flag takes precedence over all other assignments.
2. PCD assignments are not permitted in the FDF file except through SET statements or the automatic assignments from the `[FD]` section regions.
3. PCD value assignment in a module scoping section take precedence over values specified in the global section for PatchableInModule PCDs.
4. PCD value assignment in a global PCD section with an architectural modifier take precedence over assignments in a global section.
5. PCD value assignment in a global PCD section without an architectural modifier.
6. The value specified in the Binary INF has the lowest precedence; the DEC file is never used to determine a PCD value for a Binary INF.

The following rules apply to modules listed in a platform DSC file.

1. PCD value assignment from command-line using `--pcd` flag takes precedence over all other assignments.
2. PCD assignments in an FDF file are positional, with the last value taking precedence over previous assignments in the FDF file.

3. A PCD assignment in an FDF file takes precedence over PCD values assigned in the DSC file's module scoping section.
4. A PCD value of an entry listed in a module scoping section take precedence over the PCD value listed in a global section that has an architectural modifier in the DSC file.
5. A PCD value of an entry listed in a global section that has an architectural modifier takes precedence over the PCD value listed in a global section without an architectural modifier in the DSC file.
6. A PCD value of an entry listed in a global section without architectural modifiers in the DSC file takes precedence over the PCD value listed in an INF file in a section with an architectural modifier.
7. A PCD value of an entry listed in an INF file section with an architectural modifier takes precedence over an entry listed in an INF file section without an architectural modifier.
8. A PCD value of an entry listed in an INF file section without an architectural modifier takes precedence over a PCD value listed in a DEC file in a section with an architectural modifier.
9. A PCD value of an entry listed in a DEC file section with an architectural modifier takes precedence over a PCD value listed in a DEC file in a section without an architectural modifier.

Because the INF describes how a module is coded, a PCD can only be listed under one access method: a PCD section tag with architectural modifiers cannot specify a different access method for a PCD. The EDK II C Coding Standard prohibits using preprocessor directives for architectures within the C code.

- For instance, it is not possible to list a PCD as being FixedAtBuild for all architectures and PatchableInModule for X64 if the module uses only common files.

For a given platform, a PCD can only use one access method for source modules. Having different access methods for same architecture is not permitted.

- For instance, it is prohibited to have one module for IA32 and another copy of the module for X64 that use different PCD access methods.

8.4.1.4 Dynamic and DynamicEx Database Rules

This subsection covers the rules for adding Dynamic or DynamicEx PCDs to the PCD database.

1. If a PCD is listed in a `PcdsDynamicVpd` or `PcdsDynamicExVpd` section, and the PCD is not used by any module that is listed in the DSC file, the build MUST ADD the entry in the Platform's PCD Database, and the parser must not throw an error or warning message.
2. If PCD is listed in a `PcdsDynamicDefault` or `PcdsDynamicExDefault` section, and the PCD is not used by any module that is listed in the FDF file (even if a module that uses the PCD is listed in the DSC file), the build must NOT add the entry in the Platform's PCD Database.
 - The build may provide a warning message.
3. If PCD is listed in a `PcdsDynamicHii` or `PcdsDynamicExHii` section, and the PCD is not used by any module that is listed in the FDF file (even if a module that uses the PCD is listed in the DSC file), the build must NOT add the entry in the Platform's PCD Database.
 - The build may provide a warning message.
4. If a PCD is not listed in the DSC file but is listed under a `[PcdsEx]` section in a Binary INF file listed in the FDF file, then the build must add the entry to the Platform's PCD Database as `PcdsDynamicExDefault`.
5. If a PCD is not listed in the DSC file, but binary INF files used by this platform use this PCD and list the PCD in a `[PcdsEx]` section, AND any source INF files that use the PCD list the PCD in either a `[Pcds]` or `[PcdsEx]` section, then the tools MUST ADD the PCD to the Platform's PCD Database.
 - The build must assign the access method for this PCD as `PcdsDynamicExDefault`.

6. If a PCD is not listed in the DSC file, but binary INF files used by this platform all (that use this PCD) list the PCD in a `[PatchPcds]` section, AND all source INF files used by this platform the build that use the PCD list the PCD in either a `[Pcds]` or `[PatchPcds]` section, then the tools must NOT add the PCD to the Platform's PCD Database.
 - The build must assign the access method for this PCD as `PcdsPatchableInModule`.
7. If one of the Source built modules listed in the DSC is not listed in FDF modules, and the INF lists a PCD can only use the `PcdsDynamic` access method (it is only listed in the DEC file that declares the PCD as `PcdsDynamic`), then build tool will report warning message that notifies the PI of an attempt to build a module that must be included in a flash image in order to be functional.
 - These `Dynamic` PCD will not be added into the Database unless it is used by other modules that are included in the FDF file.
8. If one of the Source built modules listed in the DSC is not listed in FDF modules, and the INF lists a PCD can only use the `PcdsDynamicEx` access method (it is only listed in the DEC file that declares the PCD as `PcdsDynamicEx`), then DO NOT break the build.
 - DO NOT add the PCD to the Platform's PCD Database.
9. If a module is listed in FDF file and use a `Dynamic` or `DynamicEx` PCD, the PCD MUST be added into the PCD Database.

The build system must emit a line containing the total number of warnings from the above rules at the end of a build.

Note: Because parsing warnings may appear for only a short period prior to calling other tools that emit a copious number of informational messages, this line will ensure that the PI knows that warnings were emitted.

8.4.1.5 FeatureFlag PCDs used in conditional directive statements in code

FeatureFlag PCDs used in conditional directive statements in code have the following rules.

1. A FeatureFlag PCD cannot use any other access method. If a PCD name is listed in an FF section, and also in another section type, the build must break.
2. A PCD can only be use one access method for all modules in a platform; a PCD cannot use the patch access method in one module and fixed access method in another module in the same platform. The build parser must break with an error message if this occurs.
3. Duplicate PCD names listed within a section are positional, such that only the value of the last entry will be used.

Note: A PCD name & value listed in an architectural section takes precedence over the PCD name & value specified in a common section when build for a specific architecture. If a PCD name is not listed in a section that contains an architectural modifier, and is listed in a section that is common, the value in from the entry in a common section will be used.

8.5 Auto-generated Makefiles

The actual build actions are done via **"MAKE"** system. This system is **"nmake"** in Windows environment and **"make"** in GCC (Linux and Mac OS/X) environment. The `Makefiles` are created at the module level. For one platform, one makefile is generated for each tool chain, build target (`DEBUG` / `RELEASE` / `NOOPT`) and architecture.

In Platform mode, the `build` tool calls the build script tool (**nmake** or **make**) for each Module's `Makefile`.

In Module mode, the `build` tool calls the build script tool, giving the Module `Makefile` as an argument. However, in Module Mode, if the build tool target is "fds", after the module builds successfully, the build tool calls the GenFds tool to regenerate an FD file.

8.5.1 Module Makefile

This section describe the formats of the individual component/module Makefiles. Users may generate a custom makefile for their EDK II module based on the information provided by this section.

The module `Makefile` is composed by two parts: macro definitions and target definitions.

In the pseudo-code provided, the MACRO, `$(MODULE_BUILD_DIR)` is constructed using the following rules:

- If the .dsc file's `OUTPUT_DIRECTORY` value (path) starts with an alpha character, the value of the `OUTPUT_DIRECTORY` statement is relative to the directory specified in the system environment variable, `WORKSPACE`. Otherwise, it is considered an absolute directory path.

```
If (isalpha (getValue ("OUTPUT_DIRECTORY", DscFile)[0]) {
  MOD_BUILD_DIR = "$WORKSPACE\" + getValue("OUTPUT_DIRECTORY", DscFile)
} else {
  MOD_BUILD_DIR1 = getValue ("OUTPUT_DIRECTORY", DscFile)
}
Foreach Target in ActiveTargetList {
  Foreach ToolChainTag in ActiveToolChain {
    MOD_BUILD_DIR2 = $ (MOD_BUILD_DIR1) + "\" + Target + "_" + ToolChainTag + "\";
    foreach Arch in ActiveArchList {
      MODULE_BUILD_DIR = $ (MOD_BUILD_DIR2) + Arch + "\";
      MODULE_BUILD_DIR += getDirPart (InfFile) + "\";
      MODULE_BUILD_DIR += getValue ("BASE_NAME", InfFile) + "\";
      MAKEFILE = $ (MODULE_BUILD_DIR) + "Makefile";
      genModuleMakefile ($ (MAKEFILE));
      addModuleToList ($MAKEFILE, MakefileList);
    }
    genTopMakefile ($ (MOD_BUILD_DIR2) + "Makefile")
  }
}
```

8.5.1.1 Macro definitions

8.5.1.1.1 Platform information

These come from `[Defines]` section in the DSC file.

```
MakefileList = $ (PLATFORM_MAKEFILE)
Foreach InfFile {
  MakefileList += $ (MODULE_MAKEFILE)
}
foreach Makefile in MakefileList {
  include_statement ($ (MODULE_BUILD_DIR)\Makefile, "
    PLATFORM_NAME = getValue("PLATFORM_NAME", DscFile);
    PLATFORM_GUID = getValue ("PLATFORM_GUID", DscFile);
    PLATFORM_VERSION = getValue ("PLATFORM_VERSION", DscFile);
```

```

PLATFORM_RELATIVE_DIR = getDirPart (ActivePlatform);
PLATFORM_DIR = "$(WORKSPACE)\\" + getDirPart(ActivePlatform);
pPLATFORM_OUTPUT_DIR = getValue ("OUTPUT_DIRECTORY", DscFile);
");
}

```

Example

```

PLATFORM_NAME = NT32
PLATFORM_GUID = EB216561-961F-47EE-9EF9-CA426EF547C2
PLATFORM_VERSION = 0.3
PLATFORM_RELATIVE_DIR = Nt32Pkg
PLATFORM_DIR = $(WORKSPACE)\Nt32Pkg
PLATFORM_OUTPUT_DIR = Build\NT32

```

8.5.1.1.2 Module information

These come from `[Defines]` section in the INF file and `[Components]` section in DSC file.

```

Foreach InfFile {
  include_statement ($ (MODULE_BUILD_DIR)\Makefile, "
    MODULE_NAME = getValue ("BASE_NAME", InfFile)
    MODULE_GUID = getValue ("FILE_GUID", InfFile)
    MODULE_VERSION = getValue ("VERSION_STRING", InfFile)
    MODULE_TYPE = getValue ("MODULE_TYPE", InfFile);
    MODULE_FILE_BASE_NAME = getValue ("BASE_NAME", InfFile)
    BASE_NAME = $ (MODULE_NAME)
    MODULE_RELATIVE_DIR = getDirPart (InfFile)
    MODULE_DIR = "$(WORKSPACE)\\" + getDirPart(InfFile)

  ");
}

```

Example

```

MODULE_NAME = HelloWorld
MODULE_GUID = 6987936E-ED34-44db-AE97-1FA5E4ED2116
MODULE_VERSION = 1.0
MODULE_TYPE = UEFI_APPLICATION
MODULE_FILE_BASE_NAME = HelloWorld
BASE_NAME = $(MODULE_NAME)
MODULE_RELATIVE_DIR = MdeModulePkg\Application\HelloWorld
MODULE_DIR = $(WORKSPACE)\MdeModulePkg\Application\HelloWorld

```

8.5.1.1.3 Build configuration

These come from `$(WORKSPACE)/Conf/target.txt` , command line options, or `[Defines]` section in DSC file.

```

ARCH = IA32
TOOLCHAIN_TAG = MYTOOLS
TARGET = DEBUG

```

8.5.1.1.4 Build directories

These are determined by build tools. Macro `DEST_DIR_OUTPUT` and `DEST_DIR_DEBUG` are generated for backward compatibility.

```

PLATFORM_BUILD_DIR = $(WORKSPACE)\Build\NT32
BUILD_DIR = $(WORKSPACE)\Build\NT32\DEBUG_MYTOOLS
BIN_DIR = $(BUILD_DIR)\IA32
LIB_DIR = $(BIN_DIR)
MODULE_BUILD_DIR = $(BUILD_DIR)\IA32\MdeModulePkg\Application\HelloWorld\HelloWorld

```

```

OUTPUT_DIR = $(MODULE_BUILD_DIR)\OUTPUT
DEBUG_DIR = $(MODULE_BUILD_DIR)\DEBUG
DEST_DIR_OUTPUT = $(OUTPUT_DIR)
DEST_DIR_DEBUG = $(DEBUG_DIR)

```

8.5.1.1.5 Tools flags,

These are used to concatenate the flags from different places in the predefined order. The order makes sure that the flags defined DSC file can override flags in INF file and default ones. In the code example below, the tools will expand the values into a single line - `$(TOOLS_DEF_LZMA_FLAGS)` does not appear in the Makefile, only the flag values appear.

```

LZMA_FLAGS    = $(TOOLS_DEF_LZMA_FLAGS) $(INF_LZMA_FLAGS) $(DSC_LZMA_FLAGS) $(DSC_INF_LZMA_FLAGS)
PP_FLAGS      = $(TOOLS_DEF_PP_FLAGS) $(INF_PP_FLAGS) $(DSC_PP_FLAGS) $(DSC_INF_PP_FLAGS)
SLINK_FLAGS   = $(TOOLS_DEF_SLINK_FLAGS) $(INF_SLINK_FLAGS) $(DSC_SLINK_FLAGS) $(DSC_INF_SLINK_FLAGS)
CC_FLAGS      = $(TOOLS_DEF_CC_FLAGS) $(INF_CC_FLAGS) $(DSC_CC_FLAGS) $(DSC_INF_CC_FLAGS)
APP_FLAGS     = $(TOOLS_DEF_APP_FLAGS) $(INF_APP_FLAGS) $(DSC_APP_FLAGS) $(DSC_INF_APP_FLAGS)
VFRPP_FLAGS   = $(TOOLS_DEF_VFRPP_FLAGS) $(INF_VFRPP_FLAGS) $(DSC_VFRPP_FLAGS) $(DSC_INF_VFRPP_FLAGS)
DLINK_FLAGS   = $(TOOLS_DEF_DLINK_FLAGS) $(INF_DLINK_FLAGS) $(DSC_DLINK_FLAGS) $(DSC_INF_DLINK_FLAGS)
ASM_FLAGS     = $(TOOLS_DEF_ASM_FLAGS) $(INF_ASM_FLAGS) $(DSC_ASM_FLAGS) $(DSC_INF_ASM_FLAGS)
TIANO_FLAGS   = $(TOOLS_DEF_TIANO_FLAGS) $(INF_TIANO_FLAGS) $(DSC_TIANO_FLAGS) $(DSC_INF_TIANO_FLAGS)
MAKE_FLAGS    = $(TOOLS_DEF_MAKE_FLAGS) $(INF_MAKE_FLAGS) $(DSC_MAKE_FLAGS) $(DSC_INF_MAKE_FLAGS)
ASMLINK_FLAGS = $(TOOLS_DEF_ASMLINK_FLAGS) $(INF_ASMLINK_FLAGS) $(DSC_ASMLINK_FLAGS) $(DSC_INF_ASMLINK_FLAGS)
ASL_FLAGS     = $(TOOLS_DEF_AS_L_FLAGS) $(INF_AS_L_FLAGS) $(DSC_AS_L_FLAGS) $(DSC_INF_AS_L_FLAGS)

```

8.5.1.1.6 Tools path

These come from the file specified by `TOOL_CHAIN_CONF` definition in `$(WORKSPACE)/Conf/target.txt`.

```

LZMA    = H:\dev\AllPackagesDev\IntelRestrictedTools\Bin\Win32\LzmaCompress.exe
PP      = C:\Program Files\Microsoft Visual Studio 8\VC\bin\cl.exe
SLINK   = C:\Program Files\Microsoft Visual Studio 8\VC\bin\lib.exe
CC      = C:\Program Files\Microsoft Visual Studio 8\VC\bin\cl.exe
APP     = C:\Program Files\Microsoft Visual Studio 8\VC\bin\cl.exe
VFRPP   = C:\Program Files\Microsoft Visual Studio 8\VC\bin\cl.exe
DLINK   = C:\Program Files\Microsoft Visual Studio 8\VC\bin\link.exe
ASM     = C:\Program Files\Microsoft Visual Studio 8\VC\bin\ml.exe
TIANO  = TianoCompress.exe
MAKE    = C:\Program Files\Microsoft Visual Studio 8\VC\bin\nmake.exe
ASMLINK = C:\WINDDK\3790.1830\bin\bin16\link.exe ASL = C:\ASL\iasl.exe

```

8.5.1.1.7 Shell commands

These are used to make sure that the file operations for both nmake and GNU make system become as the same as possible.

```

# shell commands for nmake
RD = rmdir /s /q
RM = del /f /q
MD = mkdir
CP = copy /y
MV = move /y

# shell commands for gnu make
RD = rm -r -f
RM = rm -f
MD = mkdir -p
CP = cp -u -f
MV = mv -f

```

8.5.1.1.8 Source files and target files list macro

In these, `<FILE_TYPES>` macros are generated from `$(WORKSPACE)/Conf/build_rule.txt` and files listed in `[Sources]` section in INF file, `"INC"` macro is generated from `[Includes]` section in DEC file and `[Packages]` section in INF file, `"LIBS"` macro is generated from `[LibraryClasses]` section in INF file and DSC file, and `"COMMON_DEPS"` macro is generated by parsing recursively the `"#include"` preprocessor directives in source code files.

```
C_CODE_FILES = $(WORKSPACE)\MdeModulePkg\App\Hello\HelloWorld.c
DYNAMIC_LIBRARY_FILE_LIST = $(DEBUG_DIR)\$(MODULE_NAME).dll
UNKNOWN_TYPE_FILE_LIST = $(DEBUG_DIR)\$(MODULE_NAME).efi
OBJECT_FILE_LIST = $(OUTPUT_DIR)\HelloWorld.obj
STATIC_LIBRARY_FILE_LIST = $(OUTPUT_DIR)\$(MODULE_NAME).lib

INC = <include search path list>
LIBS = <dependent library file list>
COMMON_DEPS = <header file list>
```

8.5.1.1.9 Target macros

In these `CODA_TARGET` is generated according to the last rule(s) in rule chains defined in `$(WORKSPACE)/Conf/build_rule.txt`.

```
INIT_TARGET = init
CODA_TARGET = $(DEBUG_DIR)\$(MODULE_NAME).efi
```

8.5.1.2 Target definitions

8.5.1.2.1 "all" target

Default target which actually executes against the `"mbuild"` target.

8.5.1.2.2 "pbuild" target

Target which is used to build the source files of current module only. It's always used in top-level makefile because the libraries will be built above all non-library modules.

```
pbuild: $(INIT_TARGET) $(CODA_TARGET)
```

8.5.1.2.3 "mbuild" target

Actual default target which is used for single module build mode. Because in single module build mode the top-level `Makefile` will not be called, the build system has to build libraries that the current module needs in module's `Makefile`. `"mbuild"` target is used for this purpose.

```
mbuild: $(INIT_TARGET) gen_libs $(CODA_TARGET)
gen_libs:
  cd $(BUILD_DIR)\X64\MdePkg\Library\DxePcdLib\DxePcdLib && "$(MAKE)"
$(MAKE_FLAGS)
  cd $(BUILD_DIR)\X64\MdePkg\Library\BaseLib\BaseLib && "$(MAKE)"
  cd $(MODULE_BUILD_DIR)
```

8.5.1.2.4 "init" target

Target used to print verbose information and create necessary directories used for build.

```
init:
  -@echo Building ... $(MODULE_NAME) $(MODULE_VERSION) [$(ARCH)] in platform $(PLATFORM_NAME) $(PLATFORM_VERSION)
  -@if not exist $(DEBUG_DIR) mkdir $(DEBUG_DIR)
  -@if not exist $(OUTPUT_DIR) mkdir $(OUTPUT_DIR)
```

8.5.1.2.5 Miscellaneous build targets

Targets which are used to build source files to object files and then in turn into final `.lib` file, `.efi` file or other files. These targets are generated according to the rule chains in `$(WORKSPACE)/Conf/build_rule.txt`. For example:

```
$(OUTPUT_DIR)\ModuleFile.obj : $(COMMON_DEPS)
    "$(CC)" /Fo$(OUTPUT_DIR)\ModuleFile.obj $(CC_FLAGS) $(INC) $(WORKSPACE)\MyPlatformPkg\MySubDir\ModuleFile.c

$(OUTPUT_DIR)\$(MODULE_NAME).lib : $(OBJECT_FILE_LIST)
    "$(SLINK)" $(SLINK_FLAGS) /OUT:$(OUTPUT_DIR)\$(MODULE_NAME).lib $(OBJECT_FILE_LIST)

$(DEBUG_DIR)\$(MODULE_NAME).dll : \
    $(OUTPUT_DIR)\$(MODULE_NAME).lib $(LIBS) $(MAKE_FILE)
    "$(DLINK)" /OUT:$(DEBUG_DIR)\$(MODULE_NAME).dll $(DLINK_FLAGS) $(DLINK_SPATH) $(LIBS) $(OUTPUT_DIR)\$(MODULE_NAME).lib

$(DEBUG_DIR)\$(MODULE_NAME).efi : $(DEBUG_DIR)\$(MODULE_NAME).dll
    GenFw -e $(MODULE_TYPE) -o $(DEBUG_DIR)\$(MODULE_NAME).efi $(DEBUG_DIR)\$(MODULE_NAME).dll
    $(CP) $(DEBUG_DIR)\$(MODULE_NAME).efi $(OUTPUT_DIR)
    $(CP) $(DEBUG_DIR)\$(MODULE_NAME).efi $(BIN_DIR)
    -$(CP) $(DEBUG_DIR)\*.map $(OUTPUT_DIR)

$(OUTPUT_DIR)\AutoGen.obj : \
$(WORKSPACE)\Build\MyPlatform\DEBUG_ICC\X64\MyPlatformPkg\MyModDir\MyModDir\DEBUG\AutoGen.c
    "$(CC)" /Fo$(OUTPUT_DIR)\AutoGen.obj $(CC_FLAGS) $(INC) $(WORKSPACE)\Build\MyPlatform\DEBUG_ICC\X64\MyPlatformPkg\MyModDir\
    \MyMod Dir\DEBUG\AutoGen.c
```

8.5.1.2.6 clean, cleanall, cleanlib

Targets used to delete part or all files generated during build.

```
clean:
    if exist $(OUTPUT_DIR) rmdir /s /q $(OUTPUT_DIR)

cleanall:
    if exist $(DEBUG_DIR) rmdir /s /q $(DEBUG_DIR)
    if exist $(OUTPUT_DIR) rmdir /s /q $(OUTPUT_DIR)
    del /f /q *.pdb *.idb > NUL 2>&1

cleanlib:
    cd $(BUILD_DIR)\X64\MdePkg\Library\DxePcdLib\DxePcdLib && \
        "$(MAKE)" $(MAKE_FLAGS) cleanall
    cd $(BUILD_DIR)\X64\MdePkg\Library\BaseLib\BaseLib && \
        "$(MAKE)" $(MAKE_FLAGS) cleanall
    cd $(MODULE_BUILD_DIR)
```

8.6 Binary Modules

EDK II accommodates distribution of binary module code for inclusion into a firmware volume. This feature is used by vendors who have a proprietary code base, but need to provide their customers with the ability to use that code in a platform. Vendors may protect their IP by distributing only module code in either lib, bin, or efi format, without distributing debug files or sources.

No Makefile is generated for binary only modules.

A binary module must have a `[Binaries]` section. It is recommended that binary INF files not be listed in DSC file so that the build tools will not try to do a module build for a binary module. The INF file of a binary module is always put in FDF file for flash image generation. The binary files can also be referenced directly in FDF. Please refer to Section 10 (Post-Build ImageGen Stage - FLASH Images) for details.

Binary modules are used only with FDF files unless a PCD using `PatchableInModule` access method is used by the binary module and the platform developer wants to change the value for this PCD in the binary module.

The build command has an option flag, `--ignore-sources`, that will treat all INF files listed in the DSC file as though they were binary INF files. The build will not generate any Makefiles, totally ignoring any files listed in a `[Sources]` section. If a module is specified in the DSC file that does not contain a `[Binaries]` section, the build will provide an appropriate error message and terminate. This mode allows distribution of binary modules with source files that can be used during debugging.

8.7 Generated AsBuilt INF Files

The EDK II build system will generate an INF file for every module that is built from source files. Comments that would be required in the INF file for the UEFI Packaging Tool to create a distribution package must be preserved. The AsBuilt INF file must be an ASCII formatted file with DOS end-of-line (CRLF) characters. Portions of the AsBuilt INF are generated during pre-build, while other portions are determined after the images have been created during the \$Make stage. Refer to the EDK II Module Information (INF) File Specification for the exact format for content in these sections. AsBuilt INFs are only created from building source modules.

8.7.1 Header Section

The header of the AsBuilt INF file will use the same content and format as the INF file except when a comment section that follows the source header contains the following line:

```
## @BinaryHeader
```

1. If the above tag is located, then the tool must ignore the source header and used the Binary header block instead.
2. If using the Binary header block, the tools must replace `@BinaryHeader` with `@file` in the AsBuilt INF.
3. The tool must insert the following four lines between the description and copyright line regardless of the header used to create the AsBuilt INF:

```
#  
# DO NOT EDIT  
# FILE auto-generated Binary INF  
#
```

Note: The copyright date in the source INF should be updated every time a change is made to the INF file. Since every bug fix or new feature added to the source code requires that at least one of the `VERSION_STRING` values to be updated, the binary header should carry the same copyright date as the source header copyright date it was generated from.

Note: When generating the AsBuilt INF, if the source INF file contains the Doxygen tag, `@BinaryHeader`, the content from this section (which matches the format of the standard header) will replace the content from the standard header. The `@BinaryHeader` tag will be replaced with the `@file` tag as the first line of the AsBuilt INF file.

8.7.2 [Defines] Section

The following elements of the source INF will be copied into the `[Defines]` section of the AsBuilt INF file if and only if they exist in the source INF. The `INF_VERSION` in the AsBuilt INF File will be updated to match the version number in the EDK II INF Specification that was used at the time the tool code to create the AsBuilt file was updated, even if the `INF_VERSION` in the source INF was a lower version, such as `0x00010005`. If the EDK II INF Specification version in the source INF is greater than the version embedded in the tool, the tools should replace the version value with the version that is embedded in the tool, lowering the value.

Macros definitions ("DEFINE" statements) are not listed in the AsBuilt INF file. Instead, the macro value (where it was used) will be expanded in the path and value statements.

```
<TS> "[Defines]" <EOL>
<TS> "INF_VERSION" <Eq> <CurrentInfSpecificationVersion> <EOL>
<TS> "BASE_NAME" <Eq> <BaseName> <EOL>
<TS> "FILE_GUID" <Eq> <RegistryFormatGUID> <EOL>
<TS> "MODULE_TYPE" <Eq> <Edk2ModuleType> <EOL>
[<TS> "UEFI_SPECIFICATION_VERSION" <Eq> <VersionVal> <EOL>]
[<TS> "PI_SPECIFICATION_VERSION" <Eq> <VersionVal> <EOL>]
[<TS> "VERSION_STRING" <Eq> <DecimalVersion> <EOL>]
[<TS> "PCD_IS_DRIVER" <Eq> <PcdDriverType> <EOL>]
[<TS> "ENTRY_POINT" <Eq> <CName> <EOL>]*
[<TS> "UNLOAD_IMAGE" <Eq> <CName> <EOL>]*
[<TS> "CONSTRUCTOR" <Eq> <CName> <EOL>]*
[<TS> "DESTRUCTOR" <Eq> <CName> <EOL>]*
[<TS> "SHADOW" <Eq> <BoolType> <EOL>]
[<TS> "PCI_VENDOR_ID" <Eq> <UINT16> <EOL>]
[<TS> "PCI_DEVICE_ID" <Eq> <UNIT16> <EOL>]
[<TS> "PCI_CLASS_CODE" <Eq> <UINT8> <EOL>]
[<TS> "PCI_REVISION" <Eq> <UINT8> <EOL>]
[<TS> "BUILD_NUMBER" <Eq> <UINT16> <EOL>]
[<TS> "MODULE_UNI_FILE" <Eq> <Filename> <EOL>]
[<TS> "SPEC" <MTS> <Identifier> <Eq> <DecimalVersion> <EOL>]*
[<TS> "UEFI_HII_RESOURCE_SECTION" <Eq> <TrueFalse> <EOL>]
```

Parameters

MODULE_UNI_FILE

If the source module contains this entry, the tools must create a USC-2LE encoded file in the module's `OUTPUT` directory, ensuring that any of the tags that refer to BINARY content (`@BinaryHeader`) are used in place of tags that do not contain the word `BINARY` .

CurrentInfSpecificationVersion

This is the version of the EDK II INF Specification at the time the code (in the build tools) to generate the AsBuilt INF is updated.

Example

```
[Defines]
INF_VERSION      = 0x00010017
BASE_NAME        = DxeCore
MODULE_UNI_FILE  = DxeCore.uni
FILE_GUID        = D6A2CB7F-6A18-4e2f-B43B-9920A733700A
MODULE_TYPE      = DXE_CORE
VERSION_STRING   = 1.0

ENTRY_POINT      = DxeMain
```

8.7.3 [LibraryClasses] Section

This section must list (in comments) every library instances that gets linked with the module. A Doxygen tag, `@LIB_INSTANCES` in a comment must precede the list of library instances.

Example

```
[LibraryClasses]
## @ LIB_INSTANCES
# MdePkg/Library/BaseDebugLibSerialPort/BaseDebugLibSerialPort.inf
```

8.7.4 [Packages] Section

This section is required if there are PCDs listed in the `[PatchPcd]` and `[PcdEx]`, the packages that declare the PCDs that are list must be listed here. The format for the PCD entries is defined in the Module Information (INF) File Specification.

Example

```
[Packages.IA32]
MdePkg/MdePkg.dec
MdeModulePkg/MdeModulePkg.dec
```

8.7.5 [Guids] Section

All GUIDs that are listed in the source INF and their usage (if available) must be include in this section. Usage information may be modified based on feature flag expressions that are evaluated during the build. For example, the source INF may have a `SOMETIMES_PRODUCES` usage that may be changed to `PRODUCES` in the AsBuilt INF file if the build uses a feature flag to include the item.

Example

```
[Guids.IA32]
## PRODUCES ## Event
gEfiEventMemoryMapChangeGuid

## CONSUMES ## UNDEFINED
gEfiEventVirtualAddressChangeGuid

## CONSUMES ## UNDEFINED
## PRODUCES ## Event
gEfiEventExitBootServicesGuid

## CONSUMES ## HOB
gEfiHobMemoryAllocModuleGuid
```

8.7.6 [Protocols] Section

All Protocols that are listed in the source INF and their usage (if available) must be include in this section. The format for the Protocol entries is defined in the Module Information (INF) File Specification. Usage information may be modified based on feature flag expressions that are evaluated during the build. For example, the source INF may have a `SOMETIMES_PRODUCES` usage that may be changed to `PRODUCES` in the AsBuilt INF file if the build uses a feature flag to include the item.

```
[Protocols.IA32]
## PRODUCES
## SOMETIMES_CONSUMES
gEfiDecompressProtocolGuid

## SOMETIMES_PRODUCES ## Produces when PcdFrameworkCompatibilitySupport is set
gEfiLoadPeImageProtocolGuid

## SOMETIMES_CONSUMES
## SOMETIMES_CONSUMES
gEfiSimpleFileSystemProtocolGuid
```

8.7.7 [PPIs] Section

All Ppis that are listed in the source INF and their usage (if available) must be include in this section. The format for the PPI entries is defined in the Module Information (INF) File Specification. Usage information may be modified based on feature flag expressions that are evaluated during the build. For example, the source INF may have a `SOMETIMES_PRODUCES` usage that may be changed to `PRODUCES` in the AsBuilt INF file if the build uses a feature flag to include the item.

```
[Ppis.IA32]
# SOMETIMES_CONSUMES # PeiReportStatusService is not ready if this PPI doesn't exist
gEfiPeiStatusCodePpiGuid

# SOMETIMES_CONSUMES # PeiResetService is not ready if this PPI doesn't exist
gEfiPeiResetPpiGuid

## CONSUMES
gEfiDxeIplPpiGuid

## PRODUCES
gEfiPeiMemoryDiscoveredPpiGuid

## SOMETIMES_CONSUMES
gEfiPeiDecompressPpiGuid

## SOMETIMES_PRODUCES
## NOTIFY
# SOMETIMES_PRODUCES # Produce FvInfoPpi if the encapsulated FvImage is found
gEfiPeiFirmwareVolumeInfoPpiGuid
```

8.7.8 [PatchPcd] Section

All PCDs that are listed in the source INF, that are defined as `PatchableInModule` in the DSC file must be inserted into this section. The current value and the offset into the `PE32` (.efi) file must be included in the entry for each PCD listed in this section of the AsBuilt INF file. If the usage is available, that information must also be included. The format for the PCD entries is defined in the Module Information (INF) File Specification.

To support override of the Formset class GUID in a binary HII driver, the build system was enhanced as follows:

- Build tool will collect all VFR file names in one module and output them into a temp file, for example, `VfrFileName.txt`.
- After creating the EFI image, the **GenPatchPcdTable** tool will be used to create PatchPcd information with input from the MAP, EFI and `VfrFileName.txt`.
- **GenPatchPcdTable** will get HII data in the binary EFI image, and locate the reserved empty Formset class GUID slot (all zero GUID). If the empty slot is found, a Patchable PCD `PcdHiiFormSetClassGuid##VfrFileName` (type `VOID *` for GUID) will be auto generated. `VfrFileName` is obtained from the `VfrFileName.txt`.
- Usage information may be modified based on feature flag expressions that are evaluated during the build. For example, the source INF may have a `SOMETIMES_PRODUCES` usage that may be changed to `PRODUCES` in the AsBuilt INF file if the build uses a feature flag to include the item.

```
[PatchPcd.IA32]
## SOMETIMES_CONSUMES
gEfiMdeModulePkgTokenSpaceGuid.PcdLoadFixAddressBootTimeCodePageNumber|0x00000000|0xC584

## SOMETIMES_CONSUMES
gEfiMdeModulePkgTokenSpaceGuid.PcdLoadFixAddressRuntimeCodePageNumber|0x00000000|0xC588
```

8.7.9 [PcdEx] Section

All PCDs that are listed in the source INF, that are defined as `DynamicEx` in the DSC file must be inserted into this section. In general, values for the `DynamicEx` PCDs are global to a platform, and must not be inserted into the AsBuilt INF file. If the usage is available, that information must also be included. The format for the PCD entries is defined in the Module Information (INF) File Specification.

If the `DynamicEx` PCD was assigned as subtype HII, then for modules that produce IFR for setup screens, the following is required. If any of the fields of an EFI VarStore in the IFR are associated with a PCD, then the AsBuilt INF must declare that relationship. Since a module that produces IFR may not have C code that uses the PCDs we need here, the source INF file may not list those PCDs. Instead, the build tools when building a module that contains IFR must determine if there is a mapping between PCDs and an EFI VarStore and add those relationships to the AsBuilt INF. The syntax of the `[PcdEx]` for AsBuilt INF files is augmented by additional comment information for PCDs that are expected to be used with HII. The current `<Usage>` comment will be followed by Variable Name, Variable GUID C Name, and byte offset value which is the same order used in a DSC file for a `[PcdsDynamicExHii]` section, separated by the "|" field separation character.

Usage information may be modified based on feature flag expressions that are evaluated during the build. For example, the source INF may have a `SOMETIMES_PRODUCES` usage that may be changed to `PRODUCES` in the AsBuilt INF file if the build uses a feature flag to include the item.

```
[PcdEx.IA32]
## SOMETIMES_PRODUCES
## SOMETIMES_CONSUMES
gEfiMdeModulePkgTokenSpaceGuid.PcdConOutRow

## SOMETIMES_PRODUCES
## SOMETIMES_CONSUMES
gEfiMdeModulePkgTokenSpaceGuid.PcdConOutColumn
```

8.7.10 [Depex] Section

The complete dependency expression including all dependencies from the libraries linked with the module must be included in comments in this section. The format for this dependency expression is defined in the Module Information (INF) File Specification.

Example

```
[Depex]
# NOT (gEfiHiiDatabaseProtocolGuid AND gEfiHiiStringProtocolGuid)
# OR gPcdProtocolGuid
```

8.7.11 [BuildOptions] Section

The format for the build option entries is defined in the Module Information (INF) File Specification. All entries in this section appear in comments, beginning with the following line.

```
## @AsBuilt
```

Example

```
[BuildOptions.IA32]
## @AsBuilt
## MSFT:DEBUG_VS2008x86_IA32_SYMRENAME_FLAGS = Symbol renaming not needed for
## MSFT:DEBUG_VS2008x86_IA32_ASXLINK_FLAGS = /NODEFAULTLIB /ENTRY:ReferenceAcpiTable /SUBSYSTEM:CONSOLE
## MSFT:DEBUG_VS2008x86_IA32_VFR_FLAGS = -l -n
## MSFT:DEBUG_VS2008x86_IA32_PP_FLAGS = /nologo /E /TC /FIAutoGen.h
## MSFT:DEBUG_VS2008x86_IA32_GENFW_FLAGS =
## MSFT:DEBUG_VS2008x86_IA32_OPTROM_FLAGS = -e
```

```

## MSFT:DEBUG_VS2008x86_IA32_SLINK_FLAGS = /NOLOGO /LTCG
## MSFT:DEBUG_VS2008x86_IA32_ASM_FLAGS = /nologo /c /wX /w3 /Cx /coff /Zd /Zi
## MSFT:DEBUG_VS2008x86_IA32_ASX_FLAGS =
## MSFT:DEBUG_VS2008x86_IA32_CC_FLAGS = /nologo /c /wX /GS- /w4 /Gs32768 /D UNICODE /O1ib2 /GL /FIAutoGen.h /EHs-c- /GR- /GF
/Gy /Zi /Gm
## MSFT:DEBUG_VS2008x86_IA32_VFRPP_FLAGS = /nologo /E /TC /DVFRCCOMPIL /FI$(MODULE_NAME)StrDefs.h
## MSFT:DEBUG_VS2008x86_IA32_ASXCC_FLAGS = /nologo /c /FIAutoGen.h /TC /Dmain = ReferenceAcpiTable
## MSFT:DEBUG_VS2008x86_IA32_APP_FLAGS = /nologo /E /TC
## MSFT:DEBUG_VS2008x86_IA32_DLINK_FLAGS = /NOLOGO /NODEFAULTLIB /IGNORE:4001 /OPT:REF /OPT:ICF=10 /MAP /ALIGN:32 /SECTION:.
xdata,D /SECTION:.pdata,D /MACHINE:X86 /LTCG /DLL /ENTRY:$(IMAGE_ENTRY_POINT) /SUBSYSTEM:EFI_BOOT_SERVICE_DRIVER /SAFESEH:NO /
BASE:0 /DRIVER /DEBUG /PDB:$(OUTPUT_PATH)\$(PACKAGE_NAME)_$(PACKAGE_GUID)_$(PACKAGE_VERSION)\$(PACKAGE_RELATIVE_DIR)\$(MODULE_
FILE_BASE_NAME)\DEBUG\IA32\$(BASE_NAME).pdb /PDBSTRIPPED:$(OUTPUT_PATH)\$(PACKAGE_NAME)_$(PACKAGE_GUID)_$(PACKAGE_VERSION)\$(P
ACKAGE_RELATIVE_DIR)\$(MODULE_FILE_BASE_NAME)\DEBUG\IA32\$(BASE_NAME)_Stripped.pdb
## MSFT:DEBUG_VS2008x86_IA32_ASXPP_FLAGS = /nologo /E /C /FIAutoGen.h
## MSFT:DEBUG_VS2008x86_IA32_OBJCOPY_FLAGS = objcopy not needed for
## MSFT:DEBUG_VS2008x86_IA32_MAKE_FLAGS = /nologo
## MSFT:DEBUG_VS2008x86_IA32_ASMLINK_FLAGS = /nologo /tiny

```

8.7.12 [Binaries] Section

The format for the binaries section entries is listed in the Module Information (INF) File Specification. The a binary PE32 file, with the .efi extension, was created by the build, it must be listed in this section. All files listed in this section must be placed in a section with the corresponding architectural modifier, such as [Binaries.IA32], where IA32 is the architectural modifier. The examples below do not cover all of the potential file types that may appear in a binary INF file; it does show the file types that must be placed into the auto-generated INF file created during a build.

The generic format for these entries are:

```
<TS> BinaryType|[RelativePath]Filename.Extension
```

The following is an example of an EFI file format:

```
<TS> PE32|Filename.efi
```

The following is an example of a DEPEX file format:

```
<TS> DXE_DEPEX|Filename.depex
```

If the build produces a PDB or SYM file, an entry must be placed in the [Binaries.\$(ARCH)] section. The following example shows an entry for a PDB file.

```
<TS> DISPOSABLE|Filename.pdb <EOL>
```

If a filename is a fully qualified path and filename, such as a ROM filename, the build tool must copy that file into the module's OUTPUT directory, then insert the line as though it were in the directory as part of the build. For a ROM file, the entry must use the following format:

```
<TS> BIN|Filename.rom <EOL>
```

For AML files from a platform, the entry must use the following format:

```
<TS> ASL|Filename.aml <EOL>
```

For ACPI files from a platform, the entry must use the following format:

```
<TS> ACPI|Filename.acpi <EOL>
```

For a Binary or raw binary file, the entry may use either of the following two formats:

```

<TS> RAW|Filename.raw <EOL>
<TS> BIN|Filename.bin <EOL>

```

If the tools cannot determine the content, the binary type, the tools must use the BIN binary type.

In the above examples, the Filename may be preceded by a module relative path subdirectory as in the following example:

```
<TS> PE32|Ia32/Filename.efi  
<TS> RAW|Vt0/Bin/ResetVec.ia32.raw
```

Example

```
[Binaries.IA32]  
  PE32|Ia32/DxeCore.efi  
  DISPOSABLE|Ia32/DxeCore.pdb
```

8.7.13 [Sources] Section

The build tools must never add the [Sources] section or the name of the files from a sources section.

8.7.14 [UserExtensions] Section

Any [UserExtensions] sections that are present in the source INF with a UserId of "TianoCore" will be copied into the generated INF. [UserExtensions] sections with other UserId values will not be copied to the generated INF. The EDK II build tools will ignore these sections, however other vendors may provide tools that have a priori knowledge of how to process these sections.

9 BUILD OR \$(MAKE) STAGE

This chapter describes the processing of the source files into EFI files.

The make stage starts out by building required libraries, followed by EDK II modules. The outputs of this stage are linked PE32+/COFF images that have been processed to replace the standard header with an appropriate EFI header.

How a file will be processed is defined in the file specified by the `BUILD_RULE_CONF` statement in `target.txt` or the default file `$(WORKSPACE)/Conf/build_rule.txt`. The build system will use the sections in this file to convert to actions and targets in the Makefile. In the previous stage, Pre-Build AutoGen Stage, those rules were used to generate Makefiles.

9.1 Overview

From a platform point of view (remember, all builds are in the context of a platform, even when the desired output may only be a driver), what will be done in \$(MAKE) stage includes building library modules, building non-library modules and finally (if the desired output is a file to be burned into a flash part on a physical platform) generating flash image(s).

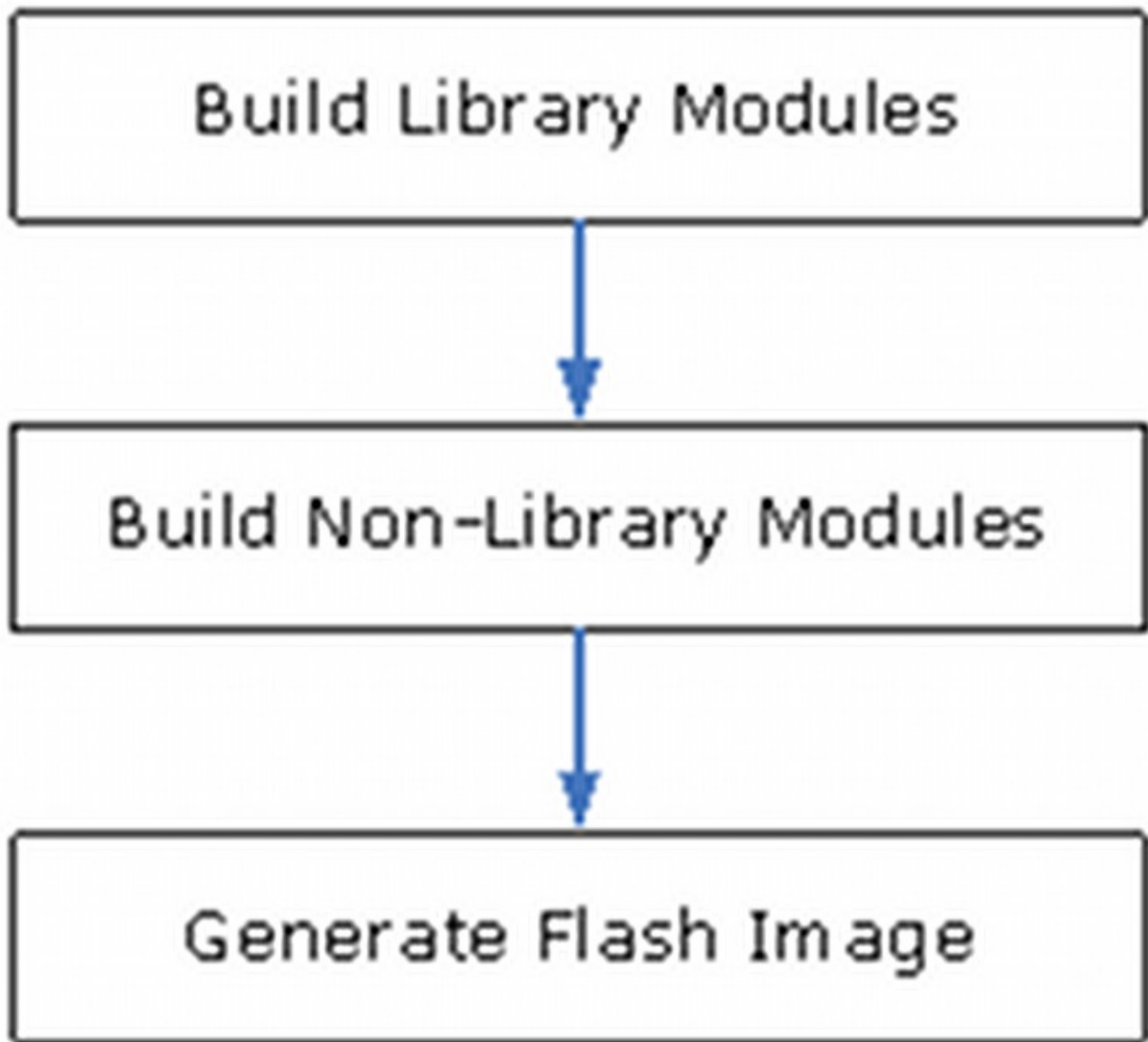


Figure 20 EDK II Build Process - Platform Point of View (PoV)

From a module point of view, things done in \$(MAKE) stage includes preprocessing, compiling or assembling, static/dynamic linking and module image generation.

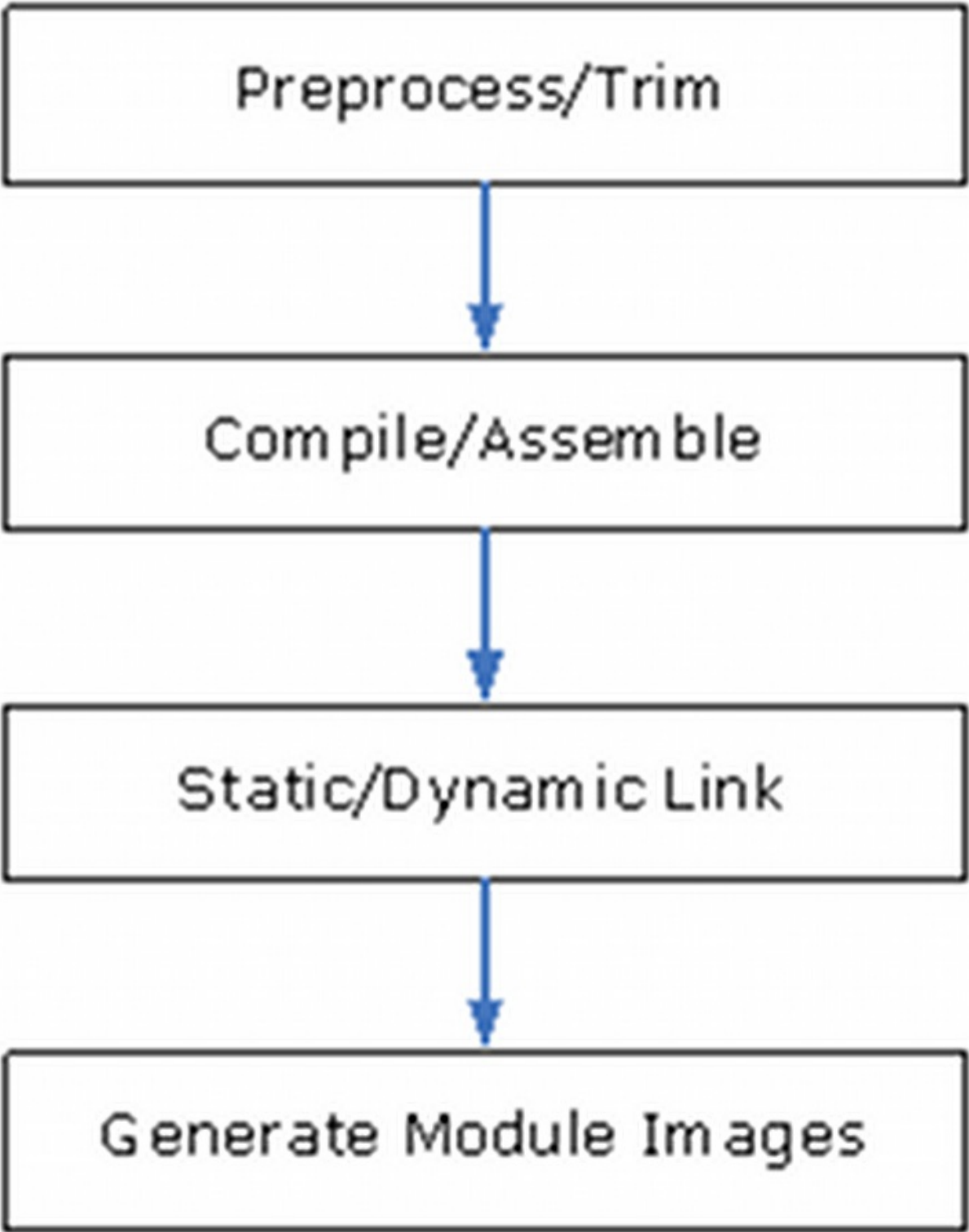


Figure 21 EDK II Build Process - Module PoV

9.1.1 File Extensions for UEFI image files.

This section details the intermediate file extensions that generated by the \$(MAKE) stage of the build process. This stage involves processing source files and generating dynamic objects which are further processed by the **GenFw** tool to create .efi files.

Table 17 \$(MAKE) Stage Intermediate Output File Extensions

Extension	Description
.obj	Object files generated by \$(MAKE) stage
.lib	Static Linked files generated by \$(MAKE) stage
.dll	Dynamically Linked files generated by \$(MAKE) stage
.aml	ACPI code files generated by \$(MAKE) stage
.i, .iii	Trim and C Pre-Processor output files
.bin	Microcode files

Table 18 \$(MAKE) Stage Output File Extensions

Extension	Description
.efi	Non UEFI Applications, DXE Drivers, DXE Runtime Drivers, DXE SAL Drivers have the Subsystem type field of the DOS/TE header set to <code>EFI_IMAGE_SUBSYSTEM_EFI_APPLICATION</code> , <code>EFI_IMAGE_SUBSYSTEM_EFI_BOOT_SERVICE_DRIVER</code> , <code>EFI_IMAGE_SUBSYSTEM_EFI_RUNTIME_DRIVER</code> and <code>EFI_IMAGE_SUBSYSTEM_SAL_RUNTIME_DRIVER</code> respectively
	For a Security Module, the Subsystem type is set to <code>EFI_IMAGE_SUBSYSTEM_EFI_BOOT_SERVICE_DRIVER</code> .
	For <code>PEI_CORE</code> , <code>DXE_CORE</code> , <code>PEIM</code> , <code>DXE_SMM_DRIVER</code> , <code>UEFI_APPLICATION</code> , <code>UEFI_DRIVER</code> , the Subsystem type is set to <code>EFI_IMAGE_SUBSYSTEM_EFI_BOOT_SERVICE_DRIVER</code> .
.acpi	ASL or IASL compiled ACPI tables
.depex	Compiled dependency sections
.mcb	Microcode Binary files

Additional modifications to the files are permitted. Modifications that recommended are as follows:

```
TimeStructure can be modified to a given date using a data structure of
tm_mon, // months since January, [0,11]
tm_mday, // day of the month [1,31]
tm_year, // years since 1900
tm_hour, // hours since midnight [0,23]
tm_min, // minutes after the hour [0,59]
tm_sec, // seconds after the minute [0,59]
```

Subsystemfield is changed to one of the following:

```
EFI_IMAGE_SUBSYSTEM_UNKNOWN          0
EFI_IMAGE_SUBSYSTEM_NATIVE           1
EFI_IMAGE_SUBSYSTEM_WINDOWS_GUI      2
EFI_IMAGE_SUBSYSTEM_WINDOWS_CUI      3
EFI_IMAGE_SUBSYSTEM_OS2_CUI          5
EFI_IMAGE_SUBSYSTEM_POSIX_CUI        7
EFI_IMAGE_SUBSYSTEM_EFI_APPLICATION  10
EFI_IMAGE_SUBSYSTEM_EFI_BOOT_SERVICE_DRIVER 11
EFI_IMAGE_SUBSYSTEM_EFI_RUNTIME_DRIVER 12
EFI_IMAGE_SUBSYSTEM_SAL_RUNTIME_DRIVER 13
```

The Machine value in the PE image file header is used to indicate the machine code type of the image. The following values are set for UEFI images:

```
EFI_IMAGE_MACHINE_IA32  0x014C
EFI_IMAGE_MACHINE_IA64  0x0200
EFI_IMAGE_MACHINE_x64   0x8664
EFI_IMAGE_MACHINE_EBC   0x0EBC
```


9.2 Preprocess/Trim

Preprocessing is an intrinsic behavior of C compiler and will be always done automatically without explicitly calling. A separate preprocessing step is needed by those non-C files which have no preprocessing supported in their compiler or assembler.

For example, in order to use macros defined in C header files, `#include` directives can be used in an assembly file. A separated preprocessing step will be used to complete macro replacement before calling assembler. The .VFR files, .asl files and .dxs files also need preprocessing step to allow using macros in them.

In addition, the preprocessed assembly files, VFR files and .dxs files need an additional Trim step to remove unnecessary contents left by preprocessor.

9.3 Compile/Assembly

For C and assembly files, the usual C compiler or assembler is used to generate object files for them.

For VFR files, there's a special VfrCompiler tool used to generate C and header files from them, then the standard compiler is used to generate object files.

There's a special C file with .aslc extension. The standard C compiler is used to generate object files.

For ASL files, the ASL compiler is used to generate ACPI machine language files.

9.4 Static Link

Static link step is used for all modules with C files. For library modules, linking all object files into static library file is the last step. A static link step for non-library modules is not necessary to generate the final image file; however, for better optimization purpose for MSFT tool chains this step is included.

For those modules with no C files, the static link step is skipped.

9.5 Dynamic Link

Dynamic link step is used for non-library modules which have .c files, .asc files and/or .asm16 (real mode assembly) files declared in their INF files.

The static library file generated in static link step will be linked (DLINK) together with other static library files generated from dependent library modules into .dll file.

Object files generated from ASLC files will be linked (DLINK) to .dll file directly without static link step.

Object files generated from real mode assembly files are linked to .com files by real mode linker (ASMLINK).

9.6 Generate Module Images

The final images generated by building a module are files which can be recognized by EFI/Framework protocols. The types of those files supported by default are EFI executable image file (.efi), ACPI machine language file (.aml), ACPI table file (.acpi), real mode executable file (.com) and microcode binary file (.bin).

The .efi file will be generated for non-library modules which have C files declared. It's converted from .dll file created during the dynamic link step by the GenFw tool. Also:

- The .aml file is generated from .asl file in Compile/Assembly step.
- The .acpi file is converted from .dll file by the **GenFw** tool.
- The .com file is generated in the Dynamic Link step by real mode linker (ASMLINK).
- The .bin file is converted from .txt file by the **GenFw** tool.

9.6.1 GenFw

This tool is used to generate UEFI Firmware Image files based on Component or Module types listed in the INF files from the PE/PE32+/COFF images generated by the third party tool chains. This takes `.dll` files created during the compile portion of the \$(MAKE) stage, converting the header and creating the .efi files. Additional functions of the GenFw tool are discussed in Post-Build ImageGen Stage - FLASH Images, Section 10.

9.7 Generate Platform Images

The final images generated by building a platform are always FVs or FDs if an FDF file is declared in platform's DSC file. The **GenFds** tool is used for this purpose. This is the final step of building a platform. For details regarding **GenFds** please refer to Post-Build ImageGen Stage - FLASH Images, Section 10.

10 POST-BUILD IMAGEGEN STAGE - FLASH

This chapter describes the processing of the EFI files generated by the \$(MAKE) Stage into FLASH binary images. Some of the PCDs defined or used in conditional directives in the FDF are set in the platform's DSC file. The tools must make at least one pass over the DSC file to get PCD values for conditional directives and other PCD entries used in the FDF file. If a FeatureFlag or FixedAtBuild PCD value, used in a conditional directive, cannot be determined the build must break.

For the remainder of this chapter, the `WORKSPACE` and `$(WORKSPACE)` refer to the ordered list of directories specified by the combination of `WORKSPACE + PACKAGES_PATH`.

10.0.1 ImageGen File Extensions

Table 19 and Table 20 describe intermediate file extensions and final file extensions in the ImageGen stage of the build for a platform. The ImageGen stage takes the output of the \$(MAKE) stage (typically the `.efi` files) and converts the files into EFI section files using the **GenSec** tool. The next step combines the section files into FFS files using the **GenFfs** tool. Once the Ffs files have been generated, they are combined into an FV image file using the **GenFv** tool. FV image files are combined into FD image files by the **GenFds** tool (which also controls all of the other steps in this stage).

Binary files listed in the FDF file's [FD] region section are included without processing. This allows for the addition of VPD data files (generated during the AutoGen Stage) to be included in the FD output file.

Table 19 GenFds Image Generation: Intermediate File Extensions

Input Extension	Output Extension	Description
.efi	.pe32	EFI_SECTION_PE32
.pe32, .ui, .ver	.com	EFI_SECTION_COMPRESSION
.ui	.ui	EFI_SECTION_USER_INTERFACE
.depex	.dpx	EFI_SECTION_PEI_DEPEX or EFI_SECTION_DXE_DEPEX
.tmp, .sec	.guided	EFI_SECTION_GUID_DEFINED
.ver	.ver	EFI_SECTION_VERSION
.acpi, .aml, .bin, .bmp	.raw	EFI_SECTION_RAW
ANY	SAME as Input	EFI_SECTION_FREEFORM_SUBTYPE_GUID
.com, .dpx, .guided, .pe32, .ui, .ver	.ffs	FFS file images
.ffs	.fv	Firmware Volume Image files
.fv	.sec	
.txt	.mcb	Microcode Binary File generated from the Microcode text files
.map	.bin	VPD binary image file (created by VPD_TOOL) where the file name is the GUID of the VPD tool.

Table 20 ImageGen Final Output File Extensions

Input Extensions	Output Extension	Description
------------------	------------------	-------------

.fv, .mcb	.fd	Firmware Device Images
.efi, .pe32	.rom	UEFI PCI Option ROM Images

For UEFI compliant PCI Option ROMs, the EfiRom tool is used to process `.efi` or `.pe32` files into the `.rom` file.

For UEFI applications, the `.efi` file generated at the end of the \$(MAKE) stage can be used directly, or, if the application will be included as part of a flash device image (all of the shell applications) the `.efi` file is processed using the standard steps for including a driver in an image.

10.1 Overview of Flash Device Layout

The **GenFds** tool is typically called after a platform build's \$(MAKE) Stage completes. The **build.exe** command will call **GenFds**. The **GenFds** program can also be executed by the developer from the command line.

In order to execute from the command-line, the tool needs to have its environment setup. The following is an example of executing GenFds as a stand-alone command.

```
GenFds -f $(WORKSPACE)\Nt32Pkg\Nt32Pkg.fdf \  
-o $(WORKSPACE)\Build\Nt32\DEBUG_MYT00LS -t MYT00LS -b DEBUG -v \  
-p $(WORKSPACE)\Nt32Pkg\Nt32Pkg.dsc -a IA32
```

GenFds

calls several other tools during the generation of an FD image:

GenSec

This application is used to generate valid `EFI_SECTION` type files from PE32/PE32+/ COFF image files or other binary files. The utility will attach a valid section or PEIM header to the input file as defined in the PI specification.

GenFfs

This application is used to generate FFS files for inclusion in a firmware volume. Rules specified in the FDF file stipulate how the FFS file will be organized (what kind of sections should reside in it and in what format).

GenFv

This application is used to generate FV image by taking what and how to place FFS into it from the corresponding FV.inf file.

GenFw

This application is used to generate UEFI Firmware Image files based on Component or Module types listed in the INF files from the PE/PE32+/COFF images generated by the third party tool chains.

GenVtf

This application generates the Boot Strap File (AKA Volume Top File, or VTF) for IA32 X64, and IPF images.

Figure 22 shows how the above tools involved in the GenFds process.

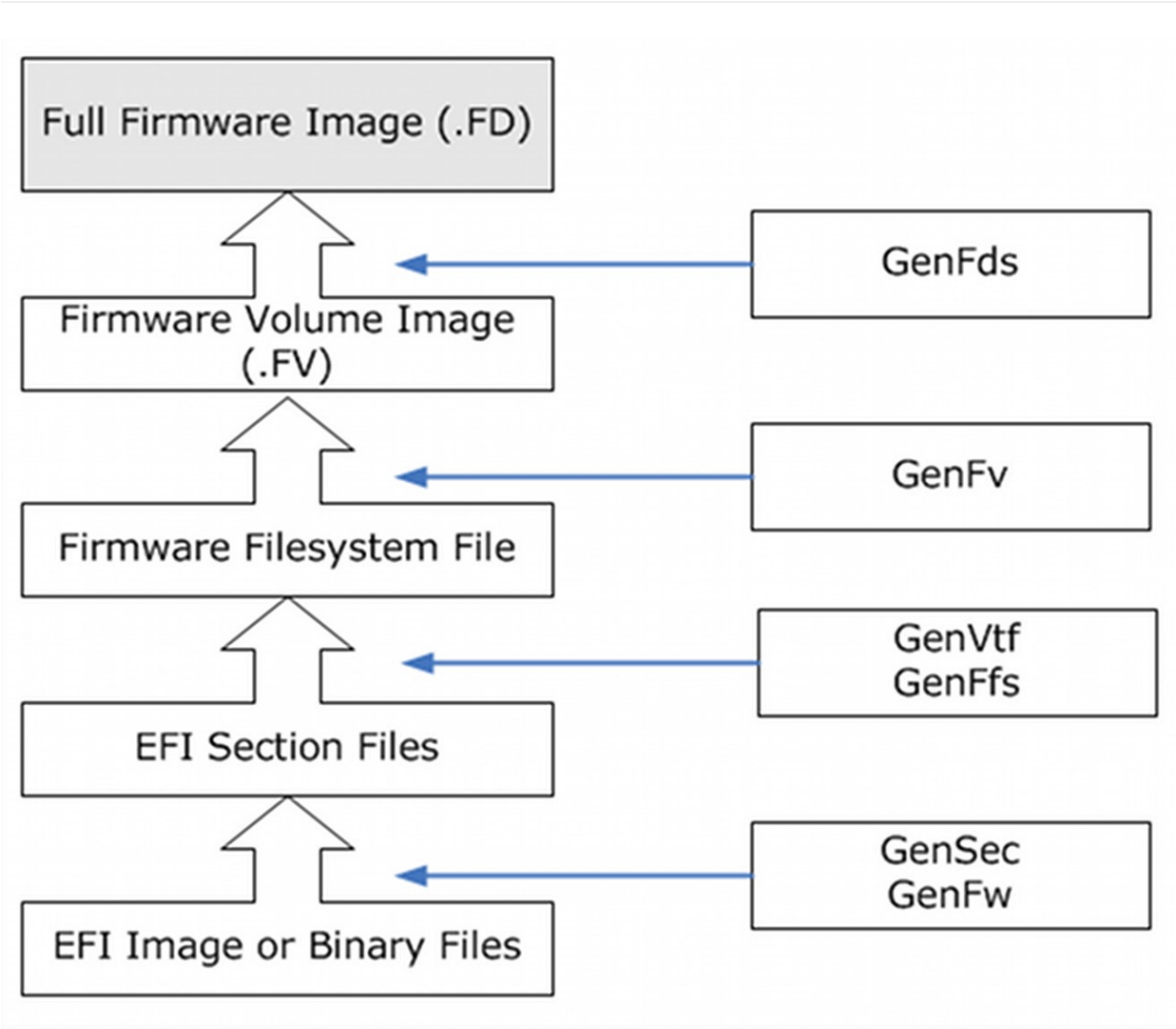


Figure 22 FD Image Generation Process

10.2 Parsing FDF Meta-Data File

GenFds get the flash image organization information from the FDF file which is specified in command line with the **-f** option. Most files that comprise the flash image are described by **INF** and **FILE** statements in FV sections of an FDF file. These files contain file name, file type and other useful information that let **GenFds** know which rule specified in FDF file must be used to generate the FFS file. The location of the output directory containing the image files created by **GenFw** or provided as binary images from \$(MAKE) stage is described in the **.dsc** file which is specified in command line using the **-p** option. As more than one architecture might be supported by the platform, the **-a** option clarifies outputs for the different architectures. Some binary files, such as a VPD binary file (not generated by the build) and INF files that specify binary files, such as a binary module containing microcode binaries, may be listed in an [FD] region section.

INF files are listed in a FV in the FDF file following the format in the example:

```
INF MyPlatform/SecCore/SecCore.inf
INF MdeModulePkg/Core/Pei/PeiMain.inf
INF MdeModulePkg/Universal/PCD/Pei/Pcd.inf
INF IntelFrameworkModulePkg/Universal/StatusCode/Pei/PeiStatusCode.inf
INF IntelFrameworkModulePkg/Universal/VariablePei/VariablePei.inf
```

All the paths in the above example are relative to the **WORKSPACE** directory. The information in the INF files determine how an FFS will be generated. If you want to customize the FFS, you can specify an override by adding an override in the INF:

```
INF RuleOverride = PICOMPRESSED Ich7Pkg/UhciPei/Ich7Uhci.inf
INF RuleOverride = PICOMPRESSED My/Bus/Pci/UhciPei/UhciPei.inf
INF RuleOverride = PICOMPRESSED My/Bus/Usb/UsbBusPei/UsbBusPei.inf
INF RuleOverride = PICOMPRESSED My/Bus/Usb/UsbBotPei/UsbBotPei.inf
INF RuleOverride = PICOMPRESSED My/Bus/Isa/IsaFloppyPei/IsaFloppyPei.inf
INF RuleOverride = PICOMPRESSED My/Universal/Disk/FileSystem/FatPei/FatPei.inf
```

In the above example, **GenFds** will use the **PICOMPRESSED** rule to generate the FFS regardless of the rules that would normally process the INF file.

While INF files may only be listed once per FV, most entries, if listed more than once in a section, only the most recent value will be used. For example, if a single **[Capsule]** section has two **OEM_CAPSULE_FLAGS** entries, the last one takes precedence.

10.2.1 FILE Format Example

If the file you want to place into flash is not built using information from an INF file (for example, a micro-code that must be placed into FV), the file can be directly specify using **FILE** statement. The following is an example of the **FILE** format:

```
FILE DRIVER = 961578FE-B6B7-44c3-AF35-6BC705CD2B1F {
  SECTION PE32 = FatBinPkg/EnhancedFatDxe/X64/Fat.efi
}
```

In this example, the **Fat.efi** file is placed into a PE32 section first and then placed into the generated 'DRIVER' FFS "named" with the specified GUID.

10.3 Build Intermediate Images

10.3.1 Binary modules

Binary modules can be inserted into flash image in one of three ways. The first way is to use the FILE statement mentioned in Section 10.2.1. The second way uses an INF file listed in an FV section that describes binary files, like the one below:

```
[Defines]
  INF_VERSION           = 0x00010017
  BASE_NAME             = Logo
  FILE_GUID             = 7BB28B99-61BB-11D5-9A5D-0090273FC14D
  MODULE_TYPE          = USER_DEFINED
  VERSION_STRING        = 1.0
  EFI_SPECIFICATION_VERSION = 0x00020000

[Binaries.common]
  BIN|Logo.bmp|*
```

This INF file shows that binary file `Logo.bmp` will be wrapped into the Logo FFS file. This kind of INF file is specified using standard `INF` statement in an FV section of the FDF file.

The third method is to list a binary INF file containing the binary data in an FD section. If the binary specified in the INF file in this section is a BIN type (BIN|Filename.bin) the tools will not process the file and will be inserted at the offset specified for the region. This is equivalent of specifying a filename using the FILE statement, but with the binary file included from a binary module. If the file is another binary file type, such as an FSP binary containing PatchableInModule PCDs, the tools will be able to patch the binary file prior to adding it to the region.

10.3.2 Creating EFI Sections

Sections are produced by `GenSec` tool using information in FDF file of what type and content the section must contain. Section information in FDF file belongs to two categories: either it is a leaf section, or it is an encapsulate section. Encapsulation sections may contain one or more leaf sections or other encapsulate section. The leaf section information appears in the FILE statement in Section 10.2.1, the PE32 section type for the Fat.efi file. Normally this information is enough for `GenSec` tool, however, more information can be specified by specifying a `[Rule]` section in the FDF file. Rules in an FDF file, look like:

```
[Rule.Common.SEC]
  FILE SEC = $(NAMED_GUID) {
    TE TE Align = 8 |.efi
    RAW BIN Align = 16 |.com
  }
```

The above rule stipulates that for file type `SEC` (Security) in all build architectures, the generated FFS must contain one TE section with 8-byte alignment and one `RAW` section with 16-byte alignment.

Different information can be specified for different section types:

```
[Rule.Common.PEIM]
  FILE PEIM = $(NAMED_GUID) {
    PEI_DEPEX PEI_DEPEX Optional |.depex
    TE TE |.efi
    UI STRING = "$(MODULE_NAME)" Optional
    VERSION STRING = "$(INF_VERSION)" Optional BUILD_NUM = $(BUILD_NUMBER)
  }
```

The above rule stipulates that for file type `PEIM` in all build architectures, the generated FFS may contain at most one optional `PEI_DEPEX` section, must contain one TE section, and may contain at most one UI section with the UI string set to the INF file's module name, and at most one `VERSION` section.

10.3.3 Create an Apriori File

Some firmware volumes may require, an `APRIORI` file to be created. An `APRIORI` file is a text file containing a GUID-named list of two or more modules in the firmware volume. The modules will be invoked or dispatched in the order they appear in the `APRIORI` file. Only one of each PEI and DXE Apriori file is permitted within a single Firmware Volume. Nested Firmware Volumes are permitted, so `Apriori` files are limited to specifying the files (and not FVs) that are within the scope of the FV image in which it is located. It is permissible for nested FV images to have one PEI and one DXE Apriori file per FV. Scoping is accomplished using the curly "{}" braces.

The following example demonstrates an example of multiple `APRIORI` files.

```
[Fv.Root]
DEFINE NT32      = $(WORKSPACE)/EdkNt32Pkg
DEFINE BuildDir  = $(OUTPUT_DIRECTORY)/$(PLATFORM_NAME)/ $(TARGET)_$(TOOL_CHAIN_TAG)
APRIORI DXE {
  FILE DXE_CORE = B5596C75-37A2-4b69-B40B-72ABD6DD8708 {
    SECTION COMPRESS {
      SECTION PE32 = $(BuildDir)/X/Y/Z/B5596C75-37A2-4b69-B40B-72ABD6DD8708-DxeCore.efi
      SECTION VERSION "1.2.3"
    }
  }
  INF VERSION = "1" ${NT32}/Dxe/WinNtThunk/Cpu/Cpu.inf
}

FILE FV_IMAGE = EF41A0E1-40B1-481f-958E-6FB4D9B12E76 {
  SECTION GUIDED 3EA022A4-1439-4ff2-B4E4-A6F65A13A9AB {
    SECTION FV_IMAGE = Dxe {
      APRIORI DXE {
        INF a/a.a.inf
        INF a/c/c.inf
        INF a/b/b.inf
      }
      INF a/d/d.inf
      ...
    }
  }
}
```

In the example above, there are three FFS files in the `Fv.Root` and one Encapsulated FV image. The build tools will create an `APRIORI` file that will dispatch the `DXE_CORE` first, then the CPU module second. In the FV image, named by the GUID `EF41A0E ...`, there will be at least five FFS files, the `APRIORI` file, named `Dxe`, listing the GUID names of modules `a.inf`, `c.inf` and `b.inf`, which will be dispatched in this order. Once complete, the `d.inf` module may be dispatched.

10.3.4 Create FFS Files from Leaf Sections

Section 9.2 shows the INF and FILE statements in an FDF to describe FFS files that will be placed into FV. The `FILE` statement is straight forward, letting you know how an FFS file is organized, as it contains section information within its scope. The `INF` statement, on the other hand, will use a particular `RULE` that is determined by the module type in the INF and specified build architecture.

The `[Rule]` section of the FDF file is used to define custom rules. Custom rules may also be applied to a given INF file listed in an `[FV]` section. The `[Rule]` section is also used to define rules for module types that permit the user to define the content of the FFS file - when an FFS type is not specified by either PI or UEFI specifications.

The Rules can have multiple modifiers as shown below.

```
[Rule.$(ARCH).$(MODULE_TYPE).$(TEMPLATE_NAME)]
```

If no `$(TEMPLATE_NAME)` is given then the match is based on `$(ARCH)` and `$(MODULE_TYPE)` modifiers. `BINARY` is a reserved `TEMPLATE_NAME` as the default rule name for binary modules. The `$(TEMPLATE_NAME)` must be unique to the `$(ARCH)` and `$(MODULE_TYPE)`. It is permissible to use the same `$(TEMPLATE_NAME)` for two or more `[Rule]` sections only if the `$(ARCH)` and the `$(MODULE_TYPE)` listed are different for each of the sections.

A `[Rule]` section is terminated by another section header or the end of file.

The content of the `[Rule]` section is based on the `FILE` and section grammar of the `FV` section. The difference is the `FILE` referenced in the `[RULE]` is a `MACRO`. The section grammar is extended to include an optional argument, `optional`. The `optional` argument is used to say a section is optional, that is to say if it does not exist it's O.K.

The generic form of the entries for leaf sections is:

```
<SectionType> <FileType> [Options] [{<Filename>} {<Extension>}]
```

When processing the FDF file, the rules apply in the following order:

1. If `<SectionType>` not defined or not a legal name, then error
2. If `<FileType>` not defined or not a legal name, then error
3. If `[FilePath/FileName]`, then:
 - Add one section to FFS with a section type of `<SectionType>`
4. Else:
 - Find all files defined by the INF file whose file type is `<FileType>` and add each one to the FFS with a section type of `<SectionType>`
 - Add files defined in `[Sources]` followed by files defined in `[Binaries]`
5. If more than 1 `UI` section in the final FFS file, then error
6. If more than 1 `VER` section in the final FFS file, then error
7. If more than 1 `DXE_DEPEX` section in final the FFS file, then error
8. If more than 1 `PEI_DEPEX` section in the final FFS file, then error
9. If more than 1 `SMM_DEPEX` section in the final FFS file, then error.

10.3.5 Create Encapsulation Sections

There are two types of encapsulation sections, a `COMPRESSION` section and the `GUIDED` section. A `COMPRESSION` section uses standard UEFI compression/decompression mechanisms. Other compression schemes must use the `GUIDED` form of encapsulation section.

The `COMPRESS` encapsulation section uses the following format.

```
SECTION COMPRESS [type] {
  SECTION EFI_SECTION_TYPE = FILENAME
  SECTION EFI_SECTION_TYPE = "string"
}
```

The `[type]` argument is optional, only `EFI_STANDARD_COMPRESSION` is supported by the PI specification. The current EDK enumerations for compression are a violation of the PI specification, and `SECTION GUIDED` must be used instead.

The `EFI_SECTION_TYPE` and `FILENAME` are required sub-elements within the compression encapsulation section. for most sections, however both the `VERSION` (`EFI_SECTION_VERSION`) and `UI` (`EFI_SECTION_USER_INTERFACE`) may specify a string that will be used to create an EFI section.

The `GUIDED` encapsulation section uses one of the following formats.

```
SECTION GUIDED $ (GUID_CNAME) [auth] {  
  SECTION EFI_SECTION_TYPE = FILENAME  
  SECTION EFI_SECTION_TYPE = "string"  
}  
SECTION GUIDED $ (GUID_CNAME) [auth] FILENAME
```

The required argument is the `GUIDED` name followed by an optional `auth` flag. If the argument `auth` flag is specified, then the attribute `EFI_GUIDED_SECTION_AUTH_STATUS_VALID` must be set.

For statements that do not use a scoping notation, (the second `SECTION` statement of the two listed above), if `FILENAME` exists, the attribute `EFI_GUIDED_SECTION_PROCESSING_REQUIRED` must be set to `TRUE`. The file pointed to by `FILENAME` is the data. If `FILENAME` does not exist `EFI_GUIDED_SECTION_PROCESSING_REQUIRED` is cleared and normal leaf sections must be used.

GenSec tool uses information from these encapsulated section definition as input parameters to generate the corresponding section format.

10.4 Create the FV Image File(s)

Once all of the EFI FFS files have been created, these images are bundled into an FV image.

GenFv needs two kinds of information about the target FV:

- The FV attributes
- The list of one or more files that will be placed into this FV.

This information is defined in the FV section of the FDF file.

If the `[FV]` section contains an `FvNameString` entry and it is set to `TRUE`, the tools will use the `FvUiName` from the section tag to create an `FvNameString` entry in the FV image's extension header.

The following example is for a FV section named "BiosUpdate."

```
[FV.BiosUpdate]
  BlockSize           = 0x10000
  FvAlignment         = 16
  ERASE_POLARITY      = 1
  MEMORY_MAPPED       = TRUE
  STICKY_WRITE        = TRUE
  LOCK_CAP            = TRUE
  LOCK_STATUS         = TRUE
  WRITE_DISABLED_CAP  = TRUE
  WRITE_ENABLED_CAP   = TRUE
  WRITE_STATUS        = TRUE
  WRITE_LOCK_CAP      = TRUE
  WRITE_LOCK_STATUS   = TRUE
  READ_DISABLED_CAP   = TRUE
  READ_ENABLED_CAP    = TRUE
  READ_STATUS         = TRUE
  READ_LOCK_CAP       = TRUE
  READ_LOCK_STATUS    = TRUE

  FILE FV_IMAGE = EDBEDF47-6EA3-4512-83C1-70F4769D4BDE {
    SECTION GUIDED {
      SECTION FV_IMAGE = BiosUpdateCargo
    }
  }
```

This FV is very simple; it contains only one `FILE`. But this file contains an entire FV image named `BiosUpdateCargo` which must be available when **GenFds** creates the `BiosUpdate` FV.

The **GenFds** tool will process the FDF file and place the FV attributes and contents in to an INF file (in this example, the `BiosUpdate.inf` file) and then processing is transferred to **GenFv** tool when creating FV images. The following example is what this generated, FV-style, INF file looks like:

```
[options]
EFI_BLOCK_SIZE = 0x10000

[attributes]
EFI_ERASE_POLARITY      = 1
EFI_WRITE_ENABLED_CAP   = TRUE
EFI_READ_ENABLED_CAP    = TRUE
EFI_READ_LOCK_STATUS    = TRUE
EFI_WRITE_STATUS        = TRUE
EFI_READ_DISABLED_CAP   = TRUE
EFI_WRITE_LOCK_STATUS   = TRUE
EFI_LOCK_CAP            = TRUE
EFI_LOCK_STATUS         = TRUE
EFI_ERASE_POLARITY      = 1
EFI_MEMORY_MAPPED       = TRUE
EFI_READ_LOCK_CAP       = TRUE
EFI_WRITE_DISABLED_CAP  = TRUE
```

```
EFI_READ_STATUS      = TRUE
EFI_WRITE_LOCK_CAP    = TRUE
EFI_STICKY_WRITE      = TRUE
EFI_FVB2_ALIGNMENT_16 = TRUE

[files]
EFI_FILE_NAME = C:\edk2BuildMyPlatformDEBUG_MYTOOLS\Ffs\EDBEDF47-6EA34512-83C1-70F4769D4BDE\EDBEDF47-6EA3-4512-83C1-70F4769D4BDE
.ffs
```

10.5 Create the FD image file(s)

The whole FD image is described by a list of `Regions` which correspond to the locations of different areas within the hardware flash device. Currently most flash devices have a variable number of blocks, all of identical size. When "burning" an image into one of these devices, only whole blocks can be burned into the device at any one time. This puts a constraint that all layout regions of the FD image must start on a block boundary. To accommodate future flash parts that have variable block sizes, the layout is described by the offset from the `BaseAddress` and the size of the section that is being described. Since completely filling a block is not probable, part of the last block of a region can be left empty. To ensure that no extraneous information is left in a partial block, the block must be erased prior to burning it into the device. Multiple devices with non-volatile memory are treated as a single device with contiguous memory space.

Regions must be defined in ascending order and may not overlap.

Each layout region starts with a eight digit hex offset (leading "0x" required) followed by the pipe "|" character, followed by the size of the region, also in hex with the leading "0x" characters.

The format for an FD Layout Region is:

```
Offset|Size
[TokenSpaceGuidName.PcdOffsetName|TokenSpaceGuidName.PcdSizeName]
[RegionType]
```

Setting the optional PCD names in this fashion is shortcut. The two regions listed below are identical, with the first example using the shortcut, and the second using the long method:

```
0x000000|0x0C0000
gEfiMyTokenSpaceGuid.PcdFlashFvMainBaseAddress|gEfiMyTokenSpaceGuid.PcdFlashFvMainSize
FV = FvMain

0x000000|0x0C0000
SET gEfiMyTokenSpaceGuid.PcdFlashFvMainBaseAddress = 0x000000
SET gEfiMyTokenSpaceGuid.PcdFlashFvMainSize = 0x0C0000
FV = FvMain
```

The shortcut method is preferred, as the user does not need to maintain the values in two different locations.

The EDK II BaseTools support the use of expressions in the offset field and size fields. When a PCD is used in either of these fields, the PCD must have been set in a statement above where it is used in an expression (tools process the file top to bottom). During the processing of the FDF file, the value of an 'offset' PCD is the offset from 0x00000000. After the processing has been completed, the tools will adjust these 'offset' PCDs to be the absolute address. For example:

```
[FD.Main]
BaseAddress = 0xFFE00000
Size = 0x00800000
#DEFINE REGION1_SIZE = 0x1000
#DEFINE REGION2_SIZE = 0x2000

0x00000000|$(REGION1_SIZE)
gMyPlatformTSGuid.PcdRegion1Base|gMyPlatformTSGuid.PcdRegion1Size
FILE = MyPlatform/Region1Bin/Region1.bin

gMyPlatformTSGuid.PcdRegion1Base + $(REGION1_SIZE)|$(REGION2_SIZE)
gMyPlatformTSGuid.PcdRegion2Base|gMyPlatformTSGuid.PcdRegion2Size
```

In the above example, during FDF processing, the `PcdRegion1Base` is `0x00000000`, while after the FDF file processing has been completed, the value of the PCD, `PcdRegion1Base`, will be `0xFFE00000`.

The optional `RegionType`, if specified, must be one of the following `FV`, `DATA`, `FILE`, `CAPSULE` or no `RegionType` at all. Not specifying the `RegionType` implies that the region starting at the `offset`, of length `Size` must not be touched. This unspecified region type is typically used for event logs that are persistent between system resets, and modified via some other mechanism (and SMM Event Log module, for example).

EDK II FDF does not use the concept of sub-regions, which existed in EDK FDF files.

10.5.1 FV Region Type

The `FV` `RegionType` is used as a pointer to either one of the unique FV names that are defined in the `[FV]` section. These are files that contains a binary FV as defined by the PI specification. The format for the `FV` `RegionType` is one of the following:

```
FV = $(UiFvName)
```

The following is an example of `FV` region type.

```
0x000000|0x0C0000
gEfiMyTokenSpaceGuid.PcdFlashFvMainBaseAddress|gEfiMyTokenSpaceGuid.PcdFlashFvMainSize
FV = FvMain
```

10.5.2 DATA Region Type

The `DATA` `RegionType` is a region that contains a hex value or an array of hex values. This data that will be loaded into the flash device, starting at the first location pointed to by the `offset` value. The format of the `DATA` `RegionType` is:

```
DATA = { <Hex Byte Data Structure> }
```

The following is an example of a `DATA` region type.

```
0x0CA000|0x002000
gEfiMyTokenSpaceGuid.PcdFlashNvStorageBase|gEfiMyTokenSpaceGuid.PcdFlashNvStorageSize
DATA = {
  0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
  0x8D, 0x2B, 0xF1, 0xFF, 0x96, 0x76, 0x8B, 0x4C
}
```

This data may need to be modified based on content of the region. In order for EFI modules to access these regions, a customized region header may be required. Tools for creating custom header information is beyond the scope of the standard build.

10.5.3 FILE Region Type

The `FILE` `RegionType` is a pointer to a binary file that will be loaded into the flash device, starting at the first location pointed to by the `offset` value. The format of the `FILE` `RegionType` is:

```
FILE = $(FILE_DIR)/Filename.bin
```

The following is an example of the `FILE` `RegionType`.

```
0x0CC000|0x002000
gEfiCpuTokenSpaceGuid.PcdCpuMicrocodePatchAddress|gEfiCpuTokenSpaceGuid.PcdCpuMicrocodePatchSize
FILE = FV/Microcode.bin
```

10.5.4 INF Region Type

The `INF` `RegionType` is a pointer to a binary INF file that will be loaded into the flash device, starting at the first location pointed to by the `Offset` value. The format of the `INF` `RegionType` is:

```
INF [Options] Package/BinModule.inf
```

The following is an example of the `INF` `RegionType`.

```
0x0CC000|0x002000
gEfiCpuTokenSpaceGuid.PcdCpuMicrocodePatchAddress|gEfiCpuTokenSpaceGuid.PcdCpuMicrocodePatchSize
INF MyPackage/MyMicrocode.inf
```

10.6 Post Build Processing

If the `[Defines]` section of the DSC file contains a `POSTBUILD = entry` statement, prior to exiting, the script specified in the `POSTBUILD` statement is executed. The entry of `POSTBUILD` support multiple arguments. And Tool will convert arguments that are `WORKSPACE` or `PACKAGES_PATH` relative paths to absolute paths. If the script file is not found, the **build** command exits with an appropriate error message. If the script fails, it must terminate with a non-zero exit code and the **build** command terminates with the exit value from the post-build script. The script is required to generate error messages that provide the reason for the termination.

All of the command line options passed into the **build** command are also passed into the script along with the options for `TARGET`, `ARCH`, `TOOL_CHAIN_TAG`, `ACTIVE_PLATFORM`, `Conf Directory`, and `build target`.

If the script terminates successfully (exit value of 0), then the **build** command terminates normally

Note: This entry may be wrapped in a conditional directive. Unlike the `PREBUILD` entry, there are no restrictions on the MACRO values used in a conditional directive.

Note: Quotes are needed when the script's additional options are present. Quotes are also required if the path to the post-build command contains space or special characters. Quotes may be used for arguments that have spaces or special characters.

11 POST-BUILD IMAGEGEN STAGE - OTHER

This chapter describes the processing of the EFI files generated by the \$(MAKE) Stage into images such as Applications or images used by PCI Option ROMs and/or Update Capsules. Creating images that do not go into a flash part directly such as stand-alone Applications and PCI Option ROM images, do not need an FDF file. This also applies to binary driver images that are to be used for a binary distribution - the files for these images are created during the \$(MAKE) stage.

11.1 EFI PCI Option ROM Images

To generate the EFI PCI Option ROM, the EFI PE32 files and optionally the legacy OptROM (from a separate tool) are needed.

The **EfiRom** tool is used on the PE32 and optionally the legacy Option ROM binary images. The tool will check the header of each file to determine the type.

- If the input file(s) are EFI PE32 image,
 - fill in EFI PCI OptROM header and PCI data structure in the output EFI PCI Option ROM image
 - then copy the input EFI PE32 file content to the output EFI PCI Option ROM image to create the EFI PCI Option ROM image.
- If the input file(s) are legacy OptROM binary image,
 - fill in EFI PCI OptROM header in the output EFI PCI Option ROM image
 - then copy the input file content to the output EFI PCI Option ROM image to create the EFI PCI Option ROM image.

The final image is placed in the FV folder of the build directory.

11.2 UEFI Applications

If a developer wants to generate only UEFI applications, verify that no FDF file is specified in the DSC file. This prevents the GenFds tool from being called after all of the modules have been built by the \$(MAKE) stage. The UEFI application files (.efi files) built from application modules are put in the following directory:

```
$(OUTPUT_DIRECTORY)/$(PLATFORM_NAME)/<BuildTarget>_<ToolChainTag>/$(ARCH)
```

11.3 Capsules

This section describes the processing of the EFI files generated by the \$(MAKE) Stage into Update Capsules. Capsule images contain a Fv Image or a FFS file to be updated.

The [Capsule] section in FDF file is parsed to get:

- Capsule Header information, including: Capsule GUID, flags and header size.
- Capsule content may be either a Fv Image or a FFS file.

A Fv Image may be specified using any FV section described in this FDF file. It will be generated same using the process described in Section 10.4. Additionally, an existing FV file created as part of an FD image may be used. These FV files can be directly integrated into a Capsule. Raw data (non-FFS files) can be included in a FV file, using `EFI_FV_FILETYPE_RAW`.

The FFS file contains EFI section files (see Table 2 for a list of `EFI_SECTION` types. All files generated by the \$(MAKE) stage, will have the output located in a build directory, either at the top of:

`$(OUTPUT_DIRECTORY)/$(PLATFORM_NAME)/<BuildTarget>_<ToolChainTag>/$(ARCH)` or a sub-directory created that replicates the INF file path. All EFI section files and encapsulated section files are created based on their description in FDF file. For a binary or raw file type, the raw data can be any binary file. One FV image or one FD image described in FV section or FD sections of the FDF file may also be treated as RAW data. The process of creating a FV image is described in Section 10.4, the process of creating a FD image is described in Section 10.5.

The \$(MAKE) stage creates EFI files. During the ImageGen stage, GenFds will create the required FFS files and FV images based on [Capsule] description in the FDF file. Finally, the capsule header will be prefixed to the capsule data to construct the complete capsule. The overview of the Capsule creation process is shown in Figure 23:

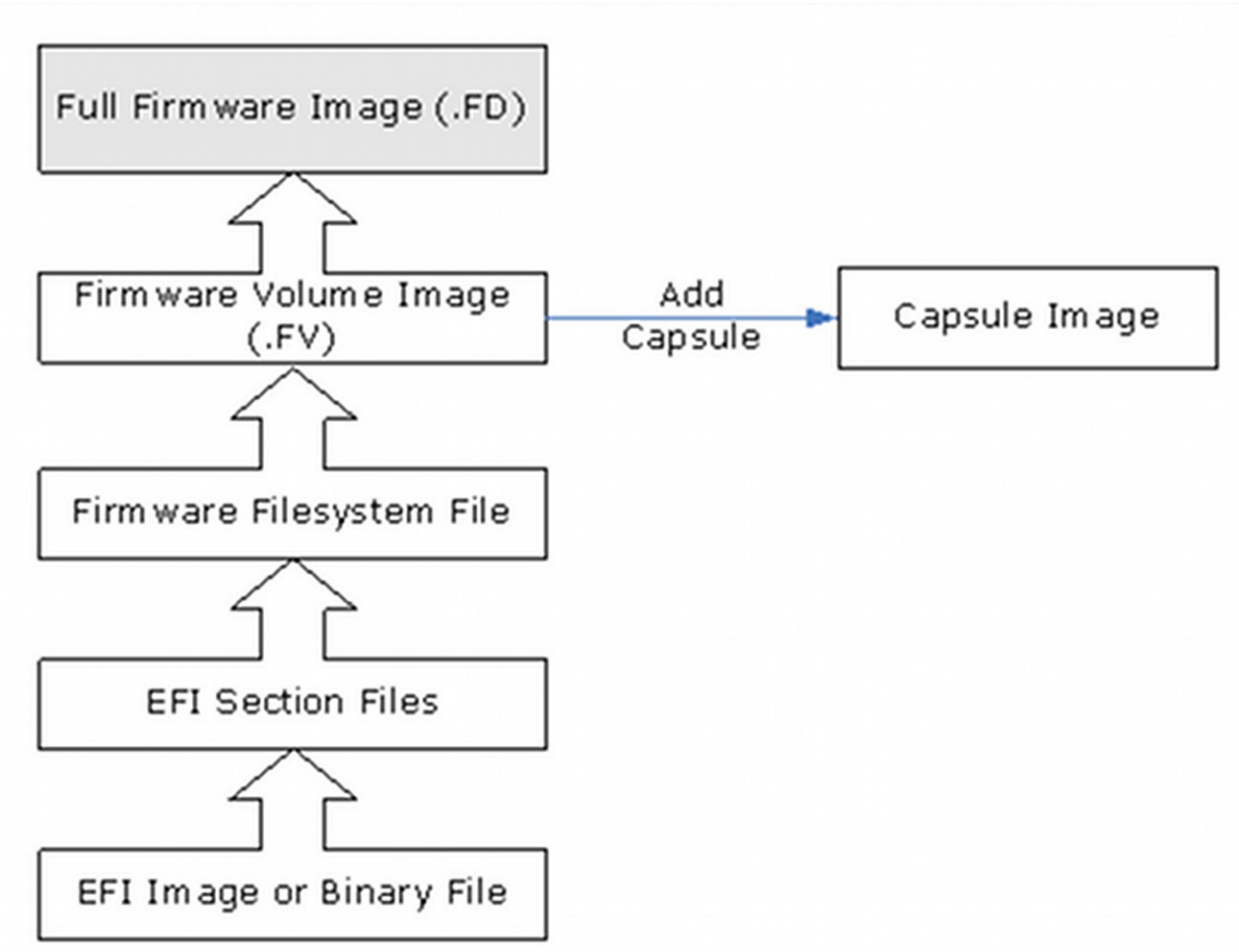


Figure 23 Capsule Creation Process.

12 BUILD CHANGES AND CUSTOMIZATIONS

This chapter deals with customizing a build, including options and settings for debugging, using custom tools.

12.1 Building for Debug

The build tool defaults support three building targets: NOOPT, DEBUG and RELEASE. This section describes how to enable DEBUG target when building and how to setup compiler flags used for DEBUG. The NOOPT target disables all optimizations in addition to setting the flags for DEBUG.

There are three ways provided in build tool to define the target that will be used in the building process.

- Situation A: Setup by overriding file "target.txt"
 - After executing "edksetup", there will be a file named "target.txt" under `$(WORKSPACE)/Conf`.
 - Users can edit this file and change the value of item `TARGET`.
 - A specific example of this is `TARGET = DEBUG`, which sets the current building method.
 - In this example, the default value of the `TARGET` is set to `DEBUG`.
- Situation B: Use a parameter `-b BUILDTARGET` when executing building command
 - Users can type a command with the format `build -b BUILDTARGET` to specify the target used in current building.
 - A specific example of this command is `build -b DEBUG`.
 - In this example, the value set in the file `target.txt` will be ignored.
- Situation C: Setup in the DSC file of a platform
 - When the `BUILDTARGET` is not specified in the command line or in the file `target.txt`, the build tool will attempt to build all valid targets specified in the DSC file.
 - This contrasts with situations A and B, where only the targets specified as valid in the DSC file can be used.

12.1.1 Debugging Files

For a debugging build, the files created will be saved to

`$(WORKSPACE)/$(OUTPUT_DIRECTORY)/$(BUILDTARGET)_$(TOOL_CHAIN_TAG)/$(ARCH)/`. For each single module, the temporary files created in `DEBUG` building process will be saved to

`$(WORKSPACE)/$(OUTPUT_DIRECTORY)/$(BUILDTARGET)_$(TOOL_CHAIN_TAG)/$(ARCH)/$(PACKAGE_NAME)/$(MODULE_NAME)/DEBUG/` and

`$(WORKSPACE)/$(OUTPUT_DIRECTORY)/$(BUILDTARGET)_$(TOOL_CHAIN_TAG)/$(ARCH)/$(PACKAGE_NAME)/$(MODULE_NAME)/OUTPUT/`, so such as `.map`, `.pdb` and other `DEBUG` files can be found in these two directories.

User can also define a specific directory to save `DEBUG` files. A detailed example is given in the next subsection.

12.1.2 Debugging Options

Build tool supports customized `DEBUG` flags in the `<BuildOptions>` section of the DSC file, the INF file and `tools_def.txt`. The highest priority for a same compiler flag is the one defined in INF file, the medium is that in DSC file and the lowest is the one in `tools_def.txt`.

For example, to generate the `.cod` files for the `.obj` files of a platform, user can add one line such as

`*_MYTOOLS_*_CC_FLAGS = /Facs /FA $(OUTPUT_DIR)\` in section `[BUILD_OPTIONS]` of DSC file. This option tells build tool to generate a `.cod` file for each `.obj` file and put them to `$(OUTPUT_DIR)`.

For only generating the .cod files for one single module, one way is to add the option in section [BUILD_OPTIONS] of the module's INF file; another way is to add the option to DSC file's <BuildOptions> for the INF file like below:

```
MdeModulePkg/Universal/PCD/Pei/Pcd.inf {  
  <BuildOptions>  
    *_MYTOOLS_*_CC_FLAGS = /FAcs /FA $(OUTPUT_DIR)\  
}
```

12.1.3 Advanced Debugging

For generating disassembly (.cod) files for debugging, the following is one way to setup **dumpbin - disasm** for individual modules as well as using it for every .efi file generated.

To generate the disasm for the efi files, the user can add two definitions in tools_def.txt :

```
DEBUG_MYTOOLS_IA32_DISASM_PATH = DEF(VS2005TEAMSUITE_BIN)\dumpbin.exe  
DEBUG_MYTOOLS_IA32_DISASM_FLAGS = -dump -disasm -out:$(DEST_DIR)
```

Then user can use build option `-D` , `--define` with a reserved MACRO name: DISASM to start building. The build tool automatically detects if a **DISASM** tool defined in the TagName of Tool Chain, then after ever link command that generates an EFI file, the tool will run the **DISASM** tool (with the flags) against the EFI file. In the example, the output file will be next to the EFI file based on the FLAGS entry, - **out:\$(DEST_DIR)** which is the same location as the .efi file.

12.2 Adding Custom Compression Tools

This section covers how to add a customized compress tool, such as **TianoCompress** tool.

First, one specific GUID is assigned to the added tool, which can be used to specify this tool and its compressed data. Then the tool path and GUID needs to be added into `tools_def.txt` file, for example `TianoCompress` tool used for all tool chains, Target and Archs can be added like:

```
* _ _ _TIANOCOMPRESS_PATH = DEF(TOOL_PATH)\TianoCompress.exe
* _ _ _TIANOCOMPRESS_GUID = A31280AD-481E-41B6-95E8-127F4C984779
```

Next, "`$(TOOLNAME)`" can be specified in `build_rule.txt` file to call this tool. And, its GUID value is supported in FDF file to call this tool, which is used to create the EFI guided section data. For **TianoCompress** tool, "`$(TIANOCOMPRESS)`" is used in `build_rule.txt` file, `A31280AD-481E-41B6-95E8-127F4C984779` is used in FDF file.

`NT32.fdf` file uses **TianoCompress** tool to create the guided data like:

```
[Rule.Common.PEIM.TIANOCOMPRESSED]
FILE PEIM = $(NAMED_GUID) DEBUG_MYTOOLS_IA32 {
  PEI_DEPEX PEI_DEPEX Optional |.depex
  GUIDED A31280AD-481E-41B6-95E8-127F4C984779 {
    PE32 PE32 |.efi
    UI STRING="$(MODULE_NAME)" Optional
    VERSION STRING = "$(INF_VERSION)" Optional|BUILD_NUM = $(BUILD_NUMBER)
  }
}
```

12.3 Using Custom Build Tools

This section introduces how to use the custom tools in EDKII build system.

The custom tools can be classified to two types. One is used to create the EFI guided section data, which must have its matched GUID value, such as the custom compression tool introduced in the last section. Another is only used to process files, which may not require its GUID, such as ASL compiler. This section focuses on the later one.

First, the custom tool path and name needs to be added into `tools_def.txt` file. For ASL compiler, it can be added like:

```
*_*_*_ASL_PATH = DEF(ASL_PATH)\iasl.exe
```

Then, "`$(TOOLNAME)`" is specified in `build_rule.txt` file to call this tool. For ASL compiler, it can be used to process ASL source file. The rule to process ASL file is added in `build_rule.txt` like:

```
[Build.Aspi-Source-Language-File]
<InputFile>
  ?.asl, ?.Asl, ?.ASL
<OutputFile>
  $(OUTPUT_DIR)(+)${s_base}.aml
<Command.MSFT, Command.INTEL>
  "$(PP)" $(APP_FLAGS) $(INC) ${src} > ${d_path}(+)${s_base}.i
  "$(ASL)" -p ${dst} ${d_path}(+)${s_base}.i
```

12.4 Customizing Compilation for a Component

There are several mechanisms for customizing the build for a firmware component. These include:

- Creating a new Platform (DSC) file from an existing platform.
- Creating a custom INF file for individual components or modules.
- Using MACRO definitions with control statements (!ifxxx) in the DSC and FDF files.
- Customizing the INF build options in the DSC file.

12.5 Platform Specific ASL Tools

The platform ACPI compilers are not all backward compatible. Typically, an ASL compiler is selected based on the ACPI version and features that are required by the platform. Different flags may also be required for different releases of the ASL compilers. One method for using different versions of the ASL compilers on Windows* systems is presented here.

The EDK II build tools expect the Microsoft ASL compilers (asl.exe) and the Intel/ACPI compiler (iasl.exe) to be located in the C:\ASL directory of the developer's workstation. This path and the compiler names are also coded in the `tools_def.txt` file. Name the compiler binaries using the ACPI Spec compliance value, for example, `C:\ASL\iasl3a.exe` and `C:\ASL\iasl5.exe`.

Use the `[BuildOptions]` section of the platform's DSC file to override the default values in the `tools_def.txt` file as shown below.

Platform1 DSC requiring ACPI 3a compliance

```
[BuildOptions]
*_*_*_ASL_PATH == C:\ASL\iasl3a.exe
```

Platform2 DSC requiring ACPI 5 compliance

```
[BuildOptions]
*_*_*_ASL_PATH == C:\ASL\iasl5.exe
*_*_*_ASL_FLAGS = -cr
```

The "==" means replace the ASL compiler specified by the `PATH` attribute in the `tools_def.txt` file with this ASL compiler.

The "=" means to append the flags to the flags specified in the `FLAGS` attribute in the `tools_def.txt` file.

No changes are required to any other files or tools in order for this method to work. Other tools may also benefit from this build system flexibility.

12.6 Build Reproducibility

The EDK II build system is designed to provide functional reproducibility, not necessarily binary reproducibility. For example, when building the `DEBUG` targets, the absolute path and file name for a PDB file is inserted into PE32 file. Building from different directory trees will result in different directory paths, and the PE32 files will not be identical bit for bit, but they will be identical in functionality.

Using the `RELEASE` build target will only result in identical files if there are no changes whatsoever to the source files. The EDK II debug libraries insert `DebugAssert` statements into binaries. These statements record line numbers from the source files. This means that inserting a blank line or a comment anywhere in a module can yield multiple different line numbers from the `DebugAssert` statements.

Fortunately, a special compiler macro, `MDEPKG_NDEBUG`, has been in the EDK II code base debug libraries. When this macro is defined, the `DebugAssert` statements are stripped completely removed from the resultant binary. So using a `RELEASE` and adding

the `MDEPKG_NDEBUG` macro will allow generating binaries that are identical, regardless of the directories or changes to comments in the source files.

The best method for adding this macro is again, using the `[BuildOptions]` section of the DSC file. The following is an example that is valid for all tool chains with a family set to MSFT in the `tools_def.txt` file.

Example

```
[BuildOptions]
MSFT:RELEASE_*_*_CC_FLAGS = -D MDEPKG_NDEBUG
```

Using this flag is also the only way to turn off `DebugAssert` statements when disabling all optimizations using the `/Od` flag for Microsoft* compilers.

13 BUILD REPORTS

This section introduces the build report generation tool functionality and its output report format. It describes the external behaviors of the tool, i.e. the accepted command line options and the detailed output report format.

Unless the quiet or silent options are given to the build command, the build system automatically reports the following in build log file:

- Each region, offset and size defined in the FD.
- The location of the GUID cross reference file.
- The size of the data in each FV region.
- The date and time the build completed.

13.1 Build Report Generation Options

The Build Report Generation (BRG) tool is part of build process to report the following platform information after platform build ends successfully.

PCD Information

Complete platform configuration database information

LIBRARY Information

Library class and instance mapping, constructor/destructor information

DEPEX Information

Module dependency information

BUILD Information

Module build tool chain tag, specific compiler or linker options

FLASH Information

Module firmware device and firmware volume information

PREDICTION Information

The predicted dispatch order of modules (PEIMs / drivers) and their notification invoking sequence; Also the predicted addresses of module image loading, entry point and notification functions. Generating this report does take a significant amount of time, more than 2x the standard build time.

Module Information

Details of the module, may include the HASH of the `.efi` file.

Note: The execution order prediction report output is an html file, separate from the rest of the reports. All remaining reports are generated in a single text file. The reports are generated in the current working directory.

The information in the reports listed above is useful for platform integrators to diagnose the platform issues in an efficient way. Integrators must specify which reports to include in the report file.

13.2 Sample Launch Steps: NT32 platform

BRG functionality is switched on by **-y** or **-Y** option from **build** command. The following steps output the build report for NT32 platform:

1. Check out edk2 packages from `git@github.com:tianocore/edk2.git` to `c:\Users\YourLogin\Documents\edk2` directory¹.
2. Run **cmd.exe**, cd to your Documents directory and enter `subst s: .`
3. Cd to `s:\edk2`
4. Run **edksetup.bat --nt32**
5. Run **build.exe -a IA32 -p Nt32Pkg\Nt32Pkg.dsc -y ReportFile.txt**

- **-y**: This option specifies the output file name for build report.
- **-Y**: This option specifies flags that control the type of build report. It must be from the set of **PCD**, **LIBRARY**, **DEPEX**, **HASH**, **BUILD_FLAGS**, **FLASH**, **FIXED_ADDRESS** and **EXECUTION_ORDER**. To specify more than one flag, repeat the option on the command line.
Example of usage:

On the command line, append the following arguments:

-y report_filename.txt -Y PCD -Y FLASH -Y DEPEX

The default set of flags (if **-Y** is not specified) is: **PCD**, **LIBRARY**, **FLASH**, **DEPEX**, **HASH**, **BUILD_FLAGS** and **FIXED_ADDRESS**.

¹ On Microsoft Windows 7, you must be an administrator to create a directory in the root of the C: drive. It is recommended that you checkout edk2 into your User directory, then use the subst command to map that directory to a virtual drive.

13.3 Output

The output is in raw text file encoded in ASCII character set so that it can be portable to all OS environments. The text file is supposed to be organized in a logical way for human readability and QA team's validation.

Note: If the EXECUTION_ORDER flag is provided as the only report type and the -y option is not provided, the tool will generate an HTML document, Report.html in the current working directory.

If any other report type is also requested, the report will be a flat text file. If the -y option is provided, the report type will also be a flat text file (even if you name the file, using -y, as "Report.html").

13.3.1 Layout

The layout of the text report file:

```
|---- Platform summary
|---- Conditional directives section
|---- Unused PCDs section
|---- Mixed PCD section
|---- Global PCD section
|---- FD section*
|---- FD Region sub-section*
|---- VPD PCD Data sub-section*
|---- Module section*
|---- Basic Information summary
|---- PCD sub-section
|---- Library sub-section
|---- DEPEX sub-section
|---- Build_flags sub-section
|---- Notification sub-section
```

Note: Items marked with * can occur more than once in one parent instance.

13.3.2 Section and Sub-section Format

The output report of BRG is divided into platform and module part. Each part may further consist of sections and sub-sections with the following rules:

1. Each section starts with marker `>=====`
2. Each section ends with marker `<=====`
3. There must be a section header after each section start marker.
4. There must a separator `=====` to separate the section header and contents if the section has non-empty contents.
5. The section contents can further be divided into one-level sub-sections.
6. Each sub-section starts with marker `>-----`
7. Each sub-section ends with marker `<-----`
8. There must be a sub-section header after each section start marker.
9. There must a separator `-----` to separate the section header and contents if the section has non-empty contents.
10. In general, each line in section will not exceed 120 characters.

Example

```
Platform Name:      NT32
Platform DSC Path:  s:\edk2\Nt32Pkg\Nt32Pkg.dsc
Architectures:     IA32
Tool Chain:        VS2008x86
Target:            DEBUG
SKUID:             DEFAULT
DefaultStore:      STANDARD
Output Path:       s:\edk2\Build\NT32IA32
Build Environment:  Windows-7-6.1.7601-SP1
Build Duration:    00:01:29
AutoGen Duration:  00:00:10
Make Duration:     00:01:02
GenFds Duration:   00:00:15
Report Contents:   PCD, LIBRARY, BUILD_FLAGS, DEPEX, HASH, FLASH, FIXED_ADDRESS

>=====<
Firmware Device (FD)
FD Name:           NT32
Base Address:      0x0
Size:              0x2A0000(2688KB)
=====
>-----<
FD Region
Type:              FV
Base Address:      0x0
Size:              0x280000 (2560K)
FV Name:           FvRecovery (65.9% Full)
Occupied Size:     0x1A6028 (1688K)
Free Size:         0xD9FD8 (872K)
Offset            Module
-----
..(List of Module in FvRecovery)
<----->
>-----<
..(List of other FD region sub-section)
>=====<
```

The following sections describe these reports and sub-sections in detail.

13.4 Platform Summary

Platform summary displays at the beginning of the output report, including the following items:

- Platform Name : % PLATFORM_NAME in DSC [Defines] section%
- Platform DSC Path : %Path of platform DSC file%
- Architectures : %List string of all architectures used in build%
- Tool Chain : %Tool chain string%
- Target : %Target String%
- SKUID : %Platform SKUID String%
- DefaultStore : %Platform DefaultStore String%
- Output Path : %Path to platform build directory%
- Build Environment : %Environment string reported by Python%
- Build Duration : %Build duration time string%
- AutoGen Duration : %AutoGen duration time string if it exists%
- Make Duration : %Make duration time string if it exists%
- GenFds Duration : %GenFds duration time string if it exists%
- Report Content : %List of flags the control the report content%

Example

```
Platform Name:      NT32
Platform DSC Path:  s:\edk2\Nt32Pkg\Nt32Pkg.dsc
Architectures:     IA32
Tool Chain:        VS2008x86
Target:            DEBUG
SKUID:             DEFAULT
DefaultStore:      STANDARD
Output Path:       s:\edk2\Build\NT32IA32
Build Environment:  Windows-7-6.1.7601-SP1
Build Duration:    00:01:29
AutoGen Duration:  00:00:10
Make Duration:     00:01:02
GenFds Duration:   00:00:15
Report Contents:   PCD, LIBRARY, BUILD_FLAGS, DEPEX, HASH, FLASH, FIXED_ADDRESS
```

Note: Platform Summary is always present and appears at the beginning of report.

13.4.1 Common PCD rules

There have several PCD sections in the report file, following rules are apply to all those PCD sections.

1. The content of PCD section in the report is grouped by token space, and the order of PCD is sorted by token space first, then by PcdCName.
2. *B means the PCD's value was obtained from build option.
3. *F means the PCD's value was obtained from the FDF file.
4. *P means the PCD's value was obtained from the DSC file PCD sections.
5. *M means the PCD's value was obtained from the INF file or from the DSC file [Components] sections scope.²
6. If no *B, *F, *P or *M is shown, the PCD's value comes from DEC file. If the value obtained from either a build option, DSC, FDF or INF is the same as the value in the DEC, then *B, *F, *P or *M will not be shown in the report.
7. If the PCD's datum type is UINT8, UINT16, UINT32 or UINT64, then in the report will display both

hexadecimal format and integer format of PCD value.

² If both INF file and DSC file [Components] sections scope have this PCD, the value would obtained from DSC file [Components] sections scope base on PCD value precedence rule.

13.4.2 PCDs in Conditional Directives

If a PCD is used in a conditional directive statement in DSC or FDF file, this PCD section is generated. This is optional section.

PCD values derived from expressions or other PCDs are not differentiated in the report. Only the final value is displayed.

The first line is required:

```
[*P|*F|*B] <PcdName>: <PcdType> (<DatumType>) = <PcdValue>
```

Note: If the Pcd is a Structure PCD, <DatumType> is the Struct Name.

Additional lines may be displayed showing default values when the value is not a default value.

Example

```
>=====<
Conditional Directives used by the build system
=====
PCD statements
>-----<
gMyTokenSpaceGuid
*B LogEnable          : FIXED   (UNIT32) = 0x1 (1)
                        DEC DEFAULT = 0x0 (0)
*P SmmEnable          : FEATURE (BOOLEAN) = 0x0
                        DEC DEFAULT = 0x1
<----->
>=====<
```

13.4.3 PCDs not used

If a PCD defined in DSC or FDF file, but the PCD is not used in a conditional directive statement and not used by any module, the not used PCD section is generated. This is optional section.

PCD values derived from expressions or other PCDs are not differentiated in the report. Only the final value is displayed.

The first line is required:

```
[*P|*F|*B] <PcdName>: <PcdType> (<DatumType>) [((<SKUID>))[((<DefaultStore>))] = <PcdValue>
```

Note: If the Pcd is a Structure PCD, <DatumType> is the Struct Name.

Additional lines may be displayed showing default values when the value is not a default value.

Since the PCDs in this section are not used by any module, the PCD value is not evaluated to determine if it is a valid value or in a value in a valid range. Instead, the PCD value from the DSC file, FDF file, or build option are displayed exactly as they were entered.

Example

```
>=====<
PCDs not used by modules or in conditional directives
=====
PCD statements
>-----<
```

```
gMyTokenSpaceGuid
*P SmmEnable          : FEATURE (BOOLEAN) = 0x0
                        DEC DEFAULT = 0x1
*B UsbEnable          : FIXED (UNIT32) = 0x1 (1)
                        DEC DEFAULT = 0x0 (0)
<----->
>=====<
```

13.5 Mixed PCD Section

There is an optional section that, when present, lists the PCDs in the platform that use multiple access methods. This section is only present if there are Binary modules included in the platform build and the binary module uses a different PCD access method than other modules in the same platform build.

The section header is:

```
>=====<
The following PCDs use different access methods:
=====
.. (List of PCDs)
<=====>
```

Format for the entries in this section:

```
<PcdTokenSpaceGuid> . <PcdCName>
```

13.6 Global PCD Section

This section contains the information for all PCDs that used for all modules in a platform. The content of global PCD section is grouped by token space:

```
gEf1MdeModulePkgTokenSpaceGuid
...
...
gEf1Nt32PkgTokenSpaceGuid
...
...
...
```

PCD values derived from expressions or other PCDs are not differentiated in the report. Only the final value is displayed.

The section header is:

```
>=====
Platform Configuration Database Report
  *B - PCD override in the build option
  *P - Platform scoped PCD override in DSC file
  *F - Platform scoped PCD override in FDF file
  *M - Module scoped PCD override
=====
.. (List of PCDs)
<=====
```

Each global PCD item contains one or more lines:

13.6.1 Required line

The first line is required:

```
[*P|*F|*B] <PcdName>: <PcdType> (<DatumType>) [(<SKUID>)][(<DefaultStore>)] = <PcdValue>
```

Note: If the Pcd is a Structure PCD, <DatumType> is the Struct Name.

Examples

```
*P PcdWinNtFirmwareVolume          : FIXED  (VOID*) = L"..\\Fv\\Nt32.fd"
*F PcdWinNtFlashNvStorageFtwWorkingBase : FIXED (UINT32) = 0x0028E000 (2678784)
                                     DEC DEFAULT = 0x0 (0)

gTokenSpaceGuid
*B LogEnable                        : FIXED  (UNIT32) = 0x1 (1)
                                     DEC DEFAULT = 0x0 (0)

*P TestDynamic                      : DYN    (VOID*) (DEFAULT) = L"COM1!COM2"
                                     : DYN    (VOID*) (SKU1)    = L"COM3!COM4"
                                     : DYN    (VOID*) (SKU2)    = L"COM5!COM6"
                                     DEC DEFAULT = L"COM1!COM0"
```

13.6.2 Optional lines

13.6.2.1 Dynamic/DynamicEx

- if <PcdType> is DYN-HII

```
<VariableGuid>:<VariableName>:<Offset>
```

Example

```
*P PcdGmchDvmtTotalSize : DYN-HII (UINT8) = 0x0 (0)
    gSysConfigGuid: L"Setup": 0xAA
```

- if <PcdType> is DYN-VPD

<Offset relative to VPD base address>

Example

```
*F PcdVpdSample : DYN-VPD (UINT32) = 0x1 (1)
    0x0001FFF
```

13.6.2.2 Default (optional) line

The second optional line is present if the value from the DSC was overridden by build option. It is formatted as follows:

```
DSC DEFAULT = <Value in PCD Section in DSC>
```

The third optional line is present if the value from the DEC was overridden. It is formatted as follows:

```
DEC DEFAULT = <Value in DEC>
```

Example

```
*P PcdWinNtFirmwareFdSize : FIXED (UINT32) = 0x2a0000 (2752512)
    DEC DEFAULT = 0x0 (0)
```

13.6.2.3 Additional optional lines

Additional lines are optional and show if the PCD's value was obtained from the INF file or DSC file components module scoped PCD section. This will be listed if the module's final PCD value is not the same as the first line.

*M means the PCD's value was obtained from the INF file or DSC file components module scoped PCD section.

These lines are formatted as:

```
*M Inf Filename = <Value>
```

Example

```
*P PcdDebugPrintErrorLevel : PATCH (UINT32) = 0x80000042 (2147483714)
    DEC DEFAULT = 0x80000000 (2147483648)
*M      Tcp4Dxe.inf          = 0x0 (0)
```

Note: Global PCD section is present when **PCD** is specified in **-Y** option.

13.6.2.4 Rules for Structure PCD

If the Pcd is a Structure Pcd, it display in the report file would have two parts. One is the final Pcd value and its field value, the field value is from FDF file, DSC file PCD Section or build option, not from the final structure byte array, and the field order is sorted by the field name. When the field value from DSC file PCD Section, it will not have additional *P Flag, when the field value from FDF file, it will have additional *F Flag before the field name, when the field value from build option, it will have additional *B Flag before the field name. The other part is the DEC default value and its field value, this part is optional. The field value in this part is from DEC file, and the field order is sorted by the field name.

Note: When the structure Pcd have some value from build option, no matter whether the value is same with FDF or DSC file or DEC file, there will display a *B Flag before the PCD name. When the structure Pcd have some value from FDF file, no matter whether the value is same with DSC file or DEC file, there will display a *F Flag before the PCD name. When the structure Pcd value have some value from DSC file PCD section, no matter whether the value is same with DEC file, there will display a *P Flag before the PCD name.

Example

```
gEfiMdePkgTokenSpaceGuid
*B TestDynamicExHii          : DEXHII    (TEST) (SKU1) (STANDARD) = {
    0xff, 0x01, 0x00, 0x2e, 0xf6, 0x08, 0x6f, 0x19, 0x5c, 0x8e, 0x49, 0x91, 0x57, 0x00, 0x00, 0x00,
    0x00, 0x64, 0x00, 0x00, 0x00}
    .A                        = 0x1
    .Array                    = {0x2e, 0xf6, 0x08, 0x6f, 0x19, 0x5c, 0x8e, 0x49, 0x91, 0x57}
*B .C                        = 0x0
    .D                        = 0x64
                                : DEXHII    (TEST) (SKU1) (Manufacturing) = {
    0xff, 0x02, 0x00, 0x2e, 0xf6, 0x08, 0x6f, 0x20, 0x5c, 0x8e, 0x49, 0x91, 0x57, 0x00, 0x00, 0x00,
    0x00, 0x68, 0x00, 0x00, 0x00}
    .A                        = 0x2
    .Array                    = {0x2e, 0xf6, 0x08, 0x6f, 0x20, 0x5c, 0x8e, 0x49, 0x91, 0x57}
*B .C                        = 0x0
    .D                        = 0x68
                                DEC DEFAULT = {0xFF, 0xFF}
    .A                        = 0xF
    .C                        = 0xF
*P TestFix                   : FIXED     (TEST) = {
    0xff, 0x02, 0x00, 0x2e, 0xf6, 0x08, 0x6f, 0x19, 0x5c, 0x8e, 0x49, 0x91, 0x57, 0x00, 0x00, 0x00,
    0x00, 0x64, 0x00, 0x00, 0x00}
    .A                        = 0x2
    .Array                    = {0x2e, 0xf6, 0x08, 0x6f, 0x19, 0x5c, 0x8e, 0x49, 0x91, 0x57}
    .C                        = 0x0
    .D                        = 0x64
                                DEC DEFAULT = {0xFF, 0xFF}
    .A                        = 0xF
    .C                        = 0xF
```

13.7 FD Section

This section contains platform flash device information and its layout.

13.7.1 FD Section Header

Given that a platform may have multi-Firmware device, this section may appear more than once in the output report. The section header lists the name of FD and its base address and size. The contents of the section consist of one or more FD region subsection.

The line format is: " %-20(key)s: %(value)s " to ensure vertical alignment.

- FD Name : %FD file base name%
- Base Address: %Base address for the FD image%
- Size : %Size of the FD image%

Example

```
>=====<
Firmware Device (FD)
FD Name:      NT32
Base Address: 0x0
Size:         0x2a0000 (2688KB)
=====
... (one or more FD Region Sub-section)
<=====
```

13.7.2 FD Region Sub-section

This sub-section contains FD region information of platform flash device. If the region is a firmware volume, it lists the set of modules and its space information; otherwise, it only lists its region type, base address and size in its sub-section header.

The line format is: " %-20(key)s: %(values)s " to ensure vertical alignment.

- Type : %The type of the FD region (FV, DATA, FILE or None)%
- Base Address : %Base address for the FD region%
- Size : %Size of the FD region%
- Fv Name*: %FV name and occupation percentage%
- Occupied Size*: %The occupied size of the FV%
- Free Size*: %The free size of the FV%

The contents of FD region sub-section contain the list:

(Offset, Module)*: %The list offset and module INF file path in the FV%

The items marked with * are only available when the region type is FV.

Example1:

```
>-----<
FD Region
Type:         FV
Base Address: 0x0
Size:         0x280000 (2560K)
Fv Name:      FvRecovery (65.9% Full)
Occupied Size: 0x1A6028 (1688K)
Free Size:    0xD9FD8 (872K)
```

```

Offset      Module
-----
0x00000078 PEI Apriori
0x000000D8 DXE Apriori
0x00000FE8 PeiCore (s:\edk2\MdeModulePkg\Core\Pei\PeiMain.inf)
0x00000FE8 PcdPeim (s:\edk2\MdeModulePkg\Universal\PCD\Pei\Pcd.inf)
...(More list of offset and modules)
<-----
>-----

```

Example2:

```

>-----<
FD Region
Type:          DATA
Base Address:  0x280000
Size:          0xc000 (48K)
<----->
>-----<
FD Region
Type:          None
Base Address:  0x28C000
Size:          0x2000 (8K)
<----->
>-----<
...(More list of FD regions)

```

13.7.3 VPD PCD Sub-section

This section lists, in Offset order, every VPD PCD specified in the DSC file. The line format for this section is PcdName Skuld Offset PcdSize PcdValue.

- Base Address : %Base address from the start of the FD file%
- Size : %Size of the FD VPD region%

For each PCD in this region:

- PcdName : PcdTokenSpaceGuidCname.PcdCname
- Skuld : The string name of the Skuld for this build (or DEFAULT if no Skuld name is defined)
- Offset : The number of bytes from the start of the FD file
- PcdSize : Number of bytes reserved for this PCD
- PcdValue : The current value of the PCD, in hex or (for `VOID*`) the byte array

Note: There may be gaps in the address map as some PCDs may not be required for this specific build, but may be required for other builds based on the same DSC file.

Example

```

>-----<
FD VPD Region
Base Address:  0x3BC000
Size:          0x04000 (16K)
-----
gNoSuchTokenSpaceGuid.NoSuchPciSubsystemVendorId | DEFAULT | 0x003BC000 | 2 | 0x8086
gNoSuchTokenSpaceGuid.NoSuchPciSubsystemDeviceId | DEFAULT | 0x003BC002 | 2 | 0x1000
gNoSuchTokenSpaceGuid.NoSuchGigabitEthernetMac | DEFAULT | 0x003BC004 | 8 | {0x80, 0x40, 0x20, 0x10, 0x08, 0x04}
gEfiMdeModulePkgTokenSpaceGuid.PcdRsa2048Sha256PublicKeyBuffer | DEFAULT | 0x003BC01C | 32 | {0x91, 0x29, 0xc4, 0xbd, 0xea, 0x
6d, 0xda, 0xb3, 0xaa, 0x6f, 0x50, 0x16, 0xfc, 0xdb, 0x4b, 0x7e, 0x3c, 0xd6, 0xdc, 0xa4, 0x7a, 0x0e, 0xdd, 0xe6, 0x15, 0x8c, 0x
73, 0x96, 0xa2, 0xd4, 0xa6, 0x4d}

```

< ----- >

Note: The whole FD section is present when **FLASH** is specified in **-Y** option.

13.8 Module Section

Module section lists all modules involved in the platform build. If the **EXECUTION_ORDER** option is specified in **-Y** option, the module sections are sorted according to their PEI or DXE dispatch order; otherwise the module sections are listed according to their DSC position.

13.8.1 Module Section Summary

This sub-section lists the module basic information: Module name, Module Arch, INF file path, File GUID, Size, hash value, module build time stamp, module build time and driver type.

- Module Name : % BASE_NAME in INF [Defines] section%
- Module Arch : %Architecture of current module%
- Module INF Path : %Path of Module INF file%
- File GUID : % FILE_GUID in INF [Defines] section%
- Size : %Module EFI image size%
- SHA1 HASH : %SHA1 hash value and efi file name with * character%
- Build Time Stamp : %The time stamp in module PE32 image% (If the time stamp is cleared to be zero, the build time stamp is 1970-01-01 08:00:00 UTC time.)
- Module Build Time : %The time string for this module's build%
- Driver Type : %The driver's file type code³ and name in firmware volume%

The following entries are options:

- UEFI Spec Version : % UEFI_SPECIFICATION_VERSION in INF [Defines] section%
- PI Spec Version : % PI_SPECIFICATION_VERSION in INF [Defines] section%
- PCI Device ID : % PCI_DEVICE_ID in INF [Defines] section%
- PCI Vendor ID : % PCI_VENDOR_ID in INF [Defines] section%
- PCI Class Code : % PCI_CLASS_CODE in INF [Defines] section%

³ The hex value in this field is the Firmware File Type value defined in Volume 3 of the PI Specification (Table 3 Defined File Types).

Example1:

```
>=====
Module Summary
Module Name:      SmbiosDxe
Module Arch:      X64
Module INF Path:  MdeModule\Universal\SmbiosDxe\SmbiosDxe.inf
File GUID:        F9D88642-0737-49BC-81B5-6889CD57D9EA
Size:             0x7000 (28.00K)
SHA1 HASH:        d94c3f180f25d6b562f477bc4a16b286cb66a8b6 *SmbiosDxe.efi
Build Time Stamp: 1970-01-01 08:00:00
Module Build Time: 1060ms
Driver Type:      0x7 (DRIVER)
=====
... (Module Section Details for SmbiosDxe)
<=====
```

Example2:

```
>=====
Module Summary
Module Name:      EbcDxe
Module Arch:      X64
Module INF Path:  MdeModule\Universal\EbcDxe\EbcDxe.inf
```

```

File GUID:      13AC6DD0-73D0-11D4-B06B-00AA00BD6DE7
Size:           0x9000 (36.00K)
SHA1 HASH:      ff4c019345614afe5c88e7fc37219c30a07f4af4 *EbcDxe.efi
Time Stamp:     1970-01-01 08:00:00
Module Build Time: 1731ms
Driver Type:    0x7 (DRIVER)
=====
... (Module Section Details for EbcDxe)
<=====

```

13.8.2 Library Sub-section

This sub-section, which follows each Module Summary section, holds the information for all libraries used in this module. If it is an EDKII style module, it further lists its correspondent library class, library constructor and destructor name if they exist. The library instances are sorted by the order of their constructor calling sequence and the reverse order of their destructor calling sequence.

- Library INF Path: Path of library instance INF file
- Class*: The library class name of the library instance
- C*: The library constructor if it exists
- D*: The library destructor if it exists
- Time: The build time of this library if it exists

The items marked with * are only available when the module is an EDKII style module and they must be listed in the next line immediately after library instance's INF path.

An example of the module's library instance section is shown below.

Following the subsection header, for each library instance that was linked, the format is:

1. The first line is the INF file path of the library instance
2. {ClassName} - the name of the library class that the above INF file provides
 - If constructors are provided, for each constructor, the following content is inserted in the curly braces after the ClassName:

```
C = ConstructorCName
```

- If destructors are provided, for each destructor, the following is inserted in the curly braces before the closing curly brace.

```
D = DestructorCName
```

- Display the build time.

```
Time = TimeString
```

Example:

```

>-----<
Library
-----
s:\edk2\MdePkg\Library\UefiDevicePathLib\UefiDevicePathLib.inf
{DevicePathLib: Time = 643ms}
s:\edk2\MdePkg\Library\BaseLib\BaseLib.inf
{BaseLib: Time = 14702ms}
s:\edk2\MdePkg\Library\BaseMemoryLib\BaseMemoryLib.inf
{BaseMemoryLib: Time = 284ms}

```

```

s:\edk2\MdePkg\Library\UefiMemoryAllocationLib\UefiMemoryAllocationLib.inf
{MemoryAllocationLib: Time = 249ms}
s:\edk2\MdePkg\Library\UefiBootServicesTableLib\UefiBootServicesTableLib.inf
{UefiBootServicesTableLib: C = UefiBootServicesTableLibConstructor Time = 219ms}
s:\edk2\MdePkg\Library\DxePcdLib\DxePcdLib.inf
{PcdLib: C = PcdLibConstructor Time = 265ms}
s:\edk2\MdePkg\Library\UefiRuntimeServicesTableLib\UefiRuntimeServicesTableLib.inf
{UefiRuntimeServicesTableLib: C = UefiRuntimeServicesLibConstructor Time = 203ms}
s:\edk2\MdePkg\Library\BaseIoLibIntrinsic\BaseIoLibIntrinsic.inf
{IoLib: Time = 702ms}
s:\edk2\MdePkg\Library\BasePciCf8Lib\BasePciCf8Lib.inf
{PciCf8Lib: Time = 345ms}
s:\edk2\MdePkg\Library\BasePciLibCf8\BasePciLibCf8.inf
{PciLib: Time = 341ms}
s:\edk2\MdePkg\Library\BasePrintLib\BasePrintLib.inf
{PrintLib: Time = 312ms}
s:\edk2\Ich9Pkg\Library\IntelIchAcpiTimerLib\IntelIchAcpiTimerLib.inf
{TimerLib: C = IntelAcpiTimerLibConstructor Time = 282ms}
s:\edk2\MdePkg\Library\UefiLib\UefiLib.inf
{UefiLib: Time = 733ms}
s:\edk2\MdePkg\Library\BaseSynchronizationLib\BaseSynchronizationLib.inf
{SynchronizationLib: Time = 920ms}
s:\edk2\MdePkg\Library\DxeHobLib\DxeHobLib.inf
{HobLib: C = DxeHobLibConstructor Time = 218ms}
s:\edk2\MdePkg\Library\UefiDriverEntryPoint\UefiDriverEntryPoint.inf
{UefiDriverEntryPoint Time = 234ms}
s:\edk2\MdePkg\Library\UefiRuntimeLib\UefiRuntimeLib.inf
{UefiRuntimeLib: C = UefiRuntimeLibConstructor D = UefiRuntimeLibDestructor Time = 265ms}
<----->

```

Note: This sub-section is present when **LIBRARY** is specified in **-Y** option.

13.8.3 PCD Sub-section

This sub-section (following the Module Summary information) holds the information for all PCDs used in this module. The content of module PCD sub-section is divided by token space such as:

```

gEfiMdeModulePkgTokenSpaceGuid
...
...
gEfiInt32PkgTokenSpaceGuid
...
...
...

```

Each PCD may contain up to following lines:

1. The first line is a mandatory line with the following format:

```
[*B|*F|*P|*M] <PcdName> : <PcdType> (<DatumType>) [((<SKUID>))[((<DefaultStore>))] = <PcdValue>
```

Note: If the Pcd is a Structure PCD, `<DatumType>` is the Struct Name.

For example:

```
*P PcdWinNtFirmwareVolume : FIXED (VOID*) = L"...\Fv\Nt32.fd"
```

2. The second line is the optional line

- o if `<PcdType>` is DYN-HII

```
<VariableGuid>:<VariableName>:<Offset>
```

For example:

```
*P PcdGmchDvmtTotalSize : DYN-HII (UINT8) = 0x0 (0)
    gSysConfigGuid: L"Setup": 0xAA
```

- o if <PcdType> is DYN-VPD

```
<Offset relative to VPD base address>
```

For example:

```
*F PcdVpdSample : DYN-VPD (UINT32) = 0x1 (1)
    0x0001FFF
```

3. The `DSC DEFAULT`, `INF DEFAULT` and `DEC DEFAULT` are option if the module's final <PcdValue> is not equal to the PCD value in the PCD common section in DSC file, the PCD value in the module INF file and the PCD value in the DEC file respectively.

```
DSC DEFAULT = <Value in PCD Common Section in DSC>
INF DEFAULT = <Value in module INF>
DEC DEFAULT = <Value in DEC>
```

For example:

```
*M PcdDebugPrintErrorLevel : FIXED (UINT32) = 0x80000042 (2147483714)
    DSC DEFAULT = 0x80000040 (2147483712)
    DEC DEFAULT = 0x80000000 (2147483648)
*P PcdPlatformBootTimeOut : DYNHII (UINT16) = 0xA (10)
    gEfiGlobalVariableGuid: L"Timeout": 0x0
    DEC DEFAULT = 0xffff (65535)
```

4. Additional lines may exist if the PCD is Structure PCD. Please refer to 13.6.2.4 Rules for Structure PCD for details.

Note: This sub-section is present when **PCD** is specified in **-Y** option.

13.8.4 DEPEX Sub-section

This sub-section (following the Module Summary information) holds module dependency expression (DEPEX) information. The sub-section header holds the module dependency expression instructions and final dependency expression. If the module is an EDK II style module and it inherits dependency from one of its library instance, it lists the inherited library dependency expression in the sub-section contents.

Note: For `UEFI_DRIVER` module types, the tools may optimize the depex to none, and therefore, a **DEPEX** report may not be output. However, some `UEFI_DRIVER` modules may produce a DEPEX section if libraries that they have been linked with have DEPEX sections.

Example1:

```
>-----<
Final Dependency Expression (DEPEX) Instructions
```

```

PUSH gEfiFirmwareVolumeBlock2ProtocolGuid
PUSH gEfiRuntimeArchProtocolGuid
PUSH gEfiPcdProtocolGuid
PUSH gEfiDevicePathUtilitiesProtocolGuid
AND
AND
AND
END
-----
Dependency Expression (DEPEX) from INF
(gEfiFirmwareVolumeBlockProtocolGuid AND gEfiRuntimeArchProtocolGuid) AND
(gEfiPcdProtocolGuid) AND
(gEfiDevicePathUtilitiesProtocolGuid)
-----
From Module INF: gEfiFirmwareVolumeBlockProtocolGuid AND
gEfiRuntimeArchProtocolGuid
From Library INF: (gEfiPcdProtocolGuid) AND
(gEfiDevicePathUtilitiesProtocolGuid)
<----->

```

Example2:

```

>-----<
Dependency Expression (DEPEX) Instructions
PUSH gEfiPciRootBridgeIoProtocolGuid
PUSH gEfiVariableArchProtocolGuid
PUSH gEfiVariableWriteArchProtocolGuid
PUSH gEfiMetronomeArchProtocolGuid
PUSH gEfiRuntimeArchProtocolGuid
PUSH gEfiHiiDatabaseProtocolGuid
AND
AND
AND
AND
AND
END
-----
Dependency Expression (DEPEX) from DXS
EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL_GUID AND EFI_VARIABLE_ARCH_PROTOCOL_GUID AND
EFI_VARIABLE_WRITE_ARCH_PROTOCOL_GUID AND EFI_METRONOME_ARCH_PROTOCOL_GUID AND
EFI_RUNTIME_ARCH_PROTOCOL_GUID AND EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL_GUID AND
EFI_HII_DATABASE_PROTOCOL_GUID
<----->

```

Note: This sub-section is present when **DEPEX** is specified in **-Y** option.

13.8.5 Build Flags Sub-section

This sub-section (following the Module Summary information) holds module build flags information. The sub-section header holds the module tool chain tag and the subsection contents list all related build flags, arranged using the tool code and flag attributes defined in the `Conf/tools_def.txt` file.

Example

```

>-----<
Build Flags
Tool Chain Tag: VS2008x86
-----
SLINK_FLAGS = /NOLOGO /LTCG
-----
DLINK_FLAGS = /NOLOGO /NODEFAULTLIB /IGNORE:4001 /OPT:REF /OPT:ICF=10 /MAP /

```

```
ALIGN:32 /SECTION:.xdata,D
/SECTION:.pdata,D /MACHINE:X86 /LTCG /DLL /ENTRY:$(IMAGE_ENTRY_POINT) /
SUBSYSTEM:EFI_BOOT_SERVICE_DRIVER /SAFESEH:NO
/BASE:0 /DRIVER /DEBUG /EXPORT:InitializeDriver=$(IMAGE_ENTRY_POINT) /
BASE:0x10000 /ALIGN:4096 /FILEALIGN:4096
/SUBSYSTEM:CONSOLE
-----
CC_FLAGS = /nologo /c /WX /GS- /W4 /Gs32768 /D UNICODE /O1ib2 /GL /FIAutoGen.h /
EHs-c- /GR- /GF /Gy /Zi /Gm
<----->
```

Note: This sub-section is present when `BUILDFLAGS` is specified in `-Y` option.

13.8.6 Fixed Address Prediction Sub-section

This sub-section (following the Module Summary information) contains module notification function information. All the notification functions are listed with the following triplet line by line:

(Type, Address, Name)

%The address type, predicted address, and function name%

The second character of the Type indicates whether the address is in Flash or Memory.

Example1:

```
>-----<
Fixed Address Prediction
*I  Image Loading Address
*E  Entry Point Address
*N  Notification Function Address
*F  Flash Address
*M  Memory Address
*S  SMM RAM Offset
TOM Top of Memory
Type Address      Name
-----
*IF 0x00ffffe6dac (Image Base)
*EF 0x00ffffe6e74 _ModuleEntryPoint
*Nf 0x00ffffe70b5 EndOfPeiCallback
*Nf 0x00ffffe83f0 MemoryDiscoveredPpiNotifyCallback
*IM 0x003ef48000 (Image Base)
*EM 0x003ef480c8 _ModuleEntryPoint
*Nm 0x003ef48309 EndOfPeiSignalPpiNotifyCallback
*Nm 0x003ef49644 EndOfPeiCallback
<----->
```

Example2:

```
>-----<
Fixed Address Prediction
*I  Image Loading Address
*E  Entry Point Address
*N  Notification Function Address
*F  Flash Address
*M  Memory Address
*S  SMM RAM address
TOM Top of Memory
Type Address      Name
-----
*IM TOM-0x00014000 (Image Base)
*EM TOM-0x00013d60 _ModuleEntryPoint
```

```
*IS  TOM-0x00034000  (Image Base)
*ES  TOM-0x00033d60  _ModuleEntryPoint
<----->
```

Note: This sub-section is present when `FIXEDADDRESS` is specified in `-Y` option.

13.9 Execution Order Prediction Section

This section contains platform level prediction for the execution flow. Each phase list the following triple in their predicted order:

(Type, Name, Module INF Path)

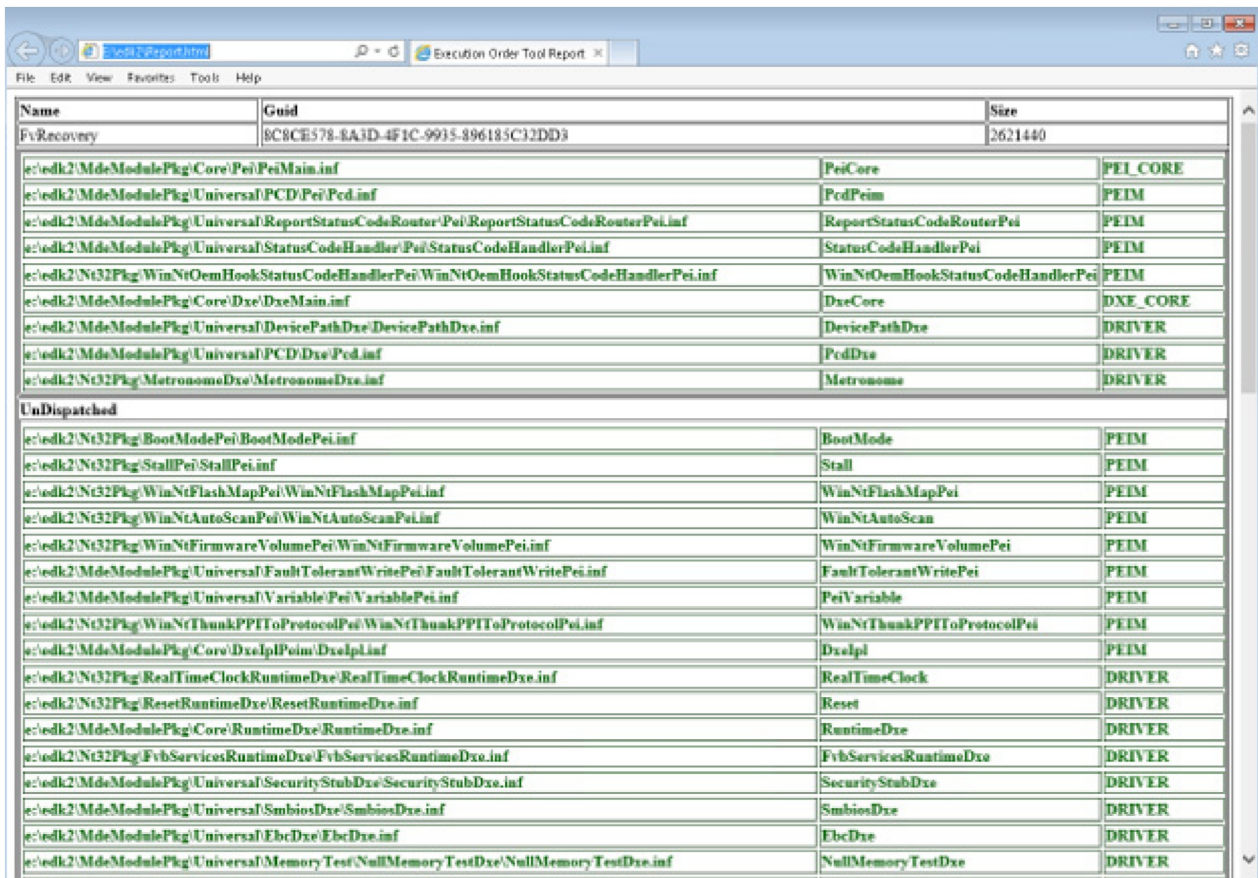
%The entry point or notification function name%

Example

```
>=====<
Execution Order Prediction
*P PEI phase
*D DXE phase
*E Module INF entry point name
*N Module notification function name
Type Symbol      Module INF Path
=====
*PE  PeiCore      s:\edk2\MdeModulePkg\Core\Pei\PeiMain.inf
*PE  PcdPeimInit  s:\edk2\MdeModulePkg\Universal\Pcd\Pei\Pcd.inf
...
*PN EndOfPeiCallback s:\edk2\MyPlatform\PlatformPei\PlatformPei.inf
*DE DxeMain       s:\edk2\MdeModulePkg\Core\Dxe\DxeMain.inf
*DE PcdDxeInit     s:\edk2\MdeModulePkg\Universal\Pcd\Dxe\Pcd.inf
...
<=====
```

Note: This section is present when **EXECUTIONORDER** is specified in **-Y** option.

The following figure shows the HTML format with an entry expanded.



The screenshot shows a web browser window titled "Execution Order Tool Report" displaying a table of execution order predictions. The table has three columns: Name, Guid, and Size. The first section lists various PEI and DXE components, followed by an "UnDispatched" section listing components that are not dispatched.

Name	Guid	Size
FvRecovery	8C8CE578-8A3D-4F1C-9935-896185C32DD3	2621440
e:\edk2\MdeModulePkg\Core\Pei\PeiMain.inf	PeiCore	PEI_CORE
e:\edk2\MdeModulePkg\Universal\PCD\Pei\Pcd.inf	PcdPeim	PEIM
e:\edk2\MdeModulePkg\Universal\ReportStatusCodeRouter\Pei\ReportStatusCodeRouterPei.inf	ReportStatusCodeRouterPei	PEIM
e:\edk2\MdeModulePkg\Universal\StatusCodeHandler\Pei\StatusCodeHandlerPei.inf	StatusCodeHandlerPei	PEIM
e:\edk2\Nt32Pkg\WinNtOemHookStatusCodeHandlerPei\WinNtOemHookStatusCodeHandlerPei.inf	WinNtOemHookStatusCodeHandlerPei	PEIM
e:\edk2\MdeModulePkg\Core\Dxe\DxeMain.inf	DxeCore	DXE_CORE
e:\edk2\MdeModulePkg\Universal\DevicePathDxe\DevicePathDxe.inf	DevicePathDxe	DRIVER
e:\edk2\MdeModulePkg\Universal\PCD\Dxe\Pcd.inf	PcdDxe	DRIVER
e:\edk2\Nt32Pkg\MetronomeDxe\MetronomeDxe.inf	Metronome	DRIVER
UnDispatched		
e:\edk2\Nt32Pkg\BootModePei\BootModePei.inf	BootMode	PEIM
e:\edk2\Nt32Pkg\StallPei\StallPei.inf	Stall	PEIM
e:\edk2\Nt32Pkg\WinNtFlashMapPei\WinNtFlashMapPei.inf	WinNtFlashMapPei	PEIM
e:\edk2\Nt32Pkg\WinNtAutoScanPei\WinNtAutoScanPei.inf	WinNtAutoScan	PEIM
e:\edk2\Nt32Pkg\WinNtFirmwareVolumePei\WinNtFirmwareVolumePei.inf	WinNtFirmwareVolumePei	PEIM
e:\edk2\MdeModulePkg\Universal\FaultTolerantWritePei\FaultTolerantWritePei.inf	FaultTolerantWritePei	PEIM
e:\edk2\MdeModulePkg\Universal\Variable\Pei\VariablePei.inf	PeiVariable	PEIM
e:\edk2\Nt32Pkg\WinNtThunkPPIToProtocolPei\WinNtThunkPPIToProtocolPei.inf	WinNtThunkPPIToProtocolPei	PEIM
e:\edk2\MdeModulePkg\Core\Dxe\IpI\Peim\Dxe\IpI.inf	Dxe\IpI	PEIM
e:\edk2\Nt32Pkg\RealTimeClockRuntimeDxe\RealTimeClockRuntimeDxe.inf	RealTimeClock	DRIVER
e:\edk2\Nt32Pkg\ResetRuntimeDxe\ResetRuntimeDxe.inf	Reset	DRIVER
e:\edk2\MdeModulePkg\Core\RuntimeDxe\RuntimeDxe.inf	RuntimeDxe	DRIVER
e:\edk2\Nt32Pkg\FvbServicesRuntimeDxe\FvbServicesRuntimeDxe.inf	FvbServicesRuntimeDxe	DRIVER
e:\edk2\MdeModulePkg\Universal\SecurityStubDxe\SecurityStubDxe.inf	SecurityStubDxe	DRIVER
e:\edk2\MdeModulePkg\Universal\SmbiosDxe\SmbiosDxe.inf	SmbiosDxe	DRIVER
e:\edk2\MdeModulePkg\Universal\EbcDxe\EbcDxe.inf	EbcDxe	DRIVER
e:\edk2\MdeModulePkg\Universal\MemoryTest\NullMemoryTestDxe\NullMemoryTestDxe.inf	NullMemoryTestDxe	DRIVER

Figure 24 Report.html

APPENDIX A VARIABLES

One of the core concepts of this utility is the notion of symbols. Use of symbols follows the makefile convention of enclosing within `$()`, for example `$(EFI_SOURCE)`. As the utility processes files during execution, it will often perform parsing of variable assignments. These variables can then be referenced in other sections of the DSC file. Variable assignments will be saved internally in either a local or global symbol table. The local symbol table is purged following processing of individual Platform (DSC) files. Global symbol values persist throughout execution of the utility. Local symbol values take precedent over global symbols. The following table describes the symbols generated internally by the utility. They can be overridden either on the command line, in the DSC file, or in individual INF files. The G/L column indicates whether the symbol is typically a global (appears in all Makefiles) or a local (to the module's Makefile) symbol.

Variable descriptions follow in Table 21.

Note: This table does not list required system environment variables or optional system environment variable.

Table 21 Variable Descriptions

Variable Name	G/L	Description
BIN_DIR	L	Specifies the directory where final component binaries are deposited during build. Typically <code>\$(BUILD_DIR)\\$(PROCESSOR)</code>
BUILD_DIR	G	Defines the build tip directory for the current platform. For example, this may be <code>\$(EFI_SOURCE)\Platform\Anacortes_870</code> .
BUILD_TYPE	L	If defined, then the utility will copy the <code>[build.\$(PROCESSOR).\$(BUILD_TYPE)]</code> section from the DSC file to the component's makefile. If not specified, then the <code>[build.\$(PROCESSOR).\$(COMPONENT_TYPE)]</code> section will be used to emit command to build the component.
DEST_DIR	L	For a component, defines the directory (typically under <code>BUILD_DIR</code>) where the component object files are to be built.
DSC_FILENAME	G	Name of the DSC file as specified on the command line. Can be used for dependencies in the Makefiles.
FILE	L	As the utility processes each source file in the Platform (DSC) file, this symbol gets assigned the name of the file, less the file extension.
FV_EXT	L	Common component type (BS driver, application, etc.) have predefined file name extensions assigned (.dxe, .app, etc.). If there is a deviation from the convention, or a new (unknown to the utility) component type is being built, then <code>FV_EXT</code> may need to be defined for the component so the utility knows the result file name extension. This information is necessary to generate dependencies in <code>makefile.out</code> .
INF_FILENAME	L	Name of the INF file for a given component. Can be used for dependencies in the Makefiles.
LIB_DIR	L	Specifies the directory where EFI libraries are deposited after building. Typically <code>\$(BUILD_DIR)\\$(PROCESSOR)</code>
MAKEFILE_NAME	L	Name of the output makefile for the component. Default is "makefile". This value can be overridden to support building different variations of a component in the same <code>DEST_DIR</code> directory.

OUT_DIR	L	Unused, but typically <code>\$(BUILD_DIR)\\$(PROCESSOR)</code>
PACKAGE	L/G	If defined, then the utility will create a package file named <code>\$(DEST_DIR)\\$(BASE_NAME).pkg</code> , and copy, with macro expansion, the <code>[package.\$(COMPONENT_TYPE).\$(PACKAGE)]</code> section from the DSC file to the output file.
PACKAGE_FILE	L	If defined, then the utility will not generate a package file. The build can then use the value <code>\$(PACKAGE_FILE)</code> to have GenFfsFile use an existing package file for creating the firmware file.
PLATFORM	L	This symbol can be used to provide more selectivity of files in the Platform (DSC) files. If assigned, then the utility will also process any files in the INF file under sections <code>[sources.\$(PROCESSOR).\$(PLATFORM)]</code> , <code>[includes.\$(PROCESSOR).\$(PLATFORM)]</code> , and <code>[libraries.\$(PROCESSOR).\$(PLATFORM)]</code> .
PROCESSOR	G/L	Defines the target processor for which the code is to be built. This symbol will typically be used to include or exclude source files in Platform (DSC) files, and to define the tool chain for building.
SOURCE_DIR	L	For a component, defines the directory of the component source files.

APPENDIX B TOOLS_DEF.TXT

The following is the default version of the `tools_def.txt` file. No line wrapping is permitted in the `tools_def.txt` file.

Line extension characters are not permitted in this file. Each entry must reside on a single line.

The `edksetup.bat` (or `edksetup.sh`) script will copy the template file from the `BaseTools/Conf` directory into the (system environment variable `WORKSPACE`) `WORKSPACE/Conf` directory and rename it if it does not already exist.

APPENDIX C TARGET.TXT

No line wrapping is permitted in the `target.txt` file.

The `edksetup.bat` (or `edksetup.sh`) script will copy the template file from the `BaseTools/Conf` directory into the (system environment variable `WORKSPACE`) `WORKSPACE/Conf` directory and rename it if it does not already exist.

APPENDIX D BUILD.EXE COMMAND

This section describes the **build.exe** command line tool's options. **Build.exe** is generated from Python code. Options on the command line may be specified in any order.

D.1 Overview

Under normal circumstances, the build tool will:

1. Process the command-line options
2. Parse the meta-data files
3. Generate the C files
4. Generate the Makefiles
5. Call the make command.

D.2 Makefile actions

The module's `Makefile` is responsible for compilation of the source code and executing the **GenFw** command on the intermediate object files in order to create the `.efi` files.

The actions taken by the `Makefile` are:

1. Create the build output directories
2. Build the libraries
3. Build the modules
4. Call the **GenFds** tool
5. The last step of the `Makefile` processing is to call the **GenFds** tool that will:
 - Process the command-line options
 - Parse the meta-data files
 - Create FFS, Capsules, FV images and the final FD image(s).

D.3 Build Targets and options

In order to provide flexibility, the build command supports stopping the build process after specific actions have taken place. These targets will ensure that all previously required actions have been completed. New for this release is the implementation of targets that permit processing files for only one given step, such that previous steps are NOT processed. Table 22 provides the descriptions of targets supported by the build, as well as the **GenFds** tools.

Note: The flag, **--skip-autogen**, is required to prevent the build tool from re-creating the auto generated C and Makefiles.

Table 22 Build Targets and Command-line Options

Target	Description
genc	Generates the C code files (AutoGen.c, AutoGen.h and ModuleName.depex) then stops
genmake	Generates the C code files (AutoGen.c, AutoGen.h and ModuleName.depex) then the Makefiles, then stops
libraries	Generates the C code files, the Makefiles, then generates the object files for libraries
modules	Generates the C code files, the Makefiles, generates the object files for libraries, generates the object files for the modules, then links them, then calls GenFw for each of the intermediate final objects to create .efi files
fds	Generates the C code files, the Makefiles, generates the object files for libraries, generates the object files for the modules, links them, calls GenFw for each of the intermediate final objects to create .efi files, generates SECTION files, generates FFS files, generates FV files and finally generates FD files

D.4 Usage

Usage: build.exe [options]

[all|fds|genc|genmake|clean|cleanall|cleanlib|modules|libraries|run]

Copyright (c) 2007 - 2017, Intel Corporation All rights reserved.

Options:

- version show program's version number and exit
- h, --help show this help message and exit
- a TARGETARCH, --arch=TARGETARCH
 ARCHS is one of list: IA32, X64, IPF, ARM, AARCH64 or
 EBC, which overrides target.txt's TARGET_ARCH
 definition. To specify more archs, please repeat this
 option.
- p PLATFORMFILE, --platform=PLATFORMFILE
 Build the platform specified by the DSC file name
 argument, overriding target.txt's ACTIVE_PLATFORM
 definition.
- m MODULEFILE, --module=MODULEFILE
 Build the module specified by the INF file name
 argument.
- b BUILDTARGET, --buildtarget=BUILDTARGET
 Using the TARGET to build the platform, overriding
 target.txt's TARGET definition.
- t TOOLCHAIN, --tagname=TOOLCHAIN
 Using the Tool Chain Tagname to build the platform,
 overriding target.txt's TOOL_CHAIN_TAG definition.
- x SKUID, --sku-id=SKUID
 Using this name of SKU ID to build the platform,
 overriding SKUID_IDENTIFIER in DSC file.
- n THREADNUMBER
 Build the platform using multi-threaded compiler. The
 value overrides target.txt's
 MAX_CONCURRENT_THREAD_NUMBER. When value is set to 0,
 tool automatically detect number of processor threads,
 set value to 1 means disable multi-thread build, and
 set value to more than 1 means user specify the threads
 number to build.
- f FDFFILE, --fdf=FDFFILE
 The name of the FDF file to use, which overrides the
 setting in the DSC file.
- r ROMIMAGE, --rom-image=ROMIMAGE
 The name of FD to be generated. The name must be from
 [FD] section in FDF file.
- i FVIMAGE, --fv-image=FVIMAGE
 The name of FV to be generated. The name must be from
 [FV] section in FDF file.
- C CAPNAME, --capsule-image=CAPNAME
 The name of Capsule to be generated. The name must be
 from [Capsule] section in FDF file.
- u, --skip-autogen Skip AutoGen step.
- e, --re-parse Re-parse all meta-data files.
- c, --case-insensitive
 Don't check case of file name.
- w, --warning-as-error
 Treat warning in tools as error.
- j LOGFILE, --log=LOGFILE
 Put log in specified file as well as on console.
- s, --silent Make use of silent mode of (n)make.
- q, --quiet Disable all messages except FATAL ERRORS.
- v, --verbose Turn on verbose output with informational messages
 printed, including library instances selected, final
 dependency expression, and warning messages, etc.
- d DEBUG, --debug=DEBUG
 Enable debug messages at specified level.
- D MACROS, --define=MACROS
 Macro: "Name [= Value]".
- y REPORTFILE, --report-file=REPORTFILE
 Create/overwrite the report to the specified filename.

```

-Y REPORTTYPE, --report-type=REPORTTYPE
    Flags that control the type of build report to
    generate. Must be one of: [PCD, LIBRARY, FLASH, DEPEX,
    HASH, BUILD_FLAGS, FIXED_ADDRESS, EXECUTION_ORDER].
    To specify more than one flag, repeat this option on
    the command line and the default flag set is [PCD,
    LIBRARY, FLASH, DEPEX, HASH, BUILD_FLAGS,
    FIXED_ADDRESS]
-F FLAG, --flag=FLAG Specify the specific option to parse EDK UNI file.
    Must be one of: [-c, -s]. -c is for EDK framework UNI
    file, and -s is for EDK UEFI UNI file. This option can
    also be specified by setting *_*_BUILD_FLAGS in
    [BuildOptions] section of platform DSC. If they are
    both specified, this value will override the setting
    in [BuildOptions] section of platform DSC.
-N, --no-cache        Disable build cache mechanism
--conf=CONFDIRECTORY Specify the customized Conf directory.
--check-usage         Check usage content of entries listed in INF file.
--ignore-sources       Focus to a binary build and ignore all source files
--pcd=OPTIONPCD        Set PCD value by command line. Format: "PcdName=Value"
-l COMMANDLENGTH, --cmd-len=COMMANDLENGTH
    Specify the maximum line length of build command.
    Default is 4096.
--hash                Enable hash-based caching during build process.
--binary-destination=BINCACHEDEST
    Generate a cache of binary files in the specified
    directory.
--binary-source=BINCACHESOURCE
    Consume a cache of binary files from the specified
    directory.

```

D.4.1 Debug Levels

The numeric debug levels are defined as integer values 0-9.

Level 0 will provide a few extra messages that might, under certain environments, cause a build to break, during later stages of the build.

Level 1 provides messages from level 0, along with information related to PCDs.

Level 2 provides messages from levels 1 and 0, along with information related to Macros.

Level 3 provides all messages from levels 0 - 2, along with information related to Library Classes as well as generating code for PCDs during AutoGen.

Level 4 provides all previous level messages - no new information is added

Level 5 provides all previous level information as well as information regarding the database that is used by the build system tools to decrease incremental build times as well as HII information.

Levels 6 and 7 provides all previous messages - no new information is added

Level 8 provides all previous messages as well as adding build process information, such as queues and threads running.

Level 9 provides the most details, displaying all previous messages and adding information about what is happening at each step during the build.

D.4.2 MACRO Option Definition

This section provides the EBNF for the `-D` option, which allows users to specify macro values on the command-line. Macro values on the command-line take precedence over Macros defined in the DSC and FDF files.

Prototype

```

<MacroOption>    ::= {<ShortOpt>} {<LongOpt>}
<SP>             ::= 0x20
<MTS>           ::= <SP>+
<ShortOpt>       ::= "-D" <SP> <MACRO> ["=" <Value>] <MTS>
<LongOpt>        ::= "--define" "=" <MACRO> ["=" <Value>] <MTS>
<MACRO>          ::= (A-Z)(a-zA-Z0-9_)*
<Value>          ::= {<Number>} {<CString>} {<TrueFalse>} {<RegFmtGUID>}
<Number>         ::= {"0x" (a-fA-F0-9)+} {(0-9)+
<CString>        ::= ["L"] <QuotedString>
<QuotedString>   ::= <DbQuote> <CChars>* <DbQuote>
<DbQuote>        ::= 0x22
<CChars>         ::= {0x21} {(0x23 - 0x5B)} {(0x5D - 0x7E)} {<EscapeSequence>}
<EscapeSequence> ::= "\" {"n"} {"t"} {"f"} {"r"} {"b"} {"0"} {"\""} {0x22}
<TrueFalse>      ::= {"TRUE"} {"True"} {"true"} {"FALSE"} {"False"} {"false"}
<H4>             ::= (a-fA-F0-9) (a-fA-F0-9) (a-fA-F0-9) (a-fA-F0-9)
<H8>             ::= <H4> <H4>
<H12>            ::= <H4> <H4> <H4>
<RegFmtGUID>     ::= <H8> "-" <H4> "-" <H4> "-" <H4> "-" <H12>

```

D.4.3 PCD Option Definition

This section provides the EBNF for the `--pcd` option, which allows users to specify PCD values on the command-line. PCD values on the command-line take precedence over PCD provided in DSC, FDF, INF, and DEC files.

Prototype

```

<PcdOption>      ::= "--pcd" <PcdName> ["=" <PcdValue>] <MTS>
<SP>             ::= 0x20
<MTS>           ::= <SP>+
<TS>            ::= <SP>*
<CommaSpace>     ::= ", " <SP>*
<HexDigit>       ::= (a-fA-F0-9)
<CName>          ::= A valid C variable name.
<PcdName>        ::= [<TokenSpaceCName> "."] <PcdCName> ["." <Field>]
<TokenSpaceCName> ::= C Variable Name of the Token Space GUID
<PcdCName>       ::= C Variable Name of the PCD
<Field>          ::= C Variable Name of the Structure PCD field
<PcdValue>       ::= {<Boolean>} {<Number>} {<String>} {<Array>}
<Number>         ::= {<Integer>} {<HexNumber>}
<Integer>        ::= {(0-9)} {(1-9)(0-9)+}
<HexNumber>      ::= {"0x"} {"0X"} (a-fA-F0-9){1,16}
<Boolean>        ::= {<True>} {<False>}
<True>           ::= {"TRUE"} {"True"} {"true"} {"1"} {"0x1"} {"0x01"}
<False>          ::= {"FALSE"} {"False"} {"false"} {"0"} {"0x0"} {"0x00"}
<String>         ::= {<QuotedStr>} {<SglQuotedStr>}
<QuotedStr>      ::= ["L"] <DbQuote> <PrintChars>+ <DbQuote>
<SglQuotedStr>   ::= ["L"] <DbQuote> "\" <SglQuote> <PrintChars>+
                  "\" <SglQuote> <DbQuote>
<PrintChars>     ::= {<TS>} {<CChars>}
<DbQuote>        ::= 0x22
<SglQuote>       ::= 0x27
<CChars>         ::= {0x21} {(0x23 - 0x26)} {(0x28 - 0x5B)} {(0x5D - 0x7E)}
                  {<EscapeSequence>}
<EscapeSequence> ::= "\" {"n"} {"t"} {"f"} {"r"} {"b"} {"0"} {"\""}
                  {<DbQuote>} {<SglQuote>}
<Array>          ::= "H" <DbQuote> "{"[<Lable>] <ArrayVal>
                  [<CommaSpace> [<Lable>] <ArrayVal>]*}" <DbQuote>
<ArrayVal>       ::= {<Num8Array>} {<GuidStr>} {<DevicePath>}
<ShortNum>       ::= (0-255)
<IntNum>         ::= (0-65535)
<LongNum>        ::= (0-4294967295)
<LongLongNum>    ::= (0-18446744073709551615)
<UINT8>          ::= {"0x"} {"0X"} (a-fA-F0-9){1,2}
<UINT16>         ::= {"0x"} {"0X"} (a-fA-F0-9){1,4}
<UINT32>         ::= {"0x"} {"0X"} (a-fA-F0-9){1,8}
<UINT64>         ::= <HexNumber>

```

```

<ArrayString>      ::= {<ArrayQuotedStr>} {<ArraySglQuotedStr>}
<ArrayQuotedStr>   ::= ["L"] "\"" <Db1Quote> <PrintChars>* "\"" <Db1Quote>
<ArraySglQuotedStr> ::= ["L"] "\"" <SglQuote> <PrintChars>* "\"" <SglQuote>
<NonNumType>       ::= {<Boolean>} {<ArrayString>} {<Offset>} {<UIntMac>}
<Num8Array>        ::= {<NonNumType>} {<ShortNum>} {<UINT8>}
<Num16Array>       ::= {<NonNumType>} {<IntNum>} {<UINT16>}
<Num32Array>       ::= {<NonNumType>} {<LongNum>} {<UINT32>}
<Num64Array>       ::= {<NonNumType>} {<LongLongNum>} {<UINT64>}
<GuidStr>          ::= "GUID(" <GuidVal> ")"
<GuidVal>          ::= {"\"<Db1Quote> <RegistryFormatGUID> "\"<Db1Quote>}
                    {<CFormatGUID>} {<CName>}
<RegistryFormatGUID> ::= <RHex8> "-" <RHex4> "-" <RHex4> "-" <RHex4> "-"
                    <RHex12>
<RHex4>            ::= <HexDigit> <HexDigit> <HexDigit> <HexDigit>
<RHex8>            ::= <RHex4> <RHex4>
<RHex12>           ::= <RHex4> <RHex4> <RHex4>
<RawH2>            ::= <HexDigit>? <HexDigit>
<RawH4>            ::= <HexDigit>? <HexDigit>? <HexDigit>? <HexDigit>
<OptRawH4>         ::= <HexDigit>? <HexDigit>? <HexDigit>? <HexDigit>?
<Hex2>             ::= {"0x"} {"0X"} <RawH2>
<Hex4>             ::= {"0x"} {"0X"} <RawH4>
<Hex8>             ::= {"0x"} {"0X"} <OptRawH4> <RawH4>
<Hex12>            ::= {"0x"} {"0X"} <OptRawH4> <OptRawH4> <RawH4>
<Hex16>            ::= {"0x"} {"0X"} <OptRawH4> <OptRawH4> <OptRawH4>
                    <RawH4>
<CFormatGUID>      ::= "{" <Hex8> <CommaSpace> <Hex4> <CommaSpace>
                    <Hex4> <CommaSpace> "{"
                    <Hex2> <CommaSpace> <Hex2> <CommaSpace>
                    <Hex2> <CommaSpace> <Hex2> <CommaSpace>
                    <Hex2> <CommaSpace> <Hex2> <CommaSpace>
                    <Hex2> <CommaSpace> <Hex2> "}" "}"
<DevicePath>       ::= "DEVICE_PATH(" <DevicePathStr> ")"
<DevicePathStr>    ::= A double quoted string that follow the device path
                    as string format defined in UEFI Specification 2.6
                    Section 9.6
<UIntMac>          ::= {<UInt8Mac>} {<UInt16Mac>} {<UInt32Mac>} {<UInt64Mac>}
<UInt8Mac>         ::= "UINT8(" <Num8Array> ")"
<UInt16Mac>        ::= "UINT16(" <Num16Array> ")"
<UInt32Mac>        ::= "UINT32(" <Num32Array> ")"
<UInt64Mac>        ::= "UINT64(" <Num64Array> ")"
<Lable>            ::= "LABEL(" <CName> ")"
<Offset>           ::= "OFFSET_OF(" <CName> ")"

*****

**Note:** The " and ' inside the string, must use escape character format (\", \').
*****

```

APPENDIX E NT32 PLATFORM EMULATION

The NT32Pkg provides a platform emulation environment that executes on windows platform. The EDK II build program is used to start the emulation environment after it has been built. The `Nt32Pkg\Nt32Pkg.dsc` file has been modified to also build a version that will run on 64-bit versions of Windows. The architectural modifier, **-a**, of the **build.exe** command is used to enable this option.

Prior to building the platform: `Nt32Pkg\Nt32Pkg.dsc`, the user may want to modify PCD settings in the file. The following PCDs control the mappings of your system environment to the emulation environment.

```
PcdWinNtSerialPort|L"COM1!COM2"|VOID*|18
```

This maps the serial port to COM1 or COM2 (if COM1 is not available).

```
PcdWinNtFileSystem|L"..\.\.\.\.EdkShellBinPkg\bin\ia32\Apps"|VOID*|106
```

This shows the location of the shell applications.

```
PcdWinNtGop|L"UGA Window 1!UGA Window 2"|VOID*|50
```

This defines label for the two windows that are started.

```
PcdWinNtConsole|L"Bus Driver Console Window"|VOID*|50
```

This defines label for the windows that are started.

```
PcdWinNtVirtualDisk|L"FW;40960;512"|VOID*|24
```

This defines the max and block sizes for the virtual disk drive that is created.

```
PcdWinNtMemorySize|L"64!64"|VOID*|10
```

This defines the memory available for the emulator in megabytes.

```
PcdWinNtPhysicalDisk|L"a:RW;2880;512!d:R0;307200;2048!j:RW;262144;512"|VOID*|100
```

This defines the available storage devices that must be present at startup, A:, D: and J: - you may want to change the drive letters to match the development environment - note that you must not use the C: drive, as you could inadvertently wipe it out.

```
PcdWinNtUga|L"UGA Window 1!UGA Window 2"|VOID*|50
```

This defines label for the two windows that are started

APPENDIX F FIRMWARE VOLUME INF

The Firmware Volume INF file is generated by the EDK II build tools as an intermediate file between the code generation stage and the final image creation stage.

F.1 Firmware Volume INF Description

The Firmware Volume INF files are generated by tool based on content from Platform description files (DSC) and Flash definition files (FDF) and may contain these three sections: [options] , [attributes] and [files] .

This file is an input to the GenFvImage utility.

```
<FIRMWARE_VOLUME_INF> ::= [ <options> ]  
                           [ <attributes> ]  
                           [ <files> ]
```

F.2 [Attributes] Section

Summary

Defines the `[Attributes]` tag is found only in Firmware Volume INF files. This file is created by the tools and is an input to the **GenFv** utility. Refer to the document, "Intel(R) Platform Innovation Framework for EFI, Firmware Volume Block Specification" for more information on these values. This is an optional section.

Prototype

```
<attributes> ::= "[attributes]" <EOL>
               <expression>

<expression> ::= ["EFI_READ_DISABLED_CAP" "=" <TrueFalse> <EOL>]
                 ["EFI_READ_ENABLED_CAP" "=" <TrueFalse> <EOL>]
                 ["EFI_READ_STATUS" "=" <TrueFalse> <EOL>]
                 ["EFI_WRITE_DISABLED_CAP" "=" <TrueFalse> <EOL>]
                 ["EFI_WRITE_ENABLED_CAP" "=" <TrueFalse> <EOL>]
                 ["EFI_WRITE_STATUS" "=" <TrueFalse> <EOL>]
                 ["EFI_LOCK_CAP" "=" <TrueFalse> <EOL>]
                 ["EFI_LOCK_STATUS" "=" <TrueFalse> <EOL>]
                 ["EFI_ERASE_POLARITY" "=" <ZeroOne> <EOL>]
                 ["EFI_STICK_WRITE" "=" <TrueFalse> <EOL>]
                 ["EFI_MEMORY_MAPPED" "=" <TrueFalse> <EOL>]
                 ["EFI_ALIGNMENT_CAP" "=" <TrueFalse> <EOL>]
                 ["EFI_ALIGNMENT_2" "=" <TrueFalse> <EOL>]
                 ["EFI_ALIGNMENT_4" "=" <TrueFalse> <EOL>]
                 ["EFI_ALIGNMENT_8" "=" <TrueFalse> <EOL>]
                 ["EFI_ALIGNMENT_16" "=" <TrueFalse> <EOL>]
                 ["EFI_ALIGNMENT_32" "=" <TrueFalse> <EOL>]
                 ["EFI_ALIGNMENT_64" "=" <TrueFalse> <EOL>]
                 ["EFI_ALIGNMENT_128" "=" <TrueFalse> <EOL>]
                 ["EFI_ALIGNMENT_256" "=" <TrueFalse> <EOL>]
                 ["EFI_ALIGNMENT_512" "=" <TrueFalse> <EOL>]
                 ["EFI_ALIGNMENT_1K" "=" <TrueFalse> <EOL>]
                 ["EFI_ALIGNMENT_2K" "=" <TrueFalse> <EOL>]
                 ["EFI_ALIGNMENT_4K" "=" <TrueFalse> <EOL>]
                 ["EFI_ALIGNMENT_8K" "=" <TrueFalse> <EOL>]
                 ["EFI_ALIGNMENT_16K" "=" <TrueFalse> <EOL>]
                 ["EFI_ALIGNMENT_32K" "=" <TrueFalse> <EOL>]
                 ["EFI_ALIGNMENT_64K" "=" <TrueFalse> <EOL>]

<TrueFalse> ::= {<ZeroOne>} {<TF>}
<TF>        ::= {<True>} {<False>}
<True>      ::= {"TRUE"} {"True"} {"true"}
<False>     ::= {"FALSE"} {"False"} {"false"}
<ZeroOne>   ::= {"0"} {"1"}
<EOL>      ::= end of line
```

Example

```
[attributes]
EFI_READ_DISABLED_CAP = TRUE
EFI_READ_ENABLED_CAP  = TRUE
EFI_READ_STATUS       = TRUE
EFI_WRITE_DISABLED_CAP = TRUE
EFI_WRITE_ENABLED_CAP  = TRUE
EFI_WRITE_STATUS      = TRUE
EFI_LOCK_CAP          = TRUE
EFI_LOCK_STATUS       = FALSE
EFI_STICKY_WRITE      = TRUE
EFI_MEMORY_MAPPED     = TRUE
EFI_ERASE_POLARITY    = 1
```

EFI_ALIGNMENT_CAP	= TRUE
EFI_ALIGNMENT_2	= TRUE
EFI_ALIGNMENT_4	= TRUE
EFI_ALIGNMENT_8	= TRUE
EFI_ALIGNMENT_16	= TRUE
EFI_ALIGNMENT_32	= TRUE
EFI_ALIGNMENT_64	= TRUE
EFI_ALIGNMENT_128	= TRUE
EFI_ALIGNMENT_256	= TRUE
EFI_ALIGNMENT_512	= TRUE
EFI_ALIGNMENT_1K	= TRUE
EFI_ALIGNMENT_2K	= TRUE
EFI_ALIGNMENT_4K	= TRUE
EFI_ALIGNMENT_8K	= TRUE
EFI_ALIGNMENT_16K	= TRUE
EFI_ALIGNMENT_32K	= TRUE
EFI_ALIGNMENT_64K	= TRUE

F.3 [Files] Section

Summary

Defines the `[files]` tag is found only in Firmware Volume INF files. This file is created by the build utility and is an input to the **GenFv** utility

Prototype

```
<files>          ::= "[files]" <EOL>
                  <expression>+
<expression>     ::= <Filename> [<COMPONENT_TYPE>] [<FVS>]
                  [<FFSEXT>] ["PROCESSOR=" <arch>] [<APRORI>]
                  [<EFN>] <EOL>
<Filename>       ::= <PATH> <Word> <Extension>
<COMPONENT_TYPE> ::= Refer to Table "Component (module) Types"
<PATH>           ::= [[".."]{0,1} "\"]* {<Word> {"\"}{0,1}}*
<arch>           ::= {IA32} {X64} {IPF} {EBC}
<FVS>            ::= "FVs=" <FvImageName>["", <FvImageName>]*
<FvImageName>    ::= <Word>
<FFSEXT>         ::= "FFSExt=" <Extension>
<APRORI>         ::= "APRORI=" <FvImageNameIdx>
                  [", " <FvImageNameIdx>]*
<FvImageNameIdx> ::= <FvImageName> ";" <PositiveInt>
<PositiveInt>    ::= Integer value greater than 0
<EFN>            ::= "EFI_FILE_NAME" "=" <Path> <Arch>
                  <FileSep> <Word> <Extension> [<XipSuffix>]
<XipSuffix>      ::= ",XIP"
<Extension>      ::= "." (a-zA-Z0-9_-)+
```

Parameters

XipSuffix

The optional `,XIP` suffix appended immediately after the file path indicates that the referenced FFS file is intended for eXecute-In-Place (XIP) operation.

This suffix is generated by the build system based on the `Xip=TRUE` keyword in the applicable FDF `[Rule]` section and should not be manually edited. The suffix is added regardless of the `FvForceRebase` setting.

The **GenFv** tool uses the `,XIP` suffix only when `ForceRebase` is `TRUE` :

- If `ForceRebase` is `TRUE` and one or more files have the `,XIP` suffix, only those files are rebased (selective rebase).
- If `ForceRebase` is `TRUE` and no files have the `,XIP` suffix, all eligible files are rebased (legacy behavior preserved).
- If `ForceRebase` is `FALSE`, no files are rebased regardless of the `,XIP` suffix.
- If `ForceRebase` is not specified, the `,XIP` suffix is not consulted. Rebase occurs for all eligible files only if `BaseAddress` is non-zero.

Example

```
[files]
EFI_FILE_NAME = C:\Build\Platform\DEBUG\FV\Ffs\PeiCore.ffs,XIP
EFI_FILE_NAME = C:\Build\Platform\DEBUG\FV\Ffs\PcdPeim.ffs,XIP
EFI_FILE_NAME = C:\Build\Platform\DEBUG\FV\Ffs\DxeIpl.ffs
EFI_FILE_NAME = C:\Build\Platform\DEBUG\FV\Ffs\PlatformPei.ffs,XIP
```

```
EFI_FILE_NAME = C:\Build\Platform\DEBUG\FV\Ffs\DxeCore.ffs
```

F.4 [Options] Section

Summary

Defines the `[options]` tag is found only in Firmware Volume INF files. This is an optional section.

Prototype

```
<options>      ::= "[options]" <EOL>
                  <expression>+
<expression> ::= <Variable> "=" <Value> <EOL>
<Variable>    ::= {"EFI_BASE_ADDRESS"} {"EFI_BLOCK_SIZE"}
                  {"EFI_FILE_NAME"} {"EFI_NUM_BLOCKS"}
                  {"EFI_SYM_FILE_NAME"} {"IA32_RST_BIN"}
<CName>       ::= A valid C variable name
<VAL>         ::= "0x" <HexDigit>{1,8}
<Value>       ::= {<String>} {<VAL>} {<Filename>}
```

Example

```
[options]
EFI_BASE_ADDRESS = 0xFFD80000
EFI_FILE_NAME    = FvRecovery.fv
EFI_NUM_BLOCKS   = 0x28
EFI_BLOCK_SIZE   = 0x10000
```

APPENDIX G VS2005 TEAM SUITE PERFORMANCE PROFILE

This appendix provides the best known method for using Microsoft Visual Studio 2005* Team Suite to get performance data. It involves "porting" the EFI code to a Win32 console application and then using the VS Performance Wizard to figure out how to tune it.

G.1 Step 1 - Create a new project

On the Menu bar, select: **File->New->Project..**

In the Project types: frame select **Visual C++ -> Win32**

In the Templates: frame then select **Win32 Console Application**

- Give the project name and a solution
- Accept the default settings from the wizard.

A `<project name>.cpp` file will be generated

```
<project name>.cpp
// Test.cpp : Defines the entry point for the console application.
//

#include "stdafx.h"

int _tmain (int argc, _TCHAR *argv[])
{
    return 0;
}
```

G.2 Step 2 - Update the project

You will need to update the new project to support reading in input files and writing data to an output file:

```
#include "stdafx.h"
#include <windows.h>
#include <stdio.h>

void
*Malloc (
    int Size
)
{
    return HeapAlloc (GetProcessHeap (), 0, Size);
}

int
_tmain (
    int argc,
    _TCHAR *argv[]
)
{
    HANDLE hFile;
    HANDLE hOutFile;
    DWORD Error;
    DWORD BytesRead;
    BY_HANDLE_FILE_INFORMATION FileInfo;
    void *Buffer;
    int Status;
    UINT32 DestinationSize;
    VOID *Destination;

    Status = 0;
    printf ("test %d\n", argc);
    if (argc <= 1) {
        return 0;
    }

    hFile = CreateFile (
        argv[1],
        GENERIC_READ,
        FILE_SHARE_READ,
        NULL,
        OPEN_EXISTING,
        0,
        0
    );
    if (hFile == INVALID_HANDLE_VALUE) {
        Error = GetLastError ();
        return Error;
    }

    if (!GetFileInformationByHandle (hFile, &FileInfo)) {
        Error = GetLastError ();
        return Error;
    }

    if (FileInfo.nFileSizeHigh != 0) {
        // Assume input file is less than 4GB in size
        return 0;
    }

    Buffer = Malloc (FileInfo.nFileSizeLow);

    if (!ReadFile (hFile, Buffer, FileInfo.nFileSizeLow, &BytesRead, NULL)) {
        Error = GetLastError ();
        return Error;
    }
}
```

```

}

// Process File ...
// DestinationSize = ...
// Destination = ...

// If a 2nd argument exists it is a file name to write data to
if ((argc >= 3) && (Status == 0)) {
    hOutFile = CreateFile (
        argv[2],
        GENERIC_WRITE | GENERIC_READ,
        0,
        NULL,
        CREATE_ALWAYS,
        FILE_ATTRIBUTE_NORMAL,
        NULL
    );
    if (hOutFile != INVALID_HANDLE_VALUE) {
        if (!WriteFile (hOutFile, Destination, DestinationSize, &BytesRead, NULL)) {
            Error = GetLastError ();
        }
        CloseHandle (hOutFile);
    }
}

CloseHandle (hFile);
return 0;
}

```

G.2.1 To pass an argument in to the console application

Do the following:

1. Update the <project name> Property Pages:
2. Right click on the <project name> in the Solution Explorer pain
3. Select preferences
4. In the configurations: window select All Configurations
5. In the left hand pain select Configuration Properties->Debugging
6. Under Command Arguments type in the command line. In my example the input file is compress and the output file is decompress.out

In this example compress is the EDK II NT32 FV (2.5MB) compressed to 707K.

So decompress.out must be 2.5MB NT32 FV.

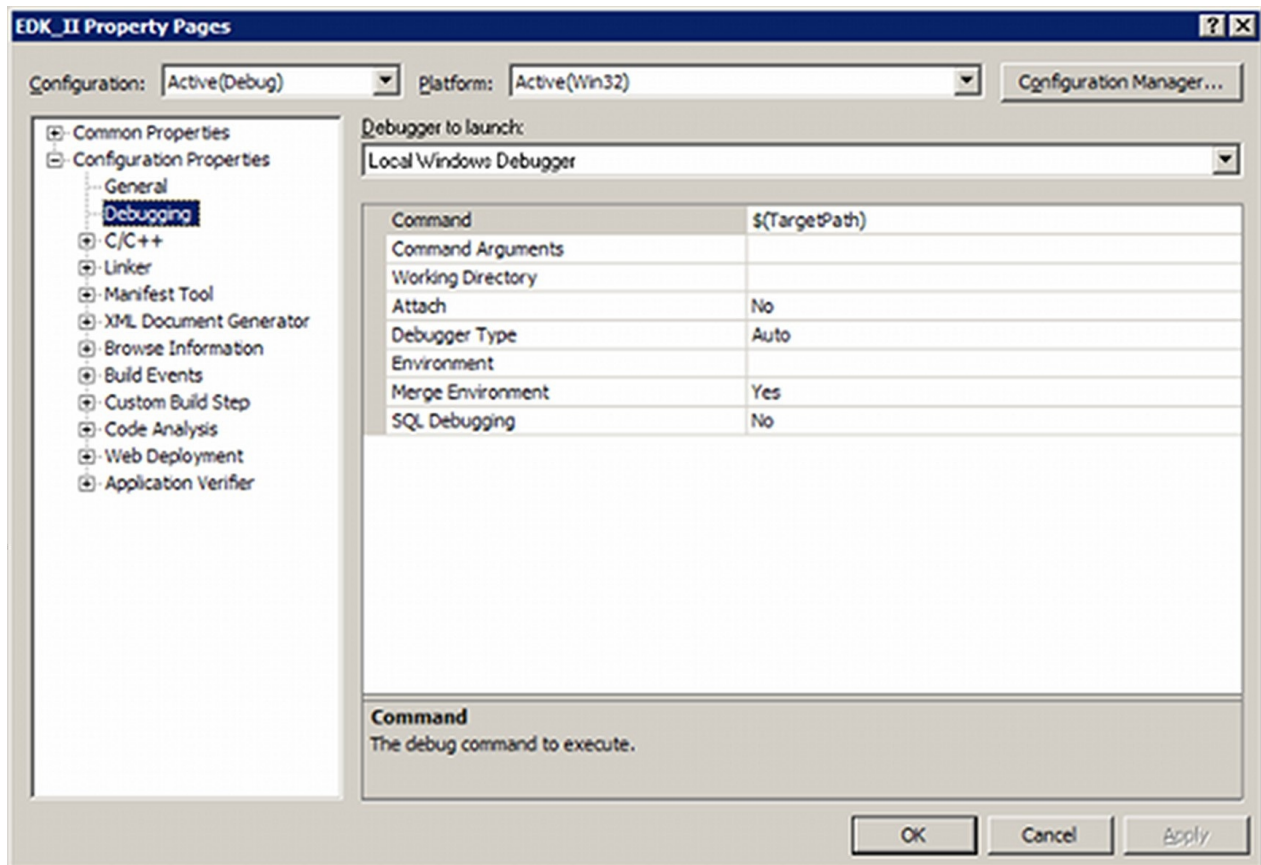


Figure 25 VS2005 Property Page

This example required the EDK II Decompress Lib be ported into this environment as follows:

1. Add EDK II EFI type definitions to get the EFI code to compile.

```
//
// Map EFI types
//
typedef unsigned __int64  UINT64;
typedef __int64          INT64;
typedef unsigned __int32  UINT32;
typedef __int32          INT32;
typedef unsigned short    UINT16;
typedef unsigned short    CHAR16;
typedef short            INT16;
typedef unsigned char     BOOLEAN;
typedef unsigned char     UINT8;
typedef char              CHAR8;

#define UINT8_MAX  0xff
```

2. Convert EFI_STATUS/RETURN_STATUS to int and removed #defines for return values to make it easier for the code to compile.
3. Glue in the EFI code into _tmain()

```
// Process File
Status = UefiDecompressGetInfo (
    Buffer,
    FileInfo.nFileSizeLow,
    &DestinationSize, &ScratchSize
);
if (Status == 0) {
    Destination = Malloc (DestinationSize);
    Scratch = Malloc (ScratchSize);
```

```

if ((Scratch != NULL) && (Destination != NULL)) {
    Status = UefiTianoDecompress (Buffer, Destination, Scratch, 2);
    if (Status != 0) {
        printf ("Decompress Failed");
    }
}
}
}

```

G.2.2 Step 3 Run the Performance Wizard

1. Tools->Performance Tools->Performance Wizard...
2. Make sure your project is selected and hit Next
3. When you are asked what method of profiling would like to use select Instrumentation.
 - The default is Sampling so you must change this
4. Then type Finish
5. A Performance Explorer pain will show up.
6. Right click on you project name and select Launch
 - This will rebuild your application with performance infrastructure.
 - Under Reports you will see a <Project Name>[date].vsp file that contains the info

Make sure you profile in the Release build and not the Debug build for best results.

The following is an example of the output you will see.

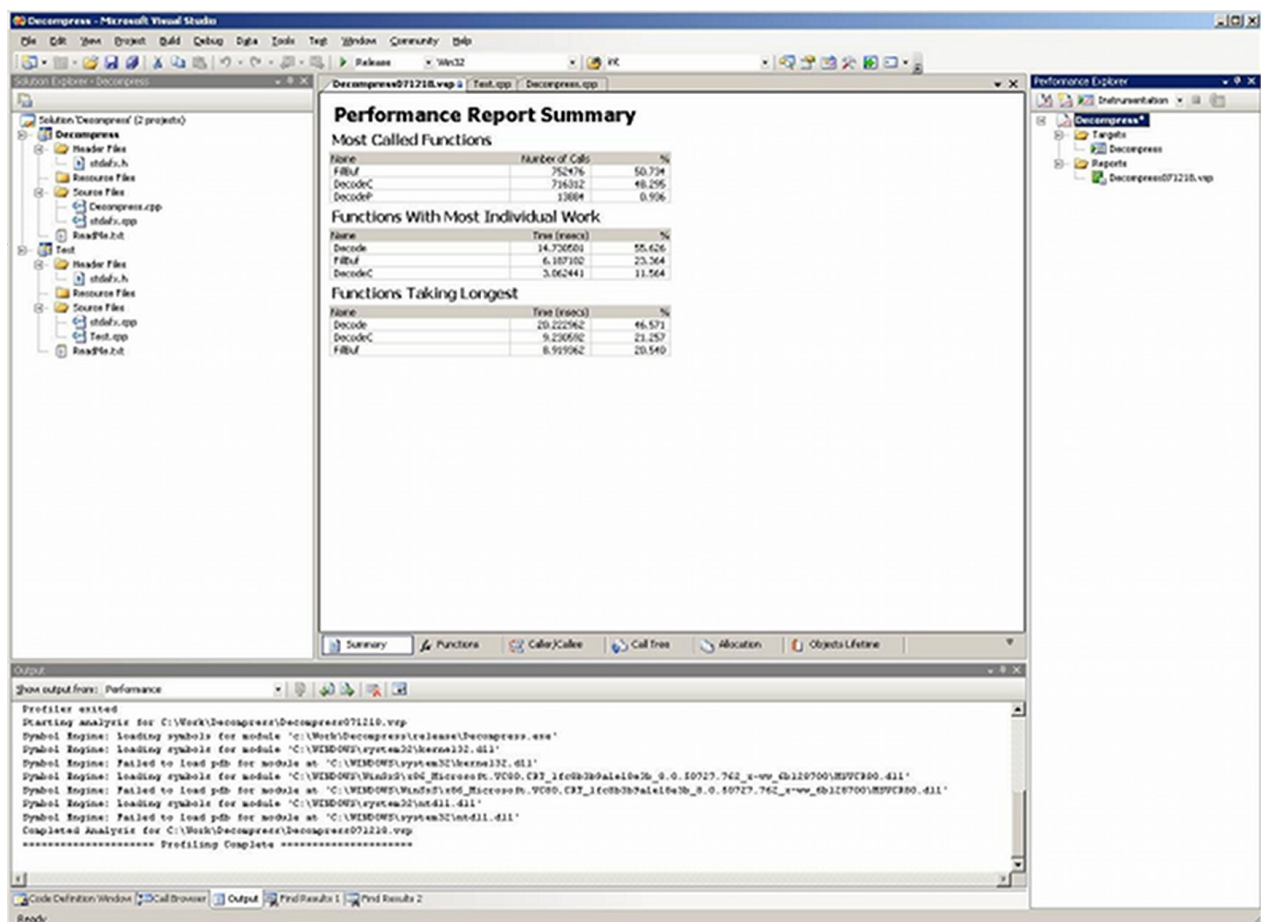


Figure 26 VS2005 Performance Summary

From the summary, it appears that Decode() must have a very hot loop in it. DecodeC and FillBuf are very simple, but they are called so many times a very small improvement will be multiplied by 100,000.

Expanding the call tree view can be very useful.

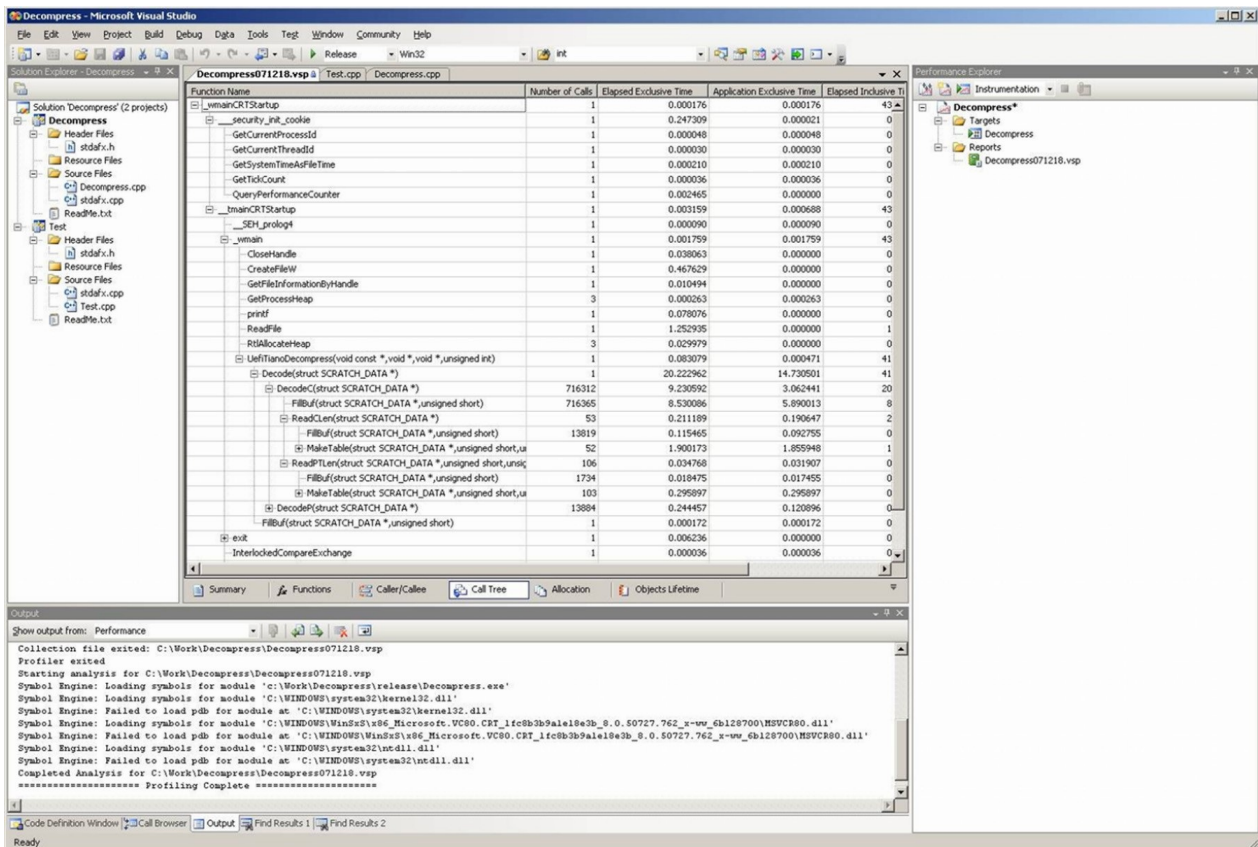


Figure 27 VS2005 Call Tree View

Definition of terms [http://msdn2.microsoft.com/en-us/library/ms242753\(VS.80\).aspx](http://msdn2.microsoft.com/en-us/library/ms242753(VS.80).aspx)

APPENDIX H MODULE TYPES

Table 23 EDK II Module Types

MODULE_TYPE	Supported Architecture Types	Description
BASE	Any	Modules or Libraries can be ported to any execution environment. This module type is intended to be used by silicon module developers to produce source code that is not tied to any specific execution environment.
SEC	Any	Modules of this type are designed to start execution at the reset vector of a CPU. They are responsible for preparing the platform for the PEI phase.
PEI_CORE	Any	This module type is used by PEI Core implementations that are compliant with the PI Specification.
PEIM	Any	This module type is used by PEIMs that are compliant with the PI specification.
DXE_CORE	Any	This module type is used by DXE Core implementations that are compliant with the PI Specification.
DXE_DRIVER	Any	This module type is used by DXE Drivers that are compliant with the PI Specification.
DXE_RUNTIME_DRIVER	Any	This module type is used by DXE Drivers that are compliant to the PI Specification. These modules execute in both boot services and runtime services environments.
DXE_SAL_DRIVER	IPF	This module type is used by DXE Drivers that can be called in physical mode before SetVirtualAddressMap() is called and either physical mode or virtual mode after SetVirtualAddressMap() has been called. This module type is only available for IPF processor types.
DXE_SMM_DRIVER	IA32, X64	This module type is used by DXE Drivers that are loaded into SMRAM.
SMM_CORE	Any	This is the SMM core.
UEFI_DRIVER	Any	This module type is used by UEFI Drivers that are compliant with the EFI 1.10 and UEFI specifications. These modules provide services in the boot services execution environment. UEFI Drivers that return EFI_SUCCESS are not unloaded from memory. UEFI Drivers that return an error are unloaded from memory.
UEFI_APPLICATION	Any	This module type is used by UEFI Applications that are compliant with the EFI 1.10 and EFI 2.0 specifications. UEFI Applications are always unloaded when they exit.

APPENDIX I VPD TOOL

This appendix describes the format of the build system created file containing the PCD information from the DSC file, and the output map file from an external tool that will be used by the EDK II build system to create header files for the PCD drivers.

I.1 Build System Output File Format

Summary

The build system will generate a text file containing a list of PCDs that have been declared as type VPD. An external tool that processes this file must be capable of reading the following format.

Prototype

```

<File> ::= <AutoGenHeading>
        [<CommentBlock>] [<PcdEntry>]*

<AutoGenHeading> ::= "## @file" <EOL> "##" <EOL>
        "## THIS IS AUTO-GENERATED FILE BY BUILD TOOLS"
        " AND PLEASE DO NOT MAKE MODIFICATION." <EOL>
        "##" <EOL>
        "## This file lists all VPD information for a"
        " platform collected by build.exe." <EOL>
        "##" <EOL>
        "## Copyright (c) 2010, Intel Corporation. All"
        " rights reserved.<BR>" <EOL>
        "## This program and the accompanying materials" <EOL>
        "## are licensed and made available under the"
        " terms and conditions of the BSD License" <EOL>
        "## which accompanies this distribution. The"
        " full text of the license may be found at" <EOL>
        "## "
        "http://opensource.org/licenses/bsd-license.php"
        <EOL> "##" <EOL>
        "## THE PROGRAM IS DISTRIBUTED UNDER THE BSD"
        " LICENSE ON AN \"AS IS\" BASIS," <EOL>
        "## WITHOUT WARRANTIES OR REPRESENTATIONS OF ANY"
        " KIND, EITHER EXPRESS OR IMPLIED." <EOL>

<CommentBlock> ::= ["#" <String> <EOL>]*

<PcdEntry> ::= <PcdName> "|" <Offset> "|" <Size> "|" <PcdValue> <EOL>

<PcdName> ::= <TokenSpaceCName> "." <PcdCName>

<TokenSpaceCName> ::= C Variable Name of the Token Space GUID

<PcdCName> ::= C Variable Name of the PCD

<Offset> ::= {"*"} {<Number>}

<HexNumber> ::= "0x" (a-fA-F0-9)8

<Size> ::= <Number>

<PcdValue> ::= if (pcddatetype == "BOOLEAN"):
        <Boolean>
        elif (pcddatetype == "UINT8"):
        <HexByte>
        elif (pcddatetype == "UINT16"):
        <HexWord>
        elif (pcddatetype == "UINT32"):
        <HexLong>
        elif (pcddatetype == "UINT64"):
        <HexLongLong>
        else:
        <StringData> [<MaxSize>]

<Number> ::= {<HexNumber>} {<NonNegativeInt>}

<PcdNumber> ::= if NumType == UINT8
        <HexByte>
        if NumType == UINT16
        <HexWord>
        if NumType == UINT32
        <HexLong>
        if NumType == UINT64
        <HexLongLong>

<HexByte> ::= "0x" (a-fA-F0-9){1,2}
<HexWord> ::= "0x" (a-fA-F0-9){1,4}
<HexLong> ::= "0x" (a-fA-F0-9){1,8}
<HexLongLong> ::= "0x" (a-fA-F0-9){1,16}

```

```

<Boolean>      ::= {<True>} {<False>}
<True>         ::= {"TRUE"} {"True"} {"true"} {"1"} {"0x1"} {"0x01"}
<False>        ::= {"FALSE"} {"False"} {"false"} {"0"} {"0x0"} {"0x00"}
<NonNegativeInt> ::= (0-9)+
<StringData>   ::= {<QString>} {<CArray>}
<QString>      ::= ["L"] <DbQuote> <String> <DbQuote>
<DbQuote>      ::= 0x22
<CArray>       ::= "{" <NList> "}"
<NList>        ::= <HexByte> ["," <HexByte>]*

```

Example

```

## @file
#
# THIS IS AUTO-GENERATED FILE BY BUILD TOOLS AND PLEASE DO NOT MAKE MODIFICATION.
#
# This file lists all VPD information for a platform collected by build.exe.
#
# Copyright (c) 2010, Intel Corporation. All rights reserved.<BR>
# This program and the accompanying materials
# are licensed and made available under the terms and conditions of the BSD License
# which accompanies this distribution. The full text of the license may be found at
# http://opensource.org/licenses/bsd-license.php
#
# THE PROGRAM IS DISTRIBUTED UNDER THE BSD LICENSE ON AN "AS IS" BASIS,
# WITHOUT WARRANTIES OR REPRESENTATIONS OF ANY KIND, EITHER EXPRESS OR IMPLIED.
#
gEfiMdeModulePkgTokenSpaceGuid.PcdVideoHorizontalResolution|*|4|800
gEfiMdeModulePkgTokenSpaceGuid.PcdVideoVerticalResolution|*|4|600
gEfiMdeModulePkgTokenSpaceGuid.PcdConOutRow|*|4|25
gEfiMdeModulePkgTokenSpaceGuid.PcdConOutColumn|*|4|80

```

I.2 VPD Tool Map File Format

Summary

The build system will expect the following format in the file generated by an external tool that processes the VPD PCDs. This format will be used by the build system to generate header files for the PCD drivers.

Prototype

```

<File> ::= <AutoGenHeader>
        [<CommentBlock>]
        [<PcdEntry>]*
<AutoGenHeading> ::= "## @file" <EOL> "##" <EOL>
        "# THIS IS AUTO-GENERATED FILE BY BUILD TOOLS"
        " AND PLEASE DO NOT MAKE MODIFICATION." <EOL>
        "##" <EOL>
        "# This file lists all VPD information for a"
        " platform collected by build.exe." <EOL>
        "##" <EOL>
        "# Copyright (c) 2010, Intel Corporation. All"
        " rights reserved.<BR>"
        "# This program and the accompanying materials"
        "# are licensed and made available under the"
        " terms and conditions of the BSD License"
        "# which accompanies this distribution. The"
        " full text of the license may be found at"
        "## "
        "http://opensource.org/licenses/bsd-license.php"
        <EOL> "##" <EOL>
        "# THE PROGRAM IS DISTRIBUTED UNDER THE BSD"
        " LICENSE ON AN \"AS IS\" BASIS,"
        "## WITHOUT WARRANTIES OR REPRESENTATIONS OF ANY"
        " KIND, EITHER EXPRESS OR IMPLIED."
<CommentBlock> ::= ["#" <String> <EOL>]*
<FS> ::= <Space>* " | " <Space>*
<Space> ::= 0x20
<PcdEntry> ::= <PcdName> <FS> <Offset> <FS> <PcdValue> <EOL>
<PcdName> ::= <TokenSpaceCName> "." <PcdCName>
<TokenSpaceCName> ::= C Variable Name of the Token Space GUID
<PcdCName> ::= C Variable Name of the PCD
<Offset> ::= "0x" (a-fA-F0-9){1,8}
<Size> ::= <Number>
<PcdValue> ::= if (pcddatumtype == "BOOLEAN"):
        "BOOLEAN" <FS> <Boolean>
        elif (pcddatumtype == "UINT8"):
        "UINT8" <FS> <HexByteZ>
        elif (pcddatumtype == "UINT16"):
        "UINT16" <FS> <HexWordZ>
        elif (pcddatumtype == "UINT32"):
        "UINT32" <FS> <HexLongZ>
        elif (pcddatumtype == "UINT64"):
        "UINT64" <FS> <HexLongLongZ>
        else:
        <Size> <FS> <StringData>
<HexByteZ> ::= "0x" (a-fA-F0-9)(a-fA-F0-9)
<HexWord> ::= "0x" (a-fA-F0-9)(a-fA-F0-9)(a-fA-F0-9)(a-fA-F0-9)
<HexLong> ::= "0x" (a-fA-F0-9)(a-fA-F0-9)(a-fA-F0-9)(a-fA-F0-9)
        (a-fA-F0-9)(a-fA-F0-9)(a-fA-F0-9)(a-fA-F0-9)
<HexLongLong> ::= "0x" (a-fA-F0-9)(a-fA-F0-9)(a-fA-F0-9)(a-fA-F0-9)
        (a-fA-F0-9)(a-fA-F0-9)(a-fA-F0-9)(a-fA-F0-9)
        (a-fA-F0-9)(a-fA-F0-9)(a-fA-F0-9)(a-fA-F0-9)
        (a-fA-F0-9)(a-fA-F0-9)(a-fA-F0-9)(a-fA-F0-9)
<Number> ::= {<HexNumber>} {<NonNegativeInt>}
<Boolean> ::= {<True>} {<False>}
<True> ::= {"TRUE"} {"True"} {"true"} {"1"} {"0x1"} {"0x01"}
<False> ::= {"FALSE"} {"False"} {"false"} {"0"} {"0x0"} {"0x00"}

```

```
<HexNumber>      ::= "0x" (a-fA-F0-9){2,16}
<NonNegativeInt>  ::= (0-9)+
<StringData>      ::= {<QString>} {<CArray>}
<QString>         ::= ["L"] <Db1Quote> <String> <Db1Quote>
<Db1Quote>        ::= 0x22
<CArray>          ::= "{" <HexByte> ["," <HexByte>]* "}"
<NList>           ::= <HexByte> ["," <HexByte>]*
```

Example

```
## @file
#
# THIS IS AUTO-GENERATED FILE BY BPDG TOOLS AND PLEASE DO NOT MAKE MODIFICATION.
#
# This file lists all VPD informations for a platform fixed/adjusted by BPDG tool.
#
# Copyright (c) 2010, Intel Corporation. All rights reserved.<BR>
# This program and the accompanying materials
# are licensed and made available under the terms and conditions of the BSD License
# which accompanies this distribution. The full text of the license may be found at
# http://opensource.org/licenses/bsd-license.php
#
# THE PROGRAM IS DISTRIBUTED UNDER THE BSD LICENSE ON AN "AS IS" BASIS,
# WITHOUT WARRANTIES OR REPRESENTATIONS OF ANY KIND, EITHER EXPRESS OR IMPLIED.
#
gEfiMdeModulePkgTokenSpaceGuid.PcdVideoHorizontalResolution | 0x0 | 4 | 800
gEfiMdeModulePkgTokenSpaceGuid.PcdVideoVerticalResolution | 0x4 | 4 | 600
gEfiMdeModulePkgTokenSpaceGuid.PcdConOutRow | 0x8 | 4 | 25
gEfiMdeModulePkgTokenSpaceGuid.PcdConOutColumn | 0xc | 4 | 80
```

APPENDIX J MAKEFILES

This appendix describes the format of the Makefiles created by the EDK II build system.

J.1 NMAKE Module Makefile Format

The build system will generate a top level Makefile for each module in the platform, using the target and tool chain as an identifier for the combination. If multiple architectures are part of the platform build, there will be a separate tree under the directory for each module that requires multi-architecture builds.

Template

```
# DO NOT EDIT
# This file is auto-generated by build utility
#
# Module Name:
#
#   %s
#
# Abstract:
#
#   Auto-generated makefile for building modules, libraries or platform
#
#
# Platform Macro Definition
#
PLATFORM_NAME      = ${platform_name}
PLATFORM_GUID      = ${platform_guid}
PLATFORM_VERSION    = ${platform_version}
PLATFORM_RELATIVE_DIR = ${platform_relative_directory}
PLATFORM_DIR        = $(WORKSPACE)${sep}${platform_relative_directory}
PLATFORM_OUTPUT_DIR = ${platform_output_directory}
#
# Module Macro Definition
#
MODULE_NAME        = ${module_name}
MODULE_GUID        = ${module_guid}
MODULE_VERSION     = ${module_version}
MODULE_TYPE        = ${module_type}
MODULE_FILE        = ${module_file}
MODULE_FILE_BASE_NAME = ${module_file_base_name}
BASE_NAME          = $(MODULE_NAME)
MODULE_RELATIVE_DIR = ${module_relative_directory}
MODULE_DIR         = $(WORKSPACE)${sep}${module_relative_directory}
#
MODULE_ENTRY_POINT = ${module_entry_point}
ARCH_ENTRY_POINT   = ${arch_entry_point}
IMAGE_ENTRY_POINT  = ${image_entry_point}
#
${module_extra_defines}
#
# Build Configuration Macro Definition
#
ARCH      = ${arch}
TOOLCHAIN = ${toolchain_tag}
TOOLCHAIN_TAG = ${toolchain_tag}
TARGET    = ${build_target}
#
# Build Directory Macro Definition
```

```
#
# PLATFORM_BUILD_DIR = ${platform_build_directory}
BUILD_DIR      = ${platform_build_directory}
BIN_DIR        = $(BUILD_DIR)${sep}${architecture}
LIB_DIR        = $(BIN_DIR)
MODULE_BUILD_DIR = ${module_build_directory}
OUTPUT_DIR     = ${module_output_directory}
DEBUG_DIR      = ${module_debug_directory}
DEST_DIR_OUTPUT = $(OUTPUT_DIR)
DEST_DIR_DEBUG  = $(DEBUG_DIR)

#
# Shell Command Macro
#
RD = rmdir /s /q
RM = del /f /q
MD = mkdir
CP = copy /y
MV = move /y

#
# Tools definitions specific to this module
#
${module_tool_definitions}
```

Note: `${module_tool_definitions}` individual lines that contain an environment name followed by the equal sign, followed by a string. The following two lines are an example.

```
TIANO = TianoCompress
TIANO_GUID = A31280AD-481E-41B6-95E8-127F4C984779

MAKE_FILE = ${makefile_path}

#
# Build Macro
#
```

Note: The value of `${macroname}` is derived from the file types identified in the `build_rule.txt` file, these are typically: `OBJECT_FILES` and `STATIC_LIBRARY_FILES` a special macro name, `INC` is also emitted listing the EDK II Include directories required from a module's dependent packages. The `INC` list entry prepends the compiler's option character sequence to include the directory in the search list.

```
${macro_name} = \
\t$(OUTPUT_DIR)${sep}${filename} \\\
```

Note: The above line is duplicated for additional files, the "\" is removed from the last line if this list. A text file for each of these macro sets (except `INC`) is generated in the module's `OUTPUT` directory.

```
${macro_name}_LIST = $(OUTPUT_DIR)${sep}${macro_name}.lst

COMMON_DEPS = \
\t${common_dependency_file} \\\
```

Note: The above line is duplicated for all header files required by the module (defined by the `#include` statements) and files included by header files required by the module

```
#
# Overridable Target Macro Definitions
#
FORCE_REBUILD = force_build
INIT_TARGET = init
PCH_TARGET =
BC_TARGET = $ {backward_compatible_target}
CODA_TARGET = $ {remaining_build_target} \\\
```

Note: The above line contains the name of the output .efi file generated by the drivers or the .lib file generated by libraries. The backslash character is added to allow extra lines after `$(CODE_TARGET)` is specified.

```
#
# Default target, which will build dependent libraries in addition to
# source files
#

all: mbuild

#
# Target used when called from platform makefile, which will bypass
# the build of dependent libraries since the platform makefile builds
# all libraries first.
#

pbuid: $(INIT_TARGET) $(BC_TARGET) $(PCH_TARGET) $(CODA_TARGET)

#
# ModuleTarget
#

mbuild: $(INIT_TARGET) $(BC_TARGET) gen_libs $(PCH_TARGET)$(CODA_TARGET)

#
# Build Target used in multi-thread build mode, which will bypass the
# init and gen_libs targets
#

tbuid: $(BC_TARGET) $(PCH_TARGET) $(CODA_TARGET)

#
# Phony target which is used to force executing commands for a target
#

force_build:
\t-@

#
# Target to update the FD
#

fds: mbuild gen_fds

#
# Initialization target: print build information and create necessary directories
```

```
#
init: info dirs

info:
\t-@echo Building ... $(MODULE_DIR)${sep}$(MODULE_FILE) [$(ARCH)]

dirs:
\t-@-@if not exist $(DEBUG_DIR) $(MD) $(DEBUG_DIR)
\t-@if not exist $(OUTPUT_DIR) $(MD) $(OUTPUT_DIR)

strdefs:
\t-@$(CP) $(DEBUG_DIR)${ds}AutoGen.h \
    $(DEBUG_DIR)${ds}$(MODULE_NAME)StrDefs.h
```

Note: The above two lines are not exact, as they will appear on the same line in the generated Makefile without the "\" line extension character.

```
#
# GenLibsTarget
#
gen_libs:
\t$@$(MAKE)" $(MAKE_FLAGS) \
    -f ${dependent_library_build_directory}${sep}${makefile_name}
```

Note: The above two lines are not exact, as they will appear on the same line in the generated Makefile without the "\" line extension character. The line is repeated for every library instance that the module requires to be linked against.

```
\t$@cd $(MODULE_BUILD_DIR)

#
# Build Flash Device Image
#
gen_fds:
\t@"$(MAKE)" $(MAKE_FLAGS) -f $(BUILD_DIR)${sep}${makefile_name} fds
\tcd $(MODULE_BUILD_DIR)

#
# Individual Object Build Targets
#
${file_build_target}
```

Note: The above line is repeated for each `CODA_TARGET` using the format from the `build_rule.txt` file to build intermediate files.

```
#
# clean all intermediate files
#
clean:
\tif exist $(OUTPUT_DIR) $(RD) $(OUTPUT_DIR)
\t$

#
```

```
# clean all generated files
#
cleanall:
\tif exist $(DEBUG_DIR) $(RD) $(DEBUG_DIR)
\tif exist $(OUTPUT_DIR) $(RD) $(OUTPUT_DIR)
\t$(RM) *.pdb *.idb > NUL 2>&1
\t$(RM) $(BIN_DIR){sep}$(MODULE_NAME).efi

#
# clean all dependent libraries built
#
cleanlib:
\t${library_build_command} cleanall
```

Note: The above `${library_build_command}` is repeated for every library instance used to link against the driver or application module. It first tests for the existence of the makefile and if it exists, runs the make command. If the module is a library, the above lines are not emitted.

```
\t@cd $(MODULE_BUILD_DIR)\n\n
```

APPENDIX K THIRD PARTY TOOL FLAGS

The following tables provide a summary of these "Best Known" options.

Note: A reserved keyword, `MDEPKG_NDEBUG`, can be used for code size reduction purposes.

Table 24 Standard C File Compiler Options

Microsoft	Intel	GCC	Description
<code>/nologo</code>	<code>/nologo</code>		Do not display compiler version information
<code>/c</code>	<code>/c</code>	<code>-c</code>	Compile C files to object (.obj) files only, do not link
<code>/WX</code>	<code>/WX</code>	<code>-Werror</code>	Force warnings to be reported as errors.
<code>/GS-</code>	<code>/GS-</code>		Disable security checks
		<code>-Wno-missing-braces</code>	Warn if an aggregate or union initializer is not fully bracketed. In the following example, the initializer for 'a' is not fully bracketed, but that for 'b' is fully bracketed.
		<code>-Wno-array-bounds</code>	Disables warnings if subscripts to arrays are out of bounds.
<code>/W4</code>	<code>/W4</code>	<code>-Wall</code>	Warning level 4 - print errors, warnings and remarks (or enable most warning messages)
<code>/Gs32768</code>			Control stack (32768 bytes) checking calls
<code>/Gy</code>	<code>/Gy</code>		Separate functions for linker.
<code>/O1b2</code>	<code>/O1</code>		Optimize for minimum space, enable intrinsic functions, enable in-line expansion.
	<code>/Oi</code>		Enable Intrinsic functions
	<code>/Ob2</code>	<code>-default-inline</code>	In-line any function, at the compiler's discretion (same as <code>/Qip</code>)
		<code>-O</code>	Optimize output file
<code>/GL</code>			Enable link-time code generation
<code>/EHs-c-</code>			Combine <code>/EHs-</code> and <code>/EHc-</code>
	<code>/EHs-</code>		Disable C++ EH (no SHE exceptions)
	<code>/EHc-</code>		Disable extern C defaults to no throw
<code>/GF</code>	<code>GF</code>		Enable read-only string pooling
<code>/GR-</code>			Disable C++ RTTI
EDK II Specific Flags			
<code>/D UNICODE</code>	<code>/D UNICODE</code>	<code>-DUNICODE</code>	define macro UNICODE
<code>/FIAutoGen.h</code>	<code>/FIAutoGen.h</code>	<code>--include AutoGen.h</code>	Always include AutoGen.h file
Debug			

Specific Flags			
/Zi	/Zi	-g	Enable debugging information
/Gm	/Gm		Enable minimum rebuild
		-fshort-wchar	Force the underlying type for "wchar_t" to be "unsigned short"
		-fno-stack-protector	
		-fno-strict-aliasing	
		-ffunction-sections	
		-fdata-sections	
IPF Specific Flags			
/Ox			Maximum Optimization (/Ogityb2 /Gs)
/X			ignore standard places
/QIPF_fr32			Do not use upper 96 Floating Point Registers
/Zx			Generates debug-able optimized code. Only available in the IPF cross compiler or IPF native compiler.

Table 25 Assembly Flags

Microsoft	GCC	Description
/nologo		Do not display assembler version information
/c	-c	Generate object (.obj) files, do not link
/WX		Treat warnings as errors
/W3		Warning level 3
/Cx		Preserve case in publics and externs
/coff		Generate COFF format object files
/Zd		Add line number debug info
/Zi		Add symbolic debug info (DEBUG target)
	-x assembler	Input files are in assembly language
	-imacros AutoGen.h	Accept definition of macros in AutoGen.h

Table 26 C Compiler's Preprocessor Options

Microsoft	Intel	GCC	Description
/nologo	/nologo		Do not display compiler version information
/E	/E	-E	Preprocess only; do not compile, assemble or link
/TC	/TC	-x assembler-with-cpp	Compile as .c files

/FIAutoGen.h	/FIAutoGen.h	--include AutoGen.h	Always include AutoGen.h file
--------------	--------------	---------------------	-------------------------------

Table 27 C Compiler's Preprocessor Options for VFR files ONLY

Microsoft	Intel	GCC	Description
/nologo	/nologo		Do not display compiler version information
/E	/E	-E	Preprocess only; do not compile, assemble or link
/TC	/TC	-x c	Compile as .c files
/D VFR_COMPILE	/D VFR_COMPILE	-DVFR_COMPILE	Used only for Preprocessing VFR files
		-P	Used only for Preprocessing VFR files - do not generate #line directives
/FI\$(MOD_NAME)StrDefs.h			Force include of the module's StrDefs.h file.

Table 28 Pre-compiled Header (PCH) Creation Flags

Microsoft	Intel	GCC	Description
/nologo	/nologo		Do not display compiler version information
/c	/c	-c	Compile C files to object (.obj) files only, do not link
/W4	/W4	-Wall	Warning level 4 - print errors, warnings and remarks (or enable most warning messages)
/WX	/WX	-Werror	Force warnings to be reported as errors.
/Gy	/Gy		Separate functions for linker.
/GS-	/GS-		Disable security checks
/O1	/O1		Optimize for Maximum Speed
/Oi	/Oi		Enable Intrinsic functions
/Ob2	/Ob2	-default-inline	In-line any function, at the compiler's discretion (same as /Qip)
/GL			Enable link-time code generation
/EHs-	/EHs-		Disable C++ EH (no SHE exceptions)
/EHc-	/EHc-		Disable extern C defaults to no throw
/GF	/GF		Enable read-only string pooling
/Gs8192	/Gs8192		Control stack (8192 bytes) checking calls
/TC	/TC		Compile as .c files
/Yc			Create the .pch file
/Gm			Enable minimal rebuilds
/FpAutoGen.h.gch			
/X	/X		Ignore standard places
/Zi	/Zi		Produce debugging information

Table 29 Static Linker Flags

Microsoft	GCC	Description

/nologo		Do not display compiler version information
/LTCG		Use link-time code generation

Table 30 Dynamic Linker Flags

Microsoft	GCC	Description
/NOLOGO		Do not display compiler version information
/NODEFAULTLIB	-nostdlib	Disable using default libraries
/IGNORE:4086	N/A	Use /Gz option instead
/OPT:ICF=10		Perform identical COMDAT folding (10 iterations) to remove duplicates.
/MAP	-Map filename.map	Create a map file.
/ALIGN:32	--section- alignment 0x20	Use 32-byte alignment instead of the default 4K
	--file- alignment 0x20	
/MACHINE:\$	N/A	Where \$ is one of: I386 , AMD64 OR IA64
/DLL	--dll	The output is a DLL
/LTCG		Use link-time code generation
/ENTRY:\$(ENTRYPOINT)	--entry _\$(ENTRYPOINT)	The function that specifies a starting address.
/SUBSYSTEM:CONSOLE	--subsystem console	Do not use the EFI_* subsystem interface, as this is EFI 1.0 compliant, not UEFI compliant.
/SAFESEH:NO		Do not produce an image with a table of safe exception handles
/BASE:0	--image-base 0x0	Base address is always 0, and will be adjusted later by the build tools when creating images.
/DRIVER		Specify Kernel mode
/DEBUG		Create debugging information
	-O2	Optimize
	--gc-sections	Enable garbage collection of unused input sections
	--export-all- symbols	All global symbols in the objects used to build a DLL will be exported by the DLL.